F-PC Technical Reference Manual

F-PC Forth for the IBM PC — Technical Reference Manual

C. H. Ting · Offete Enterprises, Inc. (1988)

Reconstructed from the on-line HTML edition at the eForth Academy Library (eforth.com.tw), November 2026.
The print edition is out of print; this PDF was compiled from the freely published web chapters.

Contents

1. Prefaces and Contents
2. Chapter 1 - The F-PC Virtual Forth Machine
3. Chapter 2 - The F-PC Kernel
4. Chapter 3 - Input, Output and Numbers
5. Chapter 4 - Interpreter and Compiler
6. Chapter 5 - System Initiation
7. Chapter 6 - Kernel Utilities
8. Chapter 7 - Basic Utilities
9. Chapter 8 - Basic Utilities (continued)

PREFACE TO THE SECOND EDITION:

OUT OF THE SHADOW OF F83

It has been almost a year since F-PC 2.25 was released. It is very encouraging to see that F-PC spread rapidly inside the Forth community. It is also surprising that F-PC reached many new users who have not considered using Forth because of its perceived weaknesses and limitations. Due to the fact that we were able to immobilize Tom Zimmer from further modifying and enhancing this version, many veteran Forth programmers were convinced that they had a solid ground to build useful applications. We have seen many substantial programs converted and developed in this environment.

However, Tom could not be held steady for long. There were reported bugs in F-PC he had to fix, and there were great ideals to be put in to enhance its performance. In the end, a compromise was stricken that he could make changes anyway he thought necessary, but the modified system could not be called F-PC and F-PC would stay at Version 2.25 until November, 1989. He distributed the modified versions under the name F-TZ, whose version number clicked from 2.25 to 3.34. The changes and the enhancements were very impressive, especially the hypertext help system and the source code browser. Bob Smith also cleaned up the help files associated with the KERNEL files. When the F-PC Working Group reconvened in its August, 1989 meeting, it decided unanimously to adopt F-TZ 3.34 as F-PC 3.5.

When F-PC was first released, it was necessary to emphasize its heritage of F83 in order to capitalized on the popularity of F83, and to acknowledge the fact that much of the source code in the kernel were simply lifted from F83. In the course of enhancing F-PC, Tom went through every word and managed to change most of them for one reason or another. He left no stone unturned, without a single thought about how difficult it is to update the documentations. One of the major reasons to change the kernel so extensively was the new target compiler Tom built. It has to compile words independently into the target code image. In F83, there were lots of shared code, which is executed by several words to reduce the size of the dictionary. The new target compiler cannot detect that a word must jump to other words, and consequently every word must be completely self-contained. Therefore, all kernel words using shared code were flattened. Another reason of modification is performance. A good example is the addressing of multiple items on the return stack. Tom developed three styles: indexing through BP register, indexing through DI register, and swapping the SP and BP registers and then using PUSH and POP instructions. The fastest method was chosen in each instance when the return stack is manipulated. The outer shell of F83 (the names of words) is preserved, but everything underneath was changed.

In F-PC 2.25, much of the F83 tools to access block file like LIST, LOAD, VIEW, etc. are retained in spite of the switch from block files to sequential files. These tools are less meaningful in the new F-PC 3.5, because of the much more powerful browsing facility and the hypertext help system. In F83, as in most other Forth systems, the text interpreter is the programming environment and the editor is an application to provide services on the manipulation of the source code. In F-PC 3.5, much of the programming environment is shifted into the editor. The editor allows you to compose source text, and to explore the system. You will spend as much time in the editor as in the interpreter.

At this point, F-PC is ready to assert its own identity, equal to if not surpassing F83. I hope that it will become a common vehicle by which many Forth programmers will use to build applications and also to communicate ideals and algorithms.

My last comment on F-PC is about its size. F-PC 3.5 has more than 2600 words, making it the largest public domain Forth system. It is also larger than most commercial Forth systems. To put it in perspective, figForth has about 350 words, and F83 has about 1000 words. The largest commercial Forth system I know is Harvard Softworks' HS/FORTH, which has about 1700 words. The growing size is demanded by the increases in functionality and also in convenience to the user. However, the convenience comes not without a price, which includes greatly increased demands on memory and disk storage, and the demand on the user in the comprehension of the entire system. F-PC is very friendly to the user in doing routine Forth programming. However, like other large language compilers, it is difficult to modify to suit specific needs. It is amazing that the whole system would work at all. The only reason that it works at all is the tireless efforts Tom Zimmer put into it.

Friendliness is all in the eyes of the beholder. F-PC proves that Forth can be as friendly as any other systems if you care to put in the features that the user needs. Incorporating all the features most programmers need result in a mammoth system, which adds little to a programming language and Forth in particular. The essence of Forth is modularity, interactivity, and simplicity. These qualities, though preserved in F-PC, are obscured by the huge volume of F-PC. Lots of tools, utilities, and conveniences (like all the earthly possessions) give you only a false sense of security. It is the complete understanding of the bare essentials which gets the job done.

A couple of years ago, Chuck Moore released his cmForth for the Novix 4000 Forth chip. Many people were amazed at what he accomplished in 30 screens: a complete Forth system with an optimized compiler and a target compiler. After studying cmForth carefully, my feeling could only be expressed using the words from Lao-Tze, the legendary progenitor of Toaism:

"For knowledge, add a little everyday;

for wisdom, erase a little everyday."

Here we have two contrasting Forth systems. By adding a little everyday, Tom Zimmer built F-PC with tons of features and utilities. Polishing off all the non-essential elements in Forth, Chuck Moore gave us the functional definition of Forth in its purest form.

So, go ahead and enjoy F-PC. While you are doing it, keep in mind that the Tao of Forth is not very far and it can always be reached with some time and efforts.

C. H. Ting

November 1989

PREFACE TO THE FIRST EDITION:

FOOLISH OLD MAN MOVING A MOUNTAIN

As I dug through the source code in the F-PC system every evening for the last four months, I was constantly reminded of this old Chinese proverb: 'Foolish Old Man moving a mountain.' Shifting through two million bytes of Forth source listings and trying to bring some sense to the whole thing is often very frustrating. My wife kept nagging me: "When are you going to finish it and be a father to the kids?"

The Foolish Old Man had one definite advantage over me. The mountain didn't move, while Tom Zimmer just kept on enhancing and sometimes made major modifications on the F-PC. Every time I met him in the office, he would say: "Want to see something new?" I definitely did not want to see anything new. I just wanted to finish the documentation, but that didn't help. Anytime I finished documenting a file, it got changed the next morning. I shot at a moving target, and always hit its shadow.

We figured that the only way to get F-PC stabilized was to shoot Tom. But he protested: "Then who's going to fix bugs in F-PC?" Well, there went our argument.

The game actually started last year, 1987. Tom distributed his systems at FIG chapters meetings. The name of his system kept changing. He ought to have exhausted the alphabets, going through HF, DF, PF, ZF, etc. Bob Smith also gave me his LaForth system. At that time I was working on my 'Forth Notebook,' Volume 2. I promised that after I finished my book, I would start working on one of their systems. As 1987 draw to a close, I kept on getting new disks and updates, so I figured that I might as well wait a bit longer till they settled down. After the new year, I got trapped in coding Bach's 'The Art of Fugue.' It was a very fascinating project and I was fully occupied and incapacitated for several months.

Just after the new year, my working conditions at Lockheed deteriorated to such a low point that I quit. My presence in Lockheed had never been truly appreciated. I was only marginally tolerated because I got jobs done. The reason I got jobs done was I insisted on doing things my way, i.e. the Forth way. People would want them done in Fortran, in C, in Pascal, and lately in Ada. My department manager and my supervisor tried to enlighten me on other avenues to achieve the same goal, but did not persist that I change my addiction to Forth. The situation became unbearable after both of them were relieved of their managerial positions early in 1987. I lost my protective shields and had to deal with reality in the company. That ability was not engraved in my genes.

I found my refuge at Maxtor Corporation. It has a very strong group of Forth programmers in its Test Engineering Department. My primary motive in going there was that I would not have to constantly justify my using Forth as programming tool versus other languages and tools. The real thrill is to work in the company of Tom Zimmer, Bob Smith, Jerry Modrow, Charles Curley, and many other Forth programmers in a very harmonious and courteous atmosphere.

After Bob Smith taught a Forth class in the company, he decided to organize a weekly technical seminar with a strong bias towards topics in Forth. I got a chance to play ' The Art of Fugue' in one of the seminars. In another session, Tom talked about XF, his then latest version of Forth. In the discussion that followed, he mentioned that Maxtor did not hold any rights on the package and he was making it public domain. I got this brilliant idea that we would release it with full documentation and call it F88. The name was very catchy because of its F83 heritage, its segmented architecture optimized for the 80x88 CPU's, and the current year of 1988. It would also force us to work hard so that it would be released before 1989 hits us.

After reaching some consensus among the Forth Mafias in Maxtor, the project was carried to the Silicon Valley FIG Chapter for support and beta testing. An F88 Working Group was organized in a Chapter meeting and the Chapter also allocated several FORML sessions to the Working Group to arrange presentations and discussions on F88. The response from chapter members was very favorable and many people took the diskette set for testing and evaluation.

However, the name F88 irritated some members because its similarity to F83 and its connotation of a Forth-88 standard. The worst news was that Martin Tracy had it as a trade mark. At one time, we toyed with the name FAT-Forth, which is quite appropriate because the system is fat, and it is best used on an AT class machine. Eventually, Tom and Bob decided to use the name F-PC, which was accepted without any objection.

I volunteered to coordinate the documentation efforts, fully aware that coordination might mean actual writing the manual myself. The original plan was to generate two documents to accompany the released version of F-PC: a User's

Manual which explains how to use the system, and a Systems Manual to explain how the system works. The documentation effort looked quite easy at that time because Tom had already generated a substantial amount of documentation, included in the system as on-line help and many related documentation files. What had to be done, it seemed, was to take the existing documentation, rearrange it a little bit, add some glue and the job would be done.

I was particularly interested in documenting the kernel in F-PC because that was one thing I had tried to avoid in all my previous books on Forth systems, including 'Systems Guide to figFORTH' and 'Inside F83.' The reason was that a Forth kernel must be written in assembly and it is messy. At the same time, you cannot write about the kernel without first choosing a specific CPU. In figFORTH, you have the choice among 8080, 6502, PACE, PDP-11, 9900, and 1802. In F83, you have 8080, 8088/86, and 68000. If you made a choice, it would certainly be the wrong one. In F-PC, there is no problem because it is for 80x88 and for 80x88 only. Since IBM with some substantial help from many Taiwanese sweat shops cornered 50% of the market place with the 80x88/86 series of personal computers, a manual focused on an 80x88 kernel should find a good size audience.

Additional encouragement came from Dr. S. Y. Tang, who suggested that a good guide on assembly programming in Forth could be a nice product he could use. F-PC is a perfect vehicle to teach 80x88 assembly programming, because the assembler in Forth is interactive and there are numerous examples of code words in the F-PC kernel. One can pull out a code word in the F-PC kernel and modify it a little to suit his problem. Load it and get it done. I even thought of a very catchy title: "How to succeed in Assembly Programming without Trying." With the large number of code words in F-PC, one should be able to find enough existing code to solve any problem.

The F-PC User's Manual was completed in July. The Systems Manual, however, overwhelmed me. We missed the perfect release date, August 8, 1988, which was supposed to be the luckiest day in this century. It was not too much of a loss, however, after losing F88 for its name. We pushed back the release date to November, and hoped that we would distribute it at the Annual Forth Convention at Anaheim in Southern California. The name was also changed to F-PC Technical Reference Manual. A thick manual needs a long name.

I also had to reduce the scope of this Manual to meet the deadline. The 8088 assembler, the multitasker, and the meta compiler will be documented in another volume later. These omissions are not detrimental because they are very close to similar utilities in F83, which were fairly elucidated in "Inside F83." Tom Zimmer's editor was still evolving and nobody wanted to do anything on it before it solidified. Bob Smith claimed that he was exhausted implementing the floating point packages and had no energy left to write a detailed documentation for it. We are trying to convince Mark Smiley that he should write the reference manual for the floating packages with his contributions on graphics. We need help.

With the exceptions of 8088 assembler, the massive editor, the multitasker, and the meta compiler, this F-PC Technical Reference Manual contains narration on all the words inside the F-PC kernel. The source code files include KERNEL1.SEQ to KERNEL4.SEQ, with many other files needed for the kernel to function, like console and printer input/output, file interface, and interface to the DOS operating system. The narration follows the loading order of the source files. Related words are grouped together in chapters and sections, which usually begin with some overviews and introductory remarks.

Each word is most often documented in an imaginary screen with three windows: The top window runs between the left and right margins, containing the name and a functional description of this word. The lower left window contains the source code presented in the vertical style. The lower right window contains comments on the corresponding code in the left window. The underlying desire was to make this manual loadable, if the text in the top window were commented out. Then we would have a system in which truly complete documentation is provided with the source code. The idea is still premature. Nevertheless, it is not impossible if everybody were to get an optical disk drive from Maxtor...

To appreciate F-PC, one has to understand the reasons why Tom developed this system. The first is to break the 64 Kbyte barrier so that large and sophisticated applicationss can be written on top of it. The second is to provide a very smooth interface among the user, the Forth system, and the DOS operating system. A tremendous amount of his time was spent on the editor, which becomes a very friendly word processor. Rough corners in the assembler, debugger and many other F83 utilities were also patiently polished. The third, but not the least, is speed. Tom's obsession to speed lead to many strange features in F-PC, such as 64 hashed threads in vocabulary, upper case conversion through a large lookup table, flattening out a load-byte-store-word loop in the video output routine, etc. It makes F-PC one of the fastest Forth systems for PC/XT/AT computers.

This Manual serves three different types of Forth users. The advanced users would most likely concentrate on Chapter 1 and 4, which describe the virtual Forth engine, the interpreter, and the compiler. Users who have had experience in using Forth might want to dip into 8088 assembly code to optimized an application. Chapters 2, 3, and 6 contains numerous low level code words which are useful as templates and examples to be borrowed to construct words in assembly. Chapters 7 and 8 are mostly utilities in colon words. Those who are new to Forth are encouraged to scan over them for guidance in Forth programming techniques and good programming style.

F-PC is much more than can be covered in a single volume. A huge number of user contributed files are included, covering a wide variety of topics. Documentation is provided in all these files, though the degree of completeness varied from file to file. With the knowledge on the F-PC kernel presented in this Manual, struggling through these files should be much less painful. We certainly hope that many users can manage to build useful, even profitable, applications with the tools supplied in this system.

As the newest member in the tradition of public domain Forth systems, following the steps of figForth, MVP-Forth, and F83, F-PC is an effort to address many needs of the current Forth users. Whether it addresses these needs successfully or not, only time can tell. I hope that this Manual will help the user climb the learning curve a little bit faster and that his project can be completed a little bit earlier.

Let's applaud Tom Zimmer for making F-PC publicly available and also his continuing efforts in polishing it and enhancing it, though against the wishes of most users and myself in particular. Bob Smith also deserves acknowledgement in contributing many code and ideas in its evolution. They have also provided inspiration and encouragement to me in writing this Manual. It is impossible to acknowledge all the people who helped in documenting the F-PC system. However, my special thanks are due to John McKnight who carefully read through the manuscript and made numerous suggestions and comments to make this Manual easier to read and comprehend.

What about that Foolish Old Man?

The story as told by Le-Tze is as follows:

The Foolish Old Man decided to move the mountain in front of his house so that he could get to places easier. The Wise Man On The Riverbank laughed at him, saying: "The mountain is so big, and you are so old. How much can you dig up in your lifetime?" The Foolish Old Man answered: "Though I am old and my strength is fading, I will dig as much as I can. After I die, my sons will continue digging. After they die, their sons will continue digging. And so on. There will be many and many of us, and this mountain does not grow. We will get rid of it eventually."

The Mountain God heard the conversation and was scared. He reported to the Emperor In Heaven. The Emperor sent two Muscle Gods and they moved the mountain out of the Foolish Old Man's way.

With this big mountain moved, what's in your way of using F-PC?

Happy Forthing!

C. H. Ting

San Mateo, California

October, 1988

CONTENTS

Preface to the Second Edition: Out of the Shadow of F83 i

Preface to the First Edition: Foolish Old Man Moving a Mountain iii

Contents vi

1. F-PC Virtual Forth Machine 1

1.1. Forth on an segmented architecture 1
1.2. The F-PC memory map 1
1.3. The virtual Forth machine 3
1.4. The code and parameter fields 4
1.5. The colon lists 6
1.6. The heads 6
1.7. F-PC virtual machine 8

2. The F-PC Kernel 16

2.1. The stack words 16
2.2. Arithmetic and logic words 22
2.3. Memory words 28
2.4. Comparison words 33
2.5. Input and output words 37
2.6. Double integer words 39

2.7. The structure words 43

3. Text Input, Output, and Numbers 57

3.1. User variables and user deferred words 57
3.2. System variables 58
3.3. String words 59
3.4. DOS words 65
3.5. Long memory words 69
3.6. Memory allocation words 71
3.7. Head and list segment words 73
3.8. Terminal output words 76
3.9. Expect words 78
3.10. Number conversions words 81
3.11. The parsing words 86

4. Interpreter and Compiler 91

4.1. The dictionary words 91
4.2. Interpreter words 96
4.3. Compiler Tool words 101
4.4. Literal word compilers 102
4.5. String literal compilers 104
4.6. Forget 106
4.7. Structure compilers 107
4.8. Colon and other compilers 112

5. System Initialization 122

5.1. User vocabulary 122
5.2. IS and SELECT 123
5.3. F-PC boot system 124
5.4. F-PC initialization 126
5.5. Initial user area 131
5.6. Debugging words 131
5.7. Resolving forward references and deferred words 134
5.8. Saving F-PC kernel 135

6. Kernel Utilities 137

6.1. Video output words 137
6.2. Values: constants as variables 141
6.3. Save and restore variables and values 145
6.4. Handles 147
6.5. Console and printer words 155
6.6. Sequential file 156
6.7. File path 167
6.8. Command tail 170

7. Basic Utilities 172

7.1. General utility words 172
7.2. Screen display words 174
7.3. Vocabulary words 179
7.4. Task chains 182
7.5. F-PC Decompiler 183
7.6. DUMP 194

7.7. CASE 196

7.8. Set directory path 198

7.9. String search 199

7.10. WORDS 200

7.11. Cursor control words 206

7.12. Monochrome video words 209

7.13. Color video words 210

7.14. Save screens 211

7.15. Timer words 213

7.16. Comments 219

7.17. Conditionally compilation 221

8. High Level Utilities 223

8.1. HELLO 223

8.2. Fast video 225

8.3. VIEW and HELP 225

8.4. Status 227

8.5. Line editing words 229

8.6. Window file selection 236

8.7. Files status words 246

8.8. NEEDS 246

8.9. Environment manipulation words 247

8.10. DOS words 250

8.11. Save .EXE file 254

8.12. Hypertext 259

Index 262

Figures

Figure 1. Memory map of F-PC. 2

Figure 2. The code fields of different types of words 5

Figure 3. Structure of a colon definition 7

Figure 4. Data structures in colon words 45

Figure 5. Control structures in F-PC 46

Figure 6. The name hash table 93

Figure 7. Vocabularies and their linkage to VOC-LINK 120

Figure 8. Handle stack and related pointers 161

CHAPTER 1. F-PC VIRTUAL FORTH MACHINE

1.1. FORTH ON A SEGMENTED ARCHITECTURE

Nowadays people think that segmented architecture was Intel's greatest sin against humanity and that IBM's adopting this architecture for its PC series of computers set the computer industry back by 10 years. However, looking back to the time when this architecture was first developed for the 8088 in the middle of 1970's, it was a real breakthrough against the tight grip of the 64K byte limit on the addressing space.

The segmented architecture was adopted by Intel long before the 8088 was even designed. The Macro Assembler developed for the Intel Microprocessor Development Systems (MDS) had the assembler directives CODE SEGMENT, DATA SEGMENT, and STACK SEGMENT to separate assembler modules so that data and code would not be mixed. The 8088 was designed to reduce this software concept to silicon. Standing at the threshold between 8 bit and 16 bit microprocessor designs, it was understandable why IBM picked the 8088 as the backbone of its new PC lines, which it shoved down our throats and practically wiped out all other microprocessors except the 68000 family.

As far as Forth is concerned, segmented architecture is no vice. There is no problem in making full use of all the resources provided by the CPU. F83 did not take advantage of the segmented architecture of 8088, nor the 32 bit addressing space of 68000. It was derived from the 8 bit figForth model, and tried to be compatible among the three implementations for 8080, 8088/86, and 68000. The primary concerns of the F83 implementers were portability and compatibility across the different CPU's, and it was thus restricted to a 16 bit Forth.

The primary concerns of F-PC are performance and addressibility of the entire memory space in the 8088 architecture. It started with F83/8086 Version 2.1.0, which is explained fully in "Inside F83" by the author, available from Offete Enterprises. All words needed by the interpreter and compiler have been recoded in assembly. The dictionary has been carefully partitioned into a code segment, a head segment, and a list segment. The list segment has been designed so that it occupies more space than a single 64K byte segment. The result is a highly specialized Forth optimized for the 8088 segmented architecture under the MS-DOS operating system.

1.2. THE F-PC MEMORY MAP

In this section, we shall explore the partitioned segment structure of F-PC and how the Forth machine is built upon the 8088 architecture. Figure 1 shows an overview of the memory map of F-PC in the 640K byte space provided by a PC/XT/AT system.

At the bottom of the memory map, the DOS occupies an area to provide the basic services like directory listing, file copying, loading and executing application programs. F-PC is loaded above the DOS area. The first part of F-PC is the Code Segment of 64K bytes. It contains all the low level executable code, constants, variables and other data structures, including the data and return stacks, terminal input buffer and other temporary data storage areas. Most of the Forth words deal with data stored in this segment because the CS, DS and SS segment registers are all pointing to the beginning of this Code Segment.

bytes long. Its length can be increased further by changing the constant #CODESEGS which specifies the size of this segment in number of 16 byte pages. The List Segment contains lists which are sequences of addresses pointing to executable code in the Code Segment. Lists are justified to the 16 byte page boundary. Each colon

Figure 1. Memory map of F-PC.

definition is represented by a list in the List Segment. In a large Forth system, most words are colon definitions, and it is necessary to provide as much space as needed to store them. In F-PC, the List segment can be made to occupied as much as 400K bytes or whatever the DOS allows.

The variable XSEG stores the base segment address pointing to the beginning of the List Segment. Lists in the List Segment are addressed by their segment offset relative to XSEG. Using the segment offset instead of address offset to address lists makes it possible to address a list space greater than 64K bytes. The trade-off is that there are bytes wasted at the end of a list before the next list starts at the next page boundary. However, the wasted memory only accounts for about 5 percent increase in the total sized of F-PC.

The third segment is the Head Segment; it is next above the List Segment. It contains the headers or heads of all the words defined in F-PC. The headers are linked into vocabularies and vocabularies are hashed into threads. This linked structure of headers allows F-PC to be executed and compiled very fast. The number of threads used in a vocabulary is specified by the constant #TTHREADS during metacompilation. Currently, a vocabulary is hashed into 64 threads with a special hashing algorithm which distributes words evenly among the 64 threads to optimize the compilation of source files.

The Code and List Segments are absolutely required for F-PC to run. In a turnkey application the Head Segment can be stripped to conserve memory space. To support user interaction, the Head Segment is necessary so that dictionary and vocabularies can be searched. For program development, the editor in F-PC requires additional space for text manipulation. The Text Buffer occupies another 64K segment. It is allocated only when the editor is invoked to edit a file. To help managing text, the editor also allocates space for a Line List Buffer, a Delete Buffer, and a Restore Display Buffer.

Space above the editor buffers is managed by DOS. If an application needs more data storage space, the user can obtain more memory from DOS using DOS service request 48H. Thus the user can access the entire 640K bytes of memory in a PC/XT/AT system.

1.3. THE VIRTUAL FORTH MACHINE

Forth is a computer model which uses two stacks for subroutine nesting, and executes a set of instructions which are represented by addresses. The execution addresses are generally arranged as lists. Lists are again represented by addresses which can be used to build more powerful lists. In Forth, the execution addresses or instructions are given

English-like names which are called words. A Forth system provides a basic set of words. The user can define new words using the existing words. The word set is extended until it encompasses the whole application. Programming in Forth is thus conceptually different from programming in other conventional languages. In Forth, you do not write programs. Instead, you design an instruction set which becomes the solution to your problem or your application.

This model of a computer consists of a set of registers, a dictionary which contains all the words and their execution procedures, and additional memory to host two stacks and other necessary buffers. Any actual computer can be programmed so that it supports the Forth model. Thus a real computer is turned into a 'Virtual Forth Machine' which executes a Forth instruction set. To host a virtual Forth machine, many of the registers in the 8088/86 are dedicated to functions required by the Forth model. The projection of Forth registers on the 8088/86 CPU is shown in Table 1.

The 8088/86 definitely is not the ideal CPU to host a virtual Forth machine. BP is not a real stack pointer and it is awkward to manipulate data on the return stack. All the registers in the 8088 have some specialized characteristics which makes coding any application a real challenge. Nevertheless, F-PC is coded to make the best use of all the registers in the 8088 and squeezes the last ounce of performance out of this CPU.

Table 1. Assignments of Forth Registers in 8088

Forth Register	8088/86 Register	Function of Forth Register
SP	SP	Data stack pointer
RP	BP	Return stack pointer
IP	ES:SI	Next instruction pointer
W	AX	Current word pointer
UP	Variable UP	User area pointer
Scratch	BX,CX,DX,DI	Scratch pad registers
---	CS,DS,SS	Registers which must be restored before NEXT

At the assembly-language level, data are stored on the data stack by the PUSH instruction, and are popped back into internal registers for manipulation by POP. Addresses or data on the return stack can be moved to/from the internal registers by MOV instruction using BP relative mode to specify items on the return stack. The ES:SI register pair is the Instruction Pointer (IP), which is used to point to an address where the execution address of the next word to be executed is stored. The self incrementing string instruction LODSW ES: is thus very useful in processing a list of execution (code field) addresses stored in the List Segment.

As the LODSW instruction reads the execution address into the AX register, a JMP AX instruction causes the referred word to be executed. An interesting consequence is that the AX register always points to the beginning of a code field when a Forth word is executed. Hence the AX register becomes the Current Word or W register in the virtual Forth machine, very naturally. If a Forth word has to access data stored in its Body or Parameter Field, the address of Body can be computed by adding 3 to AX. If the Forth word does not refer to its code field or its Body, AX can be used freely as a scratch pad register. In fact, most Forth code words uses AX as the first choice of scratch pad. The second choice of scratch pad register is DX. Special code ending words are defined to push AX or AX and DX on the data stack before executing the next Forth word.

Since the ES register is used to address the colon definitions in the List Segment, and the CS, DS, and SS registers have to point to the Code Segment, these four segment registers cannot be disturbed. If any of these registers are needed to perform some special functions like string compare and string move, their contents must be saved and restored before executing NEXT at the end of a Forth code word.

All system variables in F-PC are saved in a area at the top of the Code Segment. The beginning of this area is pointed to by the address contained in a variable UP. These variables are addressed by an address offset relative to the address in UP. This is not significant in a single task environment but is very important when you create background tasks in F-PC. Each task has its own user area and it keeps a unique copy of all user variables. The user area thus preserves the entire environment of a task, allowing tasks to be switched conveniently and very fast. It also allows Forth words to be re-entrant, so that the same code can be shared among all the tasks.

1.4. THE CODE AND PARAMETER FIELDS

The Code Segment of F-PC looks very similar to the entire F83 system. It contains a dictionary at the bottom. The return stack and the terminal input buffer are on the top. The free memory space between the top of the dictionary and the bottom of the return stack is shared by the data stack and the dictionary extension when new words are added to the dictionary.

In the dictionary, each Forth word occupies a variable length field call the Code Field. The address of the first byte in this field is called the Code Field Address (cfa) or Execution Address of this word. All Forth words are referenced by their Code Field Addresses. The Code Field contains executable machine code. A Forth word is executed by directly jumping to its Code Field. In Forth terminology, this arrangement is called 'Direct Threaded Code.' This is different from the more conventional 'Indirect Threaded Code' used by F83 and many other existing Forth systems in which the code field contains an address of executable code rather than the code itself. Direct Threaded Code is much faster than Indirect Threaded Code Because it eliminates a level of indirect jump.

Figure 2. The code fields of different types of words

In a low level code word, the Code Field contains machine code and is terminated by the NEXT instructions (LODSW ES: JMP AX), or a JMP >NEXT instruction, which allows the next Forth word to be executed. In other types of Forth words, the Code Field begins with a JMP or CALL instruction which invokes the inner interpreter specified for this type of words. Three bytes from the beginning of the Code Field, after the JMP or CALL instruction, is the Body or Parameter Field of the word. The Parameter Field contains data or other information needed by the word to perform its function. Figure 2 shows a collection of code and parameter fields for various types of Forth words, like constants, variables, colon definitions, arrays, etc.

In an indirect threaded Forth system, the Code Field is two bytes long and contains a pointer to a piece of executable code which is the inner interpreter for this type of words. The Parameter Field follows immediately after the Code Field. In a code word, the pointer in the Code Field points to its Parameter Field, which contains the executable code. The indirect jump to the code in the Parameter Field via the Code Field wastes a considerable number of machine cycles. The direct threaded code in F-PC gains much of its speed by eliminating this indirect jump. The cost is a three byte code field for words other than code words, one more byte per word than indirect threaded code.

1.5. THE COLON LISTS

Colon definitions are the most prevalent type of Forth words. In F-PC, a colon definition is compiled in three separated pieces. The head is placed in the Head Segment; it contains a view field, a link field, a name field, and a code pointer field. The executable part is compiled in the Code Segment, which is composed of a JMP NEST instruction and a list segment offset address. The list part is stored in the List Segment, which is a list of code field addresses. The physical segment address of this list is obtained by adding the contents of XSEG to the segment offset address stored in the Body (parameter field) of the colon word.

This rather interesting relationship among the three separated parts of a colon definition is shown in Figure 3. The Code Pointer in the head of a colon word points to the Code Field in the Code Segment. The Parameter Field points to the beginning of a list of code field addresses in the List Segment. To execute a colon word, the text interpreter or the outer interpreter in F-PC searches for the name of the colon word in the Head Segment. When the colon word is found, the execution address is obtained in the Code Pointer Field in its head.

A colon word is executed by a JMP AX instruction with AX pointing to its Code Field. The first instruction in its code field is JMP NEST, which saves ES:IP on the return stack, calculates the absolute page address of the list by adding the contents of its parameter field and the contents of XSEG, and then stores the page address in the ES register. After that, the NEXT instruction at the end of the NEST routine obtains the first execution address from the list of code field address and starts executing the words compiled into the colon definition.

The code field addresses in the list of a colon definition are executed in sequence. The last code field address in the list points to the code field of UNNEST. UNNEST pops the top two items on the return stack back into the ES:IP register pair and thus restores the condition before entering into this colon word. The NEXT instruction at the end of UNNEST causes the code compiled after the just-executed colon word to be executed.

Colon words are similar to subroutine calls in assembly code and procedures in conventional high level languages. Colon words are much more powerful because of the simplicity in the structure of address lists, and because parameters are passed on a separate data stack. The colon definitions can be nested with only a small overhead in nesting and un-nesting. Colon words are referred to by their execution addresses for execution and in compilation. They are structures more powerful and more versatile than subroutines or procedures. They are instructions to the virtual Forth machine, just like code words constructed from machine code.

1.6. THE HEADS

The Head or Header of a word contains its name and information about how it is to be executed, its context (vocabulary), and where its source code is. All heads are collected in the Head Segment. A head is a variable length structure with information arranged as shown in the upper left of Figure 3. The first two bytes are the View Field, which contains a byte offset address in the file containing its source definition. (The file is identified by the view field address of this word and the link field addresses of the loaded files which are linked into a file list). The view field locates the source code of this word so that the user can quickly find the source code by the VIEW command in F-PC. The ability to locate quickly the source code greatly improves the friendliness of F-PC. For new users it provides on-line documentation for inspecting and studying the system. For programmers, it gives fast access to the source code for debugging and modification.

The next two bytes are the Link Field, which contains an address pointing to the link field of the previous word in the same vocabulary thread. The link fields connect words with the same hash code into threads so that they can be searched quickly to locate a word by its name. In F-PC, the head dictionary is divided into several vocabularies. Words in a vocabulary are hashed into 64 threads by the first two characters and the lengths of their names. The last

Figure 3. Structure of a colon definition

link field in a thread is stored in a thread table in the parameter field of its vocabulary definition. The first link field in a thread is set to 0 to terminate vocabulary searching.

After the link field is a variable length Name Field. The first and last byte in the Name Field are flagged by setting the most significant bits in them. The first byte is called the Length Byte because it has the character length of the word name stored in the lower 5 bit field. Thus F-PC allows a word to have a name with up to 31 characters. Bit 6 in the Length Byte is the Immediate Bit. During compilation of a colon definition, words are normally compiled into a list. When the Immediate Bit is set, the word is not compiled into the list, but instead is executed immediately.

Bit 5 in the length byte is the Smudge Bit. When the Smudge Bit is set, the Forth text interpreter will not be able to find this word. This bit is used primarily to prevent finding words which are not compiled successfully. When a head is created, this Smudge Bit is always set to 1. After the colon word is compiled successfully, this bit is turned off so that the new word is revealed to the text interpreter. However, if compilation is unsuccessful, the word remains 'smudged' and the interpreter and compiler will not be able to find it. Thus the user is protected from compiling or executing invalid words.

Most Forth systems use the smudge bit to prevent invalid words from being executed or compiled. However, the invalid words still occupy dictionary space. F-PC does not use the smudge bit to protect the system. Instead, it only links words successfully compiled into the dictionary. Thus when a word is not compiled successfully, it does not take up memory space.

The name of a word is compiled in the Name Field. Up to 31 characters can be included. The last character is indicated by a 1 in bit 7, to delimit the name field.

The last field in a head is the Code Pointer Field, which contains the execution or code field address of this word. The address points to the code field of this word in the Code Segment. This address will be used to execute this word by the interpreter or compiled into a list by the colon compiler.

The first 256 bytes in the Head Segment are a Code Search Table. This table reduces the time to search the Code Pointer Fields in the heads by the decompiler. The vocabulary/thread structure is designed to locate the code field of a word from its name. The decompiler needs to do the search in reverse, finding the name of a word from its code field address. Because F-PC moves the heads out of the code dictionary and puts them in the Head Segment, the natural link between a code field and its name field is broken. To find the name field from a code field address, one would have to search all heads in the Head Segment. This search would be very time consuming.

To reduce the amount of searching, we divide the Code Segment into 64 1K byte regions. Each region has four bytes in the Code Search table. These four bytes define the area of heads to be searched for a code whose code field address falls into this code region. The first two bytes are the lower bound of the head space to be searched, and the last two bytes are the upper bound. Therefore, to search the name of a code, we have to identify its code region and then fetch the upper and lower bounds of the head space. Only the heads between these bounds have to be searched.

The Code Search Table is updated by the compiler when a new word is added to the dictionary.

1.7. F-PC VIRTUAL MACHINE

Source file: first fifth of KERNEL1.SEQ

We will follow the building process of the F-PC kernel according to the sequence of events coded in the KERNEL1.SEQ file. Since most Forth words have names in the upper case, you should be able to distinguish the code from the narration, which is in lower case. Code is shown in the vertical style on the left hand side of a page. In-line comment is shown on the right hand side, following a back slash \ which is also used in F-PC to start a comment which extends till the end of line.

It is difficult to follow and fully understand this part of the F-PC kernel because the loading is controlled by the F-PC metacompiler. Some of the words are compiled into the target dictionary as expected, but some words are compiled into the host dictionary as tools to aid the target compilation. Some knowledge about the F83 or F-PC metacompilers would be very useful. However, you will gain some insight into the inner structure of a virtual Forth machine with only elementary familiarity with Forth in general by casually going through this section. When you encounter problems about the inner interpreters later, you can always come back and pick up the details then.

The original design of this book was to include the F-PC metacompiler to achieve a system closure. As things stand now, the metacompiler and many other utilities will be published in separate volumes. In the mean time, you can use Chapter 25 in "Inside F83" for information about the metacompiler in F83, which is very close to the F-PC metacompiler.

The basic problem in understanding F-PC, or F83 and many other Forth systems in general, is that there is no starting point. The Forth is generated by itself with metacompilation. You cannot fully understand Forth not knowing the metacompiler. You cannot understand the metacompiler not knowing the Forth under it. Chicken or egg, which comes first?

The best place to start is really not here at the beginning. For those who are anxious to learn about assembly programming and Forth, Chapter 2 is the proper place to start. There you will find the definitions of the most useful Forth words and you can build up you knowledge of Forth from there.

The remaining of this section discusses the inner interpreters which are code that executes Forth instructions using the resources provided by the host CPU. The information about the inner interpreters is not necessary in order to use Forth. However, if you have to do special things and have to tinker the 8088 CPU and the virtual Forth machine, you need to dig through this section to find out why things are done the way the are, and how to tailor the system for your needs.

The first page of KERNEL1.SEQ contains some preparative actions before code is compiled into the target dictionary.

```
ONLY FORTH META ALSO FORTH  \ Establish the vocabulary search order as FORTH and then META
```

F-PC allows you to assemble the inner interpreter NEXT either as in-line code:

```
   LODSW ES:          \ Copy next code address from list pointed to by ES:IP.
             \ Increment IP to point to the next code in list.
   JMP AX         \ Jump and execute the code.
```

or as JMP >NEXT. The in-line NEXT takes four byte for every code definition, and the jump to >NEXT takes three bytes. In-line NEXT runs much faster with a small overhead in memory. Jumping to >NEXT, on the other hand, provides a central point of return for all code words. It is very handy for debuggging the Forth system by setting a break point at >NEXT if we use this method.

```
TRUE CONSTANT IN-LINE_NEXT  \ Enable in-line expansion of   NEXT, which makes F-PC run faster.

: ?.IN-LINE    ( -- )           \ Print state of IN-LINE_NEXT
   CR ." NEXT is currently "
   IN-LINE_NEXT >REV
   IF  [ASSEMBLER] INLINEON
      [FORTH] ."  IN-LINE. "
   ELSE    [ASSEMBLER] INLINEOFF
      [FORTH] ."  NOT "
      >NORM ."  IN-LINE. "
   THEN    >NORM CR ;
?.IN-LINE             \ Print it before compiling the kernel.
```

Dictionary pointers in the target must be initialized so that the addresses of code, heads and lists can be resolved for the target system.

```
256 DP-T !                  \ Set code dictionary pointer compatible with CP/M

0 DP-X !                    \ Set List dictionary pointer

HERE 10000 +                \ Allocate space for the host

' TARGET-ORIGIN >BODY !     \ Initialize physical address of the target core image
```

The Forth compiler] and interpreter [will be redefined for the meta compiler. However, we will still need the native Forth compiler/interpreters sometimes to manipulate data and code. They are given aliases so that we will not lose the capability.

```
IN-META             \ Put them in META vocabulary


: ]]  ]  ;              \ Save the Forth compiler


: [[  [COMPILE] [  ;        \ Save the interpreter


FORTH IMMEDIATE META

FORWARD: DEFINITIONS        \ Hard to define in code but we need it now.

                \ Use it now and resolve it later.

FORWARD: [              \ Same
```

We are now ready to start compiling code into the target core image. The first 8 bytes in the system are vectors to the COLD and WARM start routines. You can freely jump to them in code

anytime to restart F-PC.

```
LABEL ORIGIN            \ Starting target dictionary

   JMP HERE 8000 +          \ Jump to cold start, will be patched later

   JMP HERE 8000 +          \ Jump to warm start, will be patched later
```

We define a few macros here to make our life a little easier later. Using NEXT as a macro allows us to put it in-line later.

>NEXT is where all the action is. It is the guts of the Virtual Forth Machine. It must advance the interpretive pointer held in the ES:IP register pair and jump indirect through AX to what it points to.

>NEXT The centralized return point of all code words in F-PC when NEXT is not compiled in-line. This is known as THE Inner Interpreter of Forth. As a colon word is executed, the Forth machine scans through a list of code field addresses in the list space. ES:IP is pointing to the next code field address in this list. LODSW ES: fetches this address into the AX register and increments IP so that ES:IP points to the next code field address to be executed. JMP AX executes the code stored in the code segment starting at the address pointed to by AX register. This code sequence causes the next Forth word pointed by ES:IP to be executed. The Forth machine runs on this way indefinitely or until F-PC is terminated.

The DPUSH and APUSH labels are space savers. We jump to them in many code words when we want to push their contents on the data stack.

APUSH Push contents of AX on the data stack and then execute NEXT. AX generally holds the most immediate results to be returned to the data stack by a code word.

DPUSH Push contents of DX on the data stack and fall into APUSH. DX is used to hold the second piece of data to be returned on the data stack.

```
LABEL DPUSH          \ Code of 2PUSH
   PUSH DX
LABEL APUSH          \ Code of 1PUSH
   PUSH AX
LABEL >NEXT          \ Code of NEXT, the Forth inner interpreter
   LODSW ES:         \ Load next list address into AX and increment IP
   JMP AX        \ Jump to code pointed by AX
   END-CODE
```

Next, we create the FORTH vocabulary as the first definition in the target dictionary.

```
HERE-T ,-Y                    \ Valid "previous" CFA for  "CREATE to build a head
HERE-Y HERE-T CNHASH !-Y          \ First entry in >NAME hash  table
```

```
HERE-T DUP 100 + CURRENT-T !       \ Initialize CURRENT to an array of zero's.  They
                \   will be replaced by the thread table in FORTH.
HERE-Y            \ View field address of FORTH
```

FORTH The trunk vocabulary in a Forth system. Most commonly- used Forth words are linked into this vocabulary. It is also by far the biggest vocabulary in F-PC, containing about a thousand words. In the BODY of a vocabulary definition, there is a thread table containing the last link field addresses of words at the top of all the threads. F-PC allows you to vary the size of the thread table, hence the number of threads in a vocabulary. The current implementation uses 64 threads. A special hashing algorithm is used to make words distributed evenly among the threads to reduce vocabulary searching time during compilation. This hashing scheme enables F-PC to compile very fast, about 13000 lines per minute.

```
VOCABULARY FORTH       \ Create FORTH vocabulary as the first word in target
FORTH DEFINITIONS          \ Compile the kernel into the target FORTH vocabulary

0 OVER 2+ !-Y ( link )              \ Clear link field of FORTH as the end of a thread
2+              \ Link field address of FORTH
SWAP >BODY-T          \ BODY address of FORTH
'F'   2*                          \ Hashing algorithm is:
'O' + 2*             \   (1st_char*2+2nd_char)*2
5 +              \   + length
#TTHREADS 1- AND 2*              \ Determine which thread FORTH  goes in.
+ !-T               \ The first thread in the thread table of FORTH
IN-META           \ Return to META vocabulary
```

FILES A vocabulary where the names of files loaded into the F-PC system are linked together. F-PC allows you to create many small files. To keep track of all the files loaded into the system, a special vocabulary FILES is created. A file loaded will also compile its file name as a variable in this vocabulary. It thus automatically creates the linked list of files that have been loaded. This linked list will be used by VIEW to identify which file contains the source code of a word. The first file whose link field address is smaller than the link field address of a word must be the file from which the word was loaded.

```
VOCABULARY FILES          \ Define FILES vocabulary
FILES DEFINITIONS         \ Make it current
```

```
VARIABLE KERNEL1.SEQ       \ Compile name of this file

FORTH DEFINITIONS          \ Return to target


VARIABLE XSEG          \ Base segment address of the List Segment in target

VARIABLE YSEG          \ Base segment address of the Head Segment in target
```

ABNORM This routine allows you to use the system BREAK key to abort what F-PC is doing. It restores the >NEXT area, which is the most vulnerable place of the Forth machine.

```
HEX

LABEL ABNORM

   MOV AX, # AD26             \ Value to restore in >NEXT

   MOV >NEXT AX               \ Restore it

   MOV AX, # E0FF             \ Value in >NEXT + 2

   MOV >NEXT 2+ AX            \ Restore it

   XOR AX, AX

   MOV DS, AX

   MOV BX, # 471      \ Clear the key value in DOS

   MOV 0 [BX], AL

   MOV AX, CS

   MOV DS, AX         \ Restore DS to CS

   JMP ORIGIN 3 +     \ Warm start

   END-CODE
```

BIOSBK Put a JMP ABNORM into the >NEXT area to force the execution of ABNORM after any Forth word currently being executed.

```
LABEL BIOSBK

   PUSH AX
```

```
   MOV AL, # E9              \ JMP code
   MOV CS: >NEXT AL          \ Store to >NEXT
   MOV AX, # ABNORM >NEXT - 3 -    \ Offset from >NEXT to  ABNORM
   MOV CS: >NEXT 1+ AX     \ Complete the JMP code
   POP AX
   IRET
   END-CODE
```

DOSBK Call DOS keyboard service 16. The function number n must be on the data stack.

```
LABEL DOSBK     ( n -- )
   PUSH AX
   MOV AH, # 0               \ Throw away BREAK KEY
   INT 16
   POP AX
   CLC
   RETF
   END-CODE
DECIMAL
```

NEST The run-time code (the inner interpreter) for colon words . It pushes the current ES:IP register pair onto the return stack and sets the ES:IP to point to the beginning of a list of words to be executed. It allows high level words to call other high level words with almost unlimited nesting. In the code field of a colon definition, the first three bytes contain JMP NEST instruction. The next byte contains a relative segment address, pointing to the list of code field addresses compiled in the list space, offset by the contents of XSEG.

```
LABEL NEST
   XCHG RP, SP        \  4 cycles. Use SP to address return stack.
   PUSH ES        \ 10 cycles. Save ES.
   PUSH IP        \ 11 cycles. Save IP.
   XCHG RP, SP        \  4 cycles. Restore RP.
```

```
   MOV DI, AX         \  2 cycles. Code field address
   MOV AX, 3 [DI]     \ 18 cycles. Get to BODY of code and fetch relative segment

LABEL NESTPATCH       \ This location is later patched at boot time to correct for the
   ADD AX, # XSEG     \ actual value of XSEG at boot time.
   MOV ES, AX         \  2 cycles. Move it into ES
   SUB IP, IP         \  3 cycles. Clear IP
   NEXT               \ Ready to process list
   END-CODE
META
```

EXIT Pop an ES:IP pair off the return stack and place it in the ES and IP register pair to resume processing the unfinished list of code field addresses. Terminate a high level definition by nullifying what NEST had wrought.

```
CODE EXIT   ( -- )
   XCHG RP, SP        \  4 cycles. Use SP as RP.
   POP IP             \  8 cycles. Restore IP
   POP ES             \  8 cycles. Restore ES
   XCHG RP, SP        \  4 cycles. Restore RP
   NEXT
   END-CODE
```

?EXIT Exit from a definition if the flag on stack is true.

```
CODE ?EXIT  ( f1 -- )
   POP CX
   CX<>0 IF
      JMP ' EXIT
   THEN
   NEXT
   END-CODE
```

UNNEST Same as EXIT. Compiled by ; . The alias is defined to help the decompiler to recover ; .

DODOES The inner interpreter of the defining words. The first three bytes in the code field of a daughter word created by a defining word are CALL DODOES, which calls the high level interpreter specified after DOES> in the defining word. First it pushes the ES:IP pair onto the return stack and then it pushes the BODY address of the word being executed onto the data stack which is what the CPU thinks of as its return stack.

```
LABEL DODOES    ( addr1 addr2 -- addr1 )    \ addr1/2 are from two previous calls.
   XCHG RP, SP
   PUSH ES
   PUSH IP
   XCHG RP, SP
   POP DI          \ Pop the BODY address
   MOV AX, 0 [DI]      \ Get the list offset

LABEL DOESPATCH         \ This location is later patched at boot time to correct for the
   ADD AX, # XSEG      \ actual value of XSEG at boot time.
   MOV ES, AX          \ Move to ES
   SUB IP, IP          \ Clear IP
   NEXT                \ Execute high level inner
   END-CODE            \   interpreter.
```

UP Hold a pointer to the current USER area for multitasking. All the system variables and the multitasking mechanism are stored in the user area.

```
VARIABLE UP
```

DOCONSTANT The run time code (inner interpreter) for constants. It takes the contents of Body (the parameter field) and pushes it onto the data stack.

```
LABEL DOCONSTANT
   MOV BX, AX        \ AX points to he address of BODY where constant is stored
   PUSH 3 [BX]       \ Push constant value
   NEXT
   END-CODE
```

DOVALUE Same as DOCONSTANT.

DOUSER-VARIABLE The inner interpreter for USER variables. Place a pointer to the current version of this variable on the stack. It is needed for multitasking.

```
LABEL DOUSER-VARIABLE
   POP BX        \ Address of BODY
   MOV AX, 0 [BX]    \ Get user offset
   ADD AX, UP        \ Add it to UP
   1PUSH             \ Return its address
   END-CODE
```

>NEXT The address of the code inner interpreter. It is redefined as a constant in the ASSEMBLER vocabulary to facilitate assembling the inner interpreter at the end of code definitions.

```
ASSEMBLER >NEXT META CONSTANT >NEXT
ASSEMBLER  NEST META CONSTANT >NEST
```

EXECUTE Execute the word whose code field is on the stack. Very useful for passing executable routines to procedures.

```
CODE EXECUTE   ( cfa -- )
   POP AX        \ Pop execution address
   JMP AX        \ Jump to it
   END-CODE
```

PERFORM Execute the word whose code field is stored at the address pointed to by the number on the stack. Same as @ EXECUTE.

DO-DEFER The runtime code for deferred words. Fetch the code field address and executes it.

```
CODE PERFORM ( addr-of-cfa -- )
LABEL DO-DEFER
  POP BX       \ Pop address
  MOV AX, 0 [BX]    \ Fetch execution address
  JMP AX       \ Then jump
  END-CODE
```

EXEC: Select the n'th execution address in the following address table and execute it. This is a very fast and very powerful case structure compiled in the List Segment. It is similar to CASE: in F83. It is used in KEYTABLE, p. 91.

```
CODE EXEC: ( n -- )
  POP BX       \ Pop the case number
  SHL BX, # 1      \ Make it an address offset
  ADD IP, BX       \ Point it into the address table
  LODSW ES:        \ Get execution address
  XCHG RP, SP
  POP IP          \ Pop ES:IP
  POP ES       \ Prepare to leave current colon word
  XCHG RP, SP
  JMP AX       \ Execute and exit
  END-CODE
```

DOUSER-DEFER The run-time code for user deferred words. They are identical to regular deferred words except that each task has its own version.

```
LABEL DOUSER-DEFER
   POP BX         \ BODY address
   MOV BX, o [BX]     \ User variable offset
   ADD BX, UP         \ Address in user area
   MOV AX, o [BX]     \ Fetch execution address
   JMP AX         \ Do it
   END-CODE
```

GO Execute a conventional machine code subroutine at the given address. EXECUTE must be used to execute a Forth code word to manage the execution sequence.

```
CODE GO ( addr --- )
   RET          \ The address is on the data stack.
   END-CODE
```

NOOP One of the most useful words in Forth. Do nothing. It is often used as a place holder where other words can be plugged in later. One way to do forward referencing.

```
CODE NOOP
   NEXT
   END-CODE
```

PAUSE Used by the multitasker to switch tasks. It is defined as a NOOP word now. It will be patched when the multitasker is activated.

```
CODE PAUSE
   NOOP                    \ Gets patched later
   NOOP
   NOOP
   NEXT
   END-CODE
```

CHAPTER 2. THE F-PC KERNEL

There has not been a generally accepted definition of what is the Forth kernel, as people are still fighting over the more fundamental question on what is Forth. A practical definition of Forth kernel is the collection of words which are written in machine code, from which a complete, operating Forth can be built in high level without more low level code words. F83 collected all word definitions leading to Forth text interpreter and colon word compiler in a file KERNEL86.BLK. This is also a good practical definition of a Forth kernel.

F-PC, with all the enhancements over F83, has a much bigger kernel. The code needed to build a minimal kernel system is split into four files: KERNEL1.SEQ to KERNEL4.SEQ, with a few other utility files. Many high level words are also recoded in assembly code to improve its speed. The above definitions of a Forth kernel are not suitable to F-PC. What we call the F-PC Kernel is the word set which manipulates data in memory and on the stacks; and the data/control structures in colon definitions. Words to implement the text interpreter, compiler, and files are separated to form chapters of their own. In this chapter, we are thus restricting ourselves to the lowest layer of Forth words in the second half of KERNEL1.SEQ file. With only very rare exceptions, they are all code words and are excellent examples of assembly programming in F-PC.

Most of the code words are identical to those defined in F83. Changes are made when performance can be improved or when the segmented architecture in F-PC dictates modification. A major improvement is that the assembly code is written in the prefix style, which is very close to the Intel MASM syntax. It is quite different from the postfix style used in F83 and most other Forth systems. The entire F-PC is written in the prefix style to encourage users to adopt it in future programming efforts.

2.1. THE STACK WORDS

F-PC, as all other Forth systems, uses two stacks: the return stack for passing subroutine return addresses and the data stack for passing data or parameters between words or subroutines. In F-PC both stacks are located in the data segment, because the SS segment register contains the same segment address as the CS and DS registers. The regular stack pointer SP in the 8088/86 is used as the pointer for the data stack, and the BP register is used to point to the top

of the return stack. Consequently, the assembler instructions PUSH and POP are used to move data between CPU registers and the data stack. To get data to/from the return stack, F-PC uses two methods: the BP relative mode and PUSH/POP after swapping BP with SP. Either has its advantages and disadvantage.

In the stack words, input data are popped into registers AX, BX, CX, and DX. If there are output data, the first item is left in AX and 1PUSH is invoked instead of NEXT. The second item can be left in DX and the code word is terminated by 2PUSH, which first pushes DX and then AX on the data stack before executing NEXT. If more than two items are generated or they are in registers other than DX and AX, they can be pushed on to the data stack by explicit PUSH instructions.

Since AX and DX can be pushed using 1PUSH and 2PUSH, code words are often defined so that output data are left in them at the end of a code word. The BX register is very useful in moving data in and out of the memory. Addresses are generally popped into the BX register and the BX relative MOV instruction can be used to read or write the memory in the code/data segment. 8088/86 processors use the CX register to store counts for various repeated instructions and LOOP instructions. Counts are popped into CX.

NEXT in F-PC leaves the code field address of the word currently being executed in the AX register. The body or parameter field of a word is 3 bytes from this code field address. If you need this information, you can pick it up in the AX register. However, most code words do not need this information and the AX register can be used freely. Other free registers are BX, CX, DX, and DI. The registers SI, BP, SP, and all four segment registers CS, DS, SS, and ES contain vital system information, and must be saved and restored before NEXT if they are modified.

All the definitions of stack words are listed in this section. They are good examples to learn how either stack is addressed and how data are moved among the stacks, memory and the CPU registers.

DROP Discard the top element of the data stack.

```
CODE DROP  ( n -- )
  ADD SP, # 2
  NEXT
  END-CODE
```

DUP Duplicate the top element of the data stack.

```
CODE DUP  ( n -- n n )
  POP AX
```

```
   PUSH AX
   1PUSH
   END-CODE
```

SWAP Exchange the top two elements on the data stack.

```
CODE SWAP  ( n1 n2 -- n2 n1 )
   POP DX
   POP AX
   2PUSH
   END-CODE
```

OVER Push a copy of the second stack item.

```
CODE OVER  ( n1 n2 -- n1 n2 n1 )
   XCHG RP, SP
   MOV AX, 2 [RP]
   XCHG RP, SP
   1PUSH
   END-CODE
```

PLUCK Copy the third stack item to the top.

```
CODE PLUCK ( n1 n2 n3 -- n1 n2 n3 n1 )
   MOV DI, SP
   PUSH 4 [DI]
   NEXT
   END-CODE
```

TUCK Tuck the first stack item under the second one.

```
CODE TUCK  ( n1 n2 -- n2 n1 n2 )
   POP AX
   POP DX
   PUSH AX
   2PUSH
   END-CODE
```

NIP Discard the second stack item.

```
CODE NIP  ( n1 n2 -- n2 )
   POP AX
   ADD SP, # 2
   1PUSH
   END-CODE
```

ROT Rotate the top three stack items, bringing the third to the top.

```
CODE ROT  ( n1 n2 n3 -- n2 n3 n1 )
   POP DX
   POP BX
   POP AX
   PUSH BX
   2PUSH
   END-CODE
```

-ROT The inverse of ROT. Rotate the top stack item to third place.

```
CODE -ROT  ( n1 n2 n3 --- n3 n1 n2 )
```

```
   POP BX
   POP AX
   POP DX
   PUSH BX
   2PUSH
   END-CODE
```

FLIP Exchange the high and low bytes of the 16 bit integer on the top of data stack.

```
CODE FLIP  ( n1 -- n2 )
   POP AX
   XCHG AL, AH
   1PUSH
   END-CODE
```

SPLIT Splits n1 into two byte values: n2 low, and n3 high.

```
CODE SPLIT  ( n1 -- n2 n3 )
   POP BX
   SUB AX, AX
   MOV AL, BL
   PUSH AX
   MOV AL, BH
   1PUSH
   END-CODE
```

JOIN Join two bytes into one word.

```
CODE   JOIN   ( low-byte high-byte -- n )
```

```
   POP DX
   POP AX
   MOV AH, DL
   1PUSH
   END-CODE
```

?DUP Duplicate the top of the stack only if it is non-zero. This is most often used before IF so that you don't have to discard the 0 flag in the false clause.

```
CODE ?DUP  ( n1 -- n1 n1 | 0 )
   POP AX
   CMP AX, # 0
   0<> IF
      PUSH AX
   THEN
   1PUSH                \ 33 cycles
   END-CODE
```

?DROP If the first stack item is zero, discard the top two items; otherwise no change.

```
CODE ?DROP ( n false -- | n true -- n true )
   POP AX
   OR AX, AX
   0= IF  INC SP
      INC SP
   THEN
   1PUSH
   END-CODE
```

R> Pop an item off of the return stack and push it onto the data stack. As the return stack contains addresses of words which are yet to be executed, it is dangerous to mess it up. All words operating on the return stack are dangerous and can cause the system to crash. This is why all other languages and operating systems must hide the return stack from the user. Forth does not take away from you this privilege. It also expects you to use this privilege wisely.

```
CODE R> ( -- n )
   PUSH 0 [RP]
   ADD RP, # 2
   END-CODE
```

R>DROP Drop an item from the return stack.

```
CODE R>DROP ( -- )
   ADD RP, # 2
   NEXT
   END-CODE
```

DUP>R Push a copy of the top item on the data stack to the return stack.

```
CODE DUP>R    ( n -- n )
   XCHG SP, RP       \ Faster than moving things through a register.
   PUSH 0 [RP]
   XCHG SP, RP
   NEXT            \ 29 cycles
   END-CODE
```

>R Pop the top the data stack and push it on the return stack. It is generally used to save the top item on the data stack temporarily. This item must be restored to the data stack by R>. >R and R> must be used as a pair. If they do not occur as a pair in a colon definition, you can expect potential problems. Used effectively, they are powerful tools to change and manipulate execution sequences.

```
CODE >R ( n -- )
   SUB RP, # 2
   POP 0 [RP]
   NEXT                \ 26 cycles
   END-CODE
```

R@ Push a copy of top item on the return stack to the data stack. The state of the return stack is not disturbed. It is very convenient to push a number on the return stack and use R@ to get many copies of this number in a colon definition. However, you have to remove it before exiting the colon definition. Otherwise, the system will certainly crash because the F-PC system would interpret this number as a return address.

```
CODE R@ ( -- n )
   PUSH 0 [RP]
   END-CODE
```

2R> Pop two items off the return stack and push them on the data stack. This word is required in F-PC because F-PC saves the return address as an ES:IP register pair. To move return addresses from or to the return stack, you should move the register pair.

```
CODE 2R>   ( -- d )
   PUSH 2 [RP]
   PUSH 0 [RP]
   ADD RP, # 4
   NEXT                \ 50 cycles
   END-CODE
```

2>R Pop two integers off the data stack and push them on the return stack. It is used to move an ES:IP register pair from the data stack to the return stack. Used for this purpose, the ES register should be on the top of the data stack.

```
CODE 2>R   ( d -- )
   SUB RP, # 4
   POP 0 [RP]
```

```
   POP 2 [RP]
   NEXT                \ 52 cycles
   END-CODE
```

2R@ Replicate the top two items on the return stack to the data stack. Similar to 2R> except the return stack is not disturbed.

```
CODE 2R@  ( -- d )
   PUSH 2 [RP]
   PUSH 0 [RP]
   NEXT
   END-CODE
```

PICK Reach into the data stack and grab an element, copying it to the top of the stack. For example, if the stack has 1 2 3, then 0 PICK returns 3, 1 PICK returns 2, and 2 PICK returns 1.

```
CODE PICK  ( nm ... n2 n1 k -- nm ... n2 n1 nk )
   POP DI
   SHL DI, # 1
   ADD DI, SP
   PUSH 0 [DI]
   NEXT
   END-CODE
```

RPICK Reach into the return stack and grab the n1'th item, copying it to the top of the data stack.

```
CODE RPICK  ( n1 -- n2 )
   POP DI
   SHL DI, # 1
   PUSH 0 [RP+DI]
```

```
   NEXT

   END-CODE
```

ROLL Similar to PICK, except that the chosen item is moved (copied and then deleted) rather than copied. 1 ROLL is SWAP, 2 ROLL is ROT, etc. ROLL can be useful, but it is slow, and so should be avoided. It is hard to define in code.

```
: ROLL  ( n1 n2 .. nk n -- nk n1 n2 .. nk-1 )

   >R R@ PICK

   SP@ DUP 2+

   R> 1+ 2* CMOVE>

   DROP ;
```

2ROT Rotate the top three double numbers, bringing the deepest pair to the top.

```
: 2ROT     ( a b c d e f -- c d e f a b )

   5 ROLL        \ This is where ROLL is handy

   5 ROLL

   ;
```

SP@ Push the address of the top item (prior to the push) on the data stack.

```
CODE SP@    ( -- addr )

   MOV AX, SP

   1PUSH

   END-CODE
```

SP! Set the data stack pointer to the specified value. It is only used to initialize the data stack by ABORT, and not meant to be used by user.

```
CODE SP!    ( addr -- )
```

```
  POP SP

  NEXT

  END-CODE
```

RP@ Push the address of the top item on the return stack to the data stack. In F-PC, the RP or BP register is used as the return stack pointer, keeping track of the ES:IP pairs of words which are yet to be executed.

```
CODE RP@  ( -- addr )

  PUSH RP

  NEXT

  END-CODE
```

RP! Set the return stack pointer to the specified address. This is used by the system to initialize the return stack. It is not meant to be used by the user.

```
CODE RP!  ( addr -- )

  POP RP

  NEXT

  END-CODE
```

2.2. ARITHMETIC AND LOGIC WORDS

F-PC is a 16 bit Forth implementation. All numbers and addresses, if not otherwise specified, are assumed to be 16 bit integers. 32 bit double numbers are allowed, but the double integer operations are all based upon 16 bit operations. They are discussed in the section on double integers. This word set is almost identical to that in F83, except that more words are coded in assembly to increase the speed of F-PC.

8088/86 processors have a fairly strong arithmetic and logic instruction set for 16 bit integers. These operations use numbers stored in registers. To build Forth operations from these machine instructions, integers must be popped off the data stack and put into appropriate registers. Operations are then performed in the registers. The final results are then pushed back onto the data stack. The efficiency of the virtual Forth machine is degraded because of the data

shuffling between the stack and the CPU registers. F-PC runs much faster than F83 because more words are coded in assembly and more data are kept in CPU registers longer. The overhead in data shuffling is considerably reduced.

Arithmetic and logic operations can be classified according to the number of input and output items. A Unary Operator takes one integer as input and generates one integer as output. It includes operators like ABS, NEGATE, NOT, 1+, 2+, 1-, 2*, 2/, U2/, etc. Binary Operators are most prevalent, with two input integers and one output integer. AND, OR, XOR, +, -, *, and / belong to this group. Other operators may involve more than two inputs and one output. Many of these complex operators are defined in F-PC, over and above those required by Forth-83 Standards. Most of them are defined as code words in F-PC to optimize the system performance.

AND Return the bitwise AND of n1 and n2 on the data stack.

```
CODE AND  ( n1 n2 -- n3 )
  POP BX
  POP AX
  AND AX, BX
  1PUSH
  END-CODE
```

OR Return the bitwise OR of n1 and n2 on the stack.

```
CODE OR ( n1 n2 -- n3 )
  POP BX
  POP AX
  OR AX, BX
  1PUSH
  END-CODE
```

XOR Return the bitwise Exclusive OR of n1 and n2 on the stack.

```
CODE XOR  ( n1 n2 -- n3 )
  POP BX
```

```
   POP AX
   XOR AX, BX
   1PUSH
   END-CODE
```

NOT Do an one's complement of the top item on the data stack. Equivalent to -1 XOR. It is a bitwise logic complement operation. If the flag were 0=false or -1=true, then NOT could be used to complement the flag. But in Forth, the flag is 0=false and (not 0)=true. So in a general case, 0 NOT is -1 (=true); but 3 is true, and 3 NOT is -4, which is still true. To complement a flag, use the comparison word 0=.

```
CODE NOT  ( n -- n' )
   POP AX
   NOT AX
   PUSH
   END-CODE
```

Many small integers are used very often in Forth and they deserve separate word definitions instead of being compiled as literals which take more space in colon definitions. TRUE is a number with all 16 bits set and FALSE is one with all bit cleared. They are useful constants to make logical intentions clearer in the source code. 0 through 3 have previously been defined as constants. However, F-PC is very efficient in its literal structure. A literal is about three times faster than a constant. Since this discovery, 0 through 3 are no longer included as constants.

```
-1 CONSTANT TRUE
 0 CONSTANT FALSE
```

+ Add the top two numbers on the stack and return the sum.

```
CODE +   ( n1 n2 -- sum )
   POP BX
   POP AX
   ADD AX, BX
   1PUSH
```

```
   END-CODE
```

NEGATE Turn a number into its negative. A two's complement operation.

```
CODE NEGATE ( n -- n' )
   POP AX
   NEG AX
   1PUSH
   END-CODE
```

- Subtract n2 from n1 and leave the difference on the stack.

```
CODE -  ( n1 n2 -- n1-n2 )
   POP BX
   POP AX
   SUB AX, BX
   1PUSH
   END-CODE
```

ABS Return the absolute value of the 16 bit integer on the stack.

```
CODE ABS   ( n -- u )
   POP AX
   OR AX, AX
   0< IF
      NEG AX
   THEN
   1PUSH
   END-CODE
```

UM* Unsigned Mixed Multiply. Multiply top two unsigned items on the data stack and return an unsigned double integer product.

```
CODE UM*   ( u1 u2 -- ud )
   POP AX        \ Get u1
   POP BX        \ Get u2
   MUL BX        \ u1*u2
   XCHG DX, AX       \ Re-ordering parts
   2PUSH         \ Push double integer product
   END-CODE
```

* Multiply top two signed integers on the data stack and return a 16 bit signed integer product.

```
CODE *     ( n1 n2 -- n3 )
   POP AX
   POP BX
   MUL BX        \ n1*n2
   1PUSH            \ Save only the LS 16 bits of the product
   END-CODE
```

U*D Synonym for UM* .

```
: U*D      ( u1 u2 -- ud )
   UM*   ;
```

UM/MOD Unsigned Mixed Modulo. This is the division primitive in Forth. All other division operations are derived from it. It takes an unsigned double integer ud, and divides it by an unsigned integer u1. It leaves an unsigned remainder and an unsigned quotient on the stack.

```
CODE UM/MOD ( ud u1 -- Remainder Quotient )
```

```
   POP BX         \ u1
   POP DX         \ MS of ud
   POP AX         \ LS of ud
   CMP DX, BX
   U>=            \ Test for overflow
   IF             \ If overflow
     MOV AX, # -1       \ return two -1's
     MOV DX, AX
     2PUSH
   THEN
   DIV BX         \ Unsigned divide
   2PUSH            \ Push rem and quot
   END-CODE
```

*D Multiply two signed integers and leave a signed double integer product.

```
CODE *D    ( n1 n2 -- d )
   POP CX
   POP AX
   IMUL CX        \ Unsigned multiplication
   PUSH AX
   PUSH DX
   NEXT
   END-CODE
```

M/MOD Divide a signed double integer by a signed integer, leaving a single integer quotient and a single integer remainder. Division is floored towards minus infinity.

```
: M/MOD ( d n -- rem quot )
   ?DUP             \ Return d if n=0
```

```
  IF  DUP>R  2DUP XOR >R      \ Save n and sign
    >R DABS R@ ABS
    UM/MOD      \ Do unsigned multiply
    SWAP R> ?NEGATE \ restore sign of remainder
    SWAP R> 0<      \ Handle negative quotient
    IF  NEGATE OVER \ Negate quotient
      IF     1- R@ ROT -
        SWAP  \ Correct remainder and floor it towards -infinity
      THEN
    THEN    R>DROP
  THEN  ;
```

MU/MOD Divide an unsigned double integer by an unsigned integer, leaving a double integer quotient and a single integer remainder.

```
: MU/MOD    ( ud u -- rem dquot )
  >R  0  R@          \ Make MS of d a double
  UM/MOD             \ Divide it by n
  R>  SWAP >R        \ Save MS of quotient
  UM/MOD  R>         \ Divide LS of d and return MS of quotient
  ;
```

/ Divide second item on stack by the top item. Return the quotient only. Division is floored towards minus infinity.

```
CODE /  ( numerator denominator -- quotient )
  POP BX
  POP AX
  CWD           \ Extend numerator
  MOV CX, BX
```

```
   XOR CX, DX        \ Test sign of quotient
  0>= IF             \ For positive quotient
   IDIV BX    \  do signed divide.
   1PUSH
  THEN
  IDIV BX          \ For negative quotient
  OR DX, DX
  0<> IF
   DEC AX     \ Floor it towards -infinity
  THEN
  1PUSH
  END-CODE
```

/MOD Divide the top item by the second item like /. Return the quotient on top with the remainder under it. The division is floored. Quotient and remainder have the same sign as the numerator.

```
CODE /MOD   ( num denom -- rem quot )
   POP BX
   POP AX
   CWD
   MOV CX, BX
   XOR CX, DX
   0>= IF           \ Same treatment for quotient as in /
    IDIV BX
    2PUSH
   THEN
   IDIV BX
   OR DX, DX
   0<> IF
    ADD DX, BX     \ Floor the remainder
```

```
    DEC AX      \ Floor the quotient
  THEN
  2PUSH
  END-CODE
```

MOD Similar to /MOD, but return only the remainder.

```
: MOD      ( n1 n2 -- rem )
  /MOD  DROP  ;
```

*/MOD Multiply n1 with n2 and divide the product by n3. Return both the quotient and the remainder. The intermediate product is a double integer to preserve the precision. This is a very important operation to do scaling; i.e., n1 is multiplied by the ratio of n2/n3. It allows high precision scaling without using floating point representation.

```
CODE */MOD  ( n1 n2 n3 -- rem quot )
  POP BX
  POP AX
  POP CX
  IMUL CX        \ n1*n2
  MOV CX, BX
  XOR CX, DX
  0>= IF            \ Positive quotient
    IDIV BX     \ (n1*n2)/n3
    2PUSH
  THEN
  IDIV BX           \ Negative quotient
  OR DX, DX
  0<> I
    ADD DX, BX      \ Floor remainder
    DEC AX      \ Floor quotient
```

```
   THEN
   2PUSH            \ Push rem and quot
   END-CODE
```

*/ Multiply n1 by the ratio of n2/n3. The intermediate result is kept to full accuracy of 32 bits. Notice that this is not the same as * followed by /. It is a particularly useful operator, as it allows you to do accurate arithmetic on fractional quantities. See Leo Brodie's "Starting Forth" for more examples.

```
: */      ( n1 n2 n3 -- n1*n2/n3 )
   */MOD
    NIP  ;          \ Discard remainder
```

2* Double the number on the data stack. Doing an arithmetic left shift is much faster than doing a multiplication by 2.

```
CODE 2* ( n -- 2*n )
   POP AX
   SHL AX, # 1
   PUSH
   ND-CODE
```

2/ Shift the number on the stack right one bit. Equivalent to division by 2.

```
CODE 2/    ( n -- n/2 )
   POP AX
   SAR AX, # 1       \ Arithmetic right shift
   1PUSH         \ Sign is preserved
   END-CODE
```

U2/ 16 bit logical right shift.

```
CODE U2/   ( u -- u/2 )
   POP AX
   SHR AX, # 1       \ Logic right shift
   1PUSH             \ Sign bit is cleared
   END-CODE
```

U/16 Divide top unsigned integer by 16. This is needed in converting an address to a segment address.

```
CODE U16/  ( u -- u/16 )
   POP AX
   SHR AX, # 1
     SHR AX, # 1
   SHR AX, # 1
     SHR AX, # 1
   1PUSH
   END-CODE
```

U/8 Divide top unsigned integer by 8.

```
CODE U16/  ( u -- u/16 )
   POP AX
     SHR AX, # 1
   SHR AX, # 1
     SHR AX, # 1
   1PUSH
   END-CODE
```

8* Multiply the top of the stack by 8.

```
CODE 8* ( n -- 8*n )
   POP AX
   SHL AX, # 1
   SHL AX, # 1
      SHL AX, # 1
   1PUSH
   END-CODE
```

1+ Increment the top of the stack by one.

```
CODE 1+ ( n -- n+1 )
   POP AX
   INC AX
   1PUSH
   END-CODE
```

2+ Increment the top of the stack by two.

```
CODE 2+ ( n -- n+2 )
   POP AX
   ADD AX, # 2
   1PUSH
   END-CODE
```

1- Decrement the top of the stack by one.

```
CODE 1- ( n -- n-1 )
   POP AX
   DEC AX
```

```
  1PUSH
  END-CODE
```

2- Decrement the top of the stack by two.

```
CODE 2- ( n -- n-2 )
  POP AX
  SUB AX, # 2
  1PUSH
  END-CODE
```

2.3. MEMORY WORDS

Memory words move data from data stack to memory, from memory to stack, and from memory to memory. In F-PC, all memory operations assume that the data are in the Code/Data Segment to maintain compatibility with F83.

The address for memory reference is generally popped into the BX register. Data are then fetched or stored using the BX relative mode. The relative offset from BX is in most cases 0. The BX relative mode of addressing assumes the use of DS segment register. Since in F-PC the DS register is always aligned with the CS register, all addresses thus generated are restricted to the Code Segment.

Memory store words are often dangerous if data is stored into the wrong places in the dictionary, outside of declared areas for variables or arrays. Forth trusts that the user will make the best use of this capability.

Some Forth programmers prefer to avoid @ and ! by using VALUE's, which are discussed in Sec. 6.2.

@ Fetch a 16 bit value from addr.

```
CODE @  ( addr -- n )
```

```
   POP BX
   PUSH 0 [BX]
   NEXT
   END-CODE
```

! Store a 16 bit value to addr.

```
CODE !   ( n addr -- )
   POP BX
   POP 0 [BX]
   NEXT
   END-CODE
```

C@ Fetch an 8 bit value from addr. Fill high byte with zero.

```
CODE C@ ( addr -- char )
   POP BX
   SUB AX, AX
   MOV AL, 0 [BX]
   1PUSH
   END-CODE
```

C! Store the least significant 8 bits of char to addr.

```
CODE C! ( char addr -- )
   POP BX
   POP AX
   MOV 0 [BX], AL
   NEXT
```

```
   END-CODE
```

CMOVE Move a set of bytes from source-addr to dest-addr. The number of bytes to be moved is count. The bytes are moved from low address to high address, so overlap is possible and in fact sometimes desired. To move a range of data, we use the interesting 8088/86 instruction REPZ MOVSB which moves a byte pointed to by DS:SI to a destination pointed to by ES:DI. Both SI and DI are incremented while CX is decremented. The moving continues until CX is decremented to zero. The direction flag, ES, DS, SI, DI and CX registers must be initialized properly before the block move and restored afterwards. SI is renamed as IP in F-PC.

```
CODE CMOVE  (  source dest count -- )
   MOV BX, IP        \ Save IP
   MOV AX, DS        \ Copy DS for setting ES
   POP CX        \ Count
   POP DI        \ Destination address
   POP IP          \ Source address, SI
   MOV DX, ES        \ Save ES
   MOV ES, AX        \ Point ES to code segment
   REPNZ        \ Repeat until count is zero
   MOVSB        \ Copy DS:SI to ES:DI
   MOV IP, BX        \ Restore IP of SI
   MOV ES, DX        \ Restore ES
   NEXT          \ Done
   END-CODE
```

CMOVE> The same as CMOVE above except that bytes are moved in the opposite direction, i.e., from high addresses to low addresses. SI and DI must be initialize to point to the end of the source and destination string, and the direction flag must be set so that addresses are decremented during moving.

```
CODE CMOVE> ( source dest count -- )
   MOV BX, IP        \ Save IP or SI
   MOV AX, DS        \ Copy DS
   POP CX        \ Count
```

```
   DEC CX        \ Make a small correction
   POP DI        \ Destination
   POP IP         \ Source
   ADD DI, CX        \ End of destination string
   ADD IP, CX        \ End of source string
   INC CX        \ The right count
   MOV DX, ES        \ Save ES
   MOV ES, AX        \ Point ES to code segment
   STD          \ Reverse direction of copying
   REPNZ         \ Repeat
   MOVSB         \   string move
   CLD          \ Restore move direction
   MOV IP, BX        \ Restore IP
   MOV ES, DX        \ Restore ES
   NEXT          \ All done
   END-CODE
```

PLACE Copy n bytes of data from source-addr to dest-addr and lay it down as a counted string.

```
CODE PLACE  ( source-addr n dest-addr -- )
   POP DI        \ Destination
   POP CX        \ Length
   MOV 0 [DI], CL     \ Copy the length byte
   INC DI        \ First byte of dest string
   CLD          \ Now do CMOVE
   MOV BX, IP
   POP IP
   MOV DX, ES
   MOV AX, DS
```

```
   MOV ES, AX
   REPNZ
   MOVSB
   MOV IP, BX
   MOV ES, DX
   NEXT
   END-CODE
```

CSET Set the contents of a character (byte) at addr so that the bits that are 1 in b are also 1 in addr. Equivalent to DUP C@ ROT OR SWAP C!

```
CODE CSET   ( b addr -- )
   POP BX
   POP AX
   OR 0 [BX], AL          \ Use OR to set bits
   NEXT
   END-CODE
```

CRESET Set the contents of a character (byte) at addr so the the bits that are 1 in b are zero in addr. Equivalent to DUP C@ ROT NOT AND SWAP C! .

```
CODE CRESET ( b addr -- )
   POP BX
   POP AX
   NOT AX         \ Clear bits by NOT AND
   AND 0 [BX], AL
   NEXT
   END-CODE
```

CTOGGLE Flip the bits in a character (byte) at addr by the value b. Equivalent to DUP C@ ROT XOR SWAP C! .

```
CODE CTOGGLE    ( b addr -- )
   POP BX
   POP AX
   XOR 0 [BX], AL      \ Toggle bits by XOR
   NEXT
   END-CODE
```

ON Set the contents of addr to TRUE or -1. All 16 bits are set. See also ON> in Sec. 6.2.

```
CODE ON ( addr -- )
   POP BX
   MOV 0 [BX], # TRUE WORD
   NEXT
   END-CODE
```

OFF Set the contents of addr to FALSE or 0. All 16 bits are cleared. See also OFF> in Sec. 6.2.

```
CODE OFF    ( addr -- )
   POP BX
   MOV 0 [BX], # FALSE WORD
   NEXT
   END-CODE
```

-1! Same as ON. This is to make our intention clear in storing a number -1 into the address. ON is meant to set a flag, which should not be taken as a number -1.

```
CODE -1!    ( addr -- )
   POP BX
   MOV 0 [BX], # TRUE WORD
```

```
   NEXT
   END-CODE
```

0! Same as OFF. The intention is to store a zero, not to clear a flag.

```
CODE 0! ( addr -- )
   POP BX
   MOV 0 [BX], # FALSE WORD
   NEXT
   END-CODE
```

INCR Increment the contents of addr. Equivalent to 1 SWAP +! . This is a very common operation missing in most Forth systems.

```
CODE INCR   ( addr -- )
   POP BX
   INC 0 [BX] WORD
   NEXT
   END-CODE
```

DECR The reverse of INCR. Decrement contents in addr.

```
CODE DECR   ( addr -- )
   POP BX
   DEC 0 [BX] WORD
   NEXT
   END-CODE
```

0DECR Decrement contents in addr to 0 only, not below.

```
CODE oDECR  ( addr -- )
   POP BX
   DEC o [BX] WORD
   o< IF   MOV o [BX], # o WORD
   THEN
   NEXT
   END-CODE
```

D+! Add a double integer d to the double integer located at addr. Similar to +!.

```
CODE D+!   ( d addr -- )
   POP BX
   POP AX
   POP DX
   ADD o [BX], DX
   ADC 2 [BX], AX
   NEXT
   END-CODE
```

+! Add a 16 bit integer n to the 16 bit integer stored at addr.

```
CODE +! ( n addr -- )
   POP BX
   POP AX
   ADD o [BX], AX
   NEXT
   END-CODE
```

C+! Add a byte b to a byte stored at addr.

```
CODE C+!   ( b addr -- )
  POP BX
  POP AX
  ADD 0 [BX], AL
  NEXT
  END-CODE
```

2.4. COMPARISON WORDS

Comparison words test the top item or items on the data stack and return a flag (identified as f on the stack) to be used mostly for conditional jump words like IF, UNTIL, WHILE, ?LEAVE, ABORT", etc. The flag is either TRUE (-1) or FALSE (0). Sometimes these flags can be used to increment or decrement a variable or address conditionally. Using the flag to do arithmetic is permissible (F-PC does not care a bit!) but is considered bad style. Flags should be used for logic operations, not for arithmetic operations. Relying upon the side effects of logic flags will often lead you into trouble.

In Forth, the TRUE condition is represented by any integer other than 0. The 'true TRUE' is represented by -1. All comparison words return 'true TRUE' under the true condition, but words consuming the true flag require only a non-zero integer. This property allows us to use = to test for equality and - to test for inequality. Using the results of arithmetic operations as flags is powerful but sometimes very confusing.

Forth is best known to be an untyped language. The reason is that data are generally pushed on the data stack and are operated upon there. Once a piece of data is pushed on the data stack, its identity is completely lost. It can thus be treated as an address, an integer, part of a double integer, a flag, a byte, a character, or whatever you desire it to be. Much of the flexibility and power of Forth is derived from this unique property. However, to write a clear and understandable Forth program, one should avoid as much as possible the confusion in mixed data types. F-PC includes many words of identical functions but with different names to emphasize the types of data involved in the operation. You are encouraged to use the proper words to clarify your intentions in the program.

F-PC provides a very rich set of comparison words, allowing you to test most situations easily and quickly. Most comparison words belong to either the unary type or the binary type as discussed in the arithmetic words, Sec. 2.2. There are also more complicated comparison words which do not generate flags, like WITHIN, BETWEEN, MAX and MIN.

In F-PC 2.25 and earlier versions, a common point of return is defined as a label YES. This practice was adopted from F83. It pushes a TRUE flag on the stack before doing NEXT:

```
LABEL YES
   MOV AX, # TRUE      \ Copy a true flag into AX
   1PUSH               \ Push AX and do NEXT
   END-CODE
```

In F-PC 3.5, this common return method is abolished. Each comparison word generates the proper flag locally, and runs much faster.

0= Return true if top is zero, false otherwise. 0= is more reassuring than NOT, which does a bitwise complement to the item on top of the stack. Here n does not have to be -1. Any non-zero integer will cause a false flag to be returned, and only a 0 will result in a true flag. NOT behaves like a comparison operator only for 'true TRUE' and FALSE flags. It will not work properly as 0= if the integer is not -1 or 0.

```
CODE 0= ( n -- f )
   POP AX
   SUB AX, # 1         \ Only if n=0, a borrow will be generated
   SBB AX, AX          \ Subtract with borrow to produce the correct flag.
   1PUSH
   END-CODE
```

0< Return true if top is negative, i.e., sign bit is on. Otherwise return false.

```
CODE 0< ( n -- f )
   POP AX
   CWD             \ Extend sign bit into DX register.
   PUSH DX
   NEXT
   END-CODE
```

0> Return true if top is positive.

```
CODE 0> ( n -- f )
   POP AX
   NEG AX
   OV<> IF CWD    \ Use the overflow bit to test for positive condition
      PUSH DX    \ Extend 0 to DX
      NEXT
   THEN
   SHL AX, # 1
   1PUSH
   END-CODE
```

0<> Return true if the top is non-zero, false otherwise.

```
CODE 0<>   ( n -- f )
   POP AX
   NEG AX
   SBB AX, AX
   1PUSH
   END-CODE
```

= Return true if the two elements on the stack are equal, false otherwise.

```
CODE =  ( n1 n2 -- f )
   POP AX
   POP CX
   SUB AX, CX
```

```
   SUB AX, # 1
   SBB AX, AX
   1PUSH
   END-CODE
```

<> Return true if the two element are not equal, else false.

```
CODE <> ( n1 n2 -- f )
   POP AX
   POP CX
   SUB AX, CX
   NEG AX
   SBB AX, AX
   1PUSH
   END-CODE
```

?NEGATE Negate the second element if the top is negative.

```
: ?NEGATE  ( n1 n2 -- n3 )
   0<          \ Negate n1 if n2<0
   IF   NEGATE  THEN  ;
```

U< Compare the top two elements on the stack as unsigned integers and return true if the second is less than the first. Be sure to use U< whenever comparing addresses, or else strange things will happen above the 32K boundary.

```
CODE U< ( u1 u2 -- f )
   POP CX
   POP AX
   SUB AX, CX
   SBB AX, AX
```

```
   1PUSH
   END-CODE
```

U> Compare the top two unsigned integers on the stack. True if u1 > u2.

```
CODE U> ( u1 u2 -- f )
   POP AX
   POP CX
   SUB AX, CX
   SBB AX, AX
   1PUSH
   END-CODE
```

< Compare the top two elements on the stack as signed integers and return true if n1 < n2.

```
CODE <  ( n1 n2 -- f )
   POP AX
   POP BX
   CMP BX, AX
   >= IF   SUB AX, AX
     1PUSH
   NEXT
   MOV AX, # TRUE
   1PUSH
   END-CODE
```

> Compare the top two elements on the stack as signed integers and return true if n1 > n2.

```
CODE >  ( n1 n2 -- f )
```

```
   POP AX
   POP BX
   CMP BX, AX
   <= IF   SUB AX, AX
      1PUSH
   NEXT
   MOV AX, # TRUE
   1PUSH
   END-CODE
```

UMIN Return the smaller of n1 and n2, using unsigned comparison.

```
CODE UMIN   ( n1 n2 -- min )
   POP AX
   POP BX
   CMP BX, AX
   U<= IF  PUSH BX
      NEXT
   THEN
   1PUSH
   END-CODE
```

MIN Return the smaller of n1 and n2.

```
CODE MIN    ( n1 n2 -- min )
   POP AX
   POP BX
   CMP BX, AX
   <= IF   PUSH BX
```

```
    NEXT
  THEN
  1PUSH
  END-CODE
```

MAX Return the larger of n1 and n2.

```
CODE MAX  ( n1 n2 -- max )
  POP AX
  POP BX
  CMP BX, AX
  <= IF  1PUSH
  THEN
  PUSH BX
  NEXT
  END-CODE
```

0MAX Return the larger of n1 or zero, treated as signed numbers.

```
CODE 0MAX  ( n1 n2 -- max )
  POP AX
  SUB BX, BX
  CMP BX, AX
  <= IF  1PUSH
  THEN
  PUSH BX
  NEXT
  END-CODE
```

UMAX Return the UNSIGNED larger of n1 and n2.

```
CODE UMAX   ( n1 n2 -- max )
   POP AX
   POP BX
   CMP BX, AX
   U<= IF  1PUSH
   THEN
   PUSH BX
   NEXT
   END-CODE
```

BETWEEN Return true if min <= n <= max, otherwise false.

```
CODE BETWEEN    ( n min max -- f )
   XOR AX, AX
   POP BX
   POP CX
   POP DX
   <= IF   CMP DX, CX
      >= IF   DEC AX
      THEN
   THEN
   1PUSH
   END-CODE
```

WITHIN Return true if min <= n < max, otherwise false.

```
CODE WITHIN ( n min max -- f )
```

```
  POP DI
  POP CX
  POP BX
  XOR AX, AX
  < IF    CMP BX, CX
    >= IF   DEC AX
    THEN
  THEN
  1PUSH
  END-CODE
```

2.5. INPUT AND OUTPUT WORDS

Personal computers are much more fun than minicomputers and mainframes because you can do your own things with personal computers. The IBM-PC is more fun than the Macintosh because the PC has an open bus which you can use to put in your own circuits. For instance, the Prototype Card with decoding layout, JDR-PR2, from JDR Micro Devices, San Jose, CA (408)947-8881, about $30.

To do useful things with a PC, you have to get inputs from outside into the PC and also send signals from the PC to the outside world. Since the 8088/86 has a separate I/O path, we define a Forth interface to it. Use P@ and P! to read or write directly to the 8086 16 bit ports. Use PC@ and PC! to read or write to the 8088 8 bit ports.

PC@ Fetch a byte from an input port whose port number is on the data stack. The port number is first popped into the DX register which is used to address the I/O ports. Data is fetched into the AL register and then pushed on the data stack.

```
CODE PC@    ( port# -- b )
  POP DX        \ Read port# into DX.
  IN AL, DX         \ Fetch a byte.
  SUB AH, AH        \ Clear higher byte in AX.
```

```
   PUSH AX        \ Push results on the stack.
   NEXT           \ Faster than 1PUSH.
   END-CODE
```

P@ Fetch a 16 bit integer from an input port. The 8086/80286 has a 16 bit I/O bus and you can get 16 bit data from a port if the device connected to this port also supports a 16 bit data path. It is actually easier to get 16 bits of data than 8 bits because you don't have to mask the upper 8 bits.

```
CODE P@ ( port# -- n )
   POP DX
   IN AX, DX
   PUSH AX
   NEXT
   END-CODE
```

PC! Store a byte into an 8 bit output port.

```
CODE PC!  ( b port# -- )
   POP DX
   POP AX
   OUT DX, AL
   NEXT
   END-CODE
```

P! Store a 16 bit integer into a 16 bit output port.

```
CODE P! ( n port# -- )
   POP DX
   POP AX
   OUT DX, AX
```

```
NEXT

END-CODE
```

In the above Forth I/O words, port addresses are passed through the DX register. The F-PC assembler also supports the 8080 style input-output instructions. If the port address is known and fixed at the assembly time, the explicit port number can be used to direct I/O function to a specific port. The I/O instructions can be coded as follows:

```
IN AL, # port

OUT # port AL
```

In this case the port number is restricted to one byte. These I/O instructions cannot access ports whose port numbers are higher than 255. However, these fixed port I/O instructions are faster, even without the overhead to set the DX register.

2.6. DOUBLE INTEGER WORDS

F-PC is a 16 bit Forth implementation, because the width of data stored on the stacks is 16 bits and most of the arithmetic and logic operations assume that the data are 16 bits in width. However, a large set of words is included in F-PC to deal with 32 bit data. The double integer extension word set in the Forth-83 Standard is fully supported.

A double integer occupies a 4 byte of space in memory. The order of significance is Byte2, Byte3, Byte0 and Byte1, where Byte0 is the least significant part of the double integer and Byte3 is the most significant. This strange arrangement is caused by the requirement that when a double integer is on the stack, the most significant 16 bit half must be on the top of the stack and is more easily accessible. The ordering of bytes in a 16 bit integer is due to the Intel convention: the least significant byte precedes the most significant byte in memory. The byte ordering of a double integer is the same on the data stack as in the memory.

If you pop a double integer on the top of data stack, you will get the most significant 16 bits first. If you fetch a 16 bit integer from the address of a double integer, you will get the most significant 16 bit also. The least significant 16 bits are stored at address+2.

The 8088/8086 lends itself easily to do double integer operations if you put the address in the BX register. The most significant 16 bits can be accessed by 0 [BX] and the least significant part by 2 [BX]. The technique becomes obvious if you examine the following code definitions.

2@ Fetch a 32 bit value from addr.

```
CODE 2@ ( addr -- d )
   POP BX         \ Pop address
   PUSH 2 [BX]           \ PUSH LS 16 bits
   PUSH 0 [BX]        \ PUSH MS 16 bits
   NEXT
   END-CODE
```

2! Store a 32 bit value at addr.

```
CODE 2! ( d addr -- )
   POP BX         \ Pop address
   POP 0 [BX]         \ Store MS 16 bits
   POP 2 [BX]         \ Store LS 16 bits
   NEXT              \ Done
   END-CODE
```

2DROP Discard the top two elements of the stack.

```
CODE 2DROP  ( d -- )
   ADD SP, # 4
   NEXT
   END-CODE
```

3DROP Discard the top three elements of the stack.

```
CODE 3DROP  ( n d -- )
```

```
   ADD SP, # 6
   NEXT
   END-CODE
```

2DUP Duplicate the top two elements of the stack.

```
CODE 2DUP  ( d -- d d )
   MOV DI, SP
   PUSH 2 [DI]
   PUSH 0 [DI]
   END-CODE
```

3DUP Duplicate the top three elements of the stack.

```
CODE 3DUP  ( a b c -- a b c a b c )
   MOV DI, SP
   PUSH 4 [DI]
   PUSH 2 [DI]
   PUSH 0 [DI]
   END-CODE
```

4DUP Duplicate the top 4 elements of the stack.

```
: 4DUP   ( a b c d -- a b c d a b c d )
   2OVER 2OVER
   ;
```

2SWAP Swap the top two pairs of double integers on the stack. You can use this operator to swap two 32 bit integers and preserve their meaning as double integers.

```
CODE 2SWAP  ( d1 d2 -- d2 d1 )
   POP CX        \ Pop them all
   POP BX
   POP AX
   POP DX
   PUSH BX       \ Push in the right order
   PUSH CX
   2PUSH
   END-CODE
```

2OVER Copy the second pair of double integers over the top pair. Behaves like OVER for 32 bit integers.

```
CODE 2OVER  ( d1 d2 -- d1 d2 d1 )
   MOV DI, SP
   PUSH 6 [DI]
   PUSH 4 [DI]
   NEXT          \ 50 cycles
   END-CODE
```

2ROT Rotate top three double integers. It is done in high level because we do not have enough registers to store 6 items. It is also used very rarely.

```
: 2ROT    ( a b c d e f -- c d e f a b )
   5 ROLL          \ Get a
   5 ROLL          \ Get b
   ;
```

D+ Add the two double integers on the stack and return the results as a double integer.

```
CODE D+ ( d1 d2 -- dsum )
   POP AX        \ Get d2
   POP DX
   POP BX        \ Get d1
   POP CX
   ADD DX, CX        \ Add LS parts
   ADC AX, BX        \ Add MS parts with carry.
   2PUSH             \ Push sum.
   END-CODE
```

DNEGATE Same as NEGATE except for double integers.

```
CODE DNEGATE   ( d1 -- -d1 )
LABEL DNEG1           \ Shareable code with DABS
   POP AX        \ Get d1
   POP DX
   NEG AX
   NEG DX
   SBB AX, # 0       \ Adjust borrow
   2PUSH             \ Push results
   END-CODE
```

S>D Take a single integer and make it a double integer by extending the sign bit to the upper half.

```
CODE S>D   ( n -- d )
   POP AX
   CWD           \ Just the right instruction to extend the sign of n.
   XCHG DX, AX       \ Adjust the register order
   2PUSH
```

```
   END-CODE
```

DABS Return the absolute value of the 32 bit integer on the stack.

```
CODE DABS   ( d -- |d| )
   POP AX
   OR AX, AX
   0>= IF  1PUSH
   THEN
   POP DX
   NEG AX
   NEG DX
   SBB AX, # 0
   2PUSH
   END-CODE
```

D2* 32 bit left shift.

```
CODE D2*    ( d -- d*2 )
   POP AX        \ Get MS part
   POP DX        \ Get LS part
   SHL DX, # 1       \ Shift LS part left
   RCL AX, # 1       \ Rotate left with carry
   2PUSH             \ Push results
   END-CODE
```

D2/ 32 bit arithmetic right shift. Equivalent to divide by 2.

```
CODE D2/    ( d -- d/2 )
```

```
   POP AX
   POP DX
   SAR AX, # 1        \ Arithmetic shift right
   RCR DX, # 1        \ Rotate right with carry
   2PUSH
   END-CODE
```

D- Subtract the two double integers.

```
: D-   ( d1 d2 -- d3 )
   DNEGATE       \ -d2
   D+   ;          \ d1-d2
```

?DNEGATE Negate the double integer if the top is negative.

```
: ?DNEGATE  ( d1 n -- d2 )
   0<          \ n<0 ?
   IF   DNEGATE   THEN   ;
```

D0= Compare the top double integer to zero. True if d=0.

```
: D0=      ( d -- f )
   OR          \ Every bit must be 0.
   0= ;
```

D= Compare the top two double integers. True if d1=d2

```
: D=       ( d1 d2 -- f )
   D-          \ Compare
```

```
   D0= ;            \ Return the flag
```

DU< Perform unsigned comparison of two double integers. It is built upon the single integer unsigned compare U<.

```
CODE DU<   ( ud1 ud2 -- f )
   POP DX
   POP BX
   POP CX
   POP AX
   SUB AX, BX
   SBB CX, DX
   SBB AX, AX
   1PUSH
   END-CODE
```

D< Compare the top two double integers. True if d1<d2.

```
: D<      ( d1 d2 -- f )
   2 PICK OVER =      \ If MS parts are equal,
   IF     DU<         \ use DU< to compare.
   ELSE  NIP ROT DROP <    \ Otherwise compare MS parts.
   THEN  ;
```

D> Compare the top two double integers. True if d1>d2.

```
: D>      ( d1 d2 -- f )
   2SWAP
   D<   ;
```

DMIN Return the lesser of the top two double integers.

```
: DMIN    ( d1 d2 -- d3 )
   4DUP D>          \ If d1>d2,
   IF  2SWAP  THEN      \ Move the unwanted double integer to top of stack,
   2DROP ;      \  and discard it.
```

DMAX Return the greater of the the top two double integers.

```
: DMAX  ( d1 d2 -- d3 )
   4DUP D<          \ If d1<d2,
   IF  2SWAP  THEN      \  move d1 to top,
   2DROP ;      \  and discard the lesser.
```

2.7. THE STRUCTURE WORDS

In a colon definition, Forth words are compiled as a list of addresses which point to executable code. In F-PC, the list of addresses is stored in the List Segment, and the addresses are Code Field Addresses. The Code Field Addresses are offset addresses in the Code Segment, which contains executable machine code and various other data structures. When a colon definition is executed, the colon inner interpreter goes through this list of cfa's and executes the code contained in the code fields. Since the code field contains executable code, F-PC is thus based on the 'Direct Threaded Code.' F-PC is much faster than its predecessor F83 which is based on the 'Indirect Threaded Code.' In indirectly threaded code, an address compiled in a colon definition points to a code field which contains the address of executable code. F-PC gets rid of this level of indirection and the performance is greatly improved.

There are other data structures in a colon definition. The most common data besides code addresses are integer literals. A literal structure contains the code address of (LIT) and a 16 bit integer, compiled in-line with the addresses. When (LIT) is executed, it pulls the integer out of the next address, pushes it on the data stack, and continues executing the next word following the integer. Other data structures based on integer literals are double integer literal, ASCII code literal, and address literal.

Another useful class of literals is string literals, which allow strings of characters to be compiled in-line with the execution addresses. The run-time behavior of string literals depends upon the string literal word compiled before the string. The string literal will be discussed much later.

Forth is a structured language and it allows the user to build control structures inside colon definitions and code definitions. Control structures are characterized by the property that each structure has one entry point and one exit point. F-PC provides a powerful word set for building selection structures (IF-ELSE-THEN), finite loops (DO-LOOP), indefinite loops (BEGIN-UNTIL and BEGIN-WHILE-REPEAT), infinite loops (BEGIN-AGAIN), and case structures.

Building literals and control structures requires specialized tools which are generally called compiler directives or assembler directives. The directives are Forth words which are immediately executed inside colon or code definitions. They compile special words and the necessary data or addresses so that the desired control path can be accomplished at run-time. It requires very lengthy discussions to fully explain these directives because one would have to comprehend the significance of words whose run-time behavior differs from their compile-time behavior. The difficulty multiplies here in the kernel because the kernel is built by the metacompiler, which has to build an object image which is executable in a target environment, not in the host environment while the object image is being compiled.

Keeping these difficulties in mind, you will understand that it is impossible to fully explain all the intricacies in this part of the F-PC kernel. You don't have to read this section too seriously. Scan through it first and move to other parts which are more interesting. When you finish the metacompiler, come back and re-visit this section. It will then begin to make sense, I assure you.

T: defines a metacompiler directive, which compiles a data or control structure in the target image during metacompilation. The set of metacompiler directives are very similar to the set of compiler directives used for normal Forth programming, as discussed in Chapter 4 later. The difference is that the regular compiler directives build structures in executable code on the host computer, while the metacompiler directives build structures in the target code image intended to be executed on a target computer.

In Figures 4 and 5 we try to summarize different data structures and control structures used in F-PC colon definitions. It will give you an idea of how these structures are used in colon definitions and the run-time code which executes these structures.

(LIT) The run-time code for literals. Push the following two bytes in the list of cfa's onto the data stack and moves the IP over them. It is compiled by the word LITERAL. (LIT) allows constants to be stored in-line with the cfa's in the list of a colon definition. Using the special 8088 instruction LODSW ES:, (LIT) is much faster than constants which have to jump to the Code Segment to find their stored values. It is thus more advantageous in F-PC to use literals than constants.

CODE (LIT) (-- n)

```
   LODSW ES:          \ Grab next word in list
   1PUSH              \ Push it on data stack
   END-CODE
```

<'> Same as (LIT).

LITERAL Compile (LIT) into the target list dictionary and follow it by the integer on the top of data stack. (LIT) and the integer form an integer literal structure in a colon definition.

```
T: LITERAL  ( n -- )
   [TARGET] (LIT)      \ Compile (LIT)
   ,-X   T;            \ Compile n
```

DLITERAL Compile a double integer literal as two single integer literals. The (LIT) dl (LIT) dh structure will push the double integer onto the stack at run-time.

```
T: DLITERAL ( d -- )
   [TARGET] (LIT) ,-X      \ Compile lower half of d
   [TARGET] (LIT) ,-X   T;     \ Compile higher half of d
```

ASCII Compile the next ASCII character as a (LIT) literal. When executed at run-time, the ASCII code is pushed on the stack. Hence we call it an ASCII literal.

```
T: ASCII    ( -- )
   [COMPILE] ASCII        \ Translate character to ASCII
   [[ TRANSITION ]] LITERAL    \ Compile the ASCII code as a  literal.
   [META]  T;
```

Figure 4. Data structures in colon words.

Figure 5. Control structures in F-PC.

['] Address literal compiler. Look up the execution address of the next word and compile it as a literal. This address will be pushed on the stack at run-time.

```
T: [']     ( -- )
   'T            \ Find next word in target
   >BODY @       \   dictionary
   [[ TRANSITION ]] LITERAL    \ Compile the address as a
   [META]   T;       \   literal.
```

CONSTANT Define a constant in the target dictionary. An identical constant is also defined in host so that it can be referenced by other words while building the target system.

```
: CONSTANT  ( n -- )
   RECREATE          \ Create target header
   232 C,-T          \ Assemble a CALL instruction
   [[ ASSEMBLER DOCONSTANT ]] \ Find the address of
   LITERAL       \   DOCONSTANT, and assemble
   HERE 2+ - ,-T         \   its offset for CALL.
   DUP ,-T       \ Assemble n in the body.
   CONSTANT   ;          \ Create a constant in the host for referencing.
```

<(;CODE)> Start an inner interpreter coded in assembly language. Its actual behavior cannot be specified now because it needs many supporting words in the last part of the kernel. The forward reference will be resolved at the end of the kernel.

```
FORWARD: <(;CODE)>
```

DOES> Start an inner interpreter defined using high level Forth words. It is part of a defining word, specifying the run-time behavior of daughter words created by the defining word.

```
T: DOES>   ( -- )
   [FORWARD] <(;CODE)> \ Compile <(;CODE)> in list
   HERE-T ,-X        \   with current code address
   HERE-T            \ Save a copy of cfa
   ( DOES-OP ) 232 C,-T       \ Assemble a CALL instruction
   [[ ASSEMBLER DODOES ]] \ Compile the address of
   LITERAL           \   DODOES as a literal
   HERE 2+ - ,-T         \ Assemble offset for CALL
   HERE-X PARAGRAPH-X +    \ Move list segment pointer
   DUP DPSEG-X !      \   to next free segment
   SEG-X @ - ,-T         \ Assemble segment offset
   DP-X OFF T;        \ Clear list offset register
```

NUMERIC Convert a number string placed by the host interpreter in the host word buffer to a double integer. If the string contains a decimal point, compile a double integer literal into the target dictionary; otherwise, compile the lower half of the double integer as a single integer literal.

```
: NUMERIC   ( -- )
   [FORTH] HERE       \ Host word buffer
   [META] NUMBER      \ Convert string to double
   DPL @ 1+           \ Decimal point?
   IF [[ TRANSITION ]]        \ Yes.
     DLITERAL [META] \ Compile double literal
   ELSE DROP             \ No.  Discard upper half
     [[ TRANSITION ]]
     LITERAL [META]     \ Compile integer literal
   THEN  ;
```

UNDEFINED Create a forward reference to an undefined word in the kernel. All forward references are linked into a chain to ease their resolution.

```
: UNDEFINED ( -- )
   HERE-X >XREL       \ Save the current relative list address
   0 ,-X              \ Compile a dummy code address
   IN-FORWARD         \ Enable FORWARD vocabulary
   [FORTH] CREATE     \ Make a header in FORWARD
   [META] TRANSITION
   [FORTH] ,          \ Store the list address here
   FALSE ,  [META]    \  with a unresolved flag
   DOES>   FORWARD-CODE  ;   \ Link all occurrence until its resolution

[FORTH] VARIABLE T-IN META  \ Temporary storage for >IN
```

] The target colon compiler. It compiles colon definitions into the target dictionary and builds the target core image. It is similar to the regular F-PC]. However, it has to handle all the extra work involved in creating the target core image.

```
: ]   ( -- )
   STATE-T ON              \ Compilation flag
   IN-TRANSITION           \ Target environment
   BEGIN   >IN @ T-IN !          \ Save current >IN
      BEGIN   BL WORD DUP C@ 0= \ If nothing in line
         ?FILLBUFF               \ Optionally refill buffer
         INLENGTH 0> AND     \ Input buf not empty
      WHILE  DROP
         0 T-IN !
         ?LISTING       \ If ?LISTING is on, display names and addresses.
         IF  CR BASE @ >R HEX
```

```
            HERE-T 4 .R SPACE
            LINESTRT HERE-T
            OVER - 5 MIN BOUNDS
            ?DO I C@-T
              0 <# # # BL HOLD #>
              TYPE
            LOOP
            22 #OUT @ - SPACES
            TIB #TIB @ TYPE
            R> BASE !
          THEN
          FILLTIB               \   refill the buffer
          HERE-T =: LINESTRT  \ Reset
        REPEAT  ?UPPERCASE FIND \ Search next word
        IF      EXECUTE     \ Execute if found
        ELSE    COUNT NUMERIC?  \ Else, a number?
          IF      NUMERIC \ Yes.  Convert it.
          ELSE    T-IN @ >IN !   \ No.  Restore >IN.
            UNDEFINED?  \ Forward reference
          THEN
        THEN    STATE-T @ 0=        \ Exit if not compiling
      UNTIL ;

[       Target interpreter.  Stop target compilation and start executing words encountered.

T: [       ( -- )
      IN-META
      STATE-T OFF   T;        \ Simply turn off the compilation flag.
```

; Terminate a target colon definition by compiling an UNNEST to the colon definition and then turn on the interpreter.

```
T: ;       ( -- )
   [TARGET] UNNEST     \ Compile UNNEST
   [[ TRANSITION ]] [   T;     \ Turn off compiler
```

: Create a colon definition in the target dictionary. Turn on the target compiler to compile the words following until ; .

```
T: :       ( -- )
   TARGET-CREATE          \ Create headers for target and host dictionaries
   233 C,-T                \ Assemble JMP NEST
   [[ ASSEMBLER NEST ]] LITERAL    \   in target code space
   HERE 2+ - ,-T
   HERE-X PARAGRAPH-X +    \ Adjust to segment boundary
   DUP DPSEG-X !      \ Start list there
   SEG-X @ - ,-T         \ Assemble segment offset after JMP NEST
   DP-X OFF          \ Initialize list offset
   ] ;                        \ Start compiling

ASSEMBLER CLEAR_LABELS META \ Clear local labels
```

DOBEGIN Mark the beginning of an indefinite or infinite loop. It is compiled by BEGIN. This is in itself a NOOP and is not needed for execution purposes. It is defined so that the F-PC decompiler will be able to correctly reconstruct the loop structure.

```
CODE DOBEGIN    ( -- )      \ Really a NOOP
   NEXT
   END-CODE
```

DOTHEN A NOOP. Same as DOBEGIN. Mark the end of a conditional structure. Again, it is used to facilitate the decompiler.

DOCASE A NOOP. Mark the beginning of a case structure.

DOENDCASE A NOOP. Mark the end of a case structure.

DOAGAIN Unconditional jump to the offset address compiled in the next cell. Compiled by AGAIN to close an infinite loop.

```
CODE DOAGAIN   ( -- )
  MOV ES: IP, o [IP]     \ Move the address in next
  NEXT             \   cell into IP, thus effect
  END-CODE          \   an unconditional jump.
```

DOREPEAT Same as DOAGAIN. Compile by REPEAT to close a BEGIN-WHILE-REPEAT loop. Jump back to BEGIN.

```
CODE DOREPEAT  ( -- )
  MOV ES: IP, o [IP]     \ Load IP and jump
  NEXT
  END-CODE
```

?WHILE Compiled by WHILE inside an indefinite loop. Test top of stack. If the flag is true, continue executing the following word sequence. Otherwise, jump to the end of loop marked by DOREPEAT.

```
CODE ?WHILE ( f -- )
  POP CX
  CXo<> IF
    ADD IP, # 2     \ Continue on.
    NEXT
```

```
  THEN
  MOV ES: IP, 0 [IP]      \ Load IP and jump to the end of the loop.
  NEXT
  END-CODE
```

?UNTIL Compiled by UNTIL as the end of an indefinite loop. Test top of stack as a flag. If it is true, exit the loop. Otherwise jump back to BEGIN. The code is identical to >WHILE.

BRANCH Performs an unconditional branch. Compiled by ELSE.

```
CODE BRANCH ( -- )
LABEL BRAN1
  MOV ES: IP, 0 [IP]
  NEXT
  END-CODE
```

DOENDOF Identical to BRANCH. End of a case clause.

?BRANCH Performs a conditional branch. If the top of the data stack is true, take the branch. If not, skip over the branch address which is in-line. Same as ?WHILE. It is compiled by IF.

NEXT| End of a FOR...NEXT loop. This loop structure was used by Chuck Moore for NC4000 Forth engine, in which NEXT| is a machine instruction.

```
CODE NEXT| ( n1 --- )      \ Primitive form of NEXT (as in FOR - NEXT loops)
  SUB 0 [RP], # 1 WORD       \ Decrement top of return stack.
  U>= IF           \ If not negative yet, branch back to address after FOR.
    MOV IP, ES: 0 [IP]
    NEXT
  THEN
  ADD RP, # 2        \ End of loop.  Discard count on the return stack.
```

```
   ADD IP, # 2        \ Continue.

   NEXT    END-CODE
```

Following come the words that build control structures in the target image. They are executed during target compilation and cause special run-time words and addresses to be compiled into the target dictionary. They also use the data stack in the host system to resolve addresses for jumping and branching in the target system.

FOR Compile >R which moves the count from data stack to the return stack. Save the address after FOR for NEXT| to resolve.

```
T: FOR

   [TARGET]  >R          \ Compile >R.

   X?<MARK T;         \ Leave current address for NEXT|.
```

NEXT Terminate a FOR...NEXT loop structure. Decrement the loop count on the top of the return stack. If the count was zero before decrementing, exit the loop.

```
T: NEXT

   [TARGET] NEXT|      \ Compile NEXT|.

   X?<RESOLVE T;       \ Resolve the loop address.
```

BEGIN Compile DOBEGIN to start an indefinite loop. Leave the address after DOBEGIN for ?UNTIL and DOREPEAT to jump back to.

```
T: BEGIN

   [TARGET] DOBEGIN       \ Compile DOBEGIN

   X?<MARK   T;          \ Leave its address on stack to jump back to.
```

AGAIN Compile DOAGAIN and the address of BEGIN to close an infinite loop.

```
T: AGAIN
```

```
   [TARGET] DOAGAIN       \ Compile DOAGAIN
   X?<RESOLVE   T;    \   and the address of BEGIN left on the stack.
```

UNTIL Compile ?UNTIL and the address of BEGIN to close an indefinite loop.

```
T: UNTIL
   [TARGET] ?UNTIL        \ Compile ?UNTIL and the
   X?<RESOLVE   T;    \   address of BEGIN left on the stack.
```

IF Start a conditional branch structure. Compile ?BRANCH and leave address of the next cell on the stack as a forward jump address.

```
T: IF
   [TARGET] ?BRANCH       \ Compile conditional branch
   X?>MARK     T;    \ Mark next cell for forward jump.
```

FORWARD Make a unconditional forward jump. Similar of IF.

```
   [TARGET] BRANCH
   X?>MARK    T;
```

THEN Terminate a conditional structure and resolve the forward jump address at IF or ELSE.

```
T: THEN
   [TARGET] DOTHEN        \ Compile a NOOP
   X?>RESOLVE    T;       \ Resolve forward jump
```

AFT Start an AFT…THEN clause inside a BEGIN…WHILE…REPEAT loop. What is between AFT and THEN will be executed only once.

```
T: AFT
```

```
   2DROP          \ Discard address and flag left by BEGIN.
   [TARGET] BRANCH     \ Compile BRANCH to skip words after AFT.
   X?>MARK        \ Leave mark to jump forward.
   X?<MARK        \ Leave mark for backward jump.
   2SWAP          \ Put them in proper order.
   T;
```

ELSE Compile an unconditional jump but leave the jump address to be resolved by THEN. It also resolves the forward jump address left open by IF.

```
T: ELSE
   [TARGET] BRANCH        \ Compile unconditional branch
   X?>MARK           \ Leave jump address open
   2SWAP X?>RESOLVE   T;   \ Resolve forward jump address at IF.
```

WHILE Compile the selection word ?WHILE inside a BEGIN-WHILE-REPEAT indefinite loop. It is similar to IF.

```
T: WHILE
   [TARGET] ?WHILE     \ Compile ?WHILE
   X?>MARK   T;          \ Leave address to be resolved
```

REPEAT Compile DOREPEAT as the end of a BEGIN-WHILE-REPEAT loop. It jumps unconditionally to BEGIN.

```
T: REPEAT
   2SWAP          \ Resolve BEGIN first.
   [TARGET] DOREPEAT      \ Compile DOREPEAT
   X?<RESOLVE         \ Resolve BEGIN reference
   X?>RESOLVE T;       \ Resolve WHILE reference
```

CONTINUE Similar to REPEAT, but used inside a BEGIN loop. Jump to BEGIN unconditionally.

```
T: CONTINUE
   2OVER          \ intercept the address/flag pair.
   [TARGET] DOREPEAT
   X?<RESOLVE         \ Resolve backward jump.
   X?>RESOLVE T;      \ Resolves forward jump.
```

BREAK Exit a definition like EXIT. Terminate an IF...BREAK clause.

```
T: BREAK
   [TARGET] EXIT      \ Desired behavior.
   [TARGET] DOTHEN    \ Terminate IF clause.
   X?>RESOLVE T;      \ Resolve forward jump from IF.
```

UNDO Clean the return stack after exiting a DO...LOOP abnormally.

```
CODE UNDO  ( --- )
   ADD RP, # 6        \ Pop index, limit, and exit address off the return stack.
   NEXT
   END-CODE
```

(LOOP) The run-time procedure for LOOP. Branch back to the beginning of the loop if there are more iterations to do. Otherwise it exits. The loop counter is incremented.

```
CODE (LOOP) ( -- )
   INC 0 [RP] WORD    \ Increment index
   OV<> IF        \ If not yet overflowed,
      MOV ES: IP, 0 [IP]  \   loop back.
      NEXT
   THEN
```

```
   ADD RP, # 6        \ Overflow, terminate the loop.
   ADD IP, # 2
   END-CODE
```

(+LOOP) Increment the loop counter by the value on the stack and decide whether or not to loop again. Due to the weirdness of the 8088, you have to stand on your head to determine the conditions under which you loop or exit.

```
CODE (+LOOP)   ( n -- )
   AX POP         \ Get increment n
   ADD 0 [RP], AX     \ Add it to the index
   OV<> IF        \ If not yet overflowed,
      MOV ES: IP, 0 [IP] \   loop back.
      NEXT
   THEN
   ADD RP, # 6        \ Overflow, terminate the loop.
   ADD IP, # 2
   END-CODE
```

(DO) The run-time code compiled by DO. Push an in-line address onto the return stack along with values needed by (LOOP). This DO-LOOP mechanism was included in the Forth-83 Standard, attributed to Bob Smith. Three items are pushed on the return stack: the exit address which was compiled right after (DO), the loop limit lim with its most significant bit set, and a loop index which is ind-lim+8000H. The index is incremented by (LOOP). The overflow flag will indicate that the index has crossed the boundary of limit and limit-1. The overflow condition causes the loop to be terminated. This loop mechanism is faster than the old way where the unmodified limit and index are pushed on the return stack.

This DO-LOOP structure does have the peculiarity that if ind=lim, the loop is executed 65536 times, which produces a momentary pause for an empty loop, or a longer pause if a slow set of instructions is in the loop. The exit address is compiled after (DO) by DO and resolved by LOOP. It is pushed on the return stack under index and limit so that LEAVE will be able to exit the loop immediately, without having to continue executing till LOOP before exiting. This DO-LOOP structure was adopted in F83 and carried over into F-PC.

```
HEX            \ Hard to say 8000H in decimal
CODE (DO)   ( lim ind -- )
```

```
  POP DX        \ Get index ind
  POP BX        \ Get limit lim
  XCHG RP, SP       \ Prepare to push things on the return stack.
  LODSW ES:         \ Get the exit address after LOOP.
  PUSH AX       \ Push the address on the return stack.
  ADD BX, # $8000    \ Offset limit by $8000
  PUSH BX       \ Push modified limit
  SUB DX, BX        \ Take the difference of limit and index
  PUSH DX       \ Push it on the return stack.
  XCHG RP, SP       \ Restore data stack and return stack pointers
  NEXT
  END-CODE

DECIMAL
```

(?DO) The run-time code compiled by ?DO. The difference between ?DO and DO is that ?DO will not perform any iteration if the initial index is equal to the final limit.

```
CODE (?DO)  ( lim ind -- )
  POP DX        \ Get index
  POP BX        \ Get limit
  CMP BX, DX        \ limit=index?
  0= IF
    MOV ES: IP, 0 [IP]  \ Yes.  Exit immediately.
    NEXT
  THEN
  XCHG RP, SP       \ Lifted from (DO).
  LODSW ES:
  PUSH AX
  ADD BX, # $8000
```

```
   PUSH BX
   SUB DX, BX
   PUSH DX
   XCHG RP, SP
   NEXT
   END-CODE
```

(OF) Run-time code used inside a CASE structure. It is compiled by OF.

```
CODE (OF)  ( n1 n2 -- n1 )  ( or )  ( n1 n1 -- )
   POP AX        \ Get current test number
   MOV DI, SP        \ Test against the input number.  DI  relative is faster than
   CMP AX, 0 [DI]     \   popping from data stack and pushing it back.
   0<> IF           \ Not equal, jump to next case.
      MOV ES: IP, 0 [IP]
      NEXT
   THEN
   ADD SP, # 2       \ Numbers are equal, execute the OF clause.
   ADD IP, # 2
   NEXT
   END-CODE
```

BOUNDS Given a starting index and the number of repetitions of the loop, compute limit and index for a DO ... LOOP structure. Equivalent to OVER + SWAP.

```
CODE BOUNDS ( ind n --- lim ind )
   POP DX        \ Get count n
   POP AX        \ Get starting index ind
   ADD DX, AX        \ Compute limit lim
   2PUSH           \ Push limit and index
```

```
   END-CODE
```

?DO Similar to DO. However, the loop body will be skipped if limit and initial index are the same. This is to avoid the situation that the loop body in DO-LOOP is executed 65536 times if limit=index.

T: ?DO

```
   [TARGET] (?DO)        \ Compile (?DO)
   X?>MARK   T;          \ Leave address to be resolved by LOOP
```

DO Compile (DO) with a dummy address which will be resolved by LOOP. Construct a definite loop structure.

T: DO

```
   [TARGET] (DO)
   X?>MARK   T;
```

LOOP Close a definite loop structure. Compile (LOOP) with an address after (DO) and also resolve the forward address after (DO).

T: LOOP

```
   [TARGET] (LOOP)       \ Compile (LOOP)
   2DUP 2+           \ Get address after (DO)
   X?<RESOLVE        \ Resolve the forward reference at (DO)
   X?>RESOLVE   T;    \ Resolve the backward jump address
```

+LOOP Similar to LOOP, except that it will take the top item of the stack and add it to the index before testing for looping. The loop can be thus stepped through using any desired step size.

T: +LOOP

```
   [TARGET] (+LOOP)
   2DUP 2+
   X?<RESOLVE
```

```
   X?>RESOLVE   T;
```

I Return the current loop index. It now requires a little more calculation to compute it than in FIG Forth but the tradeoff is a much faster (LOOP). The loop index is stored on the return stack.

```
CODE I     ( -- n )
   MOV AX, 0 [RP]      \ Get modified index
   ADD AX, 2 [RP]      \ Un-modify it by adding
   1PUSH               \   mod-index + mod-limit
   END-CODE
```

J Return the loop index of the inner loop in a nested DO .. LOOP structure.

```
CODE J     ( -- n )
   MOV AX, 6 [RP]      \ It's easy to reach deep
   ADD AX, 8 [RP]      \   into the return stack.
   1PUSH
   END-CODE
DECIMAL
```

(LEAVE) Do an immediate exit from a DO ... LOOP structure. Unlike figForth which waits until the next LOOP is executed.

```
CODE (LEAVE)    ( -- )
LABEL PLEAVE
   ADD RP, # 4         \ Pop index and limit off
   MOV IP, 0 [RP]      \ Get the exit address
   ADD IP, # 2
   NEXT                \ Out of the loop
   END-CODE
```

(?LEAVE) Leaves the loop if the flag on the stack is true. Continues if not.

```
CODE (?LEAVE)   ( f -- )
   POP AX         \ Pop the flag
   OR AX, AX          \ True?
   JNE PLEAVE         \ Yes.  Exit the loop.
   NEXT               \ No.  Continue.
   END-CODE
```

LEAVE Compile (LEAVE).

```
T: LEAVE
   [TARGET] (LEAVE)
   T;
```

?LEAVE Compile (?LEAVE) which will terminate a loop if the flag on top of the data stack is true.

```
T: ?LEAVE
   [TARGET] (?LEAVE)
   T;
```

CHAPTER 3. INPUT, OUTPUT AND NUMBERS

KERNEL2.SEQ contains words which handle functions needed by the text interpreter and the colon compiler in F-PC. It includes system variables, user variables, terminal input and output words, tools and mechanisms to get strings from the user or from a file, converting the strings to a form usable by the system, and also words converting numbers between the external ASCII form and the internal binary form. Similar functions in F83 were all written in high level. Most of them are reduced to assembly code words in F-PC to improve execution speed.

Many code words in F-PC 2.25 had shared code fragments to save memory space. They are changed in F-PC 3.5 such that every code word is defined independently and are self-sufficient. Code words can now be compiled independently and moved around freely, making it easier for target compilation.

3.1. USER VARIABLES AND USER DEFERRED WORDS

Forth as originally developed by Charles Moore has always been a multi-user/multi-tasking system which can support many foreground tasks, and many background tasks. A foreground task interacts with the user through a terminal, and background tasks are used to control peripherals and instruments. Forth, Inc. still holds the multitasker, among other things in polyForth, as proprietary information. Respecting the pioneering efforts by Charles Moore and his associates at Forth, Inc., implementers of public domain Forth systems like figForth deliberately crippled their systems by deleting this important feature of Forth. However, over the years the round-robin task scheduling technique became public knowledge, and F83 included enough of the multi-task facility to implement a printer spooler. However, the multi-tasker in F83 is still incomplete and cannot support more than one foreground terminal task. F-PC has restored the full multi-tasker so that it is possible to run more than one terminal task.

The secret of multi-tasking in Forth is hidden in the user area, which is a data structure allocated in memory for every task declared in the system. The user area preserves the entire environment of a task so that the task can give up the CPU to another task, and continue its execution when the CPU is passed back to it. The environment includes a data stack, a return stack, a set of user variables, and some memory space for dictionary extension. Since each task has its own user area with all the system information to run the task, very few CPU registers need to be saved during task

switching. Therefore a Forth multi-tasker runs very fast. Typical task switching time is about 20 microseconds, which is incomprehensible in other languages and operating systems. IBM discovered that it could do multitasking only on AT's, while Forth, Inc. had it more than ten years ago on the 8080 class of microcomputers.

The user variables defined in F-PC are listed as follows, with some user deferred words. They are listed in the order of their sequence in the user area. The user variables are initialized during F-PC boot-up. See Section 5.5.

TOS Top of the data stack saved before passing CPU control to the next task. When a task PAUSEs, it pushes the return stack pointer RP, the instruction pointer IP, and the list segment ES on the data stack. It then saves the data stack pointer in TOS and jumps to the next task whose ENTRY address is in LINK.

ENTRY Contains the machine code to grab the CPU or pass it to the next task. If the task is awake, it contains E9CD which causes a software interrupt INT E9 to grab hold of the CPU. If the task is asleep, it contains E990 which is a jump instruction. The address to be jumped to is in the next cell of LINK, which points to the ENTRY cell in the user area of the next task.

F83 used 80CD or INT 80 as the wake-up code in ENTRY. This is a very obscure bug discovered by Mike Yantil. It crashes the system if an interrupt occurred when the byte 80 or E9 is written to ENTRY. The probability of this thing happening is very small if you are not doing a lot of interrupts, but when it happens the system will crash. Replacing INT 80 by INT E9 solves the problem cleanly. (This is the only serious bug in F83 discovered since 1983.)

LINK Point to the ENTRY field of next task in the round-robin chain of tasks.

ES0 The base address of the List Segment.

SP0 Base of the data stack for this task. It is used to initialize the data stack of this task.

RP0 Base of the return stack for this task. Used to initialize the return stack of this task.

DP Next available location of dictionary for this task.

OFFSET This is a user variable left over from F83. It is used to specify the block offset to be added to blocks accessed by this task. Not used in F-PC.

BASE The current numeric base for number input/output conversions. Each task can do its own number conversion.

HLD Point to the last converted character in the number string during conversion of a number to an ASCII string.

PRINTING Indicate whether the printer is enabled.

To support foreground terminal tasks, each task needs to handle its own terminal through the terminal I/O words like KEY and EMIT. Each task has a unique copy of these words to address the terminal assigned to it. In F83, only EMIT was defined as a user deferred word. Consequently, it can only support a printer task. In F-PC all the basic keyboard

I/O words are defined as user deferred words. Hence it is able to support several terminals for a multi-terminal, multi-user system. However, a multi-user system requires other supporting utilities so that each terminal can allow its user to edit files, interpret commands, and compile new words. These utilities in F-PC are defined for one user only. There is a lot more work still to be done to make it truly a multi-user system.

A user deferred word is defined like a user variable and compiled into the user area. Its body contains an address pointing to a piece of executable code. This address can be changed at run-time so that the deferred word may have different execution behavior. Each task has a unique copy of the user deferred word so that the word can be referred to by the same name but different code is executed in different tasks.

EMIT (char --) Send a character to the output device assigned to a task.

KEY? (-- f) Usually deferred to (KEY?), to sense keyboard status. It returns a true flag if a key is depressed on the keyboard.

KEY (-- char) Usually deferred to (KEY) to get a character from the terminal keyboard.

TYPE (addr count --) Display the given string on the terminal. Initialized to (TYPE).

TYPEL (seg addr count --) Similar to TYPE except the address is a segment:address pair.

Send a string to the terminal from any segment in memory. Initialized to (TYPEL).

3.2. SYSTEM VARIABLES

A large collection of variables is defined in F-PC to support various system functions. These variables are compiled into the main dictionary and are shared among all the tasks.

PRIOR Point to the last vocabulary that was searched.

STATE A variable indicating whether the text interpreter is currently in the interpreting state or compiling state. It is -1 when compiling and 0 when interpreting.

WARNING Checked by WARN for warnings. If it is ON, error messages are displayed on the terminal. If it is OFF, error messages are suppressed.

DPL The decimal point location for input number conversion. It shows the number of digits coming after the decimal point.

R# The cursor position during editing.

LAST Point to the name field of the most recently created word.

CSP Used for compile-time error checking. It stores the depth of data stack when compilation starts. The stack depth should not change when the compilation is done. Otherwise there is likely an error in the control structure.

CURRENT Point to a vocabulary word. New definitions are to be linked to this vocabulary.

#VOCS The number of vocabularies in the vocabulary stack. It specifies the maximum number of vocabularies which can be searched before FIND gives up the searching. It is set to 12.

CONTEXT The array specifying the vocabulary search order. It is also called the vocabulary stack. The first cell contains the vocabulary which is searched first. The last vocabulary searched is ROOT.

'TIB Point to the terminal input buffer which stores characters entered by user.

WIDTH Number of characters to keep in a name field. It is initialized to 31, allowing up to 31 characters in a name.

VOC-LINK Point to the vocabulary link field of the most recently defined vocabulary. All vocabularies are linked together to facilitate truncation by FORGET.

>IN Character offset into the input buffer. It shows the number of characters interpreted so far.

SPAN The actual number of characters received by EXPECT.

#TIB Maximum number of characters to be interpreted.

END? True if the input stream is exhausted, else false.

#OUT Number of output characters sent since last CR.

#LINE Number of CR's sent since last page or form-feed.

XDP The dictionary address offset in the current list segment. Next available location for new list entries.

XDPSEG The dictionary segment pointer pointing to the current list segment.

YDP The header dictionary pointer pointing to the next available location in the Head Segment for a new head.

YSTART Offset to the starting location of headers.

DPSTART Offset to the starting location of lists.

XSEGLEN Maximum number of pages allocated for List Space.

XMOVED Flag to tell if the List Segment has been relocated.

SSEG Search and scan segment used by SED editor.

SEQHANDLE Pointer to the handle of the current sequential file.

LOADLINE Line number of the line in the current file we are presently loading or interpreting.

CAPS If true, then convert names to upper case.

Several ASCII codes are used very often in programming the text interpreter. They are defined as constants to clarify the source code.

```
32 CONSTANT BL  ASCII code for BLANK character.

 8 CONSTANT BS  ASCII code for BACK-SPACE.

 7 CONSTANT BELL    ASCII code for BELL character.
```

3.3. STRING WORDS

This set of memory words deals with arrays and strings. Many of them address memory locations outside of the Code Segment which is where normal Forth memory words operate. A few words which deal with List Segment memory are also included in this section.

Strings are sequences of bytes. In Forth, there are three different ways to represent strings. The first is to use an address and a byte count; the address points to the first byte of the string. The second representation is called a 'counted string': the address of a counted string points to an address one byte ahead of the string. The extra byte is the count byte. Thus a counted string may have 0 to 255 characters in it. The third representation is called a 'long string.' Its address points two bytes ahead of the actual string. The two extra bytes is combined to a 16 bit length count, thus allowing a long string with up to 65355 characters. The long string is very useful in manipulating large text files. COUNT converts a counted string to the address/count representation. LENGTH converts a long string to the address/count representation.

Forth has to manipulate lots of texts, not the least of which are parsing of words from user input and comparing words with names in its dictionary. Most text and string words are coded in assembly to increase the compilation speed of F-PC.

FILL Fill the string starting at start-addr for count bytes with the character char. Both BLANK and ERASE are special cases of FILL.

```
CODE FILL      (  start-addr count char -- )
   CLD          \ Set forward direction flag
   MOV BX, DS        \ Save DS
   POP AX        \ Character to fill
   POP CX        \ Count
```

```
   POP DI        \ Starting address
   PUSH ES          \ Save ES
   MOV ES, BX        \ ES:DI address the string
   REPNZ         \ Repeat next instruction
   STOSB           \   and fill the memory
   POP ES        \ Restore ES
   NEXT            \ Done
   END-CODE
```

LFILL Fill a string with a character. It is like FILL, but the string is located in a segment seg other than the Code Segment.

```
CODE LFILL     (  seg start-addr count char -- )
   CLD             \ Set direction
   POP AX        \ Character
   POP CX        \ Count
   POP DI        \ Destination offset address
   POP BX        \ Destination segment
   PUSH ES       \ Save ES
   MOV ES, BX        \ Move in dest segment
   REPNZ         \ Repeat
   STOSB           \   filling the character in AL
   POP ES        \ Restore ES
   NEXT
   END-CODE
```

ERASE Fill a string with zeros.

```
: ERASE ( addr len -- )
   0 FILL  ;
```

BLANK Fill a string with blanks.

```
: BLANK ( addr len -- )
   BL FILL  ;
```

COUNT Given the address addr of a counted string on the stack, return the address of the string and its character count. Useful before words such as TYPE and CMOVE, which need a starting address of the text and a count as inputs.

```
CODE COUNT  ( addr -- addr+1 len )
   POP BX        \ Get address
   SUB AX, AX        \ Clear AH
   MOV AL, 0 [BX]     \ Get the count byte
   INC BX        \ Address+1
   PUSH BX       \ Push address
   1PUSH          \ Push count
   END-CODE
```

LENGTH Given the address of a long string on the stack, return the address of the string and its length. A long string is a string preceded by two bytes of length and may have up to 65355 characters.

```
CODE LENGTH ( addr -- addr+2 len )
   POP BX        \ Get address
   MOV AX, 0 [BX]     \ Get the length
   ADD BX, # 2       \ Real address of data
   PUSH BX       \ Push it
   1PUSH         \   with length
   END-CODE
```

COUNTL Like COUNT, but works with a long seg:offset address.

```
CODE COUNTL     ( seg addr -- seg addr+1 len )
   POP BX
   POP DS
   XOR AX, AX
   MOV AL, 0 [BX]
   INC BX
   PUSH DS
   PUSH BX
   MOV DX, CS
   MOV DS, DX
   1PUSH
   END-CODE
```

MOVE Move the specified bytes from source to destination without overlapping.

```
: MOVE  ( source-addr dest-addr len -- )
   -ROT              \ Hide length
   2DUP U<           \ If source < dest
   IF   ROT CMOVE>   \   move from the end
   ELSE   ROT CMOVE      \ Else from the beginning
   THEN ;
```

ATBL A table of ASCII values. It is used in UPC to translate lower case alphabets to upper case using the 8088/86 table look-up instruction XLAT. The rather complicated upper case conversion is thus greatly simplified and the speed improved. The cost is a 256 byte look-up table here. This is a good example of trading memory for speed.

```
CREATE ATBL           \ Uppercase translation table
 0  C,  1  C,  2  C,  3  C,  4  C,  5  C,  6  C,  7  C,
 8  C,  32  C,  10  C,  11  C,  12  C,  13  C,  14  C,  15  C,
```

```
16 C, 17 C, 18 C, 19 C, 20 C, 21 C, 22 C, 23 C,
24 C, 25 C, 26 C, 27 C, 28 C, 29 C, 30 C, 31 C,
32 C, '!' C, '"' C, '#' C, '$' C, '%' C, '&' C, ''' C,
'(' C, ')' C, '*' C, '+' C, ',' C, '-' C, '.' C, '/' C,
'0' C, '1' C, '2' C, '3' C, '4' C, '5' C, '6' C, '7' C,
'8' C, '9' C, ':' C, ';' C, '<' C, '=' C, '>' C, '?' C,
'@' C, 'A' C, 'B' C, 'C' C, 'D' C, 'E' C, 'F' C, 'G' C,
'H' C, 'I' C, 'J' C, 'K' C, 'L' C, 'M' C, 'N' C, 'O' C,
'P' C, 'Q' C, 'R' C, 'S' C, 'T' C, 'U' C, 'V' C, 'W' C,
'X' C, 'Y' C, 'Z' C, '[' C, '\' C, ']' C, '^' C, '_' C,
'`' C, 'A' C, 'B' C, 'C' C, 'D' C, 'E' C, 'F' C, 'G' C,
'H' C, 'I' C, 'J' C, 'K' C, 'L' C, 'M' C, 'N' C, 'O' C,
'P' C, 'Q' C, 'R' C, 'S' C, 'T' C, 'U' C, 'V' C, 'W' C,
'X' C, 'Y' C, 'Z' C, '{' C, '|' C, '}' C, '~' C, 127 C,

\ Characters above 127 are translated to those below 127
 0 C,  1 C,  2 C,  3 C,  4 C,  5 C,  6 C,  7 C,
 8 C,  9 C, 10 C, 11 C, 12 C, 13 C, 14 C, 15 C,
16 C, 17 C, 18 C, 19 C, 20 C, 21 C, 22 C, 23 C,
24 C, 25 C, 26 C, 27 C, 28 C, 29 C, 30 C, 31 C,
32 C, '!' C, '"' C, '#' C, '$' C, '%' C, '&' C, ''' C,
'(' C, ')' C, '*' C, '+' C, ',' C, '-' C, '.' C, '/' C,
'0' C, '1' C, '2' C, '3' C, '4' C, '5' C, '6' C, '7' C,
'8' C, '9' C, ':' C, ';' C, '<' C, '=' C, '>' C, '?' C,
'@' C, 'A' C, 'B' C, 'C' C, 'D' C, 'E' C, 'F' C, 'G' C,
'H' C, 'I' C, 'J' C, 'K' C, 'L' C, 'M' C, 'N' C, 'O' C,
'P' C, 'Q' C, 'R' C, 'S' C, 'T' C, 'U' C, 'V' C, 'W' C,
'X' C, 'Y' C, 'Z' C, '[' C, '\' C, ']' C, '^' C, '_' C,
'`' C, 'A' C, 'B' C, 'C' C, 'D' C, 'E' C, 'F' C, 'G' C,
```

```
'H' C,  'I' C,  'J' C,  'K' C,  'L' C,  'M' C,  'N' C,  'O' C,
'P' C,  'Q' C,  'R' C,  'S' C,  'T' C,  'U' C,  'V' C,  'W' C,
'X' C,  'Y' C,  'Z' C,  '{' C,  '|' C,  '}' C,  '~' C,  127 C,
```

UPC Convert a character to upper case by looking up the code in the table ATBL. Compare this code with that in F83 which subtracts 32 if the character is between a and z.

```
CODE UPC    ( char -- char' )
   POP AX         \ Get original character
   MOV BX, # ATBL      \ Put table base address in BX
   XLAT             \ Viola!  The character is converted.
   1PUSH             \ Push translated character
   END-CODE
```

UPPER Take a string at the specified address and convert it to upper case. It converts the string in place, so be sure to make a copy of the original if you need to use it later.

```
CODE UPPER  ( addr len -- )
   POP CX                    \ Get length
   POP DI                    \   and starting address
   PUSH SI                   \ Save IP
   MOV DX, ES                \   and List Segment pointer
   MOV BX, DS
   MOV ES, BX                  \ Set ES to DS for source
   MOV SI, DI                 \ Set SI to DI
   MOV BX, # ATBL             \ Load BX with table base
   CLD                       \ Clear direction flag
   CX<>0 IF          \ Skip if CX=0
      HERE             \ Leave address for LOOPNZ
         LODSB  \ Get a character
```

```
      XLAT       \ Convert to upper case
      STOSB      \ Store it back
    LOOPNZ           \   until all chars are done
  THEN
  MOV ES, DX               \ Restore ES=List pointer
  POP SI                   \   and SI=IP
  NEXT
  END-CODE
```

?UPPERCASE If CAPS is ON, convert the string at addr to upper case. If CAPS is OFF, do nothing. Addr points to a counted string.

```
CODE ?UPPERCASE    ( addr -- addr )
  MOV CX, CAPS       \ Test CAPS variable
  CX<>0 IF           \ Leave if CAPS is not on
    POP DI
    PUSH DI    \ Get a copy of address a1
    SUB CX, CX
    MOV CL, 0 [DI]  \ Get the count byte
    INC DI        \ Addr and count in DI & CX
    PUSH SI                  \ Following code is lifted from UPPER.
    MOV DX, ES               \   and List Segment pointer
    MOV BX, DS
    MOV ES, BX                 \ Set ES to DS for source
    MOV SI, DI                 \ Set SI to DI
    MOV BX, # ATBL             \ Load BX with table base
    CLD                      \ Clear direction flag
    CX<>0 IF           \ Skip if CX=0
      HERE             \ Leave address for LOOPNZ
        LODSB  \ Get a character
```

```
        XLAT       \ Convert to upper case
        STOSB      \ Store it back
      LOOPNZ       \  until all chars are done
    THEN
    MOV ES, DX          \ Restore ES=List pointer
    POP SI              \  and SI=IP
  THEN
  NEXT
  END-CODE
```

HERE Return the address of the top of the code dictionary in the Code Segment. The address is best known as the location of the Word Buffer where parsed words are stored for dictionary search.

```
CODE HERE  ( -- addr )
  MOV BX, UP        \ Base address of user area
  PUSH DP [BX]      \ Get the dictionary pointer
  NEXT
  END-CODE
```

PAD Floating temporary storage area. It is generally used to store text strings temporarily. Its location moves as new words are compiled into the code dictionary. Otherwise known as the Text Buffer.

```
CODE PAD  ( -- addr )
  MOV BX, UP        \ User area
  MOV AX, DP [BX]   \ Top of dictionary
  ADD AX, # 80      \ Allow 80 bytes for Word Buffer
  1PUSH
  END-CODE
```

-TRAILING Return the address and length of a given string, ignoring trailing blanks.

```
CODE -TRAILING  ( addr len -- addr len1 )
   POP CX         \ original count
   POP DI         \ Get addr while put it back on the stack
   PUSH DI
   CX<>0 IF
      MOV AX, DS
      PUSH ES
      STD         \ Search backward
      MOV ES, AX
      ADD DI, CX
      DEC DI      \ Point to th e  last character
      MOV AL, # $20
      REPE SCASB      \ Repeat till a non-blank character
      0<> IF      INC CX  \ Correct the new count
      THEN
      CLD         \ restore forward searching direction
      POP ES      \ Restore ES used for scanning
   THEN
   PUSH CX        \ New count pushed back.
   NEXT
   END-CODE
```

COMP This performs a string compare. If the two strings are equal, then COMP returns 0. If the two strings differ, then COMP returns -1 or +1. -1 is returned if string1 is less than string2. +1 is returned if string1 is greater than string2. All comparisons are relative to the ASCII order.

```
CODE COMP   ( addr1 addr2 len -- -1 | 0 | 1 )
   MOV DX, SI             \ Save IP
   POP CX            \ Length
```

```
  POP DI          \ Destination
  POP SI          \ Source
  CX<>0 IF        \ Length is not zero?
    PUSH ES       \ Save ES
    MOV ES, SSEG      \ Source string in SSEG segment
    REPZ
    CMPSB         \ Compare two strings
    0<> IF        \ If strings are not equal
      0< IF
        MOV CX, # -1    \ Return 1 if string1>string2
      ELSE
        MOV CX, # 1     \ Return -1 if string1<string2
      THEN
    THEN          \ CX=0 if string1=string2
  THEN
  MOV SI, DX          \ Restore IP
  POP ES          \ Restore ES
  PUSH CX         \ Push results
  NEXT
  END-CODE
```

CAPS-COMP This code handles the string comparison where case is not significant. Each character is converted to upper case before the comparison is made. Thus, lower case a and upper case A are considered identical.

```
CODE CAPS-COMP  ( addr1 addr2 len -- -1 | 0 | 1 )
  MOV DX, SI        \ Save IP
  POP CX        \ Length
  POP DI        \ Destination
  POP SI          \ Source
  PUSH ES       \ Save ES
```

```
   MOV ES, SSEG       \ Source segment SSEG
   BEGIN
      JCXZ  0 $       \ Exit if the string is exhausted.
      MOV     AH, 0 [SI] \ Get a character from string1
      INC SI
      MOV ES: AL, 0 [DI]  \ Get a character from string2.
      INC DI
      OR AX, # $02020 \ Make both characters lower case before comparing.
      CMP AH, AL
      JNE 1 $     \ Exit if a character is not equal
      DEC CX
   AGAIN
1 $: 0< IF            \ Unequal exit
      MOV CX, # -1       \ string1 < string2
   ELSE
      MOV CX, # 1     \ string1 > string2
   THEN
0 $:    MOV SI, DX         \ Equal exit
   POP ES
   PUSH CX
   NEXT
   END-CODE
```

COMPARE Performs a string compare. If CAPS is true, characters from both strings are converted to upper case before comparing.

```
: COMPARE   ( addr1 addr2 len -- -1 | 0 | 1 )
   CAPS @
   IF   CAPS-COMP
   ELSE   COMP
```

```
  THEN  ;
```

3.4. DOS WORDS

OSF A variable to indicate that an operating system call is in progress. This flag is provided so that interrupt routines can test it to avoid clashing against DOS services.

FCDOS A subroutine called by other assembler code to do INT 21H (DOS service call). BP and SI are the critical FORTH registers which must be saved and restored. OSF is set during the DOS call, and it can be tested by interrupt routines which call non-reentrant DOS functions.

```
LABEL FCDOS
  PUSH SI       \ IP must be saved
  PUSH BP       \ BP is the return stack pointer
  INC CS: OSF WORD      \ Set OSF flag
  INT 33          \ DOS system service interrupt
  DEC CS: OSF WORD      \ Reset OSF flag
  POP BP        \ Restore BP and SI
  POP SI
  RET
  END-CODE
```

XFDOS Call INT 21H with almost all possible register combinations. This is the most general form of DOS service call in F-PC.

```
CODE XFDOS  ( DX CX BX AX ES DS -- CX BX AX CY)
  POP DI        \ Get all the input data
  POP DS
  POP AX
```

```
   POP BX
   POP CX
   POP DX
   PUSH ES        \ Save ES
   PUSH DS          \ Make ES=DS
   POP ES
   PUSH CS        \ Save a copy of CS for DS later
   MOV DS, DI         \ Real DS popped from the stack
   CALL FCDOS         \ Do the DOS 33 call
   POP DS         \ DS=CS
   POP ES         \ Restore ES
   MOV DX, # -1          \ Initialize DX to false
   U>= IF           \ Test the flag left by DOS call
      XOR DX, DX      \ If no error, clear the error flag
   THEN
   PUSH CX        \ Push the output data on stack
   PUSH BX
   PUSH AX
   PUSH DX
   NEXT             \   and exit
   END-CODE
```

BDOS Load DOS function into AH and a parameter into DX, and then do a DOS service call. Return the result placed in the AL register on the stack. The BDOS calls were inherited from CP/M. It is defined as a deferred word and vectored to <BDOS> to make it more flexible.

```
CODE <BDOS> ( n fun -- m )
   POP AX         \ Function spec
   MOV AH, AL
   POP DX         \ Parameter
```

```
  INT 33
  SUB AH, AH        \ Always return a no error flag
  1PUSH
  END-CODE

DEFER BDOS     ' <BDOS> IS BDOS
```

BDOS2 A second version of BDOS call. This one pops the top three item on the stack to AX, BX, and CX registers. After executing the DOS service call, results in the same three registers are pushed back on the stack.

```
CODE BDOS2  ( CX DX AX -- CX DX AX )
  POP AX        \ Pop inputs
  POP DX
  POP CX
  MOV AH, AL        \ Adjust the function number
  INT 33            \ Call service
  PUSH CX       \ Push results
  PUSH DX
  PUSH AX
  NEXT
  END-CODE
```

OS2 Similar to BDOS2, but the last item from AX register is masked to an 8 bit number. Supposedly, it will make F-PC compatible with OS/2 system.

```
: OS2   BDOS2 255 AND ;
```

The KEY and KEY? words in F-PC were designed to do much more than simply looking at the keyboard and returning a character or a flag. They call the DOS keyboard service and do some preliminary work on the character before giving it back to the user. A key filter is built-in so that the user can install special processes responding to different key strokes. A background task chain is also built in KEY. You can insert functions into this chain to be executed when a key is pressed.

Following are the words supporting the key filter and the key task chain.

BIOSCHAR A variable holding the character from BIOS as scanned by BIOSKEY?

BIOSKEYVAL A variable holding the key value returned by BIOSKEY

BIOSKEY? Call the DOS keyboard service and return a flag. If any key is pressed, the flag is true.

```
CODE BIOSKEY?  ( --- f )
  BEGIN
    MOV AH, # 1    \ Question the keyboard
    PUSH SI    \ Save important Forth registers
    PUSH BP
    INT 16       \ 16H
    POP BP
    POP SI
    MOV BIOSCHAR AX \ Put the character in BIOSCHAR
    0= IF        \ If no key is pressed
      MOV AX, # 0 \  return with a false flag
      1PUSH
    THEN
    CMP AX, # 0    \ Is is 'Control Break'?
  0= WHILE         \ Exit if not.
    MOV AH, # 0    \ Throw away Ctrl Break
    PUSH SI
    PUSH BP
    INT 16       \ Rearm the keyboard service
    POP BP
    POP SI
```

```
   REPEAT         \ Scan again
   MOV AX, # -1           \ A key was pressed
   1PUSH             \   return with a true flag
   END-CODE
```

BIOSKEY Wait on the keyboard until a key is pressed. Return the key code. Ignore Control Break.

```
CODE BIOSKEY    ( -- key-code )
   BEGIN
      MOV AH, # 0     \ Arm the keyboard and scan for key
      PUSH SI
      PUSH BP
      INT 16
      POP BP
      POP SI
      CMP AX, # 0     \ Ignore Control BREAK, 00 Hex.
   0<> UNTIL          \ Repeat until a valid key arrives
   MOV BIOSKEYVAL AX   \ Store it in BIOSKEYVAL
   1PUSH             \   and push it on the stack
   END-CODE
```

```
DEFER KEYFILTER           \ Pre-filter a key before passing  it on.
' NOOP IS KEYFILTER
```

BGSTUFF This is a deferred word which allows background tasks to be chained to KEY. The task chain can be extended by user.

```
DEFER BGSTUFF          \ Beginning of task chain
' NOOP IS BGSTUFF
```

(KEY?) Return true if the user pressed a key, otherwise return false.

```
: (KEY?)   ( -- f )
   BGSTUFF          \ First execute all the tasks in the task chain
   BIOSKEY? ;       \ Then look at the keyboard
```

(KEY) Pause until a key is depressed, and return ASCII code on the stack.

```
: (KEY) ( -- char )
   BEGIN
      PAUSE      \ Pause for multi-tasker switching
      KEY?       \ Wait for a key
   UNTIL
   BIOSKEY       \ Recall the key code
   DUP 127 AND 0=     \ Is it a scan code?
   IF FLIP DUP 3 =       \ ETX?
      IF  DROP 0  | Allow a NUL
      ELSE   127 AND     \ The ASCII code is in the the upper byte of scan code.
      128 OR     \ Set the MSB
   ELSE 127 AND          \ No.  Get the ASCII code directly
   THEN
   KEYFILTER ;       \ Filter and do any other work
```

OUTPAUSE A deferred word. Pause on terminal output while sending characters to the screen.

```
DEFER OUTPAUSE  ( ' PAUSE )     \ Vector to PAUSE in a multi-tasking system.
' NOOP IS OUTPAUSE         \ Vector to NOOP for the moment.
```

CONSOLE A deferred word to output a character to the terminal display. The same character is also sent to printer if PRINTING is ON. Deferred to (CONSOLE).

3.5. LONG MEMORY WORDS

F-PC uses only a few 64K segments of the memory in a PC, but it allows you to use the entire memory space available to a PC. To address memory locations outside of the Code Segment, two 16 bit words are needed for the address specification. The first word in the address pair specifies the segment and the second specifies the address offset in that segment. Memory words for addressing extended or 'long' memory space have the letter 'L' prefixed or appended to the corresponding name of a regular memory word. To facilitate accessing the memory locations in the List Segment and Head segment, the memory words are prefixed with X for List Segment or Y for Head Segment.

?CS: Push the CS register on the data stack. It is the base segment address of the Code Segment.

```
CODE ?CS:  ( -- CS )
   PUSH CS       \ Push CS register
   NEXT          \ That's all
   END-CODE
```

?ES: Push the ES register on the data stack stack. It is the current segment register pointing to the beginning of a colon word list.

```
CODE ?ES:  ( -- ES )
   PUSH ES
   NEXT
   END-CODE
```

@L Fetch a 16 bit value from extended memory.

```
CODE @L ( seg addr --- n )
   POP BX        \ Get address offset
   POP DS        \ Pop segment address into DS
```

```
   MOV AX, o [BX]     \ Fetch data at DS:BX
   MOV BX, CS         \ Restore DS
   MOV DS, BX
   1PUSH          \ Push results on stack
   END-CODE
```

C@L Similar to @L except returning only a byte value.

```
CODE C@L   ( seg addr --- byte )
   POP BX
   POP DS
   MOV AL, o [BX]     \ Fetch a byte here
   XOR AH, AH
   MOV BX, CS
   MOV DS, BX
   1PUSH
   END-CODE
```

C!L Store a byte into the extended memory space.

```
CODE C!L   ( byte seg addr -- )
   POP BX         \ Pop offset
   POP DS         \ Pop segment
   POP AX         \ Pop byte value
   MOV o [BX], AL     \ Store to DS:BX
   MOV BX, CS         \ Restore DS
   MOV DS, BX
   NEXT
   END-CODE
```

!L Store a 16 bit value into a location specified by the seg:addr address pair.

```
CODE !L ( n seg addr -- )
   POP BX         \ Pop all parameters
   POP DS
   POP AX
   MOV 0 [BX], AX         \ Store 16 bit value
   MOV BX, CS         \ Restore DS
   MOV DS, BX
   NEXT
   END-CODE
```

CMOVEL Move a block of bytes from one extended memory location to another.

```
CODE CMOVEL ( sseg saddr dseg daddr count -- )
   CLD           \ Move forward
   MOV BX, SI         \ Save IP
   POP CX         \ Pop count
   POP DI         \ Pop destination offset
   POP AX         \ Pop destination segment
   POP SI           \ Pop source offset
   POP DS           \ Pop source segment
   PUSH ES        \ Save ES
   MOV ES, AX         \ Copy daddr to ES
   OR CX, CX          \ If count is zero, exit.
   0<> IF
      REPNZ       \ Otherwise, block move.
      MOVSB
```

```
  THEN
  POP ES        \ Restore ES to saved value
  MOV AX, CS        \ Restore DS
  MOV DS, AX
  MOV SI, BX        \ Restore IP
  NEXT            \ Done
  END-CODE
```

CMOVEL> Similar to CMOVEL except that the last byte is moved first to avoid overwriting moved data.

```
CODE CMOVEL>   ( sseg saddr dseg daddr count -- )
  STD           \ Reverse move direction
  MOV BX, SI        \ Save IP
  POP CX        \ Pop all data
  POP DI
  POP AX
  POP SI
  POP DS
  PUSH ES
  MOV ES, AX        \ Destination segment
  OR CX, CX         \ Skip if count=0
  0<> IF
    DEC CX
    ADD DI, CX     \ Offset to last destination byte
    ADD SI, CX     \ Offset to last source byte
    INC CX     \ Restore count
    REPNZ      \ Move from end to head
    MOVSB
  THEN
```

```
  POP ES        \ Restore registers
  MOV AX, CS
  MOV DS, AX
  MOV SI, BX
  CLD           \ Restore direction flag
  NEXT
  END-CODE
```

3.6. MEMORY ALLOCATION WORDS

F-PC allocates and uses its memory according to the rules laid down by MS-DOS so that it can take advantages of all the services that MS-DOS provides. This set of words is used by the system to arrange the various segments needed for F-PC to operate. The user can also use this word set to request additional memory for his own application. Memory not needed can be de-allocated and given back to DOS.

```
HEX
1000 CONSTANT #CODESEGS     \ Number of segments needed for Code Segments.
1000 CONSTANT #LISTSEGS     \ Number of segments needed for definitions.
1000 CONSTANT #HEADSEGS     \ Number of segments needed for Heads. 64K bytes each.
```

MEMCHK Abort if there was error in memory allocation.

```
: MEMCHK   ( f -- )
  IF                \ If memory error
    ." Insufficient Memory"
    0 0 BDOS        \ Cold start
  THEN ;
```

DEALLOC De-allocate a block of memory previously allocated by DOS. The block number n must be that returned by ALLOC. The flag returned is 0 if de-allocation is successful.

```
CODE DEALLOC  ( n -- f )
  MOV AH, # 49        \ De-allocation service number
  POP DX       \ Block number
  PUSH ES      \ We need ES to pass block number
  MOV ES, DX
  INT 21       \ Call DOS service
  u< if
    sub ah, ah     \ Not successful.  Error code.
  else
    mov ax, # 0    \ Success.  Clear error code.
  then
  POP ES       \ Restore ES
  1PUSH        \ Push flag.
  END-CODE
```

ALLOC Request n 16- byte pages of memory from DOS. If memory is successfully allocated, f will be 0, n2 is the pages allocated, and n1 is the base segment address of the allocated block of memory. If f is 8, memory is not sufficient. N2 is the pages actually available and n1 is garbage.

```
CODE ALLOC  ( n -- n1 n2 f )
  MOV AH, # 48      \ Allocation function number
  POP BX       \ Request segments
  INT 21       \ Please?
  PUSH BX      \ Segment base address
  PUSH AX      \ Allocated segments
  u< if
    sub ah, ah     \ Not successful.  Error code
  else
```

```
    mov ax, # 0     \ Successful.
  then
  1PUSH             \ Push error flag
  END-CODE
```

SETBLOCK Re-adjust the size of a memory block. Memory block starts a 'seg' with a new size 'size' in 16 byte pages. f is the error flag. If DOS failed to allocate, the error code will be 7, 8 or 9.

```
CODE SETBLOCK   ( seg size -- f1 )
  POP BX                    \ get new size
  MOV AH, # $4A             \ setblock call
  POP DX
  PUSH ES
  MOV ES, DX
  INT $21
  U< IF   SUB AH, AH      \ Error occurred, keep error code in AL.
  ELSE    MOV AX, # 0     \ No error, clear AX.
  THEN
  POP ES
  1PUSH
  END-CODE
```

DOSVER Get DOS version number.

```
: DOSVER    ( -- n )
  0 30 BDOS         \ Ask DOS for its
  0FF AND ;         \   version number
```

```
DEFER CURSORSET         \ Initialize the cursor characteristics.
```

+XSEG Add XSEG to segment offset on the data stack to get an absolute segment address.

```
CODE +XSEG     ( n1 -- n2 )
   POP AX
   ADD AX, XSEG              \ Add XSEG to n1, returning n2.
   1PUSH
   END-CODE
```

SETYSEG In the .EXE file, the head dictionary is saved after the code dictionary. The head dictionary has to be relocated to the proper segment.

```
: SETYSEG   ( --- )           \ Set Head Segment + more space
   [ LABEL 'SETYSEG ]     \ Define a label here for bootstrape process.
   ?CS: SSEG !        \ Initialize Scan/Search segment
   XSEGLEN @ +XSEG XDPSEG !    \ List Segment
   XDP OFF        \ List Segment offset
   DPSTART @ DP !      \ Code Segment dictionary
   DOSVER 2 <          \ Check DOS version
   IF ." Must have DOS 2.x or higher."
      0 0 BDOS        \ Abort if DOS version too low
   THEN
   ?CS: #CODESEGS #LISTSEGS +
   #HEADSEGS + SETBLOCK    \ Need that much space
   MEMCHK         \ Can DOS afford it?
   #OUT 0!           \ Move cursor to left edge of page
   $18 ( 24 DECIMAL) #LINE !   \ Move to bottom of page
   CURSORSET ;        \ Initialize the cursor
```

3.7. HEAD AND LIST SEGMENT WORDS

Here are all the words that manipulate memory in the Head Segment. This segment is also called the Y Space in F-PC, and the prefix 'Y' is used to identify this set of words. The base segment address of the Head Segment is stored in variable YSEG. All addresses are offset from this segment address. The dictionary in F-PC is broken into three parts: the code dictionary in the Code Segment, the head dictionary in the Head Segment, and the colon list dictionary in the List Segment. The regular Forth memory words operate on memory in the Code Segment. Similar memory words are defined for both the Head Segment and the List Segment. These segment-implicit words are convenient to use in building the head and the colon list dictionaries.

YHERE Return the address of the next available location above the head dictionary.

```
CODE YHERE  ( -- addr )
   PUSH YDP          \ Head dictionary pointer
   NEXT
   END-CODE
```

YS: Tuck the base segment address of Head Segment under the address on top of the stack.

```
CODE YS:   ( n -- YSEG n )
   POP AX        \ Retain n
   PUSH YSEG         \ Tuck YSEG
   1PUSH            \ Push n on top
   END-CODE
```

Y@ Fetch an integer in the Head Segment.

```
CODE Y@ ( addr -- n )
   POP BX        \ Pop addr into BX for relative addressing
   MOV DS, YSEG       \ DS must point to YSEG
   PUSH 0 [BX]        \ Fetch and push
   MOV BX, CS         \ Restore DS
```

```
   MOV DS, BX
   NEXT
   END-CODE
```

Y! Store an integer in the Head Segment.

```
CODE Y! ( n addr -- )
   POP BX          \ Pop address
   MOV DS, YSEG        \ Use right segment in DS
   POP o [BX]          \ Pop n and store it
   MOV BX, CS          \ Restore DS
   MOV DS, BX
   NEXT
   END-CODE
```

YC@ Fetch a byte in the Head Segment.

```
CODE YC@    ( addr -- char )
   POP BX
   SUB AX, AX          \ Clear the throat
   MOV DS, YSEG
   MOV AL, o [BX]      \ Get the byte
   MOV BX, CS
   MOV DS, BX
   1PUSH           \ Push on stack
   END-CODE
```

YC! Store a byte in the Head Segment.

```
CODE YC!    ( char addr -- )
   POP BX
   POP AX
   MOV DS, YSEG
   MOV 0 [BX], AL      \ Store one byte
   MOV BX, CS
   MOV DS, BX
   NEXT
   END-CODE
```

Y, Compile an integer on the top of the head dictionary.

```
CODE Y, ( n -- )
   MOV BX, YDP         \ Top of head dictionary
   ADD YDP # 2 WORD        \ Increment head pointer
   POP CX          \ Integer to be compiled
   MOV DS, YSEG        \ DS must point to YSEG
   MOV 0 [BX], CX      \ Write n to dictionary
   MOV BX, CS          \ Restore DS
   MOV DS, BX
   NEXT
   END-CODE
```

YCSET Set bits in a memory location in the Head Segment according to the bits set in byte.

```
CODE YCSET  ( byte addr -- )
   POP BX          \ Get address
   POP AX          \ Get bit pattern
   MOV DS, YSEG        \ DS points to YSEG
```

```
   OR 0 [BX], AL          \ Set bits
   MOV BX, CS         \ Restore DS
   MOV DS, BX
   NEXT
   END-CODE
```

YHASH Given a string and a vocabulary, return the address which holds the last link field address in a thread. The hashing algorithm is: 2*(char1*2 + char2) + length . This special hashing method distributes word definitions evenly among all the threads so that the least amount of time will be spent on searching for a word.

```
CODE YHASH  ( ystr vocaddr -- thread )
   POP DX         \ Address of vocabulary code
   POP BX         \ Address of string
   MOV DS, YSEG       \ Point DS to Head Segment
   MOV AX, 1 [BX]     \ Get first and second chars
   SHL AL, # 1        \ 2*char1
   MOV CL, 0 [BX]     \ Get count
   AND CX, # 31           \ Retain only word length
   DEC CX         \ Length=1?
   CX<>0  IF              \ If length>1,
     ADD AL, AH      \   add char2.
   ELSE    MOV AH, # 32       \ Else, add 32.
     ADD AL, AH
   THEN SHL AX, # 1       \ Multiply sum by 2
   ADD AL, 0 [BX]     \ Plus count byte
   AND AX, # #THREADS 1-   \ Limit it by # of threads
   SHL AX, # 1        \ Address offset
     ADD AX, DX         \   into thread table
   MOV CX, CS         \ Restore DS
   MOV DS, CX
```

```
   1PUSH            \ Push address of thread
   END-CODE
```

The List Segment is called X space in F-PC. Memory words directed into this segment are prefixed with 'X'. This words set is similar to that for the Head Segment.

The starting (base) segment address of the List Segment is stored in the variable XSEG. The word list of a colon definition is addressed by its segment offset from this base segment address. The code field address list in this definition is addressed by an address offset. Thus we need both a segment address and an address offset to locate memory in the List Segment. This method allows F-PC to use more than 64K bytes for the List Space.

XHERE Return the current segment pointer and the address offset pointing to the top of list dictionary.

```
CODE XHERE  ( -- seg addr )
   PUSH XDPSEG     \ Current segment pointer
   PUSH XDP          \ Current offset address in the current segment
   NEXT
   END-CODE
```

X, Compile an integer to the top of list dictionary.

```
CODE X, ( n -- )             \ XHERE !L  2 XDP +!
   POP AX         \ n
   MOV BX, XDP        \ Address offset
   MOV DS, XDPSEG      \ Current segment
   MOV 0 [BX], AX      \ Store value
   MOV BX, CS         \ Restore DS
   MOV DS, BX
   ADD XDP # 2 WORD       \ Increment address offset
   NEXT
   END-CODE
```

XC, Compile a byte to end of the list dictionary.

```
CODE XC,  ( n -- )          \ XHERE C!L 1 XDP +!
  POP AX
  MOV BX, XDP
  MOV DS, XDPSEG
  MOV 0 [BX], AL    \ store byte
  MOV BX, CS
  MOV DS, BX
  INC XDP WORD
  NEXT
  END-CODE
```

3.8. TERMINAL OUTPUT WORDS

This set of words is the interface between a user and the F-PC system. The words to be supported at the top level are EXPECT and QUERY, which ask the user to type in a line of command, which will be passed to the text interpreter for parsing and execution. As the user is typing, each character typed must be echoed to the screen and to the printer if the printer is enabled. Some characters must be acted upon immediately, like DEL or back-space. The user terminates the command line by an <enter>, which is also a character needing special attention. This word set shows how F-PC handles characters passed from the CPU to the screen and/or printer.

PR-STATUS Call DOS and get printer status n2 for printer n1..

```
CODE PR-STATUS  ( n1 -- n2 )
  POP DX              \ Printer number n1
  MOV AH, # 2       \ Printer function 2
  PUSH SI       \ Save IP
```

```
   PUSH BP        \ Save RP
   INT 23         \ Call printer service 17H
   POP BP         \ Restore IP and RP
   POP SI
   MOV AL, AH         \ Return printer status n2
   MOV AH, # 0
   1PUSH
   END-CODE
```

?PRINTER.READY Return a true flag if the printer is selected, not busy and no errors are reported. Otherwise return a false flag.

```
HEX
: <?PTR.READY>  ( --- f )
   0 PR-STATUS
   90 = ;          \ 90 is printer busy and not ready code

DEFER ?PRINTER.READY       ' <?PTR.READY> IS ?PRINTER.READY
DECIMAL
```

PEMIT   Send a character to the printer. It is usually vectored to (PRINT) which will be defined later.

```
DEFER PEMIT             \ ' (PRINT) IS PEMIT
```

(EMIT) Send a character to both the console and the printer if PRINTING is ON. Otherwise, send the character to the console only.

```
: (EMIT)    ( char -- )
   PRINTING @         \ If PRINTING is on
   IF PEMIT       \ Emit to printer
```

```
   ELSE    CONSOLE ;       \ Otherwise, emit to console
   THEN  ;
```

CRLF Send a carriage-return and line-feed sequence.

```
: CRLF  ( -- )
   13 EMIT 10 EMIT        \ CR and LF
   #OUT OFF           \ Clear character count
   #LINE DUP @ 1+      \ Increment output line number
   PRINTING @ 0=
   IF     ROWS MIN       \ Not printing, limit line number
   THEN SWAP ! ;       \   to 24 to fit in CRT display
```

CR A deferred word typically set to CRLF above.

```
DEFER CR ' CRLF is CR
```

FEMIT Fast EMIT. Send the character to screen via TYPE. It does not go through DOS which tends to slow down everything. Control characters are shown in their graphic equivalence.

```
: FEMIT ( char -- )
   SP@              \ That's where char is
   1 TYPE         \ Type it directly
   DROP ;         \ Drop it after done
```

SPCS A table containing 80 SP (space characters) to be used by SPACES. This is the fastest way to print up to 132 spaces. This is another example of trading memory for speed.

```
CREATE SPCS ( -- addr )     \ An array of 80 spaces used by SPACES.
HEX 2020
```

```
   DUP , DUP , DUP , DUP , DUP , DUP , DUP , DUP ,
   DUP , DUP , DUP , DUP , DUP , DUP , DUP , DUP ,
   DUP , DUP , DUP , DUP , DUP , DUP , DUP , DUP ,
   DUP , DUP , DUP , DUP , DUP , DUP , DUP ,      ,
DECIMAL
```

SPACE Send a space to the terminal.

```
: SPACE ( -- )
   SPCS 1 TYPE ;          \ TYPE is the principal output routine.
```

SPACES Send n spaces to the terminal. n is clipped to 80.

```
: SPACES   ( n -- )
   0MAX   DUP 80 <
   IF  SPCS SWAP TYPE
   ELSE   80 /MOD 0      \ Send 80 spaces as a packet.
      ?DO    SPCS   80 TYPE
      LOOP   SPCS SWAP TYPE
   THEN   ;
```

BACKSPACES Send n Backspaces to the terminal.

```
: BACKSPACES   ( n -- )
   0 ?DO   BS EMIT        \ Send one backspace
      -2 #OUT +!     \ Adjust output character count
   LOOP  ;
```

BEEP Ring the bell on the terminal.

```
: %BEEP ( -- )
   BELL (EMIT)
   #OUT DECR ;        \ BELL cannot be printed

DEFER BEEP  ' %BEEP IS BEEP
```

3.9. EXPECT WORDS

EXPECT is the word by which F-PC gets a line of command from the user. As it is waiting for the user to complete entering the line, a limited amount of editing is also allowed. In addition, F-PC maintains a table for all the control characters, which can cause specific actions immediately when a control character is entered. Thus an application can use the control character table to design very powerful user interfaces like menu help system, hot keys, etc. The words in this section show how this control character table is constructed and the implementation of several commonly used control characters like carriage-return, back-space and delete.

BS-IN Backspace. Beep if at the beginning of a line; otherwise, back up 1. The second item on data stack, the character count in a line, must be decrmented by 1 until it becomes zero. The top item is usually a character which is not used but preserved.

```
: BS-IN    ( n char -- 0 char | n-1 char )
   >R  DUP        \ Preserve char
   IF        1- BS EMIT     \ n>0, backspace
      #OUT @ 2- 0MAX
      #OUT !        \ Adjust #OUT
   ELSE   BEEP          \ n=0, ring the bell
   THEN   R> ;       \ Restore char
```

DEL-IN Delete a character. Beep if at beginning of a line; otherwise, back up and erase the previous character. DEL-IN is a deferred word, usually vectored to (DEL-IN).

```
: (DEL-IN)  ( n char -- 0 | n-1 )
   >R DUP
   IF     1-  BS EMIT        \ Back up
      SPACE BS EMIT   \ Erase previous character
      #OUT @ 4 -
      0MAX    #OUT !     \ Save current #OUT
   ELSE    BEEP
   THEN    R> ;

DEFER DEL-IN    ' (DEL-IN) IS DEL-IN
```

BACK-UP Wipe out the current line by overwriting it with spaces. The current line has n characters.

```
: BACK-UP   ( n c -- 0 c )
   >R
   DUP BACKSPACES      \ Back up a line
   DUP SPACES          \ Erase the line
   BACKSPACES          \ Back up again
   0  R> ;
```

RESET-IN   Reset the system to a relatively clean state and start over again.

```
: RESET-IN  ( c -- )
   FORTH           \ Restore FORTH vocabulary
   TRUE ABORT" Reset"  ;   \ Abort

DEFER RES-IN    ' RESET-IN IS RES-IN
```

P-IN Toggle the printer on or off.

```
: P-IN   ( c -- c )
   PRINTING @ 0=      \ Complementing PRINTING
   PRINTING !  ;        \ Store back
```

ESC-IN Used to process Escape sequence of characters.

```
: (ESC-IN)  ( addr count c -- addr count+1 c )
   >R
   2DUP + @ EMIT      \ Print a character in string
   1+           \ Bump count
   R> ;
```

```
DEFER ESC-IN   ' (ESC-IN) IS ESC-IN
```

CR-IN Finish input and remember the number of characters in SPAN.

```
: CR-IN ( m a n c -- m a m C )
   >R SPAN !        \ Store n in SPAN
   OVER          \ Replace n by m
   BL EMIT R>  ;        \ Print a space
```

CHAR Process an ordinary character by appending it to the input buffer. A string of n bytes is already in the buffer at addr. It is deferred to (CHAR).

```
: (CHAR)   ( addr n char -- a n+1 char )
   dup>r        \ Copy char to return stack
   3DUP EMIT         \ Emit char to screen
   + C!           \ Store char to input buffer
```

```
   1+              \ Bump buffer count
   R> ;            \ Recall char

DEFER CHAR     ' (CHAR) IS CHAR
DEFER ^CHAR    ' CHAR   IS ^CHAR
```

KEYTABLE Similar to a case structure; it processes the 32 control characters received from keyboard. It is deferred to NORM-KEYTABLE. This case structure can be changed to respond to control characters selected by plugging in appropriate words in the table. Later features, especially the key commands in the editor, replace the KEYTABLE functions.

```
: NORM-KEYTABLE
   EXEC:
  ^CHAR   ^CHAR  ^CHAR  RES-IN ^CHAR  ^CHAR   ^CHAR  ^CHAR
   DEL-IN  ^CHAR  ^CHAR  ^CHAR  ^CHAR  CR-IN   ^CHAR  ^CHAR
   P-IN    ^CHAR  ^CHAR  ^CHAR  ^CHAR  BACK-UP ^CHAR  ^CHAR
   BACK-UP ^CHAR  ^CHAR  ESC-IN ^CHAR  ^CHAR   ^CHAR  ^CHAR ;

DEFER KEYTABLE  ' NORM-KEYTABLE IS KEYTABLE
```

EXPECT Get a string from the terminal and place it in the buffer provided. Perform a limited amount of line editing. Save the number of input characters in the variable SPAN. It uses NEXPECT which can accept a string into the middle of the input buffer.

```
: NEXPECT   ( addr len start -- )
   dup>r           \ Save the starting count
   IF OVER R@ TYPE THEN    \ If not zero, type prior string
   DUP SPAN !          \ Save len in SPAN, maximum characters expected
   SWAP R>       \ len addr start --
   BEGIN      2 PICK OVER -  \ len addr #so-far #left --
   WHILE      2>R >R KEY  \ Accept one character
```

```
        SWAP 2R> ROT    \  Silly way to ensure stack is clean in doing
              \ a task switch.
        DUP BL <    \ A control character?
      IF     DUP KEYTABLE    \ Yes. Process it.
        DROP
      ELSE    DUP 127 =   \ Is it a DEL?
        IF   DEL-IN \ Yes.  Delete previous character
        ELSE   CHAR \ Put it into the input buffer
        THEN  DROP
      THEN
   REPEAT  3DROP ;     \ Repeat for next character

: (EXPECT)  ( addr len -- ) 0   NEXPECT  ;

DEFER EXPECT   ' (EXPECT) IS EXPECT
```

TIB Leave the address of the text input buffer on the data stack.

```
CODE TIB    ( --- addr )
   PUSH 'TIB          \ Address is in a user variable
   NEXT
   END-CODE
```

MORE? Return a true flag if there are still characters un-interpreted in the input buffer.

```
CODE MORE?      ( -- flag )              \ Is words left in input stream?
   MOV AX, >IN
   SUB AX, #TIB
   SBB AX, AX
```

```
   1PUSH
   END-CODE
```

QUERY Get a line of input from the user and place it in the terminal input buffer at TIB.

```
: QUERY ( -- )
   TIB COLS EXPECT        \ Wait for a line from keyboard
   SPAN @ #TIB !      \ Store line length in #TIB also
   >IN OFF  ;         \ Clear interpreter character pointer

VARIABLE DISK-ERROR     \ Leftover from F83.  Not used in F-PC.
-2 CONSTANT LIMIT          \ Last addressable word in Code Space
LIMIT 10 - CONSTANT FIRST       \ Another leftover pointer from F83
FIRST 10 - CONSTANT INIT-R0 \ Bottom of return stack
```

3.10. NUMBER CONVERSION WORDS

All data are stored as binary integers in F-PC. Numbers input from the keyboard and output to console have to be represented in ASCII strings for the user's benefit. This set of words converts binary numbers to ASCII strings and also converts ASCII number strings to binary numbers. It also gives the user tools to control the formats of the input/output ASCII number strings.

The user variable BASE contains the current number base which is used in converting numbers between the internal binary form and the external ASCII form. It gives the Forth user great power in choosing the best way to present numbers in the most convenient form.

HEX All subsequent numeric conversions will be done in hexadecimal.

```
: HEX     ( -- )     16 BASE ! ;
```

DECIMAL All subsequent numeric conversions will be done in decimal.

```
: DECIMAL   ( -- )     10 BASE ! ;
```

OCTAL   All subsequent numeric conversions will be done in octal.

```
: OCTAL ( -- )     8 BASE ! ;
```

DEFER DEFAULT   Command tail processor which executes words given after F-PC in a DOS command line. The first word in the command tail is a file name. DEFAULT opens this file when F-PC is invoked.

DIGIT   Return a flag indicating whether or not a character is a valid digit in the given base. If so, return converted value and true; otherwise, return the original char and false.

```
CODE DIGIT  ( char base -- n f )
   POP DX         \ base
   POP AX         \ char
   PUSH AX        \ Save a copy of char
   SUB AL, # ASCII 0      \ Less than 0?
   JB 0 $             \ Yes.  Return false.
   CMP AL, # 9        \ Greater than 9?
   > IF
      CMP AL, # 17       \ Less than 'A'?
      JB 0 $         \ Fail again
      SUB AL, # 7     \ Adjust for 'A' and above
   THEN
   CMP AL, DL         \ Greater than base?
   JAE 0 $            \ Yes.  Fail.
```

```
   MOV DL, AL         \ Converted value
   POP AX         \ Discard char
   MOV AX, # TRUE      \ Get true flag
   2PUSH             \ Push them on stack
0 $:   SUB AX, AX         \ Common failure exit.
   1PUSH
   END-CODE
```

DOUBLE? Returns true if a period was encountered in the input number string.

```
: DOUBLE?   ( -- f )
   DPL @ 1+           \ DPL was initialized to -1
   0<> ;              \ True if anything else
```

CONVERT Starting with an unsigned double number ud1 and a counted string at addr1, convert the string to a number in the current base and add it to ud1. Leave results and the address of the unconvertable digit on stack.

```
: CONVERT   ( ud1 addr1 -- ud2 addr2 )
   BEGIN   1+  dup>r      \ Skip count
      C@  BASE @  DIGIT   \ Convert one digit
   WHILE   SWAP       \ Exit if not a digit
      BASE @ UM*  DROP    \ Multiply d1 with base
      ROT  BASE @ UM*  D+     \ Add digit*base
      DOUBLE?        \ Increment DPL if digit is
      IF  DPL INCR THEN  \   after decimal point
      R>         \ Restore incremented addr1
   REPEAT  DROP  R>  ;     \ d2 is the converted double integer.  Addr2 points to
                \   the non-digit.
```

(NUMBER?) Given a string containing at least one digit, convert it to a double integer. The string must be terminated with an ASCII space.

```
: (NUMBER?) ( addr -- d flag )
   0 0  ROT          \ Place 0.0 under addr
   DUP 1+ C@         \ Get first character in string
   ASCII -  =        \ Is it a '-' sign?
   DUP  >R  -        \ Save the sign on return stack and back up if not a - sign.
   DPL -1!       \ Initialize DPL to -1
   BEGIN   CONVERT       \ Convert to a double integer
     DUP C@        \ Is non-digit a decimal point
     ASCII , ASCII /    \   or something like it,
     BETWEEN    \   between , and / ?
   WHILE   DPL 0!     \ Yes.  Mark it in DPL.
   REPEAT          \ Convert the part after decimal point.
   -ROT  R>          \ Conversion done. Retrieve sign.
   IF  DNEGATE  THEN         \ Negate if sign was -.
   ROT C@ BL =  ;     \ Make sure that the string is delimited by Blank.
```

NUMBER? Convert a counted string at addr to a double integer. It takes into account a leading minus sign, and stores a pointer to the last delimiter in DPL. The string must end with a blank. Leave a true flag if successful.

```
: NUMBER?   ( addr -- d flag )
   FALSE         \ Push a false flag for testing
   OVER COUNT BOUNDS   \ Scan characters in the string
   ?DO    I C@          \ Get one character
     BASE @ DIGIT NIP    \ Is it a digit?
     IF DROP TRUE LEAVE \ Yes.  Leave the loop with a true
     THEN
   LOOP            \ addr f --
```

```
   IF  (NUMBER?)      \ If valid number, convert.
   ELSE  DROP  0 0 FALSE  \ Else leave 0.0 and false
   THEN  ;
```

NUMBER Convert a counted string at a1 into a double number d1. It will be nice if we can specify hex numbers without changing BASE. A simple method used in F-PC is to make the interpreter to accept numbers prefixed with $ as hex numbers, ASCII code with a ' (tick) prefix, and control code with a ^ prefix.

```
: %$NUM       ( a1 -- d1 f1 )          \ process as a hex number $A123
   dup>r DUP COUNT 1- 0MAX >R
   DUP 1+ SWAP R> CMOVE       \ Extract the $.
   DUP C@ 1- OVER C!          \ Shorten count by 1.
   BL OVER COUNT + C!         \ Append a blank to string.
   BASE @ >R                  \ Save the base for later restoration.
   HEX NUMBER?                \ Try to convert the number in HEX
   R> BASE !                  \ Restore the BASE.
   DUP 0=                     \ If its not a number, restore the $.
   IF     R@ COUNT >R DUP 1+ R> CMOVE>
   R@ C@ 1+ R@ C!
   ASCII $ R@ 1+ C!
   THEN    r>drop ;

: %'NUM       ( a1 -- d1 f1 )          \ process as an ascii char 'A'
   STATE @
   IF         DROP
      TIB >IN @ 3 - + C@ 0
   ELSE    2+ C@ 0
   THEN    TRUE DPL ON ;

: %^NUM       ( a1 -- d1 f1 )       \ process as a control char ^A
```

```
   2+ C@ $1F AND 0 TRUE DPL ON ;


DEFER $NUM     ' %$NUM   IS $NUM
DEFER 'NUM     ' %'NUM   IS 'NUM
DEFER ^NUM     ' %^NUM   IS ^NUM
DEFER #NUM     ' NUMBER? IS #NUM


CODE %NUMBER    ( a1 -- d1 f1 ) \ Call colon words inside a code word.
\ Convert count delimited string at a1 into double number.  Special prefixes allowed.
   MOV DI, SP
   MOV BX, 0 [DI]
   MOV AL, 1 [BX]
   CMP AL, # ASCII $          \ test for leading $
   0= IF   JMP ' $NUM         \ process as HEX
   THEN
   MOV AL, 1 [BX]
   MOV AH, 3 [BX]
   CMP AX, # ASCII '  dup flip +   \ test for lead & trail '
   0= IF   JMP ' 'NUM      \ process as ascii char
   THEN
   MOV AX, 0 [BX]
   CMP AX, # ASCII ^ flip $02 +    \ test for lead ^ & count = 2
   0= IF   JMP ' ^NUM      \ process as control char
   THEN
   JMP ' #NUM         \ else process as a number
   END-CODE


: (NUMBER)     ( a1 -- d1 )       \ Convert count delimited string at a1 into a double number.
   %NUMBER NOT ?MISSING ;
```

```
DEFER NUMBER    ' (NUMBER) IS NUMBER
```

When we convert a binary number to an ASCII number string, the number string is constructed starting at PAD. New digits converted are added to PAD buffer towards the low memory. The least significant digit is placed at PAD, the next digit at PAD minus 1, and so forth. The pointer pointing to the last converted digit is stored in the system variable HLD. When the conversion is completed, the number string between (HLD) and PAD can be sent to the output device.

HOLD Save char in the numeric output string under construction.

```
: HOLD     ( char -- )
   HLD DECR          \ Move number string pointer
   HLD @ C!  ;          \ Add char to number string
```

<# Start numeric conversion. Set up the output number string below PAD.

```
: <#       ( -- )
   PAD  HLD  ! ;        \ Init number string pointer
```

#> Terminate numeric conversion. Return addr and count of the number string constructed.

```
: #>       ( d -- addr len )
   2DROP         \ Discard d.  Conversion completed.
   HLD  @        \ First digit of number string
   PAD  OVER  -  ;    \ Character count
```

SIGN If n is negative, insert a minus sign into the string.

```
: SIGN     ( n -- )
   0<
```

```
  IF  ASCII - HOLD  THEN  ;   \ Add '-' to number string
```

Convert a single digit in the current base and add the digit to the number string under construction.

```
: #   ( -- )
  BASE @ MU/MOD       \ Convert a digit
  ROT 9 OVER <        \ Adjust for alphabetic character
  IF  7 + THEN
  ASCII 0  +  HOLD  ;     \ Add digit to number string
```

#S Convert a double integer d1 until it is finished, or reduced to 0.

```
: #S      ( d1 -- 0 0 )
  BEGIN
    #         \ Convert a digit
    2DUP  OR      \   until d1 is reduced to 0
  0=  UNTIL  ;
```

(U.) Convert an unsigned 16 bit integer to a string.

```
: (U.)     ( u -- addr len )
  0   <# #S #>   ;
```

U. Output u as an unsigned single integer with a trailing space.

```
: U.      ( u -- )
  (U.)  TYPE SPACE   ;
```

U.R Output u as an unsigned single integer right justified in a field of l columns.

```
: U.R      ( u l -- )
   >R   (U.)            \ Convert unsigned integer
   R> OVER - SPACES       \ Pad with enough spaces
   TYPE   ;          \   so that number is right justified
```

(.) Convert a signed 16 bit integer to a string.

```
: (.)      ( n -- addr len )
   DUP ABS 0
   <# #S   ROT SIGN   #>   ;
```

. Output n as a signed single integer with a trailing space. Free-format output.

```
: .    ( n -- )
   (.)   TYPE SPACE   ;
```

.R Output n as a signed single integer right justified in a field of l columns. Formatted output.

```
: .R       ( n l -- )
   >R   (.)
   R> OVER - SPACES
   TYPE   ;
```

(UD.) Convert an unsigned double integer ud to a string.

```
: (UD.)    ( ud -- addr len )
   <# #S #>   ;
```

UD. Output ud as an unsigned double integer with a trailing space.

```
: UD.      ( ud -- )
   (UD.)   TYPE SPACE   ;
```

UD.R Output ud as an unsigned double integer right justified in a field of l columns.

```
: UD.R     ( ud l -- )
   >R   (UD.)
   R> OVER - SPACES   TYPE  ;
```

(D.) Convert a signed double integer d to a string.

```
: (D.)     ( d -- addr len )
   TUCK DABS   <# #S   ROT SIGN  #>   ;
```

D. Output d as a signed double integer with a trailing space.

```
: D.       ( d -- )
   (D.)   TYPE SPACE   ;
```

D.R Output d as a signed double integer right justified in a field of l columns.

```
: D.R      ( d l -- )
   >R   (D.)
   R> OVER - SPACES
   TYPE   ;
```

3.11. THE PARSING WORDS

Parsers are very profound subjects in computer science. You must go to a graduate school in computer science to learn about them. They are complicated because they are tools to decompose source code in a high level programming language to some form that a computer can do things about. On the contrary, the parser in Forth is very simple because the language syntax in Forth is very simple. All the F-PC parser does is to extract a word, which is a string delimited by blank characters or some other characters you specified, and place it in the Word Buffer just above the code dictionary. The text interpreter will then search the dictionary for this word so that it will be executed or compiled. If the word is a number string, the text interpreter will convert it to a number and either push it on the data stack or compile it as a literal. This set of words supports the F-PC parser, which is a standard Forth word WORD.

In F-PC 3.5, WORD is defined completely in assembly. SKIP, SCAN, /STRING, and PARSE are not used to build WORD. They are preserved for F83 compatibility, and for some applications where specialized parsers have to be handcrafted.

SKIP Given the address and length of a string, and a character to look for, run through the string while we continue to find the character. Leave the address of the first mismatch and the length of the remaining string.

```
CODE SKIP   ( addr1 len1 char -- addr2 len2 )
   POP AX        \ Pop char
   POP CX        \ Pop len1
   JCXZ 0 $         \ Done if len=0
   POP DI        \ Pop addr1
   PUSH ES          \ Save ES
   MOV ES, SSEG       \ Use Scan/Search Segment
   REPZ             \ Repeat if character=char
   SCASB         \ Skip over char's
   POP ES        \ Restore ES
   0<> IF           \ If string is not yet exhausted
     INC CX        \   Back up len of string
     DEC DI      \   and addr to the character
   THEN             \   different from char
   PUSH DI       \ Push addr2
```

```
   PUSH CX        \ Push len2
   NEXT
0 $:   PUSH CX        \ len=0 case
   NEXT
   END-CODE
```

SCAN Given the address and length of a string, and a character to look for, run through the string until we find the character. Leave the address of the match and the length of the remaining string.

```
CODE SCAN   ( addr len char -- addr' len' )
   POP AX
   POP CX
   JCXZ 0 $
   POP DI
   PUSH ES
   MOV ES, SSEG
   MOV BX, CX
   REPNZ          \ Same as SKIP except test for
   SCASB          \   not equal.
   POP ES
   0= IF
      INC CX
      DEC DI
   THEN
   PUSH DI
   PUSH CX
   NEXT
0 $:   PUSH CX
   NEXT
   END-CODE
```

/STRING Index into the string by n. Return addr+n and len-n.

```
CODE /STRING    ( addr len n -- addr' len' )
   POP AX         \ n
   POP BX         \ len
   PUSH BX        \ Save len in case of return
   CMP BX, AX         \ n > len ?
   U<= IF             \ If so select the smaller one
      XCHG BX, AX        \ AX =smaller of AX and BX
   THEN
   POP BX
   POP DX
   ADD DX, AX         \ addr'
   PUSH DX
   SUB BX, AX         \ len'
   PUSH BX
   NEXT
   END-CODE
```

SOURCE Return the address and length of the string to be scanned from the input stream which is stored at TIB. It is not used by F-PC but kept here for F83 compatibility.

```
CODE SOURCE ( -- addr len ) \ TIB #TIB @
   MOV DX, 'TIB
   MOV AX, #TIB
   2PUSH
   END-CODE
```

PARSE Scan the input stream until a delimiting char is encountered. Update >IN pointer. Leave the address and length of the enclosed string on the stack. It function is now absorbed into SOURCE-PARSE-WRD, but kept here to be compatible with F83.

```
: PARSE ( char -- addr len )
   >R  SOURCE          \ Source address and buffer length
   >IN @ /STRING       \ Start searching from >IN
   OVER SWAP R> SCAN   \ Scan for char
   >R OVER -           \ Leave addr and len
   DUP R>
   0<> -  >IN +!  ;          \ Update >IN to end of found string
```

WORD Parse the input stream for a string delimited by char and return a counted string at HERE. In F-PC 2.25, the word buffer was defined in a deferred word 'WORD. However, since 'WORD was initialized to HERE and never got changed, it is deleted in F-PC 3.5 and HERE is used in its place. Note there is always a blank following the string. Scan the input stream until char is encountered. Skip over leading char's. Update >IN pointer, and leave address and length of the enclosed string. This is an all-code realization of WORD to speed up the F-PC 3.5 interpreter/compiler. Earlier versions of WORD in F-PC 2.25 and F83 were colon definitions and were much slower. By defining WORD in code, many intermediate words are also eliminated, making F-PC 3.5 simpler.

```
CODE WORD   ( char -- addr )
   MOV DI, 'TIB          \ Terminal input buffer
   MOV CX, #TIB          \ Length of TIB
   POP BX         \ char to match
   PUSH ES        \ Save ES for later restoration
   MOV DX, DS
   MOV ES, DX         \ ES=DS from now to the end
   MOV AX, >IN        \ Current character pointer
   CMP CX, AX         \ Select smaller of len and >IN
   U<= IF
      MOV AX, CX      \ AX = smaller of AX and CX
   THEN
   ADD DI, AX         \ Starting address
```

```
SUB CX, AX          \ Remaining string length
MOV AX, BX          \ Char for searching
CX<>0 IF            \ Search only if string is not null
   REPZ             \ Skip over leading char's
   SCASB
   0<> IF
      INC CX  \ Remaining count
      DEC DI  \ First character of parsed string
   THEN
THEN
MOV DX, DI
MOV AX, BX
CX<>0 IF            \ Some more characters left?
   REPNZ       \ Yes.  Scan to terminating char
   SCASB
   0= IF           \ If char is found
      INC CX  \   backup pointer and count
      DEC DI
   THEN
THEN
SUB DI, DX          \ Update string length
MOV BX, #TIB
MOV AX, DX
CX<>0 IF            \ If TIB is not yet exhausted
   DEC CX      \   count must be adjusted
THEN
SUB BX, CX          \ >IN must be adjusted also
MOV >IN BX          \ Update >IN to end of found string
MOV BX, UP
```

```
MOV DX, DP [BX]     \ Get HERE, address of the word buffer on top of code dictionary
MOV CX, DI
MOV DI, DX
MOV 0 [DI], CL      \ Copy count byte
INC DI
MOV BX, IP          \ Save SI, which we need to move string
MOV IP, AX          \ Address of source string
REPNZ                 \ Copy string from input buffer to word buffer
MOVSB
MOV AL, # 32          \ Append a space to end of parsed string
STOSB
MOV IP, BX          \ Restore SI
POP ES                    \ Restore ES
PUSH DX
NEXT
END-CODE
```

CHAPTER 4. INTERPRETER AND COMPILER

KERNEL3.SEQ is the most interesting part of the F-PC system, containing all the words which build up the text interpreter and many compilers for various types of Forth words in F-PC. In a sense, these words form the Forth operating system which allows the user to enter commands for execution and also compile source code to extend the Forth system.

Many of these words are built from words that were defined in KERNEL1.SEQ. Furthermore, you will find that many of the compiler directive words are the same as those defined under the metacompiler in KERNEL1.SEQ. However, the compiler directives defined here are those which we normally use in a target system like F-PC. The ones in KERNEL1.SEQ are used to build the target system. Word names are the same but the functions are quite different. Since the compiler words defined in this file are the ones you will use in F-PC, they deserve more attention as you start learning the internal structures of F-PC. After you fully understand the whats, whys and hows here, you can go back and take a closer look at the target compiler words which build a target core image, not words executable in F-PC.

4.1. THE DICTIONARY WORDS

>TYPE Type a string in a multitasking system. The string is copied from common memory to PAD, which is private to a task, and sent out to the console device from there. Each task must manage its own input and output processes.

```
: >TYPE ( addr len -- )
  TUCK            \ Hide a copy of len under addr
  PAD SWAP CMOVE     \ Copy string to PAD
  PAD SWAP TYPE      \ Type from PAD
  ;
```

.(Type the following string on the terminal. This is useful in printing messages from a file while the file is being interpreted or compiled.

```
: .(    ( -- )
   ASCII )          \ The string terminator
   PARSE >TYPE      \ Parse out string and type it
   ; IMMEDIATE      \ Must be executed, not compiled.
```

(The Forth comment command. The following input stream up to and including) is ignored.

```
: (   ( -- )
   ASCII )          \ Scan till )
   PARSE
   2DROP        \ Ignore the string
   ; IMMEDIATE
```

TRAVERSE Run through a name field in the specified direction. Terminate when we detect a byte whose most significant bit is set. The name field in a header is delimited by characters with the most significant bit set. If the direction is 1, scan forward; if it is -1, scan backward. It allows us to move from the link field before the name field to the code pointer field after the name field and vice versa.

```
CODE TRAVERSE     ( addr direction -- addr' )
   POP CX       \ Direction flag
   POP BX       \ Address of first character
   ADD BX, CX       \ Incr/decr address
   PUSH ES      \ Save ES
   MOV ES, YSEG     \ ES needed for Head Segment
   BEGIN            \ Scan the name field
     MOV ES: AL, 0 [BX]  \ Get next character
     AND AL, # 128      \ Bit 7 cleared?
   0= WHILE         \ Yes.
```

```
    ADD BX, CX      \ Continue with next character
  REPEAT          \ No.  End of name field
  POP ES          \ Restore ES
  PUSH BX         \ Return new address
  NEXT            \ Done
  END-CODE
```

DONE? Return a true flag if the input stream is exhausted or if the number n on the data stack does not match that in STATE. This is used to turn off the compiler and return to the text interpreter. It is recoded in assembly for speed.

```
CODE DONE?  ( n -- f )     \ STATE @ <>   END? @ OR   END? OFF   .
  POP AX          \ n
  CMP AX, STATE       \ n <> STATE ?
  0<> IF
    MOV END? # 0 WORD   \ Clear END?
    MOV AX, # -1        \ Leave true in AX
    1PUSH
  THEN
  PUSH END?           \ Push flag
  MOV END? # 0 WORD
  NEXT
  END-CODE
```

CNHASH Code to Name Hash. Given a code field address, get a pointer into the name hash table in YSEG. Since heads are separated from code in F-PC, it is difficult to find the head of a word from its code field address. The name hash table in the Head Segment limits the ranges of heads to be searched for a given code field address. The table has 64 entries of 4 bytes each. Each entry has a high address and a low address, which delimits the memory range in the Head Space to be searched. The Code Segment is divided into 64 blocks of 1024 bytes. Each block of code has a name hash table entry pointing to an area in the head space where the heads of words in the corresponding block are to be found. The relationship among the code dictionary, the head dictionary, and the name hash table is shown in Figure 6.

```
HEX
```

```
: CNHASH    ( cfa -- yaddr )

   0FE00 AND          \ Code Segment is divided into 64 blocks of 1024 bytes each

   FLIP ;             \ (n/1024)*4, offset into the hash table
```

CNSRCH Code to Name Search. Search for the code field address cfa in the code pointer fields of headers between ya and maxya in YSEG. Return the name field address nfa and a false flag, true if the cfa is not found. CNSRCH is recoded in assembly to increase its speed. However, the high level definition is retained here as the documentation of this rather complicated function

Figure 6. The Name Hash Table

```
: CNSRCH    ( cfa ya maxya -- nfa flag )
   SWAP 2+ 2+          \ From ya to nfa of first head
   BEGIN 2DUP          \ Compare nfa to the limit maxya
   U> WHILE ( cfa mxy nfa )    \ If still in range, search.
      DUP YC@ 31 AND \ Get the length of name field
      + 1+ DUP Y@     \ Fetch cfa in code pointer field
      3 PICK =        \ Compare code field addresses
      IF -ROT 2DROP   \ Found.
         1- -1 TRAVERSE  \ Move back and recover nfa
         FALSE EXIT \ Add flag and quit
      THEN
       6 +             \ Not matched. Continue with the
   REPEAT   2DROP      \   next name field
   TRUE ;          \ Failed.  Push a true flag.
```

CNSRCH was re-coded in assembly to improve searching speed. The new code definition is as follows:

```
CODE CNSRCH ( cfa ya maxya -- nfa f )
   pop dx             \ maxya
   pop bx             \ ya
   add bx, # 4         \ Move to name field
   pop di             \ cfa
   mov ds, yseg           \ Align DS to Head Segment
HERE    cmp dx, bx          \ Ya is still less than maxya?
   U> IF   mov ax, 0 [bx]  \ Yes.
      and ax, # 31        \ Skip over the name field
      add bx, ax
      inc bx          \ Move to code pointer field
```

```
    mov ax, 0 [bx]      \ Get cfa
    cmp ax, di      \ If they match, then we found it
    0= if
       sub bx, # 2     \ 1 before last char in name
       begin   mov al, 0 [bx]  \ Test high bit
          and al, # 128   \ Loop till high set
       0= while   dec bx   \ Backup one char
       repeat
       push bx     \ Push nfa of found name
       mov ax, cs      \ Restore DS
       mov ds, ax
       mov ax, # false     \ Push false flag.  Success.
       1push
    then
    add bx, # 6     \ Step to nfa of next header
    JMP ROT     \ Jump to  HERE above to
             \   repeat the search
  THEN
  mov ax, cs         \ Search failed
  mov ds, ax         \ Restore DS
  mov ax, # true         \ Push flag
  push ax
  1push              \   and return
  end-code
```

The following words are necessary to find th e address of a field in a definition from the address of another field. F-PC is more complicated then F83 because a word definition has fields scattered in different segments.

N>LINK Go from a name field address nfa to the link field address lfa in a header.

: N>LINK (nfa -- lfa) 2- ;

L>NAME Go from a link field address lfa to the name field address lfa in a header.

: L>NAME (lfa -- nfa) 2+ ;

BODY> Go from a body address or parameter field address pfa to code field address cfa in the code field of a word in the Code Segment.

: BODY> (pfa -- cfa) 3 - ;

NAME> Go from a name field address nfa in Head Segment to the code field address cfa in the Code Segment.

```
: NAME> ( nfa -- cfa )
  1 TRAVERSE        \ To end of name field
  1+            \ To code pointer field
  Y@ ;              \ Fetch code field address
```

LINK> Go from a link field address lfa to the code field address cfa.

```
: LINK>     ( lfa -- cfa )
  L>NAME            \ To name field
  NAME>   ;         \ To code field
```

>BODY Go from a code field address cfa to the body address or parameter field address pfa.

: >BODY (cfa -- pfa) 3 + ; \ 3 bytes after cfa

NO-NAME A dummy word, whose name field address is returned by >NAME when the real name field can not be found. Therefore, if >NAME failed to find the name of a word, 'NO-NAME' will be printed by .ID.

```
HERE-Y 4 +            \ Leave this nfa for >NAME
: NO-NAME ;
```

>NAME Go from a code field address cfa to the name field address nfa in the Head Segment.

```
: >NAME ( cfa -- nfa )
   DUP CNHASH         \ cfa ya -- , get hash table addr
   DUP Y@ SWAP     \ Get head low limit
   2+ Y@          \ cfa sya maxya --
   CNSRCH         \ nfa flag --
   IF  DROP          \ cfa not found
      (LIT) [ ROT ,-X ]   \ Compile nfa of NO-NAME so that
   THEN    ;         \   it will be the nfa returned.
```

>LINK Go from a code field address cfa to the link field address lfa.

```
: >LINK    ( cfa -- lfa )
   >NAME            \ First find the name
   N>LINK   ;         \ Then the link
```

>VIEW Go from a code field address cfa to the view field address vfa.

```
: >VIEW ( cfa -- vfa )
   >LINK            \ View field is 2 bytes below
   2-   ;           \   the link field
```

VIEW> Go from a view field address vfa to the code field address cfa.

```
: VIEW> ( vfa -- cfa )
   2+              \ To link field first
   LINK>   ;       \ Then to code field
```

4.2. INTERPRETER WORDS

HASH Given a string address and a pointer to a vocabulary thread table, return the actual thread. Use the first two characters and the count of the string to determine which thread. The hashing algorithm is 2*(char2 + 2*char1) + count. This special algorithm gives the best distribution of words in the FORTH vocabulary among the 64 threads. The function of HASH is incorporated into the code word OVER.SWAP.HASH.@(FIND). It is not used by F-PC, but preserved for F83 compatibility.

```
CODE HASH   ( str-addr voc-ptr -- thread )
   POP CX        \ Vocabulary pfa
   POP BX        \ String address
   MOV AX, 1 [BX]     \ First and second characters
   SHL AL, # 1       \ 2*char1
   ADD AL, AH         \ Add second character
   SHL AL, # 1       \ 2* again
   ADD AL, 0 [BX]     \ Add count
   AND AX, # #THREADS 1-   \ Only 64 thread
   SHL AX, # 1       \ Offset address
   ADD AX, CX         \ Address of selected thread
   1PUSH             \   in vocabulary thread table
   END-CODE
```

(FIND) Search the dictionary based on the pointer to a vocabulary thread table and a string. If it finds the string in a thread, return the code field address of the word. If the word is an immediate word the flag returned is a 1. If it is

non-immediate the flag returned is a -1. If the name is not found, the string address is returned along with a flag of zero. Note that links point to links, and lfa's are absolute addresses in the Head Segment.

```
CODE (FIND) ( here lfa -- cfa flag | here false )
   POP BX              \ Link field address
   OR BX, BX
   0= IF               \ If it is 0, exit with
      SUB AX, AX       \   a false flag.
      1PUSH
   THEN
   POP CX              \ HERE has the string
   PUSH ES
   MOV ES, YSEG        \ ES must point to Head Segment.
   MOV DI, CX
   BEGIN
      MOV ES: AX, 2 [BX]      \ First cell in name field
      XOR AX, 0 [DI]          \ Compare to 1st cell in string
      AND AX, # ( 63 ) $7F3F  \ Mask off irrelevant bits
      0= IF               \ A match!
         MOV DX, BX       \ Save link field address
         ADD BX, # 2      \ Point BX to name field
         BEGIN            \ Scan through name
            INC BX        \ Next char in name
            INC DI        \ Next char in string
            MOV ES: AL, 0 [BX]
            XOR AL, 0 [DI]    \ Match?
         0<> UNTIL        \ Exit if characters not same
         AND AL, # 127    \ Mask off delimiting bit
         0= IF
            MOV ES: CX, 1 [BX]  \ Pick up cfa
```

```
        MOV BX, DX      \ Link field address
        MOV ES: AL, 2 [BX]  \ Count byte in name
        AND AL, # 64        \ Test for immediacy
        0<> IF
          MOV AX, # 1 \ 1 for immediate word
        ELSE
          MOV AX, # -1    \ -1 for normal word
        THEN
        POP ES      \ Restore ES
        PUSH CX     \ Cfa just found
        1PUSH           \ Push flag
      THEN        \ Fail to match last char
      MOV BX, DX  \ Restore link field address
      MOV DI, CX  \ Restore HERE pointer
    THEN          \ Fail to match 1st cell
    MOV ES: BX, 0 [BX]  \ Get next link in thread
    OR BX, BX       \ Exit loop at end of thread
  0= UNTIL          \ Not end.  Compare next name.
  POP ES        \ Restore ES
  PUSH CX       \ Push back HERE
  SUB AX, AX        \   with a false flag.
  1PUSH
  END-CODE
```

FIND Run through the vocabulary stack searching for the name whose address is supplied on the stack. If the name is found, return the code field address of the name and a non-zero flag. The flag is -1 if the word is non-immediate and 1 if it is immediate. If the name is not found, the string address is returned along with a false flag.

F-PC has being optimized to be very fast in compiling source files. One reason is that almost all the words involved in compilation were recoded in assembly. However, FIND is really too big to be reduced to machine code. It was

partially recoded by extracting a number of phrases for re-coding. These phrases were given strange names to reflect their actions rather than functions.

```
CODE DROP.CONTEXT.I2*+@DUP      ( addr -- n n )
   \ Consolidate the phrase DROP CONTEXT I 2* + @ DUP in FIND.
   POP AX         \ DROP
   MOV AX, 0 [RP]     \ I
   ADD AX, 2 [RP]
   SHL AX, # 1        \ 2*
   MOV BX, # CONTEXT      \ CONTEXT
   ADD BX, AX         \ +
   PUSH 0 [BX]        \ n
   PUSH 0 [BX]        \ DUP
   NEXT
   END-CODE

CODE PRIOR.CHECK    ( n -- n f )    \ Consolidate DUP PRIOR @ OVER PRIOR ! = in FIND
   POP AX         \ Pop n
   PUSH AX        \ Push n
   MOV BX, PRIOR      \ PRIOR @
   MOV PRIOR AX       \ OVER PRIOR !
   CMP BX, AX         \ Now, figure out the flag.
   0<> IF
      MOV AX, # FALSE
      1PUSH
   THEN
   MOV AX, # TRUE
   1PUSH
   END-CODE
```

```
CODE OVER.SWAP.HASH.@       ( str-addr voc-addr -- str-addr thread )
                \ Consolidate OVER SWAP HASH @ (FIND) in FIND.
    POP AX          \ Vocabulary address
    POP BX          \ String address
    PUSH BX         \ Save String address
    MOV CL, 0 [BX]      \ HASH
    MOV BX, 1 [BX]      \ HASH
    SHL BL, # 1         \ HASH
    ADD BL, BH          \ HASH
    SHL BL, # 1         \ HASH
    ADD BL, CL          \ HASH
    AND BX, # #THREADS 1-   \ HASH
    SHL BX, # 1         \ HASH
    ADD BX, AX          \ HASH
    MOV BX, 0 [BX]      \ HASH
    END-CODE            \   Tail recursion



: %%FIND    ( addr  false #vocs 0 -- cfa flag | addr false )
    DO  DROP.CONTEXT.I2*+@DUP   \ Get thread in vocabulary.
                \ Last flag 0 if end of thread.
        IF  PRIOR.CHECK     \ Update PRIOR with vocabulary
                \   being searched now.
            IF          \ PRIOR is same as current vocabulary,
                DROP FALSE  \ Do not search this thread.
            ELSE    OVER.SWAP.HASH.@  (FIND)
                \ Else, search this thread.
                DUP ?LEAVE  \ If TOS is true, a word was  found.
                \   Exit the loop.
```

```
        THEN
      THEN
   LOOP   ;

CODE %FIND     ( addr -- cfa flag | addr false )
   MOV DI, SP
   MOV BX, 0 [DI]
   CMP 0 [BX], # 0 BYTE
   0<> IF
      MOV PRIOR # 0 WORD \ prior off
      MOV BX, # 0
      PUSH BX              \ false
      MOV CX, # #VOCS
      PUSH CX              \ #vocs
      PUSH BX              \ 0
      MOV AX, # ' %%FIND     \ Call colon word
      JMP AX
   THEN
   MOV END? # TRUE WORD
   MOV 0 [DI], # ' NOOP WORD
   MOV AX, # 1
   1PUSH
   END-CODE

DEFER FIND  ' %FIND IS FIND
```

DEFINED Look up the next word in the input stream. Return true if it exists; otherwise false. May ignore case if CAPS is ON.

```
: DEFINED   ( -- here 0 | cfa [ -1 | 1 ] )
```

```
   BL WORD          \ Parse out next word
   ?UPPERCASE       \ Change to upper case if CAPS<>0.
   FIND             \ Search the dictionary.
   ;
```

?STACK Check for parameter stack underflow or overflow and issue appropriate error message if detected. It is broken into smaller pieces to optimize for speed. The critical part is coded in assembly. The original high level definition is:

```
: (?STACK)  ( -- )
   SP@ SP0 @ OVER  U<   ABORT" Stack Underflow"
   PAD   2DUP  U<   ABORT" Stack Overflow"
   200 + U<
   IF     CR ."  Running out of CODE memory! "
   THEN    ;
```

It is broken down into a few high level definitions and re-assembled into a code definition. It is an interested example showing you how high level words are called from a code word.

```
: STACKUNDER    ( --- )
   TRUE             \ Abort always
   ABORT" Stack Underflow" ;

: STACKOVER ( --- )
   TRUE
   ABORT" Stack Overflow" ;

: WARNOVER  ( --- )
   CR           \ Avoid crowding a line.
   ."  Running out of CODE memory! " ;
```

Now, we will assemble the above high level definitions into a code definition, which will run much faster than the original ?STACK colon definition. The testing is done in machine code and runs very fast. When an error condition is detected, the aborting routines can take all the time they can use because they are outside of the critical path.

```
CODE (?STACK)      ( --- )
   MOV CX, SP         \ Get current stack pointer
   MOV BX, UP
   MOV BX, SP0 [BX]       \ Get the bottom of stack
   CMP BX, CX         \ Underflow?
   U< IF              \ Yes.  Call STACKUNDER.
      MOV AX, # ' STACKUNDER
      JMP AX
   THEN
   MOV DX, DP [BX]     \ Get top of dictionary
   ADD DX, # 80           \ Allow some buffer
   CMP CX, DX         \ Overflow?
   U< IF              \ Yes.  Call STACKOVER.
      MOV AX, # ' STACKOVER
      JMP AX
   THEN
   ADD DX, # 200          \ Add some more buffer space
   CMP CX, DX         \ Near overflow?
   U< IF              \ Yes.  Send out a warning.
      MOV AX, # ' WARNOVER
      JMP AX
   THEN
   NEXT
   END-CODE

DEFER ?STACK    ' (?STACK) IS ?STACK
```

INTERPRET The Text Interpreter Loop in F-PC. If the next word is found in the dictionary, execute it; otherwise, convert it to a number and push it onto the stack. It interprets the commands typed on the terminal or read from a file one line at a time.

```
: INTERP   ( -- )
  BEGIN            \ Process a line of commands
    ?STACK         \ Abort if stack over- or under-flows.
    DEFINED        \ Parse out next word and search the dictionary.
    IF    EXECUTE     \ A word is found.  Execute it.
    ELSE  NUMBER      \ Otherwise, convert word to number
      DOUBLE? NOT     \ If not a double integer,
      IF  DROP  THEN  \   drop the most significant half.
    THEN
    FALSE DONE?       \ Exit if at end-of-line.
  UNTIL  ;

DEFER INTERPRET ' INTERP IS INTERPRET
```

DEFER STATUS Display current status of F-PC. See Sec. 8.4 and STATUS.SEQ.

PRINT Interpret the following words, and send output characters to the printer.

```
: PRINT   ( --- )
  PRINTING ON       \ Turn on printer
  INTERPRET         \ Execute words following
  PRINTING OFF ;    \ Turn off printer
```

4.3. COMPILER TOOL WORDS

ALLOT Allocate n bytes of space in the code dictionary.

```
: ALLOT ( n -- )       DP +!  ;  \ Add n to dictionary pointer
```

, Comma. Compile integer n to the top of the code dictionary. Increment the dictionary pointer so that it will point to the next free cell on the top of the code dictionary. Comma and C, are the two fundamental tools of the Forth compiler, as they are used to build new words and new data structures in the dictionary.

```
CODE ,     ( n -- )
   MOV BX, UP         \ DP is a user variable
   MOV AX, DP [BX]    \ Get dictionary pointer
   ADD DP [BX], # 2  WORD  \ Increment dictionary pointer by 2.
   MOV BX, AX
   POP CX         \ Get n
   MOV 0 [BX], CX     \ Compile it to dictionary
   NEXT
   END-CODE
```

C, Similar to , except it compiles an 8-bit value.

```
CODE C, ( b -- )
   MOV BX, UP
   MOV AX, DP [BX]    \ Get dictionary pointer
   INC DP [BX] WORD       \ Increment by 1
   MOV BX, AX
   POP CX
   MOV 0 [BX], CL     \ Compile byte to dictionary
```

```
   NEXT
   END-CODE
```

PARAGRAPH In the List Space, a paragraph is one 16 byte page. The word list of a colon definition must start at a page or paragraph boundary. PARAGRAPH returns the segment address of the next available paragraph.

```
: PARAGRAPH ( offset-addr -- paragraph )
   15 +              \ Offset into next page
   U/16 ;            \ Convert to segment address
```

ALIGN Used to force even addresses. A noop for 8086.

```
: ALIGN ; IMMEDIATE     \ NOOP indeed
```

EVEN Make the top of the stack an EVEN number. A noop for 8086.

```
: EVEN  ;  IMMEDIATE
```

COMPILE Compile the code field address of the following non-immediate word into the list at run-time.

```
: COMPILE   ( -- )
   2R@           \ Get ES:IP from return stack
   R> 2+ >R          \ Increment IP over next word in the list following COMPILE
   @L            \ Fetch cfa of this word.
   X,   ;            \ Compile it to List dictionary at run-time.
```

CCOMPILE Compile the cfa of the following non-immediate word into the Code dictionary when this definition executes. This is a special purpose version of COMPILE, used to compile something into the Code Space at run-time.

```
: CCOMPILE  ( -- )
```

```
  2R@SWAP          \ Get ES-IP
  R> 2+ >R         \ Increment IP
  @L , ;           \ Compile next word to Code Space.
```

IMMEDIATE Mark the last Header as an Immediate word.

```
: IMMEDIATE ( -- )
  64           \ Precedence bit
  LAST @       \ Name field address of latest word just defined.
  YCSET ;      \ Set its immediate bit
```

4.4. LITERAL WORD COMPILERS

In a colon definition, the list part of the definition is in the List segment and contains mainly a list of execution addresses pointing to executable words in the Code segment. However, other data types and structures are also allowed in the list. Optional data structures include integer literals, double integer literals, string literals, and address literals. A data structure must be preceeded by a pointer to the interpreter which converts the data stored in the list to data on the data stack or a pointer pushed on the stack. Here we shall see the set of words which compile various data structures in a colon definition. The low level words which interpret the data structures were discussed in Chapter 2. Words compiling control structures will be discussed in Section 4.7.

LITERAL Compile a single integer on the stack to the List dictionary as a literal.

```
: LITERAL  ( n -- )
  COMPILE (LIT)     \ Compile literal interpreter
  X,          \   with the literal value.
  ; IMMEDIATE       \ Must act during compiling.
```

DLITERAL Compile a double integer on the stack as a double integer literal.

```
: DLITERAL  ( d# -- )
   SWAP              \ Rearrange the order of double
   [COMPILE] LITERAL      \ Compile LS integer
   [COMPILE] LITERAL      \ Compile MS integer
   ; IMMEDIATE        \ Make it immediate
```

ASCII Compile the next character in the input stream as an ASCII literal.

```
: ASCII    ( -- n )
   BL WORD           \ Get next word
   1+ C@             \ Only use its first character
   STATE @           \ If compiling,
   IF [COMPILE] LITERAL THEN   \  compile ASCII literal
   ;             \ If interpreting, leave code on the stack.
   IMMEDIATE         \ Make it immediate
```

CONTROL Compile the next character in the input stream as a ASCII literal of a Control Character, i.e., between 0 and 31.

```
: CONTROL   ( -- n )
   BL WORD           \ Get next word
   1+ C@  31 AND      \ Convert the first character to a control character
   STATE @           \ Compile or interpret it
   IF [COMPILE] LITERAL   THEN
   ; IMMEDIATE
```

CRASH Default routine executed by an un-initialized deferred word.

```
: CRASH ( -- )
   2R@ 2-         \ ES:IP of the current word
```

```
   @L >NAME CR .ID        \ Display its name and abort with a message.
   TRUE ABORT" <- is an Uninitialized execution vector."
   ;
```

?MISSING Tell user the word does not exist if flag f is true.

```
: ?MISSING  ( f -- )
   IF  'WORD COUNT TYPE    \ Display the parsed string with an error message.
      TRUE ABORT"  <- What? "     \ Used to be " <- huh?, I'm confused."
   THEN    ;         \    Tom is learning.  Give him some time.
```

' Return the code field address of the next word in the input stream.

```
: '    ( -- cfa )
   DEFINED           \ Parse next word and search it in the dictionary.
   0= ?MISSING   ;     \ Return the cfa.  If not found, display error message.
```

['] Similar to ' but used only in a colon definition while compiling.

```
: [']      ( -- )
   '            \ Return cfa of next word.
   [COMPILE] <'> X,       \ Compile it as an address literal.
   ; IMMEDIATE        \ Make it immediate
```

[COMPILE] Force compilation of the following immediate word in a colon definition. It overrides the rule that an immediate word must execute in a colon definition, thereby allowing it to be compiled now as a regular Forth word and executed at run-time.

```
: [COMPILE] ( -- )
   '            \ Find cfa of next word, even if it is immediate
```

```
   X,              \ Force its compilation into list.
   ; IMMEDIATE     \ Itself an immediate word
```

4.5. STRING LITERAL COMPILERS

F-PC uses three different types of string literals: a dot-quote string is compiled in-line with a list of cfa's in the List Space to be typed out at run-time; an x-quote string is compiled in the List Space and copied to "BUF at run-time; and a quote string is compiled to the Code Space, whose address will be returned at run-time. Putting dot-quote strings in the List Space greatly increases the amount of memory in which user can store large amounts of text data as dot-quote strings, while conserving the code space for executable code. However, it requires that the quoted strings be used sequentially since there is only one buffer "BUF in the Code Space to hold the string for manipulation.

"BUF A string buffer to store dot-quote strings copied out of the list space. F-PC compiles ." strings in-line with the cfa's in the word list of a colon definition. The string is moved into the Code Space for manipulation.

```
VARIABLE  "BUF 132 ALLOT
```

XEVEN Adjust list dictionary pointer xdp to the next even number.

```
: XEVEN ( xdp -- even-xdp )     DUP 1 AND + ;
```

XALIGN Align the list dictionary pointer to the 16 bit cell boundary.

```
: XALIGN    ( -- )
   XHERE          \ Current XDP-SEG:XDP
   NIP 1 AND          \ If XDP is odd,
   XDP +! ;           \   increment it to make it even.
```

X>"BUF Copy an in-line string from List Space to the "BUF buffer in the Code Space. Leave the address of "BUF on the stack.

```
: X>"BUF    ( -- "BUF )

   2R>          \ Save the return ES:IP from the return stack.  X>"BUF will                \   be used by (.") which interprets a dot quote string literal.

   2R@          \ Get ES:IP which points to the in-line string.

   2DUP C@L         \ Length of string

   1+ DUP XEVEN       \ Adjust string length for moving

   R> + >R        \ Increase IP to skip the string

   ?CS: "BUF ROT CMOVEL    \ Copy string to X"BUF

   2>R            \ Restore ES:IP.

   "BUF ;
```

(") Return the address and length of an in-line quote string, and continue execution after the string.

```
: (")      ( -- addr len )

   2R@          \ Get ES:IP pointing to the next word

   @L           \   which points to the quote string in the code segment

   COUNT        \ Return address and count

   R> 2+ >R ;        \ Increment IP past the string address.
```

(X") X-quote string interpreter, which copies an x-quote string to the "BUF buffer in the Code Space and returns the address and count of the string in "BUF buffer.

```
: (X")          ( -- addr len )             X>"BUF COUNT ;
```

(.") Type an in-line dot-quote string, and continue execution after the string. It is compiled by the string literal word ." .

```
: %(.")    ( -- )

   2R@          \ Get ES:IP pointing to in-line string.

   2DUP C@L >R        \ Save the string count
```

```
   1+ R@  TYPEL       \ Type the string directly out of the list space.
   R> 1+ XEVEN          \ Actual length of compiled string
   R> + >R ;          \ Skip over the string and continue executing next word.

DEFER (.")   ' %(.") IS (.")

,"     Compile the following quote string till a " to the code dictionary in the Code Space.  Quote-string compiler.

: ,"       ( --- )
   ASCII " PARSE      \ Parse the string till " .
   TUCK HERE PLACE        \ Copy string to Word Buffer
   1+ ALLOT ;         \ Allocate dictionary space

X,"    Compile the following x-quote string till a " to the list dictionary in the List Space.  X-quote string compiler.

: X,"      ( -- )
   ASCII " PARSE      \ Parse string till " .
   HERE PLACE         \ Copy string to Word Buffer
   ?CS: HERE
   DUP C@ 1+ >R       \ Source-addr and actual length
   XHERE R@ CMOVEL    \ Copy to List dictionary
   R> XEVEN XDP +! ;      \ Allocate List space

."     Compile the following string till a " as a dot-quote string literal.  Type out the string at run-time.

: ."   ( -- )
   COMPILE (.")          \ Compile dot-quote string interpreter first.
   X,"              \ Compile the dot-quote string, complete the string literal.
   ;  IMMEDIATE      \ Compiler directive.  Must be executed, not compiled.
```

" Compile the following string as a quote string literal. Return its address and length at run-time.

```
: "    ( -- )
   COMPILE (")        \ Compile quote-string interpreter
   HERE X,            \ Compile string address to list
   ,"                 \ Compile string to code dictionary
   ;  IMMEDIATE       \ Must be executed
```

"" Force compilation of an x-quote string in the List Space. At run-time, the string is copied out to "BUF in the Code Space.

```
: ""       ( -- )
   COMPILE (X")          \ Compile an x-quote string interpreter
   X,"                   \ Compile string to List Space
   ;  IMMEDIATE          \ Must be executed
```

">$ Convert the address addr and length count of a quote string created by " into the address of a counted string. This word relies on the fact that " really creates counted strings, and converts them to address and length on execution. The string must be either used or moved before another "" string is encountered, or it will be over-written.

```
: ">$      ( addr count -- addr-1 )    DROP 1- ;
```

4.6. FORGET

FORGET is a very powerful overlay mechanism in Forth. A portion of the dictionary can be removed and the memory space reclaimed to load other applications. In F-PC, FORGET is much snorter because the dictionary is partitioned into code, head, and list dictionaries. All three dictionaries are trimmed back to a given word definition.

VARIABLE FENCE Contains an address below which the dictionary is protected.

TRIM Change the thread pointers in a vocabulary so that they are all less than a specified value, addr. It trims back the vocabulary to addr.

```
: TRIM   ( addr voc-addr -- )
  #THREADS 0      \ Go through all threads
  DO              \  in a vocabulary.
    2DUP @    \ Pick up one thread
    BEGIN
      2DUP U> NOT   \ If link address is greater than  addr,
    WHILE  Y@ \  follow to the next link address
    REPEAT    \ Continue until trimmed to addr
    NIP OVER !    \ Store the last link address into the thread entry.
    2+      \ Do the same for next thread
  LOOP
  2DROP  ;       \ All done.  Vocabulary trimmed.
```

(FRGET) Forget part of the dictionary. Both the code address and the header address must be specified, and may be trimmed independently. (FRGET) resets all of the links and releases the dictionary space.

```
: (FRGET)  ( code-addr view-addr -- )
  DUP FENCE @      \ Forget below FENCE?
  U< ABORT" Below fence"    \ Not allowed.
  OVER VOC-LINK @      \ First trim the vocabularies
  BEGIN  2DUP U<      \ Vocabulary above code-addr?
  WHILE  @       \ Yes.  Follow to next
  REPEAT      \  vocabulary.
  DUP VOC-LINK !  ( ca va ca pt )   \ Vocabulary trimmed.
  NIP        \ For the remaining vocabularies
```

```
  BEGIN   DUP           \ Trim all of them.
  WHILE   2DUP          \ Exit if at end of vocabulary link.
    #THREADS 2* - TRIM   \ Trim one vocabulary
    @             \ Continue to next vocabulary
  REPEAT
  DROP    YDP !         \ Move back the Head dictionary pointer.
  DUP 1+ @ OVER       \ cfa NEST-addr cfa --
  >BODY +           \ Code address of NEST
  (LIT) TRIM         \ cfa of TRIM, a colon definition
  DUP 1+ @ SWAP >BODY +   \ Must also be NEST
  =                 \ If code-addr is a : def
  IF      DUP >BODY @
    +XSEG XDPSEG !      \ Set back XHERE too!
    XDP OFF     \ Trim List dictionary
  THEN    DP ! ;        \ Trim Code dictionary
```

FORGET Forget all of the code and headers defined after and including <name>. FORGET must be used with caution; if you DEFER a word that is not forgotten, and vector it to a word that is forgotten, the first call to the original word will cause the system to crash. Similarly, if you are using DEFERS task chaining, FORGET will not know about these chains, so they must be unlinked before using FORGET or disaster will strike. Similarly, you have to unlink tasks before forgetting them. List Space is recovered only if you FORGET colon definitions.

```
: FORGET  <name>    ( -- )
  BL WORD           \ Get next word
  ?UPPERCASE         \ Make sure case-insensitive
  DUP CURRENT @ HASH @    \ Get the CURRENT vocabulary
  (FIND)         \   and locate the word
  0= ?MISSING           \ Abort if word is not in the CURRENT vocabulary
  DUP >VIEW (FRGET) ;     \ Trim the CURRENT vocabulary and other overlapping
            \   vocabularies.
```

4.7. STRUCTURE COMPILERS

IF, ELSE, THEN, DO ?DO, LOOP, +LOOP, BEGIN, UNTIL, AGAIN, REPEAT, and WHILE are compiler directives needed to properly compile classical conditional structures in F-PC. They are immediate words and they must compile their run-time routines along with whatever addresses they need. A modest amount of error checking is done. If you want to rip out the error checking change the ?> and ?< words to > and < words, and all of the 2DUPs to DUPs and the 2SWAPs to SWAPs. The rest should stay the same. On the other hand, you can also institute more elaborate error checking like what fig-Forth had by leaving specific error numbers on the stack and checking the numbers to determine if the control structures are properly nested.

A global error checking mechanism is built in F-PC using !CSP and ?CSP. When a colon definition is entered, the stack pointer is saved in the variable CSP by !CSP. When the colon definition is completed, the contents of CSP is compared with the stack pointer. Since the compilation of a colon definition should have no effect on the data stack, any discrepancy discovered by ?CSP could only be contributed to errors in building control structures. This global stack check and the simple structure checking in F-PC are in most cases sufficient to catch control structure errors in a colon definition.

DEFER WHERE Locate source code when an error occurred.

?ERROR A deferred word vectored to (?ERROR). If a true flag is on the top of the data stack, clear the data stack and type the error message located at addr.

```
: (?ERROR)  ( addr len f -- )
   IF           \ If flag is true, reset F-PC.
      2>R          \ Save error message
      SP0 @ SP!         \ Clear the data stack
      PRINTING OFF       \ Don't let printer go wild
      2R> SPACE TYPE SPACE    \ Display the error message
      QUIT          \ Restart F-PC text interpreter
   ELSE   2DROP      \ No error.  Continue.
   THEN  ;
```

(ABORT") The run-time code compiled by ABORT". Use ?ERROR to type an error message, and clear the return stack.

```
: (ABORT")  ( f -- )
   ?DUP IF
      X>"BUF        \ Grab the error message
      COUNT         \ Get its count
      ROT ?ERROR        \ Abort if flag is true
   ELSE   2R@ C@L 1+ XEVEN       \ Otherwise, step over the message
      R> + >R       \ Continue executing
   THEN ;
```

ABORT" Compile the following string as an x-quote string. If a flag on top of the stack is true at run-time, display the string as an error message and abort; i.e., return to F-PC text interpreter.

```
: ABORT"   ( -- )
   COMPILE (ABORT")       \ Also a string literal
   X,"             \ Compile x-quote string
   ;  IMMEDIATE      \ Compiler directive
```

ABORT Clear the data and return stacks and QUIT. No message is displayed.

```
: ABORT ( -- ) TRUE ABORT" "  ;
```

?CONDITION Simple compile-time error checking. It is usually adequate for most situations. Like F83, F-PC institutes minimal compiler security checks. A compiler directive starting a control structure would leave a true flag on the data stack at run-time, with other structural information underneath. The compiler directive resolving the control structure would test this flag. If the flag is false, the control structure is not properly constructed and the compiler would abort.

```
: ?CONDITION   ( f -- )
   NOT ABORT" Conditionals Wrong"   ;
```

```
>MARK   Set up a Forward Branch.

: >MARK ( -- addr )
   XHERE          \ Get XDPSEG:XDP
   NIP            \ Discard XDPSEG
   0 X,   ;       \ Compile a dummy address in the list dictionary,
                  \   whose address is left on the stack.

>RESOLVE    Resolve a Forward Branch.

: >RESOLVE  ( addr -- )
   XHERE          \ Get current list address pair
   -ROT SWAP      \ xhere xdpseg addr --
   !L   ;         \ Store xhere to addr, thus resolve the forward branch.

<MARK   Set up a Backward Branch.

: <MARK ( -- addr )
   XHERE NIP ;    \ Simply leave the current address on the stack.

<RESOLVE    Resolve a Backward Branch.

: <RESOLVE  ( addr -- )
   X, ;           \ Compile the addr on stack.

?>MARK  Set up a Forward Branch with Error Checking.

: ?>MARK    ( -- f addr )
```

```
   TRUE            \ Leave a flag for error checking
   >MARK
```

?>RESOLVE Resolve a Forward Branch with Error Checking.

```
: ?>RESOLVE ( f addr -- )
   SWAP ?CONDITION       \ Error checking
   >RESOLVE ;       \ Then resolve the branch
```

?<MARK Set up for a Backward Branch with Error Checking.

```
: ?<MARK   ( -- f addr )
   TRUE            \ Leave flag
   <MARK  ;         \ Mark current address
```

?<RESOLVE Resolve a Backward Branch with Error Checking.

```
: ?<RESOLVE ( f addr -- )
   SWAP ?CONDITION       \ Check error
   <RESOLVE ;       \ Then resolve
```

IF Select one of two execution pathes. Used in the form:

    flag IF <true> ELSE <false> THEN

where the ELSE <false> clause is optional. If flag is false, branches forward to <false> or after THEN.

```
: IF      ( f -- )
   COMPILE  ?BRANCH
   ?>MARK
   ; IMMEDIATE
```

ELSE Unconditional jump to the word after THEN. Used in the form:

flag IF <true> ELSE <false> THEN

If flag is false, branches forward to <false> clause.

```
: ELSE  ( -- )
  COMPILE  BRANCH   \ Compile unconditional branch
  ?>MARK            \ Mark here
  2SWAP         \ Get the marker of IF
  ?>RESOLVE         \ Resolve its forward branch
  ; IMMEDIATE
```

THEN Terminate a branch structure. Used in the form:

flag IF ... ELSE ... THEN

flag IF ... THEN

```
: THEN
  COMPILE DOTHEN    \ Compile marker
  ?>RESOLVE         \ Resolve forward jump
  ; IMMEDIATE
```

DO Initialize a loop structure with index running from index to limit-1. Used in the form:

DO ... LOOP

DO ... +LOOP

```
: DO    ( limit index -- )
  COMPILE (DO)          \ Start do-loop
  ?>MARK        \ Push addr and flag
  ; IMMEDIATE
```

?DO Same as DO except that the loop will not be executed if index = limit.

```
: ?DO      ( limit index -- )
   COMPILE (?DO)
   ?>MARK
   ; IMMEDIATE
```

LOOP Terminate a loop structure. Increment loop index by one and repeat <loop-body> until loop index crosses the boundary between limit and limit-1. Used in the form:

```
      DO <loop-body> LOOP
```

```
: LOOP     ( -- )
   COMPILE (LOOP)     \ Compile end-of-loop
   2DUP 2+            \ Duplicate address and flag
   ?<RESOLVE          \ Resolve jump from DO
   ?>RESOLVE          \ Resolve jump to DO
   ; IMMEDIATE
```

+LOOP Terminate a loop structure. Increment loop index by n and repeat <loop-body> until loop index crosses the boundary between limit and limit - 1. Used in the form:

```
   DO <loop-body> n +LOOP
```

```
: +LOOP ( n -- )
   COMPILE (+LOOP)
   2DUP 2+
   ?<RESOLVE
   ?>RESOLVE
   ; IMMEDIATE
```

LEAVE Exit a DO-LOOP immediately. LEAVE and ?LEAVE could be non-immediate in this system, but the Forth-83 Standard specifies an immediate LEAVE, so they both are defined as immediate words for compliance and uniformity.

```
: LEAVE          \ Exactly equivalent to (LEAVE)
   COMPILE (LEAVE)
   ; IMMEDIATE
```

?LEAVE Immediately exit a DO-LOOP if the flag is true; else continue looping.

```
: ?LEAVE
   COMPILE (?LEAVE)
   ; IMMEDIATE
```

BEGIN Start an infinite or indefinite loop. Used in the form:

```
      BEGIN ... AGAIN
      BEGIN ... flag UNTIL
      BEGIN ... flag WHILE ... REPEAT
```

```
: BEGIN
   COMPILE DOBEGIN    \ Mark the beginning of loop
   ?<MARK        \ Leave address and flag on stack
   ; IMMEDIATE
```

UNTIL Mark the end of a BEGIN ... UNTIL loop. Terminate the loop if flag f is true.

```
: UNTIL ( f -- )
   COMPILE ?UNTIL
   ?<RESOLVE         \ Backward jump to BEGIN
   ; IMMEDIATE
```

AGAIN Unconditional jump to just after BEGIN in an infinite loop:

BEGIN ... AGAIN

```
: AGAIN ( -- )
   COMPILE  DOAGAIN       \ Unconditional jump
   ?<RESOLVE         \   back to BEGIN
   ; IMMEDIATE
```

WHILE Conditional exit from a BEGIN...REPEAT loop. Used in the form:

BEGIN <loop> flag WHILE <true> REPEAT

Repeat <loop> and <true> clauses while the flag is true or non-zero. Exit loop if flag is false.

```
: WHILE ( f -- )
   COMPILE ?WHILE
   ?>MARK
   ; IMMEDIATE
```

REPEAT Unconditional backward branch to just after BEGIN in an indefinite loop:

BEGIN <loop> flag WHILE <true> REPEAT

```
: REPEAT    ( -- )
   2SWAP         \ Resolve backward jump first
   COMPILE DOREPEAT       \ Mark end of loop
   ?<RESOLVE         \ Backward jump to BEGIN
   ?>RESOLVE         \ Forward jump from WHILE
   ; IMMEDIATE
```

The following structure related words were suggested by Bill Muench to extend the classic Forth structures. They give the user much flexibility in exiting a loop under various conditions and are useful in cases where the classic structures are too restrictive.

FORWARD Unconditional forward branch. Use it very carefully because the words following may not get executed at all.

```
: FORWARD      ( -- )
   COMPILE BRANCH      \ Similar to IF, except using unconditional jump.
   ?>MARK
   ; IMMEDIATE
```

CONTINUE Unconditional jump back to BEGIN to repeat a loop.

```
: CONTINUE     ( -- )
   2OVER          \ Preserve the marks left by BEGIN.
   [COMPILE] REPEAT       \ Jump back.
   ; IMMEDIATE
```

BREAK Used as THEN with IF, in an IF...BREAK phrase to exit a word.

```
: BREAK        ( -- )
   COMPILE EXIT       \ Terminate a colon word prematurely.
   [COMPILE] THEN      \ Complete the IF...BREAK structure.
   ; IMMEDIATE
```

AFT Cause the words between BEGIN and AFT to be executed only once in a BEGIN ...UNTIL/REPEAT loop.

```
: AFT          ( -- )
   2DROP          \ Discard address and flag left by BEGIN.
   [COMPILE] FORWARD   \ Compile a forward jump
   ?<MARK 2SWAP       \ Leave its own marks for end-loop words to resolve
   ; IMMEDIATE
```

FOR Start a single index decrementing loop. Used with NEXT in the following fashion: n FOR ... NEXT . Repeating index n is pushed on the return stack. This loop structure was recently advocated by Chuck Moore because FOR and NEXT are native machine instructions of the NC4000 Forth chip.

```
: FOR         ( n -- )
   COMPILE >R         \ Push loop index on the return stack.
   ?<MARK         \ Leave marks for NEXT to resolve.
   ; IMMEDIATE
```

NEXT Decrement the count on the return stack. If the count becomes negative, exit the FOR...NEXT loop and discard the count. If the count is not negative, jump to the word after FOR.

```
: NEXT        ( -- )
   COMPILE NEXT|       \ Compile low level loop function.
   ?<RESOLVE          \ Resolve jump address to FOR.
   ; IMMEDIATE
```

4.8. COLON AND OTHER COMPILERS

,VIEW Compile the view field in a header.

```
: ,VIEW ( -- )
   LOADLINE @          \ Byte offset of current line in the file under loading
   Y, ;               \ Lay down the view field
```

"HEADER Compile the following string to make a header, and initialize the code field. First check for duplicated name and make an entry in >NAME hash table if appropriate. Next lay down the view field and link it to the correct thread and build the link field. Set up LAST so that it points to the new name field. Copy the name to YSEG and delimit the name field bits. Compile the code field address into the code pointer field in the YSEG. Finally add a stopper entry to >NAME hash table in case of a large ALLOT or at the end of dictionary. It does all of its work in the Head Space, and has no effect on Code Space whatsoever.

A few words are split out for optimization. The critical path is coded in assembly. Non-critical parts are coded in high level.

```
: NOHEADROOM  ( --- )
  TRUE ABORT" Out of HEAD memory!" ;

: NOLISTROOM  ( --- )
  TRUE ABORT" Out of LIST memory!" ;

CODE SPCHECK  ( -- f1 f2 )  \ Head and List space check
  MOV AX, YDP       \ Get head DP
  SHR AX, # 1       \ Convert to segment
  SHR AX, # 1
  SHR AX, # 1
  SHR AX, # 1
  ADD AX, # 6       \ Add 6 segments for head room
  CMP AX, ' #HEADSEGS >BODY  \ Are we out of space yet?
  >  IF             \ If so, abort.
    MOV AX, # ' NOHEADROOM
    JMP AX
  THEN
  MOV AX, XDPSEG    \ Load List segment
  SUB AX, XSEG      \ Convert to size of list so far
  ADD AX, # 6       \ Add 6 for head room
  CMP AX, ' #LISTSEGS >BODY  \ Are we out of List space yet?
  >  IF           \ Yes.  Abort.
    MOV AX, # ' NOLISTROOM
    JMP AX
  THEN
```

```
   NEXT              \ Everything in order.
   END-CODE

: %ALREADY_DEF     ( a1 -- a1 )   \ Is the word at A1 already defined?
   WARNING @
   IF     DUP FIND NIP
     IF     DUP  CR  COUNT TYPE ."  isn't unique "
     THEN
   THEN   ;      ( str )

: "HEADER   ( str-addr -- )
   SPCHECK        \ Check Head and List Spaces
   DUP C@ 31 >         \ Name too long?
   ABORT"  Name TOO LONG, > 31 chars!"
   /ALREADY_DEF
   ALIGN
   YHERE 2- Y@            \ cfa of last word
   CNHASH         \ Head hash table entry
   HERE CNHASH  <>     \ Is it the same as the entry for current code word?
   IF  YHERE HERE CNHASH   \ No.  Update hash entry
     DUP Y@ ROT UMIN SWAP    \ Find the minimum head address
     Y!         \   store it in hash entry
   THEN
   ,VIEW              \ Lay down the view field
   YHERE OVER CURRENT @    \ Get CURRENT vocabulary
   HASH DUP @  Y,      \ Lay the link field.  Update thread in CURRENT
   YHERE LAST !       \ Remember nfa
   YHERE ?CS: ROT      \ yhere cs str --
   DUP C@  WIDTH @ MIN \ Length of name
```

```
   1+ >R            \ yhere cs str --
   YHERE YS: R@ CMOVEL \ Copy string to name field
   R> YDP +! ALIGN     \ Allocate name field
   128 SWAP YCSET      \ Delimit count byte in name
   128 YHERE 1- YCSET      \ Delimit last byte in name
   HERE Y,         \ Lay code pointer field
   YHERE HERE CNHASH 2+ Y! \ Valid stopper in next hash entry
   ;
```

,CALL Compile a CALL to address 0. The actual subroutine address is set later by ;USES or ;CODE. See CREATE, VARIABLE, CONSTANT and : . CALL pushes the parameter field address or the address of BODY on the data stack, which is quite convenient if we have to access data stored in the parameter field.

```
: ,CALL
   232 C,            \ Assemble an 8088 machine instruction CALL, decimal 232.
   0 HERE 2+ - ,         \ Assemble a dummy address 0000
   ;
```

,JUMP Assemble a JMP 0 instruction in the code field. The address 0 will be patched with the address of an inner interpreter.

```
: ,JUMP
   233 C,            \ Assemble a JMP instruction, which is decimal 233.
   0 HERE 2+ - ,         \ Assemble dummy address 0
   ;
```

HEADER Create a named header in the Head Space, but compile absolutely nothing in Code Space. The code pointer field in the head thus created does point at HERE though. It is a deferred word vectored to <HEADER>.

```
: <HEADER>  ( -- )
   BL WORD        \ Get next word in the input stream
```

```
   ?UPPERCASE        \ Make it upper case if so desired
   "HEADER ;         \ Build a header

DEFER HEADER    ' <HEADER> IS HEADER
```

CREATE Make a header for the next word in the input stream. The word created by CREATE returns its parameter field address on the data stack due to CALL which thinks that the F-PC data stack is its return stack.

```
: CREATE <name> ( -- )
   HEADER        \ Build a header
   ,CALL         \ Assemble CALL o
   ;USES >NEXT ,-X     \ Patch o to >NEXT, which does nothing.  However, CALL
                \   pushes pfa or BODY address on data stack.

: "CREATE      ( str-addr -- ) \ Compile a header with the name at str-addr.
   "HEADER           \ Build a header
   ,CALL         \ Assemble CALL o
   ;USES >NEXT ,-X

!CSP       Save current stack pointer in CSP for ?CSP to check if the stack is disturbed.

: !CSP        ( -- )     SP@ CSP !   ;

?CSP       Compare the current data stack pointer with the contents saved in CSP.  Abort if they are not equal, indicating compilation error.

: ?CSP     ( -- )
   SP@ CSP @ <>       \ Compare stack pointer and CSP
   ABORT" Stack Changed"       \ Abort if not equal
   ;
```

HIDE Remove the last definition from the Header Dictionary. In F83, HIDE smudges the last header and makes it invisible to FIND because of the smudge bit in the first byte of the name field. This technique works but it leaves garbage headers in the dictionary. HIDE in F-PC actually un-links the last header from the current vocabulary, resulting in a clean header dictionary.

```
: HIDE    ( -- )
   LAST @         \ Name field address of last word
   DUP N>LINK Y@      \ Find link field address of next word in the thread
   SWAP CURRENT @     \ Thread head in CURRENT
   YHASH ! ;          \ Re-link to remove last word from current vocabulary
```

REVEAL Activate the last definition in the Header Dictionary by re-linking it to the current vocabulary. Remove the effects of HIDE. See HIDE for difference between F-PC and F83.

```
: REVEAL   ( -- )
   LAST @ DUP N>LINK      \ Lfa of last word
   SWAP CURRENT @     \ Thread in CURRENT vocabulary
   YHASH ! ;          \ Link last word into CURRENT vocabulary, thus reveal it.
```

;USES Similar to the traditional ;CODE except that it is used when the run-time code or the inner interpreter has been previously defined already. It sets the code field of current word to the contents of following cell.

```
: (;USES)  ( -- )
   2R> SWAP @L        \ Fetch contents of next cell, address of an interpreter
   LAST @ NAME>       \ Get code field address of the last word
   DUP >R 3 + -           \ Patch the address of JMP or
   R> 1+ ! ;          \   CALL in the code field
```

(;CODE) Set the code field of the current word under construction to the address of the following routine coded in assembly which is the inner interpreter of this type of word.

```
: (;CODE)   ( -- )
   2R> SWAP @L        \ Fetch contents of next word
   LAST @ NAME>       \ Get its code field address
   DUP >R
   232 ( CALL ) R@ C!         \ Assemble a CALL not JUMP
   3 + - R> 1+ ! ;          \   to the inner interpreter
```

DOES> Specify the run-time behavior of a defining word in high level Forth. It is similar to ;CODE but defines the inner interpreter using high level Forth word sequence, making the code easy to write and portable to other Forth systems.

```
: DOES>    ( -- )
   COMPILE (;CODE)    \ Mark the inner interpreter
   HERE X,        \   and its code field address
   232 ( CALL ) C,    \ Start the inner interpreter with CALL DODOES
   [ [FORTH] ASSEMBLER DODOES META ]
            \ Find the address of DODOES in ASSEMBLER vocabulary.
   LITERAL        \ Make it a literal to compile daughter words
   HERE 2+ - ,        \ Assemble the relative address
   XHERE PARAGRAPH +   \ Justify the list dictionary
   DUP XDPSEG !       \   segment pointer
   XSEG @ - ,         \ Compile the segment offset in the BODY of the daughter                    \   word so it can find its interpreter in the List Space.
   XDP OFF ; IMMEDIATE \ Clear list offset pointer ready to compile the inner
            \   interpreter in high level.
```

ASSEMBLER A vocabulary that contains all of the assembler definitions.

```
VOCABULARY ASSEMBLER
```

SETASSEM Set up the assembler environment to construct a code word. It allows the Forth environment to be restored after the code word is completed.

```
DEFER SETASSEM  ' NOOP IS SETASSEM
```

[Stop compiling and start interpreting.

```
: [   ( -- )
   STATE OFF          \ Turn off the compilation flag
   ;  IMMEDIATE       \ The compiler will release control to text interpreter
```

;USES Define the run-time behavior of a defining word by vectoring its inner interpreter to a routine already defined.

```
: ;USES ( -- )
   ?CSP               \ Save current stack pointer for error checking
   COMPILE (;USES)    \ Compile (;USES) so that the pointer to the inner interpreter
                      \  can be accessed at run-time
   [COMPILE] [        \ Enter the Forth interpreter to  assemble machine code
   REVEAL             \ Make current word available
   ASSEMBLER          \ Need ASSEMBLER to provide tools
   ; IMMEDIATE
```

;CODE Used for defining the run-time portion or an inner interpreter of a defining word in low level assembly code.

```
: ;CODE    ( -- )
   ?CSP               \ For error checking
   COMPILE (;CODE)    \ Compile an inner interpreter which executes the code starting
   HERE X,            \  from HERE in the Code Space
   [COMPILE] [        \ Turn on assembler
   REVEAL             \ Make word available
```

```
  SETASSEM          \ Set up assembler environment
  ; IMMEDIATE
```

] The Compiler Loop. First set Compile State. Look up the next word in the input stream and either execute it if it is an immediate word, or compile it if it is not an immediate word. If the word is not in the dictionary, convert it to a number, either single or double precision depending on whether or not any punctuation was present. Continue until the input stream is exhausted or the compilation STATE is changes.

```
: (])     ( -- )
  STATE ON          \ Turn on compilation flag
  BEGIN   ?STACK     \ Check the stacks
    DEFINED DUP     \ Find the next word in input
    IF  0>          \ Immediate word?
      IF  EXECUTE     \ Yes.  Execute it.
      ELSE    X,        \ No.  Compile it.
      THEN
    ELSE DROP           \ Not a valid word.
      NUMBER        \ Convert it to a double integer
      DOUBLE?       \ Any punctuation marks?
      IF  [COMPILE] DLITERAL  \ Yes.  Compile double.
      ELSE    DROP          \ No.  Compile single.
        [COMPILE] LITERAL
      THEN
    THEN
    TRUE DONE?      \ Input exhausted?
  UNTIL   ;         \ Yes. Exit. Otherwise, compile the next word.

DEFER ]       ' (]) IS ]
```

MAKEDUMMY Make a dummy colon definition using next word as its name. It is effectively a NOOP, used by ANEW for forgetting words before recompiling a file.

```
: MAKEDUMMY ( -- )     \ MAKEDUMMY <word> <enter>
   HEADER         \ Build a header
   ,JUMP          \ Compile JMP in code field
   XHERE PARAGRAPH +   \ Absolute paragraph of new def
   DUP XDPSEG !       \ Set new XHERE segment
   XSEG @ - ,         \ Compile relative paragraph of def
   XDP OFF        \ Clear List offset
   COMPILE UNNEST     \ End it like a colon word
   ;USES  NEST ,-X    \ Install colon interpreter as its own interpreter.
```

ANEW Define a NEW dummy definition. If the word already exists, then forget it before making a new one. A nice utility to allow re-compiling a file again and again for debugging purposes. This technique, attributed to Ray Isaac at Calos, is a real neat trick.

```
: ANEW  ( -- )         \ ANEW <word> <enter>
   >IN @ >R          \ Save interpreter pointer to re-read the word name
   DEFINED NIP       \ Is if already defined?
   R@ >IN !          \ Restore >IN pointer
   IF FORGET THEN    \ Forget it if defined.
   R> >IN !          \ Make a new definition.
   MAKEDUMMY   ;
```

: Defines a colon definition. The definition is hidden until it is completed, unless the user desires recursion. At run-time : adds a nesting level.

```
: (:)           ( -- )
   !CSP            \ Store stack pointer in CSP for error checking
   CURRENT @ CONTEXT ! \ Make CURRENT vocabulary the current one for searching
   HEADER          \ Build header in Head Segment
   ,JUMP           \ Assemble JMP 0 in Code Segment
```

```
   XHERE PARAGRAPH +   \ Justify List dictionary pointers
   DUP XDPSEG !
   XSEG @ - ,
   XDP OFF
   HIDE              \ Hide the header
   ;USES   NEST ,-X       \ Install colon interpreter

: : ( -- ) (:) ] ;
```

; Terminate a colon definition. Compile the run-time code UNNEST to remove a nesting level, and change STATE so that compilation will terminate.

```
: ;    ( -- )
   STATE @        \ If not compiling, must be
   0= ABORT" Not Compiling!"   \   in error.
   ?CSP             \ Check the stack pointer
   COMPILE UNNEST     \ Compile unnesting routine
   REVEAL         \ Make colon word available
   [COMPILE] [        \ Revert to interpreting mode
   ; IMMEDIATE        \ Must execute in colon word
```

RECURSIVE Allow the current definition to be self referencing.

```
: RECURSIVE ( -- )
   REVEAL         \ Make last header available for searching and compiling
   ;  IMMEDIATE
```

CONSTANT A defining word that creates constants. At run-time the value of the constant is placed on the stack.

```
: CONSTANT  ( n -- )
```

```
   CREATE        \ Build a header and compile  JMP 0 in code field
   ,             \ Compile n into parameter field
   ;USES DOCONSTANT ,-X    \ Install inner interpreter for constants
```

VALUE Identical to CONSTANT above. The value stored in the body (parameter) field of a VALUE word can be changed at run-time.

```
: VALUE ( n -- )
   CREATE ,
   ;USES DOVALUE ,-X       \ DOVALUE is identical to DOCONSTANT.
```

ARRAY Define an array of n1 bytes.

```
: ARRAY ( n -- )
   CREATE ALLOT
   ;USES >NEXT   ,-X
```

VARIABLE A defining word to create variables. At run-time the address of the variable or its parameter field address is placed on the stack.

```
: VARIABLE  ( -- )
   CREATE        \ Make header and compile JMP 0
   0 ,           \ Compile 0 as the initial value into parameter field
   ;USES >NEXT ,-X     \ Install >NEXT as its inner interpreter.
```

DEFER Define a vectored execution word. These are initially set to display an error message. They are initialized by IS. (See Sec. 5.2).

```
: DEFER ( -- )
   CREATE        \ Build header
   ['] CRASH ,        \ Initialize it to execute CRASH
```

```
   ;USES DODEFER  ,-X      \ Install DODEFER as the inner interpreter
```

```
DODEFER RESOLVES <DEFER>    \ <DEFER> was a forward reference in meta compilation
```

VOCABULARY Defines a new Forth vocabulary. In its parameter field, initiate an array of #THREADS cells, which is the thread table of this vocabulary, and a cell for the vocabulary link which links all vocabularies together as a linked list to facilitate the removal of words by FORGET. The vocabularies and their linking to VOC-LINK are shown in Figure 7.

```
: VOCABULARY    ( -- )
   CREATE         \ Make header
   #THREADS 0         \ Initialize all threads to 0
   DO   0 ,  LOOP

   HERE  VOC-LINK @ ,      \ Extend the VOC-LINK chain
   VOC-LINK !         \   to include this vocabulary
   DOES>          \ At run-time
      CONTEXT !       \   store thread table address into CONTEXT, making it the
   ;              \   first vocabulary to be searched

RESOLVES <VOCABULARY>       \ Resolve its forward reference in meta compiler
```

DEFINITIONS Make the context vocabulary the current vocabulary. Subsequent definitions will be linked to the CURRENT vocabulary.

Figure 7. Vocabularies and their linkage to VOC-LINK.

```
: DEFINITIONS   ( -- )
   CONTEXT @          \ Get pfa of context vocabulary
   CURRENT !   ;          \ Store it in CURRENT.

2CONSTANT   Create a double integer constant.

: 2CONSTANT ( d -- )
   CREATE         \ Build header
   , ,            \ Compile double as two singles
   DOES>   ( -- d# )      \ High level inner interpreter:
      2@          \ Fetch double integer from pfa
```

```
  ;
  DROP            \ DOES> in meta compiler leaves the address of inner
                  \ interpreter on the stack for resolving forward
                  \ reference.  It is of no use here.
```

2VARIABLE Create a double integer variable.

```
: 2VARIABLE ( -- )
  0 0 2CONSTANT     \ Build it like a double constant initialize to 0
  DOES>  ( -- addr ) \ At run-time return the parameter field addr
  ;
  DROP              \ See comments in 2CONSTANT
```

RUN This is the F-PC operating system used in QUIT (see Sec. 5.3). Interpret or compile whatever is in the Terminal Input Buffer. Test the system variable STATE, and do the appropriate thing (interpret or compile). If STATE is ON, compile the words in the input buffer to the dictionary; otherwise, execute them.

```
: <RUN>          ( -- )
  STATE @        \ In compiling state?
  IF ]           \ Yes.  Compile text in TIB.
    STATE @ NOT     \ End of line.  Test again.
    IF INTERPRET THEN   \ Interpret if STATE changes.
  ELSE   INTERPRET THEN  \ Interpreting state.  Interpret text in TIB.
  ;

DEFER RUN      ' <RUN> IS RUN
```

CHAPTER 5. SYSTEM INITIATION

KERNEL4.SEQ is the last kernel file. The most important part in this file is the initialization of the F-PC system. F-PC is saved as an EXE file under DOS. It contains the code dictionary, the head dictionary, and the list dictionary. These dictionaries must be moved to their appropriate segments and then the F-PC can be booted. Many system and user variables must also be initialized to proper values for F-PC to operate.

Many loose ends left over by the meta compiler will also have to be tied together. Forward references must be resolved. Deferred words must be vectored to executable words. Lastly, the target core image must be saved in an EXE file so that the kernel can be booted and executed later.

There are also a number of utility words included in this file. Some of them are provided for debugging purposes. The decompiler and a fully featured debugger will be loaded on top of the kernel later as tools. However, some minimal debugging facility is handy in case there are problems with the kernel and higher level utilities cannot be loaded successfully.

5.1. USER VOCABULARY

Forth has always been a multi-tasking, multi-user system from its beginning. There are two very important concepts which make it very easy for Forth to support multi-tasking: re-entrancy of Forth code and the user area. All the resources and information specific to a user or task are isolated and put into the user area. Forth words in the dictionary are fully re-entrant so that the same set of words can be used by all tasks or users. Thus each task can operate independent of each other while sharing all the tools and utilities from a centralized dictionary. A multi-tasking Forth system does not require huge amounts of memory and can switch among the tasks at very high speed.

The user area contains the following resources: a data stack, a return stack, optional dictionary extend memory, input and output buffers, and a set of user variables. The user variable area can also hold user deferred words. The same user deferred word can be vectored to different words in different tasks. F-PC uses the same names VARIABLE and

DEFER to define user variables and user deferred words. They are redefined and collected in the USER vocabulary so that they can be used to define user variables and user deferred words. This section describes all the words defined in the USER vocabulary.

#USER Count of how many user variables are defined in the current system. It is incremented when a new user variable is defined. F-PC leaves space for 10 more user variables to be added by user.

```
VARIABLE #USER
```

USER Vocabulary that holds versions of defining words which define new user variables and user deferred words. They have the same names as those in the FORTH vocabulary, but may have different functions in different tasks.

```
VOCABULARY USER        \ Create USER vocabulary

USER DEFINITIONS       \ The following words are compiled into the USER vocabulary
```

ALLOT Allocate some space in the user area for a task. When used with CREATE, you can define arrays this way. However, be aware that F-PC gives you only 20 bytes in the user area. It can be enlarged by adding more memory space in the system user variable area at the end of KERNEL4.SEQ and modifying the words which define tasks in MULTASK.SEQ.

```
: ALLOT ( n -- )
   #USER +!          \ Add n to #USER
   ;
```

CREATE Define a word that returns the address of the next available memory location in the user area.

```
' CREATE              \ Get the address of the regular CREATE for ,-X in this
: CREATE    ( -- )    \    CREATE to avoid recursion.
   [ ,-X ]            \ Compile addr of the regular CREATE
   #USER @ ,          \ Compile the user area pointer
   ;USES
   DOUSER-VARIABLE ,-X \ Install DOUSER-VARIABLE as its inner interpreter
```

VARIABLE Define a user variable. Allocate two bytes in the user area. When the user variable is executed, the address of that location in the user area is returned.

```
: VARIABLE  ( -- )
   CREATE        \ New user CREATE just defined
   2 ALLOT   ;       \ User ALLOT
```

DEFER Define an execution vector that is local to every task. The same user deferred word can be vectored to different words in different tasks.

```
: DEFER ( -- )
   VARIABLE          \ Define it as a user variable
   ;USES
   DOUSER-DEFER ,-X       \ Install the user defer inner interpreter
```

5.2. IS AND SELECT

These two words are not related to each other and to the words before and after them. It would probably be a good idea to move these words to Section 5.6 as miscellaneous utility words. However, in order not to disturb the loading sequence of words in the F-PC kernel, a little section is opened here for them.

IS is used to assign an actual function to a deferred word. It is built so that it can be used interactively or in a colon definition. It is very convenient to the user but not quite along the thinking of Forth-83, which demanded the separation of interpretive behavior and compiling behavior into separate words like ' and ['] . Since IS is not a Forth-83 standard word, it is exempted from this restriction.

SELECT is used to define words to select different drives in the DOS system.

>IS Map a code field into a parameter field. If the word is in the USER class of words, then the parameter field address must be calculated relative to the current user pointer. Otherwise it just returns the parameter field address.

```
FORTH DEFINITIONS          \ Return from USER to compile FORTH

META IN-META           \   words to the target system


: >IS      ( cfa -- pfa )

   DUP 1+ @           \ Fetch the relative jump address

   OVER >BODY +       \ Compute the absolute jump address

   DUP [  [ASSEMBLER] DOUSER-VARIABLE META ] LITERAL = SWAP

               \ Is it a user variable?

   DUP [  [ASSEMBLER] DOUSER-DEFER    META ] LITERAL = SWAP

   DROP OR        \ Or perhaps a user deferred word?

   IF   >BODY @ UP @ +     \ Either way, find the actual address in the user area

   ELSE    >BODY      \ Otherwise, move to parameter field

   THEN   ;
```

(IS) The code compiled by IS. Set the execution address of the following DEFERred word to the address on the data stack.

```
: (IS)     ( cfa --- )

   2R@  @L        \ Fetch cfa of next word in list

   >IS !          \ Store cfa in its parameter field

   R> 2+ >R   ;          \ Bump IP, which passes the next word
```

IS Depending on STATE, either set the following DEFERred word immediately or compile the setting for later resolution.

```
: IS       ( cfa --- )

   STATE @        \ If compiling,

   IF  COMPILE (IS)       \   compile (IS) which will resolve next word at run-time.

   ELSE  ' >IS !          \ Else, resolve it now.
```

```
   THEN ; IMMEDIATE
```

SELECT Select drive n as the current disk drive to use as the DEFAULT drive when no drive is specified. N ranges from 0 which is drive A:, to a legal value up to 255.

```
: SELECT    ( n -- )
   14 bdos drop       \ Select drive
   SEQHANDLE >hndle @ -2 = \ If Handle field in current file is -2,
   if -1 SEQUANDLE >hndle !    \   change it to -1.
   then    ;

: A:            ( --- )          0 SELECT ;
: B:         ( --- )          1 SELECT ;
: C:            ( --- )          2 SELECT ;
: D:            ( --- )          3 SELECT ;
```

Select drive A:, B:, C:, or D: as the default drive.

5.3. F-PC BOOT SYSTEM

The operating system of F-PC is the Text Interpreter INTERPRET, which accepts user commands from a keyboard and executes them in an input-execute infinite loop. The Boot System is the part of the code which takes the object image in a DOS .EXE file, loads it into memory, and sets up things so that the F-PC Text Interpreter can be brought up alive. The Head and List Segments must be moved to their appropriate locations in the DOS memory map. All registers, variables, and vectored words must be initialized with their proper values. All these activities must be performed at the machine code level until the Forth word COLD is executed, which brings up the Text Interpreter. This section contains all the high level words taking F-PC from COLD to INTERPRET. In the next section we will see all the low level code needed to boot F-PC up to COLD.

Another important task at boot time is to set up the interrupt system so that DOS and F-PC can pass control from one to the other smoothly. F-PC thus has to save the Control Break and Divide by 0 interrupt vectors and install its own interrupt handlers. The DOS interrupt handlers must be restored before passing control back to DOS. These things if

handled improperly will prevent an application from being smoothly integrated into the DOS environment. F-PC is very careful in this respect. The boot-up code is thus worthy of studying to learn how to do this DOS integration.

QUIT The main loop in Forth. Get more input from the terminal and interpret it. Respond with 'ok' if all input commands are executed successfully. This is often called the Forth Text Interpreter Loop, where user gives commands via the terminal keyboard. The Text Interpreter carries out these commands and comes back for more. Forth is running this loop until the power is turned off or it is forced to execute a BYE command to go back to DOS.

```
: QUIT     ( -- )
   SP0 @ 'TIB !          \ Select terminal input buffer
   [COMPILE] [           \ Start interpreting
   BEGIN                 \ Outer loop
      BEGIN              \ Inner loop
         RP0 @ RP!       \ Initialize the return stack
         STATUS          \ Display system status
         QUERY           \ Wait for a line of commands
         RUN             \ Execute them
         STATE @ NOT     \ If still interpreting,
      UNTIL              \   exit to the outer loop.
                         \   Remain in inner loop to do compiling.
      ."  ok"            \ Display 'ok' message and
   AGAIN  ;              \   loop back again.
```

BOOT A high level word executed during cold start. User may install functions into BOOT that he wants done when F-PC is loaded.

```
DEFER BOOT               \ Vectored to START in Sec. 5.7.
```

INITSTUFF A deferred word chain, used by many system utilities that need to specify a function to be performed at cold start time.

```
DEFER INITSTUFF   ' SEQINIT IS INITSTUFF     \ See Sec. 6.5 for SEQINIT.
```

SEGSET A deferred word that contains the current function used to set up the segment registers at cold start time. It typically contains SETYSEG.

```
DEFER SEGSET     ' SETYSEG IS SEGSET
```

WARMSTRT The default function to be performed on a WARM start. This word is plugged into the deferred word WARMFUNC, to specify what is done when the CONTROL BREAK key is pressed. See also WARMFUNC and WARM below.

```
: WARMSTRT  ( -- )
   FORTH          \ Select FORTH as the context vocabulary
   TRUE ABORT" Warm Start" \ Abort
   ;
```

WARMFUNC A deferred word that is invoked when a warm start occurs. This function is also called whenever the CONTROL BREAK key is pressed.

```
DEFER WARMFUNC  ' WARMSTRT IS WARMFUNC
```

WARM The WARM entry point for Forth. It simply calls the deferred word WARMFUNC. A warm start is invoked whenever the CONTROL BREAK key is pressed.

```
: WARM  ( -- )
   [ LABEL WARMBODY ]  \ Reference point for warm start for assembly code
   WARMFUNC ;         \ Do all the warm start functions
```

COLD The high level cold start code. To activate Forth, BOOT should initialize the virtual Forth Machine and pass control to QUIT.

```
: COLD     ( -- )
   [ LABEL COLDBODY ]     \ Reference point of cold start for assembly code
```

```
  1STCODE       \ Only do this stuff once on cold start
  IF  SEGSET    \ Initialize the segments.  If heads and lists are not moved
          \   to proper locations, move them.
    VMODE.SET     \ Select the CRT display mode
    INITSTUFF     \ Do other initiation functions
  THEN
  %OFF> 1STCOLD     \ Turn value in 1STCOLD off
  BOOT              \ Initialize Forth registers and user initiation work
  QUIT  ;           \ Fall into Text Interpreter
```

5.4. F-PC INITIALIZATION

The initialization process consists of all the code from the JMP CORIG at the very beginning of the F-PC code to the high level word COLD. This process in F-PC is more complicated than other PC Forth systems because of the segregation of lists and heads from the code and data. The code also provides much insight into how F-PC is constructed and some of its unique characteristics.

START Initialize the user variables needed by the Text Interpreter. BOOT is vectored to START. (See Sec. 5.7).

```
: START ( -- )
  SP0 @ 'TIB !          \ Terminal Input Buffer is placed above the data stack.
  >IN OFF       \ Clear interpreter pointer
  SPAN OFF          \ Clear input character count
  #TIB OFF          \ Clear input buffer length
  LOADING OFF       \ Clear file loading flag
  DEFAULT       \ Handle command line input from DOS
  INTERPRET ;           \ Execute user commands
```

RESTORE_VECTORS Restore the CONTROL BREAK and DIVIDE by 0 DOS interrupt vectors to their original value as when Forth was first entered. The CONTROL BREAK and DIVIDE by 0 vectors are saved by the assembly language cold start routines before Forth is entered.

```
HEX

VARIABLE BIOSBKSAVE  0 ,-T      \ Save DOS control break vector

VARIABLE DIV0SAVE    0 ,-T      \ Save DOS divide by 0 vector


CODE RESTORE_VECTORS    ( --- ) \ Restores the saved vectors to their saved values.

                \ This word is called just prior to returning to DOS.

   MOV AX, CS         \ Make DS=CS

   MOV DS, AX

   MOV DX, CS: BIOSBKSAVE  \ Get CS part of saved vector

   MOV DS, CS: BIOSBKSAVE 2+ \ Get the PC part of saved vector

   MOV AX, # 251B      \ Ask DOS to restore the control

   INT 21              \   break interrupt vector

   MOV AX, CS         \ Re-initialize DS to CS

   MOV DS, AX

   MOV DX, CS: DIV0SAVE    \ Get saved CS

   MOV DS, CS: DIV0SAVE 2+ \ Get saved PC

   MOV AX, # 2500      \ Ask DOS to restore divide by 0

   INT 21              \   interrupt

   MOV AX, CS         \ Restore DS again

   MOV DS, AX

   NEXT

   END-CODE
```

DIV0STRT The default function to perform when a DIVIDE BY 0 trap occurs. This routine does an ABORT. A divide by 0 trap calls DIV0FUNC, which is vectored to this routine.

```
: DIV0STRT  ( -- )
```

```
   TRUE ABORT" Divide OVERFLOW error" ;
```

DIV0FUNC Divide by 0 error causes a trap to the interrupt vector 0, which calls this deferred word. You can change the contents of this deferred word to handle divide by 0 errors in your own program.

```
DEFER DIV0FUNC  ' DIV0STRT IS DIV0FUNC
```

BYEFUNC A deferred word which normally contains NOOP, provided a mechanism for you to specify functions to be performed before leaving F-PC back to DOS.

```
DEFER BYEFUNC   ' NOOP IS BYEFUNC
```

BYE Return control to DOS. Perform the deferred word BYEFUNC and restore the control break and divide by 0 interrupt vectors before actually leaving.

```
: BYE     ( -- )
   RESTORE_VECTORS
   BYEFUNC        \ Bye task chain
   ." Leaving.." CR
   0 0 BDOS  ;        \ Exit to DOS
```

DIVIDE0 The actual entry point of the divide by 0 trap. This word just calls the deferred word DIV0FUNC. Normally the registers on the stack are simply discarded, but you can install your own routine into DIV0FUNC to handle the divide by 0 error.

```
: DIVIDE0   ( STATUS_reg, CS, IP, AX, BX, CX, DX, SI, BP --- )
   [ LABEL DIV0BODY ]
   DIV0FUNC          \ Abort to QUIT as default
   BYE ;             \ This word is generally not seen by F-PC, unless you
              \   re-vectored DIV0FUNC.
```

```
LABEL DIV0BK          \ Handle a Divide by 0 interrupt
   STI          \ Set flag to enable interrupts
   PUSH AX       \ Push all relevant registers
   PUSH BX       \   on the data stack
   PUSH CX
   PUSH DX
   PUSH SI
   PUSH DI
   PUSH BP
   MOV AX, # DIV0BODY 5 -  \ Jump to DIVIDE0 above the F-PC way.
   JMP AX
   END-CODE

TRUE VALUE RESTNEXT     \ A flag to determine if we want the next code restored.

LABEL SETBRK          \ Set control break to do warm start
   PUSH ES       \ Save ES.  Need it for list addressing
   MOV AX, CS        \ DS=CS
   MOV DS, AX
   CMP ' RESTNEXT >BODY # 0 WORD   \ If RESTNEXT is NOT = 0
   0<> IF  MOV AX, # AD26  \ LODSW ES: in >NEXT
      MOV >NEXT AX    \ Restore it
      MOV AX, # E0FF  \ JMP AX    in >NEXT + 2
      MOV >NEXT 2+ AX \ Restore it
   THEN
   MOV DX, # BIOSBK
   MOV AX, # 251B      \ Install BIOS Break 1B
   INT 21
   MOV DX, # DOSBK
```

```
   MOV AX, # 2523      \ Install DOS Break 23
   INT 21
   MOV DX, # 1
   MOV AX, # 3301      \ Enable DOS Break
   INT 21
   MOV DX, # DIV0BK
   MOV AX, # 2500      \ Install DIV by 0 Break 0
   INT 21
   POP ES         \ Restore ES
   RET            \ Done
   END-CODE

LABEL SAVEVECTORS      \ Save Divide by 0 & Ctrl Brk vectors for now.
   PUSH ES
   MOV AX, # 351B      \ Get the interrupt vector for
   INT 21            \   BIOS control break vector
   MOV BIOSBKSAVE BX      \ Save PC vector
   MOV BIOSBKSAVE 2+ ES    \ Save CS vector
   MOV AX, # 3500      \ Get the interrupt vector for
   INT 21            \   DIVIDE by 0
   MOV DIV0SAVE BX     \ Save PC vector
   MOV DIV0SAVE 2+ ES      \ Save CS vector
   POP ES
   MOV AX, # $3300     \ Control BREAK flag status
   NT $21
   SUB DH, DH
   MOV CTRLBKSAVE DX   \ Save it away for later restoration
   RET
   END-CODE
```

SET_VECTORS Set the CONTROL-BREAK and DIVIDE-BY-0 traps to point to the functions provided by F-PC, so we can handle them smoothly.

```
DECIMAL

CODE SET_VECTORS    ( --- )

   CALL SETBRK        \ Define as a subroutine

   NEXT

   END-CODE


CODE @REL>ABS   ( a1 --- a2 )   \ Convert JMP address in a1+1 to an absolute memory address

   POP BX

   ADD BX, 1 [BX]

   ADD BX, # 3

   PUSH BX

   NEXT

   END-CODE
```

WORIG The warm start routine which is entered when the user pushes the Control Break key on the keyboard. This generally allows F-PC to recover if it is trapped in an infinite loop or doing some very time-consuming task. The warm start can recover only if the F-PC system is intact. If the List Segment or the Code Segment were written over, F-PC would not be able to recover. If the Head Segment was written over, F-PC will still respond to carriage-returns, but it will not be able to find words in the dictionary. Under these circumstances, using Alt-Ctrl-Del or hitting the power switch are the only alternatives.

```
[FORTH] ASSEMBLER

LABEL WORIG

   HERE ORIGIN 6 + -       \ Patch ORIGIN with warm start

   ORIGIN 4 + !-T          \   address

   MOV AX, # WARMBODY 5 -  \ Jump to WARMBODY

   JMP AX

   END-CODE
```

CORIG The cold start routine which builds the F-PC from the loaded target core image. The Head dictionary is moved to the Head Segment and the List dictionary is moved to the List Segment. All Forth registers and pertinent user and system variables are initialized. Finally, COLD is executed and the Forth Text Interpreter is invoked to take commands from the user.

```
LABEL CORIG           \ Cold start routine
  HERE ORIGIN 3 + -      \ Patch ORIGIN with cold start
  ORIGIN 1+ !-T          \   address
  MOV AX, CS         \ Initialize DS and SS segment registers to CS
  MOV DS, AX
  MOV SS, AX
  MOV BX, YSTART     \ Read YSTART the relocate flag
  OR BX, BX          \ If not reset, then move head stuff
  0<> IF
    ADD AX, ' #CODESEGS >BODY   \ Add Code segments and
    ADD AX, ' #LISTSEGS >BODY      \   List segments to get to head space.
    MOV ES, AX     \ Move Head segment to ES
    MOV CX, YDP    \ Bytes in head dictionary
    MOV DI, # 0    \ Clear DI
    MOV SI, YSTART  \ Move YSTART to SI
    OR CX, CX 0<>   \ If YDP was not zero (0)
    IF  CLD
      REPZ
      MOVSB   \ Move heads to Head Space
      CLD
    THEN
    MOV YSEG ES    \ Initialize YSEG to base of Head Space
  THEN
  MOV BX, XMOVED     \ Has LIST been moved?
  OR BX, BX          \ If not, then move List Segments.
```

```
    0= IF
      MOV AX, DS      \ Copy DS (CS) to AX
      ADD AX, ' #CODESEGS >BODY   \ Add Code Segments
      MOV ES, AX      \ Copy Head Segment base to ES
      MOV CX, XSEGLEN \ Segments of list dictionary
      SHL CX, # 1     \ Multiply by 16 decimal
      SHL CX, # 1
      SHL CX, # 1
      SHL CX, # 1
      MOV DI, # 0     \ Clear DI
      MOV SI, DPSTART \ MOV source offset to SI
      OR CX, CX       \ if head length was not zero (0)
      0< IF   CLD     \ Forward move
        REPZ
        MOVSB   \ move list to List Space
        CLD
      THEN
      MOV XSEG ES     \ set XSEG base of List Segment
    THEN
\ The following few instructions patch two ADD instructions in KERNEL1, so
\ we can do an ADD IMMEDIATE rather than an ADD MEMORY in NEST and DODOES.
    MOV ES, XSEG        \ Initialize ES in case we haven't??
    MOV NESTPATCH 1+ ES \ Patch ADD instruction in NEXT
    MOV DOESPATCH 1+ ES \ Patch ADD instruction in DODES
    CALL SAVEVECTORS        \ Save existing vectors
    MOV ' RESTNEXT >BODY # TRUE WORD    \ We want NEXT restored
    CALL SETBRK         \ Install F-PC Break vectors and restore NEXT
    MOV AX, ' #CODESEGS >BODY   \ Length of code dictionary
    SUB AX, # 1         \ One less than max
```

```
SHL AX, # 1        \ * 16 to length in bytes
SHL AX, # 1
SHL AX, # 1
SHL AX, # 1
MOV ' LIMIT 3 + AX      \ Initialize LIMIT
SUB AX, # 10
MOV ' FIRST 3 + AX      \ FIRST = LIMIT - 10h
SUB AX, # 10
MOV RP, AX         \ RP = FIRST - 10h
MOV BX, # RP0
ADD BX, UP         \ Add user area
MOV 0 [BX], RP      \ RP0 = RP
SUB AX, # 200
MOV 'TIB AX        \ TIB = RP - 250 decimal
MOV BX, # SP0       \ Stack pointer
ADD BX, UP
MOV 0 [BX], AX      \ SP0 = TIB
MOV SP, AX         \ SP = TIB
MOV ' 1STCOLD >BODY # TRUE WORD     \ Make COLD to do its initialization
MOV AX, COLDBODY 2- \ Cold start routine
ADD AX, XSEG        \ Add List Segment base
MOV ES, AX         \ Execute COLDBODY as a colon word
MOV IP, # 0
NEXT
END-CODE
```

5.5. INITIALIZE THE USER AREA

Here we initialize all the USER variables with their default values.

```
IN-META         \ Meta compiler environment

HERE UP !-T     \ UP is initialized to HERE, where the user variables of the system     \  task is set up using values known so far.

   0 ,          \ TOS      Save data stack pointer before task switching

   0 ,          \ ENTRY Switch task on or off

   0 ,          \ LINK     Address of the next task

   0 ,          \ ES0      Base address of List Segment

   INIT-R0 256 - ,    \ SP0      Bottom of data stack

   INIT-R0 ,      \ RP0      Bottom of return stack

   0 ,        \ DP       Top of code dictionary

   0 ,        \ OFFSET   Not used in F-PC

   10 ,           \ BASE Number conversion base

   0 ,        \ HLD      Output string pointer

   FALSE ,    \ PRINTING Flag for printer

   ' (EMIT) ,     \ EMIT Send character to console

   ' (KEY?) ,     \ KEY? Test keyboard activity

   ' (KEY)  ,     \ KEY  Receive character from console

   ' (TYPE) ,     \ TYPE Send string to console

   ' (TYPEL) ,    \ TYPEL    Send string from any segment to console

   0 , 0 , 0 , 0 , 0 ,    \ Room for 10 additional

   0 , 0 , 0 , 0 , 0 ,    \  USER variables
```

5.6. DEBUGGING WORDS

Here is a small set of tools which allows you to exercise the F-PC kernel system if you modified the kernel and have trouble in loading the full F-PC system with all the sophisticated debugging tools. You can look at memory contents, display word names, examine the data stack, etc. They are better than nothing when you have problems, and are generally powerful enough to solve most problems.

DEPTH Return the number of items on the data stack not including itself.

```
: DEPTH ( -- n )
   SP@              \ Current stack pointer
   SP0 @            \ Bottom of data stack
   SWAP - 2/  ;         \ Difference in cells
```

VARIABLE MAX.S Maximum items on the stack to be displayed by .S . Initialized to 4.

.S Display the contents of the data stack non- destructively. Very useful when debugging.

```
: .S      ( -- )
   DEPTH 0<          \ Abort if stack underflow
   ABORT" Stack UNDERFLOW !! "
   DEPTH ?DUP        \ If stack is not empty,
   MAX.S @ 1 <       \ If MAX.S is less than 1,
   IF 4 MAX.S ! THEN      \   initialize it to 4.
   IF  DUP ." ["     \   display stack depth,
     1 .R ." ]"
     0 SWAP 1-
     MAX.S @ 1- MIN  \   and up to MAX.S items
     DO I PICK 7 U.R   \   on the top of the data
        SPACE  \   stack.
     -1 +LOOP
   ELSE   ." Stack Empty. "     \ Else, say so.
   THEN ;
```

.ID A deferred word. Display the variable length name whose name field address is on the stack. If the name is shorter than its count, it is padded with underscores. Only valid ASCII character is typed.

```
: %.ID     ( nfa -- )
   DUP 1+         \ nfa nfa+1 --
   DUP YC@            \ Get first character of name
   ROT YC@            \ Get the count byte
   31 AND 0           \ Mask off irrelevant bits
   ?DO DUP 127 AND FEMIT   \ Display one character
      128 AND     \ Is it the last character in name?
      IF  ASCII _ 128 OR  \ Yes.  Leave pad character _ .
      ELSE    1+ DUP YC@  \ No.  Get the next character.
      THEN
   LOOP              \ Display as many as count.
   2DROP SPACE ;       \ trailing blank

DEFER .ID   ' %.ID  IS  ID      \ Deferred to allow for colorizer to display names in color.
```

DUMP A primitive little dump routine to help you debug after you have changed the system source and nothing works any more. A more elaborate DUMP is defined in DUMP.SEQ.

```
: DUMP  ( addr len -- )
   0             \ Do len times
   DO  CR DUP 6 .R SPACE   \ Display address
      16 0          \ Dump 16 bytes to a line
      DO  DUP C@ 3 .R 1+  \ Display bytes
      LOOP
   16 +LOOP   DROP   ;     \ Next line
```

RECURSE Force the compilation of the word under construction. We prefer that you will use RECURSIVE rather than RECURSE. (See RECURSIVE)

```
: RECURSE   ( -- )
   LAST @         \ Get the name field address from LAST.
   NAME> X,       \ Compile its cfa into list
   ;  IMMEDIATE   \ Must be executed during compilation
```

H. Display the unsigned number in hex, with trailing blank. It does not change the number base.

```
: H.       ( n -- )
   BASE @ >R         \ Save BASE
   HEX U.            \ Display n in hex
   R> BASE ! ;       \ Restore base
```

LMARGIN The left margin setting used by ?LINE and ?CR. When a line-wrap occurs, LMARGIN specifies how many spaces are printed on the following line. Default value is 0.

```
VARIABLE LMARGIN    0 LMARGIN !-T
```

RMARGIN Control the right margin, used by ?LINE and ?CR. It specifies where to wrap the line. Default value is 70.

```
VARIABLE RMARGIN   70 RMARGIN !-T
```

TABSIZE Control the TAB increment for TAB. Default is 8.

```
VARIABLE TABSIZE    8 TABSIZE !-T
```

?LINE Break the line at the cursor if there are less than n characters till RMARGIN is encountered.

```
: ?LINE ( n -- )
   #OUT @ +          \ Add n to output character counter
   RMARGIN @ >     \ If it exceeds RMARGIN,
   IF CR        \   do carriage-return and
      LMARGIN @ SPACES    \   skip to left margin.
   THEN ;
```

?CR Break the line at the cursor, if we have reached the right margin as specified by RMARGIN.

```
: ?CR     ( -- )  0 ?LINE  ;
```

TAB Print spaces to get to the next TAB increment as specified by TABSIZE.

```
: TAB     ( --- )
   #OUT @        \ Output character count
   TABSIZE @ MOD      \ Characters printed after TAB
   TABSIZE @
   SWAP - SPACES ;     \ Pad spaces till next tab.
```

\ Comment till the end of this line.

```
: \    ( --- )
   SPAN @        \ Move interpreter point >IN to
   >IN !           \   end of input buffer, and
   ; IMMEDIATE       \   ignoring all characters in between.
```

```
CODE SET-CURSOR ( n1 --- )     \ Set the cursor shape to value n1
   POP CX
   MOV AH, # 1
```

```
   PUSH SI
   PUSH BP
   INT $10        \ Call DOS to deposit the color code
   POP BP
   POP SI
   NEXT C;

: GET-CURSOR    ( --- n1 )          \ return n1 the shape of the cursor.
   0 $460 @L ;
```

5.7. RESOLVING FORWARD REFERENCES AND DEFERRED WORDS

Here is the list of all the forward references and deferred words which were not already resolved or vectored to executable words. This list usually does not need your attention unless you made substantial modifications to the F-PC kernel. The word is in this list are also the most difficult ones to understand. Most of them are inner interpreters to classes of words generated by defining words. The defining words are very difficult to handle in meta compilation. Each defining word must be hand coded. When you are developing an application and intend it to be meta compiled, it is best to avoid using defining words if you can help it. If you have to use them, then study the F-PC meta compiler and this kernel carefully. Try to mimic the way F-PC handles these defining words.

:RESOLVES resolves forward references in the List space, while RESOLVES resolves forward reverences in the Code space. It does not matter whether the word you are resolving is a CODE word or a colon definitions, what matters is where it is being resolved, typically in the List space but not the Code space. All this is to say you should normally use :RESOLVES rather than RESOLVES to resolve forward reverences.

```
' (.")     :RESOLVES <(.")>
' (")      :RESOLVES <(")>
' (;CODE)   :RESOLVES <(;CODE)>
' (;USES)   :RESOLVES <(;USES)>
' (IS)        :RESOLVES <(IS)>
' (ABORT") :RESOLVES <(ABORT")>
```

```
[ASSEMBLER]   >NEXT    META            RESOLVES <VARIABLE>
[ASSEMBLER]   DOUSER-DEFER META       RESOLVES <USER-DEFER>
[ASSEMBLER]   DOUSER-VARIABLE META    RESOLVES <USER-VARIABLE>

' DEFINITIONS :RESOLVES DEFINITIONS
' [           :RESOLVES [
' ?MISSING     :RESOLVES ?MISSING
' QUIT         :RESOLVES QUIT
' .ID          :RESOLVES .ID
' @REL>ABS  :RESOLVES @REL>ABS
' >IS      :RESOLVES >IS

\ Fill in some deferred words
' CRLF            IS CR
' CR              IS STATUS
' START           IS BOOT
' (PRINT)         IS PEMIT
' (CONSOLE)     IS CONSOLE
```

System variables are hot patched using the target store word !-T.

```
' FORTH >BODY-T CURRENT !-T \ Make FORTH vocabulary both
' FORTH >BODY-T CONTEXT !-T \   context and current.

HERE-T  DP UP @-T + !-T     \ User DP
#USER-T @ #USER !-T              \ Number of user variables
TRUE  CAPS !-T         \ Case insensitive
TRUE WARNING !-T           \ Display error messages
```

```
31 WIDTH !-T              \ Allow 31 character names

VOC-LINK-T @ VOC-LINK !-T       \ Vocabulary link
```

5.8. SAVING F-PC KERNEL

Following is a list of words executed after the meta compilation is completed. They display a set of system parameters and other information relevant to the meta compilation and to the target Forth system, like the sizes of various segments. After that, the object image of the resulting Forth kernel is saved in a file KERNEL.EXE. This is a minimally useful F-PC system, which serves as a basis to build useful applications. One of the very useful application is the full F-PC system, which can be constructed by type EXTEND under the DOS prompt. EXTEND compiles a set of utility files on top of the F-PC kernel system to give the user a complete Forth programming environment in which very large application can be coded and debugged.

```
CR .( Unresolved references: )         \ Forward references

CR   .UNRESOLVED ?NEWPAGE

CR .(     Statistics: )           \ Code sizes

CR .( Last  Host Address:       )

   [FORTH] HERE U.

CR .( First Target Code Address: )

   META 256 THERE U.

CR .( Last  Target Code Address: )

   META HERE-T THERE U.

   META 256 THERE               \ Starting address

   SVXSEG     DPSTART !-T      \ Base List Segment

   HERE-X DROP 1+

   0 XS: DROP - XSEGLEN !-T       \ List Segment length

CR .( CODE space used:          )

   HERE-T U.             \ Code Space

CR .( LIST space used:          )

   HERE-X SWAP 0 XS: DROP - 16 * + U.  \ List Space
```

```
CR .( HEAD space used:          )
   HERE-Y U.              \ Head Space
   HERE-X DROP 1+ 0 XS: DROP -
   DUP 16 * ALLOT-T DROP       \ Allocate list space in  target
   0 XDP !-T              \ Clear list dict. ptr.
   SVYSEG DUP YSTART !-T       \ Base head segment
   0 XMOVED !-T               \ List must be moved
   HERE-Y +
   HERE-Y YDP !-T         \ Head dictionary pointer
   THERE ONLY FORTH ALSO
CR .( Free Target Program room: )  U.
    SP@ HERE -
CR .( Free Symbol Table bytes:   ) U.
.COMPSTAT                  \ Print compiler status
( A1 N1 --- )   ZSAVE KERNEL.COM        \ Copy target image to file
ONLY FORTH ALSO DEFINITIONS
CR .( Now type EXTEND <enter> at the DOS prompt.)
CR
```

CHAPTER 6. KERNEL UTILITIES

Kernel Utilities are words loaded into the F-PC Kernel with the KERNEL1-4.SEQ files. They provide I/O and file words needed by the Kernel but generally not considered as part of the Kernel because they are specific to the host computer and the underlying operating system. Since they are loaded from a separate set of files, it is convenient to group them in one place.

The loading sequence of these files in building the F-PC Kernel is shown at the end of the META86.SEQ file:

```
FLOAD KERNEL1.SEQ
FLOAD VIDEO.SEQ
FLOAD KERNEL2.SEQ
FLOAD VIDEO2.SEQ
FLOAD KERNEL3.SEQ
FLOAD EQUCOLON.SEQ
FLOAD SAVEREST.SEQ
FLOAD HANDLES.SEQ
FLOAD SEQREAD.SEQ
FLOAD FPATH.SEQ
FLOAD DEFAULT.SEQ
FLOAD KERNEL4.SEQ
```

The discussion in this Chapter follows the above loading order except the KERNEL files, which were discussed in the previous two Chapters.

6.1. VIDEO OUTPUT WORDS

VIDEO1.SEQ and VIDEO2.SEQ contain the word set which allow F-PC to access the video displays in an PC or its clones. One of the problems is that there are so many different video display cards for PC. F-PC tries its best to identify the video card used in the system and to deal with it appropriately.

VIDEO-SEG A variable that holds the segment address of the source from where text is sent to the output devices. It is normally set to the value of ?CS: to display text stored in the Code Segment, but the editor changes it to display text from an external edit buffer segment. It is defined here with other variables and values controlling various attributes of the video display:

```
VARIABLE ATTRIB   7 ATTRIB !-T          \ display attribute value
VARIABLE VIDEO-SEG                      \ Segment of display
VARIABLE VMODE-VAR              \ Video mode save variable
VARIABLE BLANKING         \ Are we blanking during screen drawing?
0 VALUE COLS              \ Screen columns
0 VALUE ROWS                \ Screen rows
0 VALUE CROWS           \ Cursor DOT rows
```

?VMODE Read the current video mode from the BIOS. A value of 7 is normal for monochrome card, and a value of 2 is for a CGA card.

```
CODE ?VMODE ( -- n )          \ Get the video mode from DOS
   MOV AH, # $0F
   INT $10
   SUB AH, AH
   MOV VMODE-VAR AX             \ Save video mode in VMODE-VAR
   PUSH AX
   MOV AX, # $40                 \ absolute segment $40:00
   MOV DS, AX                   \ goes into DS
   MOV BL, $4A                   \ get the # of COLS from    $40:$4A
```

```
   MOV CL, $84                  \ get the # of ROWS FROM    $40:$84
   MOV BH, $60                   \ get the Cursor bottom row $40:$60
   MOV AX, CS                   \ restore DS from CS
   MOV DS, AX
   MOV ' CROWS >BODY BH
   MOV ' COLS  >BODY BL
   CMP CL, # 24
   < IF    MOV CL, # 25          \ At least 25 rows
   ELSE    INC CL
   THEN
   MOV ' ROWS >BODY CL
   NEXT
   END-CODE
```

INITMONO A deferred word used to initialize the monochrome display attribute by setting the >ATTRIBn words. See the file MONOCROM.SEQ.

INITCOLOR A deferred used to initialize the color display attribute by setting the >ATTRIBn words. See the file MONOCROM.SEQ, and COLOR.SEQ.

```
DEFER INITMONO     ' NOOP IS INITMONO
DEFER INITCOLOR    ' NOOP IS INITCOLOR
```

VMODE.SET Set the VIDEO-SEG variable after testing the current video mode, and perform any needed initialization required by executing the INITMONO, or INITCOLOR as needed.

```
HEX
: VMODE.SET ( -- )
   ?VMODE 7
   IF  0B000   INITMONO       \ Monochrome text buffer
   ELSE    0B800   INITCOLOR      \ Color card text buffer
```

```
   THEN    VIDEO-SEG ! ;
DECIMAL
```

CGA video cards were the oldest video card available on IBM-PC's. It is extremely dumb and needs lots of software attentions. You have to wait for vertical retrace interval to write characters into the video memory buffer. Otherwise, you will cause snow noise to appear on the screen. Other more advanced video displays do not have this problem. BLANK.COLOR will write to CGA only during vertical retrace.

```
CODE BLANK.COLOR    ( -- )              \ Blanks screen if in COLOR mode
   MOV BX, # 1                      \ Bit mask for retrace
   MOV AX, VMODE-VAR
   CMP AX, # 7                      \ If video mode <> 7 (mono)
   0<> IF  MOV AX, BLANKING
      OR AX, AX
      0<> IF  MOV DX, # 986   \ CGA status port
         BEGIN   IN AL, DX   \ Wait for retrace
            TEST AL, BX
         0<> UNTIL
         MOV DL, # 216       \ Address control reg
         MOV AL, # 37        \ Disable the CRT
         OUT DX, AL
      THEN
   THEN
   next
   end-code

CODE SHOW.COLOR     ( -- )              \ Re-enable the display
   MOV AX, VMODE-VAR
   CMP AX, # 7
   0<> IF  MOV AX, BLANKING
      OR AX, AX
```

```
    0<> IF  MOV DX, # 984          \ CGA mode control reg
      MOV AL, # $2D          \ Enable byte for mode 3
      OUT DX, AL
    THEN
  THEN
  NEXT
  END-CODE
```

VARIABLE NOSETCUR Set to zero if we are writing a new line. Do not waste time in asking BIOS where the cursor is.

VIDO-TYPEL Send a string at seg:addr to the video screen. It is optimized for speed with some penalty in memory. It is lightening fast though. The actual code was modified by Robert Berkey to access segmented memory and is lightening fast.

```
CODE VIDEO-TYPEL    ( seg str-addr length -- )
  MOV AX, #LINE
  MOV DX, #OUT
  POP BX
  cmp bx, # 0                    \ if length = 0 then exit
  <= if   add sp, # 4
    next
  then
  PUSH BX                        \ Put length and #OUT on stack
  PUSH DX
   cmp ax, ' rows >body              \ Clip #LINE to ROWS
  u>= if  mov ax, ' rows >body
     dec ax
  then
  mov #line ax
  PUSH AX
```

```
ADD DX, BX                    \ Set length + X position to DL
cmp dx, ' cols >body          \ Clip #OUT to 80 characters or whatever
u>= if  mov dx, ' cols >body
   dec dx
then
mov #out dx
cmp nosetcur # 0              \ Do we want to spend time setting
0= if                         \ the cursor position?
   MOV DH, AL                 \ Move Y to DH
   XOR BX, BX                 \ Clear BX
   MOV AH, # 2
   PUSH SI       PUSH BP
   INT $10                    \ Move cursor to end of typed string
   POP BP        POP SI
then
MOV AX, ' cols >body          \ calculate actual cursor position
ADD AX, AX                    \ 2 bytes per character
POP BX        MUL BX          MOV DI, AX
POP AX        SHL AX, # 1     ADD DI, AX
POP CX        POP BX
POP DX  ( ds seg )
XCHG SI, BX                   \ SI is the source
PUSH BX                       \ Save SI for later restoration
PUSH ES                       \ save ES
PUSH DX ( ds seg )
MOV BX, # 1                   \ Bit mask for retrace
 MOV AX, VMODE-VAR
CMP AX, # 7                   \ If video mode <> 7 (mono)
0<> IF  MOV AX, BLANKING
```

```
    OR AX, AX
    0<> IF  MOV DX, # 986       \ CGA status port
      BEGIN   IN AL, DX       \ Wait for retrace
        TEST AL, BX
      0<> UNTIL
      MOV DL, # 216   \ Address control reg
      MOV AL, # 37    \ Disable the CRT
      OUT DX, AL
    THEN
  THEN
  MOV AH, ATTRIB          \ display attributes
  MOV ES, VIDEO-SEG          \ destination segment
  cmp cx, ' cols >body
  u>= if                        \ Clip line to COL chars
    mov cx, ' cols >body
  then
  mov dx, # 132             \ ABSOLUTE limit of output
  sub dx, cx
  xchg dx, cx
  shl cx, # 1
  POP DS
  add cx, # here $06 +            \ Add base addr of array
  jmp cx                      \ JUMP to the right instruction
```

We are blowing out the loop here for speed: it only cost 160 bytes. This is what you do when memory is cheap and cycles are expensive.

```
  LODSB   STOSW   LODSB   STOSW   LODSB   STOSW   LODSB   STOSW
  LODSB   STOSW   LODSB   STOSW   LODSB   STOSW   LODSB   STOSW
  LODSB   STOSW   LODSB   STOSW   LODSB   STOSW   LODSB   STOSW
```

```
LODSB  STOSW  LODSB  STOSW  LODSB  STOSW  LODSB  STOSW
```

```
    THEN
  THEN
  NEXT
  END-CODE
```

VIDEO-TYPE Display a string from the Code segment to the video display.

```
CODE VIDEO-TYPE  ( str-addr length -- )
  POP AX
  POP BX
  PUSH CS
  PUSH BX
  PUSH AX
  JMP ' VIDEO-TYPEL
  END-CODE
```

6.2. VALUES: CONSTANTS AS VARIABLES

Source File is EQUCOLON.SEQ. There are two advantages in treating a constant like a variable: the first is that a constant is faster in getting the data because you don't have to first get the address and then fetch it; and the second is that program becomes cleaner and easier to understand by eliminating @'s. Variables are more often fetched than stored. Defining a variable as a constant will thus improve the execution speed, with a small improvement on program size.

However, the Forth-83 Standard clearly dictates that the location where constants are stored is not known to the user, and he should not attempt to modify constants after they are defined. If you like to maintain your program according to Forth-83, skip this section. If you want to really squeeze out all the loose cycles from your PC, you will enjoy reading it and make some good use of this word set.

Many people evaluating F-PC system objected to the practice in changing the values of constants. Tom Zimmer compromised by calling these variable constants VALUEs. VALUE is simply an alias of CONSTANT. All words

operating on a VALUE will operate on a CONSTANT identically. This is not is new practice. European Forth users have been using it for years, but they call it the 'TO' concept. VALUE was defined in Sec. 4.8.

Following are code words which manipulate data stored in the parameter field of the following word in a colon definition. They are very fast because the code field address of the following word can be obtained by a LODSW ES: machine instruction. The data stored in the parameter field can then be accessed by a BX relative mode instruction.

%@> It must be compiled into a colon definition. At run-time, it pushes the value in the parameter field of the following definition onto the data stack.

```
CODE %@>  ( --  n )
   LODSW ES:        \ 12 + 2 cycles. Get cfa of next word.
   MOV BX, AX       \  2    cycles.
   PUSH 3 [BX]      \ 17 + 9 cycles. Fetch data in pfa.
   NEXT
   END-CODE
```

%!> Primitive used by the following definitions. It is compiled into a colon definition, and stores value on the stack into the parameter field of the following definition, which must be a constant, a variable, or a VALUE. Only to be used in a colon definition.

```
CODE %!>  ( n -- )
   LODSW ES:        \ 12 + 2 cycles. Get cfa of next word.
   MOV BX, AX       \  2    cycles.
   POP 3 [BX]       \ 17 + 9 cycles. Store n to its pfa.
   NEXT
   END-CODE
```

```
CODE   %U@>   ( -- n1 )     \ Fetch the DATA field of a USER definition in user space.
   LODSW ES:
   MOV BX, AX
   MOV BX, 3 [BX]
```

```
   ADD BX, UP
   PUSH 0 [BX]
   NEXT
   END-CODE

CODE   %U!>   ( n1 -- )      \ Store into the DATA field of a USER definition in user space.
   LODSW ES:
   MOV BX, AX
   MOV BX, 3 [BX]
   ADD BX, UP
   POP 0 [BX]
   NEXT
   END-CODE
```

%+!> Increment the value stored in the parameter field of the following word by the amount n on the data stack. Used in colon definitions only.

```
CODE %+!>   ( n -- )
   POP CX        \ Increment value
   LODSW ES:
   MOB BX, AX        \ Get pfa of the next word
   ADD 3 [BX] WORD    \   and increment it by n.
   NEXT
   END-CODE
```

I%NCR> Increment contents in the parameter field of the following word by one. It is used to modify the following value, constant or variable. Used in colon definitions only.

```
CODE %INCR> ( -- )
   LODSW ES:         \ Get cfa
```

```
   MOV BX, AX
   INC 3 [BX] WORD     \ Increment parameter
   NEXT
   END-CODE
```

%DECR> Decrement contents in the parameter field of the following word by one. It is used to modify the following value, constant or variable in a colon definition. Used only in a colon definition.

```
CODE %DECR> ( -- )
   LODSW ES:
   MOV BX, AX
   DEC 3 [BX] WORD     \ Decrement parameter field
   NEXT
   END-CODE
```

%OFF> Zero the following value/constant/variable. Somewhat faster than OFF. Store 0 into the parameter field of following definition. Only to be used in a colon definition.

```
CODE %OFF>  ( -- )
   LODSW ES:          \ Get cfa of next word
   MOV BX, AX         \ Move to BX for relative addressing
   MOV 3 [BX], # 0  WORD   \ Store 0 into parameter field
   NEXT
   END-CODE
```

%ON> Turn on the following value/constant/variable. Somewhat faster than <name> ON. Store -1 into the parameter field of the following definition. Only used in a colon definition.

```
CODE %ON>  ( -- )
   LODSW ES:
   MOV BX, AX
```

```
   MOV 3 [BX], # -1 WORD

   NEXT

   END-CODE


CODE   %&>  ( -- a1 )       \ Return the BODY a1 address of the following word in a definition.

   LODSW ES:

   ADD AX, # 3

   1PUSH

   END-CODE
```

Following are high level words operating on VALUEs. They can be used in colon definitions and can also be executed interactively. Used inside a colon definition, they compile low level words shown above. These words can be omitted in a stand-alone application. F-PC 3.5 allows you to set values in user variables as well. To support user variables, %U@>,%U!>, ?USER, and ?NOUSER are added.

```
: ?NOUSER   ( a1 -- a1 )       \ Test a1 to verify it is not a USER definition, if it is, abort.

   DUP >IS OVER >BODY <>

   ABORT" Won't work on USER words." ;


: ?USER ( a1 -- a1 f1 )     \ test a1, is it a user word, return TRUE if it is else FALSE.

   DUP >IS OVER >BODY <> ;
```

@> Fetch the contents from the parameter field of the next word to the stack. May be used outside of a colon definition.

```
: @>  <name>       ( -- n )

   STATE @

   IF     ' ?USER

      IF     COMPILE %U@>

      ELSE    COMPILE %@>

      THEN    X,
```

```
   ELSE   ' >IS @
   THEN   ; IMMEDIATE
```

!> Store the value n on the stack into the parameter field of the following value, constant or variable. Identical to =: . May be used outside of a colon definition.

```
: !>  <name>      ( n -- )
   STATE @
   IF    ' ?USER
     IF    COMPILE %U!>
     ELSE   COMPILE %!>
     THEN   X,
   ELSE   ' >IS !
   THEN   ; IMMEDIATE
```

+!> Increment the value of the following word by n.

```
: +!>  <name>      ( n -- )
   ?NOUSER
   STAT @
   IF  COMPILE %+!>
   ELSE   ' >BODY +!
   THEN   ; IMMEDIATE
```

INCR> Increment the following value by 1.

```
: INCR>  <name> (  -- )
   ?NOUSER
   STATE @
   IF COMPILE %INCR>
```

```
   ELSE    ' BODY INCR
   THEN    ; IMMEDIATE
```

DECR> Decrement the following value by 1.

```
: DECR>  <name> ( -- )
   ?NOUSER
   STATE @
   IF  COMPILE %DECR>
   ELSE    ' BODY DECR
   THEN    ; IMMEDIATE
```

OFF> Clear the following value to 0.

```
: OFF>  <name>  ( -- )
   ?NOUSER
   STATE @
   IF COMPILE %OFF>
   ELSE    ' BODY OFF
   THEN    ; IMMEDIATE
```

ON> Set the following value to TRUE or -1.

```
: ON>  <name>      ( -- )
   ?NOUSER
   STATE @
   IF  COMPILE %ON>
   ELSE    ' BODY ON
   THEN    ; IMMEDIATE
```

```
: &>  <name>          ( t1 -- a1 )    \ Return the BODY address a1 of word t1, a VALUE of VARIABLE.
   ' ?NOUSER
   STATE @
   IF     COMPILE %&> X,
   ELSE    >IS
   THEN    ; IMMEDIATE
```

6.3. SAVE AND RESTORE VARIABLES AND VALUES

The words defined in SAVEREST.SEQ allow you to quickly save the contents of a variable of value on the return stack, and set it to some other value temporarily with SAVE!>. The previous contents may then be restored with RESTORE>. These utility words are used extensively in F-PC 3.5, when file parameters or screen parameters have to be saved, new environment established for many operations, and then the original environment must be restored to continue on temporarily interrupted operations.

Here is an example of how to use SAVE!> and RESTORE>:

```
   : DEF   ( --- )
      32 SAVE!> MYSTUFF
       ... Do something with the new MYSTUFF ...
      RESTORE> MYSTUFF ;
```

%SAVE!> Save the BODY contents of the following definition on the return stack and set its BODY to n1.

```
CODE %SAVE!>    ( n1 --- )
   LODSW ES:          \ Get code field address of next word.
   MOV BX, AX         \ Save it to BX.
   ADD BX, # 3        \ Move it to the body field.
```

```
   SUB RP, # 2        \ Prepare to push on to the return stack.
   MOV AX, 0 [BX]     \ Extract contents in the body field.
   MOV 0 [RP], AX     \ Copy it to the return stack.
   POP 0 [BX]         \ Store n1 into the body field, changing its value.
   NEXT
   END-CODE
```

%SAVE> Save the BODY contents of the following definition on the return stack.

```
CODE %SAVE>    ( --- )
   LODSW ES:          \ See comments above.
   MOV BX, AX
   SUB RP, # 2
   MOV AX, 3 [BX]
   MOV 0 [RP], AX
   NEXT
   END-CODE
```

%RESTORE> Restore the BODY of the following definition from the return stack.

```
CODE %RESTORE>  ( --- )    \ Reverse of above.
   XCHG RP, SP
   POP DX
   XCHG RP, SP
   LODSW ES:
   MOV BX, AX
   MOV 3 [BX], DX
   NEXT
   END-CODE
```

%USAVE!> Save the USER data area for the following definition on the return stack and set it to n1.

```
CODE %USAVE!>  ( n1 --- )
   LODSW ES:
   MOV BX, AX
   MOV BX, 3 [BX]              \ get body contents
   ADD BX, UP                  \ add to USER table
   SUB RP, # 2
   MOV AX, 0 [BX]              \ save contents in AX
   MOV 0 [RP], AX              \ move AX to return stack
   POP 0 [BX]                  \ pop data stack into USER table
   NEXT
   END-CODE
```

%USAVE> Save the USER data area for the following definition on the return stack.

```
CODE %USAVE>  ( --- )
   LODSW ES:
   MOV BX, AX
   SUB RP, # 2
   MOV BX, 3 [BX]              \ get body contents
   ADD BX, UP                  \ add to USER table
   MOV AX, 0 [BX]              \ get USER table contents
   MOV 0 [RP], AX
   NEXT
   END-CODE
```

%URESTORE Restore the USER data area of the following definition from the return stack.

```
CODE %URESTORE> ( --- )
   XCHG RP, SP
   POP DX
   XCHG RP, SP
   LODSW ES:
   MOV BX, AX
   MOV BX, 3 [BX]              \ get body contents
   ADD BX, UP                  \ add to USER table
   MOV 0 [BX], DX
   NEXT
   END-CODE
```

SAVE!> Leaves value in <name> on return stack and sets body of <name> to n1, the equivalent of "VARIABLE-NAME DUP @ >R !".

```
: ?COMP       ( --- )       \ Abort if we are not in compiling.
   STATE @ 0= ABORT" Use only while compiling" ;

: SAVE!>      ( n1 | <name> --- )
   ?COMP
   ' DUP @REL>ABS DUP
   [  [ASSEMBLER] DOUSER-VARIABLE META ] LITERAL = SWAP
   [  [ASSEMBLER] DOUSER-DEFER    META ] LITERAL = OR
   IF       COMPILE %USAVE!>
   ELSE    COMPILE %SAVE!>
   THEN
   X, ; IMMEDIATE
```

SAVE> Saves value in <name> body on the return stack. Equivalent to "VARIABLE-NAME @ >R".

```
: SAVE>       ( | <name> --- )
   ?COMP
   ' DUP @REL>ABS DUP
   [  [ASSEMBLER] DOUSER-VARIABLE META ] LITERAL = SWAP
   [  [ASSEMBLER] DOUSER-DEFER    META ] LITERAL = OR
   IF       COMPILE %USAVE>
   ELSE    COMPILE %SAVE>
   THEN    X, ; IMMEDIATE
```

RESORE> Restore body of <name> from the return stack. Equivalent to "R> VARIABLE-NAME !".

```
: RESTORE>     ( | <name> --- )
   ?COMP
   ' DUP @REL>ABS DUP
   [  [ASSEMBLER] DOUSER-VARIABLE META ] LITERAL = SWAP
   [  [ASSEMBLER] DOUSER-DEFER    META ] LITERAL = OR
   IF       COMPILE %URESTORE>
   ELSE    COMPILE %RESTORE>
   THEN    X, ; IMMEDIATE
```

6.4. HANDLES

HANDLES.SEQ contains the code to talk to a file using the DOS 2.00+ file handle service routines. Handle is a better device than File Control Block to interface to the DOS file system. The very long file name field permits complete pathnames to be specified. File handles can be created, deleted, copies, and referenced through the DOS 21H service interrupts. This section describes all the F-PC words related to file handles.

The DOS definition of a handle is "a 16 bit number returned by the operating system when a file is opened or created". In F-PC, on the other hand, the DOS handle is called the 'Handle number', and the word HANDLE refers to a data array of 70 bytes. The address of this array is given by executing the handle name. The zero'th byte is a count. Bytes 1 to 65 are used to store path, file name, and extension as a long counted string. The name string is terminated by a NUL character. The count and name fields are structured like a Forth counted string, and many F-PC string words can be used to manipulate the name in a handle. Bytes 66 and 67 are for the file attributes and bytes 68 and 69 hold the handle number. After a file handle is established, the file is referenced in DOS by its handle number. F-PC refers to a handle by the address of the first byte in the handle array.

```
70 CONSTANT B/HCB       \ Bytes in a handle

68 CONSTANT HNDLOFFSET  \ Offset to >HNDL field

VARIABLE RWERR          \ Flag for read/write error
```

>attrib Step to the attribute field of the handle. Attribute is normally zero (0) for Read/Write. It may be set to one (1) for a write-only file, and to two (2) for read-only.

```
: >ATTRIB   ( h-addr -- attrib-addr )   66 + ;
```

>HANDLE Step to the handle storage field in the handle array.

```
: >HNDLE    ( h-addr -- handle-number ) HNDLOFFSET + ;
```

>NAM Step to the null terminated name field in the handle.

```
: >NAM  ( h-addr -- name-string-addr )  1+   ;
```

CLR-HCB Clear a handle at addr.

```
: CLR-HCB   ( addr - )
   DUP B/HCB ERASE     \ Erase handle body
   -1 SWAP >HNDLE ! ;      \ Initialize >HNDL field
```

HANDLE Using the following word for a name, create a handle array, which holds the file attribute, handle number, and a null terminated name string. Executing <name> will leave the handle address on the stack.

```
: HANDLE <name> ( -- )
   CREATE         \ Build a header for the handle
   HERE           \ Leave the handle addr on stack
   B/HCB ALLOT    \ Allocate enough bytes
   CLR-HCB ;      \ Clear the handle first
```

DEFEXT An array holding the default F-PC source file extension "SEQ".

```
CREATE DEFEXT
   3 C,-T         \ 3 characters in SEQ
   ASCII S C,-T   \ The characters
   ASCII E C,-T
   ASCII Q C,-T
   4 ALLOT        \ 4 bytes in counted string
```

?DEF.EXT If the specified file name has no "." indicating the extension, then supply one from the default extension in DEFEXT. It may add an extension to the file name.

```
: ?DEF.EXT  ( h-addr -- )
   dup c@ 60 >          \ Exit if name is too long
   if drop exit then
   >r true r@ count bounds     \ Scan the name string
   ?do i c@ ascii . =      \ Leave if a period is found
      if  drop false leave
      then
   loop            \ Return true if no decimal point found
   if  defext c@
```

```
      if  defext count
        r@ count +
        1+ swap cmove   \ Copy SEQ
        ascii . r@
        count + c!
        defext c@ 1+
        r@ c@ + r@ c!   \ Insert period before SEQ
      then
   then    r>drop ;
```

$>HANDLE Move a counted file name string at a1 into a handle array at a2 for use by the following words.

```
: $>HANDLE  ( a1 a2 -- )
   dup>r CLR-HCB
   count 64 min          \ Source of name
   dup r@ c!          \ Store count byte
   r@ 1+ swap         \ To name field
   0 max cmove           \ Copy to handle
   0 r@ count + c!       \ Add NUL terminator
   > ?DEF.EXT ;          \ Add .SEQ extension if needed
```

!HCB Get a handle name from the input stream and move it into the handle at addr.

```
: !HCB  <name>  ( h-addr -- )
   BL WORD           \ Accept name string from input
   CAPS @         \ Uppercase?
   IF  DUP COUNT UPPER \ Yes.  Convert string to uppercase
   THEN
   SWAP $>HANDLE ;     \ Put string in handle
```

FCB>HANDLE Extract the file name and extension from the specified FCB at a1 to a handle at a2.

```
: FCB>HANDLE    ( a1 a2 -- )
   DUP CLR-HCB        \ Clear destination handle
   1+ dup>r SWAP 1+       \ >nam a1+1 --
   dup>r 8 OVER + SWAP     \ >nam a1+1 a1+9 a1+1 --
   DO  I C@ BL = ?LEAVE    \ Scan till a blank
      I C@ OVER C! 1+ \ Copy characters before it
   LOOP
   ASCII .  OVER C! 1+     \ Insert a period before extension
      R> 8 + 3 OVER + SWAP    \ Now move to extension field in
   DO   I C@ BL = ?LEAVE   \    FCB
      I C@ OVER C! 1+ \ Copy file extension
   LOOP
   0 OVER C!          \ Append NUL terminator
   R@ - R> 1- C! ;     \ Store the count
```

HANDLE>EXT Move the address pointer from the count field in the handle array to the decimal point in the file name, if it exists. Otherwise it steps to the null immediately following the file name.

```
: HANDLE>EXT    ( h-addr -- a1 )
   count +            \ End of file name
   dup dup 4 -        \ end end-4 -- ,
   do  i c@ ascii . =      \ Where is the period?
      if  drop       \ If found, leave its address
         i leave
      then
   loop    ;          \ Point to final period
```

$>EXT Move the specified string at a1 to the extension field of the handle at a2.

```
: $>EXT ( a1 a2 -- )
   dup c@ 60 >           \ Exit if name is too long.
   if 2drop exit then
   dup>r handle>ext      \ Move pointer to extension
   ascii . over c!       \ Put a period there
   1+ >r count r@
   swap cmove            \ Copy the extension string
   r> 3 + 0 over c!      \ Append NUL
   r@ - 1- r> c! ;       \ Store count
```

HDOS1 Define a DOS call assembly word, which is later used by HOPEN and HCREATE.

```
HEX
CODE HDOS1  ( cx dx fun -- ax cf | error-code 1 )
   pop ax           \ Get parameters
   pop dx
   pop cx
   int 21           \ Do DOS call
   push ax          \ Push returned parameter
   u< if   mov al, # 1    \ If error, leave error code.
   else    mov al, # 0
   then
   sub ah, ah
   1push            \ Push error flag
   end-code
```

HDOS3 Yet another DOS call using more parameters.

```
CODE HDOS3  ( bx cx dx ds fun -- ax cf | error-code 1 )
   pop ax            \ Function code
   pop ds            \ Data segment
   pop dx
   pop cx
   pop bx
   int 21            \ Do DOS call
   push ax           \ Primary returned data
   u< if   mov al, # 1     \ Error flag
   else    mov al, # 0
   then
   sub ah, ah
   push ax
   mov ax, cs        \ DS must be restored
   mov ds, ax
   next
   end-code
```

HDOS4 Still another native DOS call.

```
CODE HDOS4  ( bx cx dx fun -- ax cf | error-code 1 )
   pop ax            \ Function code
   pop dx            \ All other data
   pop cx
   pop bx
   int 21            \ Call
   push ax           \ Return
   u< if   mov al, # 1
```

```
   else    mov al, # 0
   then
   sub ah, ah
   1push            \ Error flag
   end-code
```

MOVEPOINTER Move the file pointer to the position specified by the double-offset in a file already open by the handle.

```
CODE MOVEPOINTER    ( double-offset h-addr -- )
   pop bx           \ Handle address
   ADD bx, # HNDLOFFSET    \ >HNDL
   mov ax, 0 [bx]         \ Get handle number
   mov bx, ax
   pop cx           \ File pointer offset
   pop dx
   mov ax, # 4200         \ Move from start of file
   int 21           \ MovePointer
   next             \ Done
   end-code
```

ENDFILE Return the double-integer end pointer for the file open in handle. Also set the pointer to the end of this file. Useful for finding the end of a file, and for appending to the end of a file.

```
CODE ENDFILE    ( h-addr -- double-end )
   pop bx           \ Handle address
   ADD bx, # HNDLOFFSET
   mov ax, 0 [bx]         \ Handle number
   mov bx, ax
   mov cx, # 0        \ Initial offset
```

```
   mov dx, # 0
   mov ax, # 4202          \ To end of file
   int 21              \ Please do it, Ms. DOS.
   u< if   sub ax, ax      \ If error, return 0.
   then
   push ax             \ Push double offset
   push dx
   next
   end-code
DECIMAL
```

PATHSET Copy the current drive:path specification into a handle. Return a flag true if an error occurred while performing this operation. Code loaded later is plugged into PATHSET, to prepend the current path to a handle specified by the address on the top of the stack. The returned value is zero if the path was set properly, or non-zero if an error occurred while setting the path.

```
DEFER PATHSET   ( h-addr -- f )
' 0= IS PATHSET
```

<HRENAME> Code primitive for the HRENAME function below. Return 18 if renaming was successful.

```
HEX
CODE <HRENAME>  ( h-addr1 h-addr2 -- ax cf=0 | error-code 1 )
   pop bx              \ Handle2
   add bx, # 1         \ Name address
   mov di, bx          \ Destination register
   pop bx              \ Handle1
   push es             \ Save ES for later restoration
   mov ax, ds
   mov es, ax          \ Set ES to DS
   add bx, # 1         \ Name field of source handle
```

```
   mov dx, bx
   mov ax, # 5600         \ Rename Handle function
   int 21             \ Call DOS
   pop es             \ Restore ES
   push ax            \ Return code
   u< if   mov al, # 1
   else    mov al, # 0
   then
   sub ah, ah
   1push              \ Push error flag
   end-code
```

HRENAME Change the name of the file specified in handle1 to the name specified in handle2. Can be used to move a file from one directory to another on the same drive. Returns 18 if success.

```
: HRENAME      ( h-addr1 h-addr2 -- return-code )
   DUP PATHSET DROP       \ Apply path if needed
   OVER PATHSET DROP   \ Do same for handle1
   <HRENAME>          \ Rename file 1
   if     0FF and        \ Error?
   else    drop 0
   then    ;
```

HCREATE Create the file specified in a handle. If the file already exists, then it is ZEROed !! Return zero if no error occurred. The file is automatically opened for read and write operations.

```
: HCREATE   ( h-addr -- error-code )
   DUP PATHSET ?dup       \ Apply drive/path
   if  swap drop exit then     \ Exit if file exists.
   dup >hndle >r          \ Save handle address
```

```
   dup >attrib @          \ hndl -- bx hndl attrib
   swap >nam          \ -- bx attrib name
   3C02 hdos1 0=          \ Create it.
   if r@ ! 0         \ Stuff handle, return 0
   else    0FF and     \ Error code
   then    r>drop ;
```

R/W-MODE This value controls the read/write mode of the file being opened. A value 2 is read/write, a value of 1 specifies write only, and a value of 0 specifies read only. It picks up its default value from R?W-DMODE.

```
0 VALUE R/W-MODE !-T    \ Current read/write mode
0 VALUE R/W-DMODE !-T   \ default to read and write
```

DEF-RWMODE This word allow you to set the default read/write mode used by F-PC. It is used as follows:

```
   READ-WRITE DEF-RWMODE
```

or READ-ONLY DEF-RWMODE

All further file open operations will be in the newly specified mode.

```
: DEF-RWMODE    ( -- )  \ use current mode as the default.
   r/w-mode %!> r/w-dmode ;
```

The following words effect only the next HOPEN operation to be performed. After the open is done, R/W-MODE reverts to the the default mode for later file opens.

```
: READ-ONLY     ( -- )              \ open a file for read only
    0 %!> r/w-mode ;

: READ-WRITE    ( -- )              \ open a file for read and write operations
```

```
   2 %!> r/w-mode ;

: WRITE-ONLY    ( -- )              \ open a file for write only.
   1 %!> r/w-mode ;
```

HOPEN Open the file specified in a handle address. Return error-code zero if the file was opened properly. The file is opened for both read and write operations.

```
: HOPEN ( h-addr -- error-code )
   DUP PATHSET ?dup       \ Apply path
   if  nip exit then          \ Path error, exit.
   dup >hndle >r          \ Save handle address
   dup >attrib @          \ hndl -- hndl attrib
   swap >nam          \ -- attrib name
   $3D00 rwmode @ or
   hdos1 0=           \ read/write attribute
   if     r@ ! 0         \ Stuff handle, return 0
   else    $0FF and           \ Else error code
   then    r>drop         \ Clean rstack
   r/w-dmode %!> r/w-mode ;    \ revert to default mode
```

HCLOSE Close the file specified by a handle address, and return non-zero flag if an error occurred.

```
: HCLOSE    ( h-addr -- return-code )
   >hndle dup @ -1 rot !      \ Reset HNDL field
   dup -1 >           \ If file was open
   if  0 0 $3E00 hdos4 \ Close it
      if $0FF and    \ Return proper code
      else    drop 0
      then
```

```
   else    drop 0          \ File already closed
   then    ;
```

HDELETE Delete the file specified by a handle address, and return a non-zero error code if an error occurred.

```
: HDELETE   ( h-addr -- return-code )
   0 0 rot >nam            \ handle name field
   $4100 hdos4         \ Delete file
   if $0FF and         \ Return code
   else    drop 0 then ;
```

EXHREAD Extended Read. Read from the file specified by a handle address to an extended segment area specified by seg:addr and length n. Return the length in bytes actually read.

```
: EXHREAD   ( addr n h-addr segment -- length-read )
   >r >hndle @             \ Get handle number
   -rot swap r>
   $3F00 hdos3         \ Extended Read
   if $0FF and         \ Error return
      rwerr ! 0
   then ;
```

EXHWRITE   Extended Write. Write n bytes from memory addr and segment seg to the file specified by a handle address.

```
: EXHWRITE  ( add n h-addr seg -- length-written )
   >r >hndle @
   -rot swap r>
   $4000 hdos3         \ Extended Write
   if  $0FF and
```

```
    rwerr ! 0
  then ;
```

HWRITE Write specified bytes to a file specified by a handle address from addr in the Code Segment. Return the length actually written.

```
: HWRITE   ( addr length h-addr -- length-written )
  >hndle @ -rot swap     \ handle count addr --
  $4000 hdos4            \ Handle Write
  if     $0FF and rwerr ! 0
  then ;
```

HREAD Read from a file specified by a handle to a string buffer addr in the Code Segment. Returns n2 the length actually read.

```
: HREAD          ( addr length h-addr -- length-read )
  >hndle @ -rot swap        \ handle count addr
  $3F00 hdos4
  if     $0FF and rwerr ! 0
  then ;
```

FINDFIRST Begin a search for files specified by filespec string. String is a null terminated un-counted string. The flag f returned indicates whether any file matched. The name of a file thus found is placed in the Data Transfer Area (DTA).

```
: FINDFIRST       ( string -- f1 )
  $010 swap $4E00 hdos1
  drop $0FF and ;
```

FINDNEXT Continue the file search using a specified string for a file name. Returns a true flag if another match was found. The name of file found is placed in the Data Transfer Area (DTA).

```
: FINDNEXT  ( -- f )
   0 0 $4F00 hdos1
   drop $0FF and ;
```

SET-DTA Set addr as the Data Transfer Area.

```
: SET-DTA   ( addr -- )
   $1A BDOS DROP ;
```

6.5. CONSOLE AND PRINTER WORDS

The words handling the console and printer input/output were in the HANDLES.SEQ in F-PC 2.25. They are moved into SEQREAD.SEQ file in F-PC 3.5. A major improvement in these words from 2.25 to 3.5 is that the basic I/O words can take characters from any segment in the memory, like CONSOLEL, PRINTL, AND (TYPEL). The words working on the Code segments are derived from the more powerful long memory words.

DOS defines two special handles for the console display and the printer. The DOS devices CON: and PRN: are derived from these handles. Output characters and strings using these handles are not particularly fast. DOS is not known for its speed anyway. F-PC uses optimized output routines to raise its performance. However, if you have to move F-PC to compatible machines or clones, the hardware may prevent you from using those optimized routines. You can re-vector the output routines to this set of handle output I/O words, which should be usable under any MS-DOS systems Version 2.0 and later.

```
CREATE CONHNDL B/HCB  ALLOT \ Handle for console CON device
   1 CONHNDL >HNDLE !-T
CREATE PRNHNDL B/HCB  ALLOT \ Handle for printer PRN device
   4 PRNHNDL >HNDLE !-T
```

(CONSOLE) Output a character to the console through the CON handle. This word is used where DOS compatibility is the major concern.

```
: CONSOLEL  ( seg char -- )
   OUTPAUSE          \ Pause for task switching
   SWAP >R        \ Save segment to avoid deep ROLL
   SP@ 1 CONHNDL >R       \ Gather parameters for an
   EXHWRITE 2DROP      \   extended file write
   #OUT INCR ;        \ Characters in a line

: (CONSOLE)    ( char -- )     \ Use CONSOLEL to send character in Code segment.
   ?CS:  SWAP  CONSOLEL ;
```

(PRINT) Output a character to the printer. Use the printer handle to send out the character.

```
: PRINTL    ( seg char -- )
   OUTPAUSE          \ Similar to (CONSOLE)
   SWAP >R
   SP@ 1 PRNHNDL >R       \   except using printer handle
   EXHWRITE 2DROP
   #OUT INCR ;

: (PRINT)   ( char -- )
   ?CS:  SWAP  PRINTL ;
```

(TYPE) Output a string at addr to the console device through the CON handle. String length is len. The string may be sent to the printer if PRINTING is ON.

```
: PRNTYPEL      ( seg addr length -- )
   OUTPAUSE
   0MAX ROT
   PRNHNDL SWAP EXHWRITE
```

```
   #OUT +! ;


: CONTYPEL     ( seg addr length -- )
   OUTPAUSE
   oMAX ROT
   CONHNDL SWAP EXHWRITE
   #OUT +! ;


: (TYPEL)      ( seg addr length -- )      \ External type, from other segment
   PRINTING @
   IF  PRNTYPEL
   ELSE    CONTYPEL
   THEN    ;


: (TYPE)       ( addr length -- )
   ?CS: -ROT TYPEL ;
```

6.6. SEQUENTIAL FILES

Source file is SEQREAD.SEQ. F-PC uses sequential files for source code and documentation. The most important task in using text in a sequential file is to read the text in lines terminated by carriage-return and line-feed character-pairs. The compiler will also have to be able to compile source code line by line. The editor will have to read lines, write lines and modify lines. These functions are provided in the file SEQREAD.SEQ.

To allow a file to load other files, F-PC uses a handle stack to nest the files for loading. The handle stack has space for 7 handles. Thus you can nest file loading to 7 levels deep. This is more than enough for most applications.

In SEQREAD.SEQ, many variables are re-defined as values for speed and cleanness of code. This is good place to make use of the very powerful VALUE technique.

```
0 VALUE SEQHANDLE          \ Pointer to the currently opened handle on the handle stack
0 VALUE WITHPATH           \ Should the PATH be included in the file name?
16384 VALUE IBLEN          \ Current length of input buffer
16384 VALUE IBFULL      \ Maximum length of input buffer
0 VALUE SCREENCHAR      \ Also for editor
$0A VALUE DELIMITER     \ Line-Feed as line terminator
255 VALUE OBLEN         \ Length of output buffer.  It can be reduced to 64 to read block
                \   files.
0 VALUE INSTART         \ Starting offset of input
0 VALUE INLENGTH           \ Length of input text
0 VALUE INBSEG          \ Segment of input buffer
VARIABLE TOTALLINES     \ Total number of lines
CREATE OUTBUF OBLEN 1+ ALLOT    \ Line output buffer array
7 B/HCB * CONSTANT MAXNEST  \ Array for maximum of 5 hcb's
CREATE HNDLS           \ An array of handles, holds 5 handles in a stack.
   MAXNEST B/HCB + ALLOT
   HNDLS ' SEQHANDLE >BODY !-T \ SEQHANDLE is first on the handle stack
VARIABLE FILEPOINTER 2 allot    \ Most recent read character position
VARIABLE LOADING           \ Are we in the process of loading a file?
   0 LOADING !-T           \ Initialize to not loading.

DEFER LOADSTAT  ' NOOP IS LOADSTAT
```

SEQHANDLE+ Return an address addr of the address of the next available handle on the handle stack.

```
: SEQHANDLE+    ( -- h-addr )
   seqhandle          \ Fetch address of the current handle
   b/hcb + ;          \ Offset to next handle array
```

.SEQHANDLE Display the file name in the current handle.

```
: .SEQHANDLE    ( -- )
   seqhandle          \ Address of current handle
   count type ;           \ Type the file name
```

CURPOINTER Return the current double-integer file pointer of the file specified by a handle.

```
CODE CURPOINTER ( h-addr -- double-offset )
   pop bx             \ Handle address
   add bx, # hndloffset
   mov bx, 0 [bx]         \ Get the handle number
   sub cx, cx
   mov dx, cx
   mov ax, # $4201     \ Get character pointer
   int 21
   push ax            \ Push double integer results
   push dx
   next
   end-code
```

SAVEPOINTER Save the file pointer of the current file into a double variable FILEPOINTER for later restoration.

```
: SAVEPOINTER   ( -- )
   seqhandle          \ Handle address
   curpointer         \ Cursor location
   inlength 0 d-          \ Discount input buffer length
   filepointer 2! ;           \ Store file offset
```

GET_ALINE Get a line of text from the current file, and place it in the output buffer at OUTBUF. Return address of OUTBUF.

```
CODE GET_ALINE  ( -- addr )
   push es                        \ Save ES for later restoration
   mov di, ' instart >body        \ Searching from INSTART
   mov ax, ' DELIMITER >body      \ Searching for line delimiter
   mov cx, ' inlength >body       \ for INLENGTH clipped to OBLEN
   cmp cx, ' oblen >body          \ if INLENGTH > OBLEN
   > if     mov cx, ' oblen >body \ clip search length to OBLEN
    then    mov dx, cx            \ save search length in DX
   cx<>0 if
      mov es, ' inbseg >body     \ searching INBSEG segment
      repnz   scasb              \ Scan for Linefeed char
   then    sub dx, cx            \ DX = length of line
   sub ' inlength >body dx       \ subtract line from remaining
   mov outbuf dl byte            \ set the length of OUTBUF
   mov bx, si                    \ save IP for later restoration
   mov si, ' instart >body       \ moving from INSTART
   add ' instart >body dx        \ set start to after line
   mov cx, dx                    \ cx = length to move
   mov di, # outbuf 1+           \ moving to OUTBUF
   mov ds, ' inbseg >body        \ from INBSEG segment
   mov ax, cs
   mov es, ax                    \ to CODE segment
   cx<>0 if     repnz   movsb \ move the line to OUTBUF
   then
   mov ax, cs
   mov ds, ax                    \ restore DS
```

```
   mov si, bx                    \ restore IP
   inc loadline word             \ bump line counter
   pop es                        \ restore ES
   mov ax, # outbuf              \ return address of buffer
   1push
   end-code
```

FILLBUFF Refill the input buffer with text from file.

```
: FILLBUFF  ( -- )
   inbseg instart            \ Seg:addr of remaining text
   over 0 inlength cmovel    \ Copy to the beginning of input buffer
   %off> instart             \ INSTART points to beginning of the input buffer
   inlength iblen inlength -       \ addr len --
   seqhandle inbseg          \ addr len h-addr seg --
   exhread                   \ Read file.  Return bytes read.
   %+!> inlength             \ Add it to INLENGTH
                  \ strip off any Control Z's at end of file by
   -6 -1                     \ scanning the last 6 chars of the read buffer
                  \ when a FILLBUFF occurs.
   do inlength i + 0< ?leave         \ leave if begin buf
     inbseg inlength i + c@L 26 =    \ is it ^Z?
     if     bl
       inbseg inlength i + c!L \ change to BLANK
     then
   -1 +loop
   savepointer ;             \   and save it.
```

?FILLBUFF Refill the input buffer INBUF if it needs it.

```
CODE ?FILLBUFF  ( -- )
   cmp ' inlength >body # oblen word
   u>= if            \ If string is longer than output buffer
      mov bx, # filepointer   \ Set BX to point to FILEPOINTER
      sub cx, cx              \ clear CX
      mov cl, outbuf          \ read byte length of OUTBUF
      add 2 [bx], cx          \ Add to 32 bit contents
      adc 0 [bx], # 0
      next
   then              \ If we got here, then
   mov ax, # ' fillbuff       \   go and re-fill the buffer.
   jmp ax
   end-code
```

LINEREAD Read a line from the current file, put it in the output buffer, and return the address of the buffer.

```
: LINEREAD  ( -- addr )     \ Read a line delimited by CRLF
   ?fillbuff          \ Re-fill buffer if needed.
   get_aline ;        \ Return line including CRLF
```

DOERROR Handle the error condition when the compiler encounters trouble. It displays the file name, line number, and the word which causes compilation error. The file location is saved so the editor can be invoked to correct the error.

```
: (DOERROR) ( addr n -- )
   2>r           \ Save the location and length of the error message
   cr ." file= " .seqhandle       \ Print file name
   ."  at Line " loadline @ u.     \ Print line number.
   cr outbuf count type cr     \ Display the line under processing.
```

```
   >in @ 1- here c@ - 0max     \ First character of the offending word.
   dup %!> screenchar      \ Save its position in SCREENCHAR
   0 do  ascii - emit  loop        \ Draw a line towards it.
   ." ^-- "              \ Mark it with an arrow.
   2r> type space          \ Type the error message
   true %!> byte|line      \ Set flag in BYTE|LINE
   quit              \ Start over.
   ;

DEFER DOERROR   ' (DOERROR) IS DOERROR
```

Forth allows the user to select any base value for number conversions. This great freedom eliminate the need to buy a hex calculator which is a necessity for most assembly programmers. However, not knowing the current base while loading a file or doing something unusual accounts for a large percentage of headaches for Forth practitioners. F-PC 3.5 tries to help in a default base mechanism. The user can specify a default base, which will be assumed by the system whenever an error occurred.

```
$0A VALUE DEFBASE       \ Default base value.  Decimal is assumed.

: HEXBASE      ( --- )        \ Select HEX as the default base.
   hex base @ %!> defbase ;

: DECIMALBASE   ( --- )     \ Select DECIMAL as the default base.
   decimal base @ %!> defbase ;

: NOBASE       ( --- )        \ Do not change base when an error occurs.
   %off> defbase ;

: .BASE        ( --- )        \ Display the current base in DECIMAL.
   ." Current BASE in DECIMAL is "
   base @ dup decimal . base ! ;
```

```
: ?BASE_RESTORE ( f1 --- )              \ Restore the base to the default
   if       defbase ?dup          \ base if f1 = true, and defbase
      if    base !        \ is not zero
      then
   then   ;
```

(?SERROR) If the Boolean flag is true, then print message at addr and display the error line in the current file where the error occurred.

```
: (?SERROR) ( addr len f -- )
   dup ?base_restore      \ Restore the default base if so specified.
   %@> loading        \ Fetch flag in LOADING
   if           \ Loading error.
      if  2>r sp0 @ sp!   \ Re-init data stack
         printing off    \ Turn off printer
         loading off \ Return to interpreter
         ['] <run> is run    \ Re-install text interpreter
         2r> doerror
         quit       \ Back to the interpreter
      else    2drop      \ No error comes here
      then
   else    (?error)          \ Interpreter line error
   then   ;

' (?SERROR)  IS  ?ERROR
```

Following words use the handle stack to open multiple files. The handle stack and related pointers and operators are show in Figure 8.

SEQUP Push one more handle on the top of the handle stack and make it the current handle.

```
: ?SEQRANGE    ( --- )       \ Verify seqhandle points to a handle in \ the HNDLS array
   seqhandle hndls maxnest
   over + between 0=      \ Compare current handle address against the limits
   abort" SEQHANDLE is not set to the HANDLE stack" ;

: SEQUP ( -- )
   ?seqRange          \ Abort if out of range.
   seqhandle          \ Get current handle
   >hndle @ -1 >          \ It must be open now.
   if  seqhandle b/hcb +   \ Next handle array
      dup hndls maxnest + \ Does it exceed max handle?
      U< 0=          \ If so, abort.
      abort" Nested too deeply on FLOAD, use CLOSEALL"
      dup %!> seqhandle   \ Make it the current handle
      clr-hcb        \   and clear its contents.
   then    ;
```

Figure 8. Handle stack and related pointers.

SEQINIT Initialize the handle stack, in preparation for its use.

```
: SEQINIT  ( -- )
   noop hndls %!> seqhandle    \ Make the first handle on stack the current handle
   hndls maxnest over + swap
   do  i clr-hcb       \ Clear all handles
   b/hcb +loop
   iblen paragraph alloc       \ Allocate that much memory
   8 = memchk nip      \ Abort if allocation error
   %!> inbseg          \ Store the base segment address
   CONHNDL CLR-HCB     \ Init console handle
   " CON." ">$
   CONHNDL $>HANDLE       \ Name it CON.
   1 CONHNDL >HNDLE !  \ Handle 1
   PRNHNDL CLR-HCB     \ Init printer handle
   " PRN." ">$
   PRNHNDL $>HANDLE       \ Name it PRN.
```

```
   4 PRNHNDL >HNDLE ! ;    \ Handle 4
```

IBRESET Reset the input buffer.

```
: IBRESET   ( -- )
   %off> instart          \ Store 0 to INSTART
   0%ff> inlength ;           \ Store 0 to INLENGTH
```

$HOPEN Open a file specified by the counted string at addr. Return a Boolean false flag if the file was opened successfully.

```
: $HOPEN    ( str-addr -- f )
   seqhandle hclose drop       \ Close current handle
   seqhandle $>handle      \ Get the file name
   seqhandle hopen         \ Open the new handle
   ibreset ;          \ Reset input buffer
```

```
DEFER GETFILE   ( -- addr f )   \ Return file name addr and flag.
```

```
' FALSE IS GETFILE          \ Default to failed
```

FILE>TIB   Move the file name string from addr to the terminal input buffer TIB, making it available for !HCB .

```
: FILE>TIB  ( addr -- )
   count              \ addr+1 len --
   2dup true -rot         \ Bury a true flag underneath
   over + swap        \ Scan the name string for '.'
   do  i c@ ascii . =
      if drop false leave \ If found, reset the flag.
      then
```

```
   loop
   >r dup #tib !          \ Adjust #TIB.
   >in off            \ Reset >IN
   tib swap cmove         \ Copy the name to TIB
   r>             \ Retrieve the flag
   if  ascii . #tib @ tib + c! \ No period found.  Insert it.
      #tib incr
   then    ;
```

GFL Get file name from the next word in the input stream. Abort if no name was given.

```
: GFL      ( -- )
   more? o=           \ End of a line?
   if  getfile            \ Yes.  Ask a file name
      o=         \ Abort if no name was given
      abort" No filename specified"
      dup count type space
      file>tib           \ Move file name to TIB
   then    ;
```

SEEK Move the filepointer in SHNDL to the position d in the current file.

```
: SEEK     ( d -- )
   seqhandle movepointer ;     \ Specified by d.
```

LISTVAR A variable controlling the displaying of source code while loading a file.

```
0 VALUE LISTVAR
```

SHOWLINES Turn on listing of lines while loading.

```
: SHOWLINES ( -- )
   -1 %!> listvar ;          \ Set LISTVAR to true
```

HIDELINES Turn off listing of lines while loading.

```
: HIDELINES ( -- )
   0 %!> listvar ;        \ Reset LISTVAR to false
```

CRLF>BL'S Change CRLF at end of string to blanks. Same as DUP COUNT + 2- 8224 SWAP !

```
CODE CRLF>BL'S  ( addr -- addr )
   pop bx            \ Leave the address on the stack
   push bx
   mov al, 0 [bx]        \ Get count
   sub ah, ah
   add bx, ax
   dec bx            \ Last cell of string
   cmp 0 [bx], # $0A0D word    \ If line ends in CRLF,
   0= IF   mov 0 [bx], # 8224 word \ Fill in two blanks
   then
   next
   end-code
```

SETTIB Set TIB to counted string at addr. TIB used to be a buffer at a fixed location in F83 and other Forth system. F-PC moves the TIB itself to where the input string is by setting TIB to point at the string.

```
CODE SETTIB ( addr -- )
   pop bx            \ Counted string address
```

```
   mov al, 0 [bx]         \ Count
   inc bx             \ Point to real string
   mov 'tib bx        \ 'TIB points to string
   sub ah, ah         \ Limit string to 255 bytes
   mov #tib ax        \   and #TIB
   mov >in # 0        \ Init >IN for interpreter
   next
   end-code
```

.LOADLINE Display the line currently being loaded.

```
: .LOADLINE ( addr -- addr )
   cr loadine @           \ Current line number
   5 u.r space        \ Show it
   dup count type ;           \   with the line
```

?.LOADLINE Conditionally display the source lines while loading. The condition flag is taken from LISTVAR.

```
CODE ?.LOADLINE ( addr -- addr )
   mov cx, ' listvar >body     \ List flag on?
   inc cx
   cx<>0 if   next   then      \ Exit if LISTVAR = -1.
   mov ax, # ' .loadline       \ Otherwise, display source lines.
   jmp ax
   end-code
```

LENGTH.CHECK Return a true flag if there are characters remaining in the input buffer.

```
CODE LENGTH.CHECK ( addr --- addr flag )
```

```
   mov ax, # true
   mov cx, ' inlength >body          \ if read length <> 0
   cx<>0 if      1push               \ then we aren't done
   then
   pop di                           \ get a copy of line buf ptr
   push di
   cmp 0 [di], # 0 byte             \ if line buffer <> 0
   0<> if      1push                \ then we aren't done
   then
   mov ax, # false
   1push
   end-code                    \ else we are done
```

FILLTIB Set the terminal input buffer to point to a line just read from the current file.

```
: FILLTIB  ( -- )
   %@> span +!> loadline       \ Add SPAN to LOADLINE
   lineread crlf>bl's      \ Read a line and blank CR and LF
   ?.loadline settib ;     \ Display line and point TIB at it.
```

SEQDOWN Step down one handle in the handle stack. Close the current file, and select the next lower handle as the current handle.

```
: SEQDOWN  ( -- )
   /SEQRANGE
   seqhandle hclose drop       \ Close current handle
   seqhandle clr-hcb          \ Clear the handle array
   seqhandle b/hcb -       \ Select next handle
   hndls maxnest + h/hcb - umin    \ Make sure the handle stack will not underflow.
   hndls umax %!> seqhandle    \   and make it current
```

```
   seqhandle >hndle @ -1 <>    \ If the file is open
   loading @ and
   if  filepointer 2@      \   get its file pointer
      seqhandle movepointer   \   and move it to position
      IBRESET    \ Reset input buffer
      >in @ filltib >in ! \ Refill input buffer.  Preserve >in.
   then    ;
```

CLOSE A pseudonym for SEQDOWN.

```
: CLOSE ( -- )  seqdown ;
```

CLOSEALL Close all files whose handles are piled up on the handle stack.

```
: CLOSEALL     ( --- )
   ?SEQRANGE
   begin   seqhandle hndls u>
   while   seqdown
   repeat  seqdown ;
```

<LOAD> Load the current file starting at the current file pointer.

```
: <LOAD>    ( -- )
   loadstat              \ Display loading status
   ibfull %!> iblen        \ set maximum length read buffer
   true  %save!> loading
   %save>  'tib
   %save>  >in
   false %save!> #tib
```

```
   %save>  loadline
   0 %!> screenchar
   begin              \ Now, load the file
     lineread       \ Read one line from file
     length.check       \ If not a blank line
   while              \   interpret or compile it
     crlf>bl's ?.loadline    \ Blank CR-LF
     settib run      \ RUN it and check status
   repeat  drop           \ Repeat till end of file
   %@> loadline +!> totallines \ Keep total line count
   %restore> run          \ Now, restore everything.
   %restore> loadline
   %restore> #tib
   %restore> >in
   %restore> 'tib
   %restore> loading
   loading @                   \ if we are still loading,
   if 256   %!> iblen        \ use a small read buffer
   else    ibfull %!> iblen    \ else use the full size buffer
   then    ;

LOADER  A deferred word that loads the current file.

DEFER LOADER    ' <LOAD> IS LOADER

>LINE   Step to line n in the current file.

: >LINE ( n -- )
   0.0 seqhandle movepointer   \ To beginning of file
```

```
   loadline off
   IBRESET        \ Reset input buffer
   1-  0max   ?dup     \ Check n for range
   if  cr ." Stepping to line "    \ Display line number
      dup 1+ u. ." .."
      0 ?do   lineread c@     \ Read one line
         0= ?leave   \ Leave if end of file
      loop
   then    ;
```

LOADED, Compile the name of the current file as a variable in the FILES vocabulary. The variable are linked into a list of which is used to locate words in a file by VIEW.

```
TRUE VALUE ?LOADED,         \ Allow disabling LOADED, for one time.
FALSE VALUE ?NOLOADED,

: LOADED,      ( --- )
   ?noloaded, ?exit
   ?loaded, 0=
   if     %on> ?loaded, exit
   then
   %save> 'tib
   %save> >in
   %save> #tib                  \ save interpretation status
   seqhandle count
   withpath 0=                  \ Should the PATH not be
   if                       \ included in the file VARIABLE?
      begin   2dup
         ASCII \ scan dup   \ skip the leading path
      while   2swap 2drop
```

```
        1 -1 d+
      repeat  2drop
   then
   %!> #tib                  \ set #tib
   %!> 'tib                  \ set tib to seqhandle
   %off> >in                 \ clear >in
   %save> context            \ save current context
   %save> current            \  and current vocab state
   files definitions         \ select files vocabulary
   variable                  \ make the header
   %restore> current
   %restore> context         \ restore vocabulary state
   %restore> #tib
   %restore> >in
   %restore> 'tib ;
```

<FLOAD> Primitive word that loads the file just opened. It allows nested file loading.

```
: <FLOAD>   ( -- )
   #tib @ >r
   0 %save!> loadline      \ Save current line position
   loaded,           \ Link file name
   0.0 seqhandle movepointer   \ Reset file pointer
   loader            \ Load the file
   %restore> loadline      \ Restore previous loadline
   r> %!> loadline ;       \ Update loadline of the parent file
```

CHARREAD Read a character char from the current file.

```
: CHARREAD  ( -- char )
   loading @
   if  begin   %@> >in
         %@>  #tib = \ If nothing in line
         inlength 0> and \ and input buf not empty
       while      ?fillbuff   \ Optionally refill buffer
         filltib     \ refill the TIB
       repeat
   then
   %@> >in   %INCR>   >in      \ Adjust pointers
   tib + c@ ;          \ Return the character read in
```

OK Load all source code in the current file.

```
: ?FILEOPEN     ( --- )                  \ Verify a file is open.
   seqhandle >hndle @ 0<
   abort" A file MUST be open to perform this operation." ;

: OK       ( -- )
   ?fileopen
   IBRESET
   <fload> ;
```

\S Stop loading the current file with the line that contains this word. Ignore the rest of the file.

```
: \S       ( -- )
   seqhandle endfile 2drop     \ Move to end of file
   loadline off
   IBRESET
```

```
   %@> #tib %!> >in  ;     \ Ignore rest of line
```

6.7. FILE PATH

The file FPATH.SEQ contains a set of words which allows F-PC to keep its system source files in a different directory from your work directory. It also allows you to specify 2 library directories to store other utility files.

```
CREATE FPATH$     160 ALLOT        \ Room for a looooong path.
CREATE PATHHNDL B/HCB ALLOT \ A temporary handle
0 VALUE PATHPTR        \ A pointer into the string of paths
0 VALUE PATHLEN
```

FPATH Set the current library path for file searches. Convert the following path string and store the path in FPATH$ for subsequent file searches.

```
: PATH1        ( -- addr )           \ return the first path, always the current dir
   fpath$ count %!> pathlen           \ and length
   %!> pathptr           \ Reset the path pointer
   pathhndl dup>r clr-hcb            \ clear out the handle
   r@ pathset drop r> ;              \ install current directory

: NPATH        ( -- addr flag )         \ addr = next path name; flag = true if end of path list
   pathptr pathlen                  \ addr n1
   2dup ASCII ; scan                 \ addr n1 a2 n2
   1- 0MAX %!> pathlen        \ set new length
   dup 1+ %!> pathptr                \ and pointer
   nip over - dup pathhndl c!         \ set handle length
   pathhndl 1+ swap cmove            \ move the data
```

```
    pathhndl c@ >r
    pathhndl count + 1- c@ ASCII \ <>
    if  ASCII \ pathhndl count + c!
       1 pathhndl c+!
    then
    pathhndl r> 0= ;

: SKIP.BLANKS   ( -- )
    source >in @ /string
    tuck bl skip
    nip - >in +! ;

: <FPATH+> <path-string>    ( -- flag )        \ flag = true if null string
    skip.blanks
    ASCII ; word c@                  \ we get a word
    fpath$ c@ 159 < and               \ and total length less than 160
    if fpath$ c@ 0>
       if ASCII ; fpath$
          count + c!
          1 fpath$ c+!       \ add ;
       then
       here count -trailing
       dup here c!
       fpath$ count + swap cmove   \ add path
       here c@ fpath$ c+!
    then
    here c@ 0= ;

: FPATH+  <paths-string>    ( -- )
```

```
   %save>  #tib
   >in @
   bl word drop
   >in @ #tib ! >in !
   begin   <fpath+>
   until
   %restore> #tib ;

: FPATH  <path-string>  ( -- )
   fpath$ off                  \ initialize the path string
   fpath+ ;
```

.FPATH Display the current library path.

```
: .FPATH       ( -- )  fpath$ count type space ;
```

PREPEND.APATH   Prepend the path in hndl to name at addr.

```
: PREPEND.APATH ( addr hndl -- addr )
   dup c@ >r                   \ save length of handle
   swap                        \ bring name to top
   dup  2+ c@ ASCII : =        \ do we already have a drive?
   over 1+ c@ ASCII \ = or     \ or a path specified?
   r> 0= or 0=                 \ or handle is empty skip prepend if we do
         if dup c@ >r >r       \ save name location & length
   r@ 1+ over c@
   r@ + 1+ r@ c@ cmove>    \ make room for path
   dup r@ over c@ 1+ cmove     \ move in path
   r> r> over c+!              \ correct count
```

```
   then  nip ;
```

?OPEN.ERROR If flag is true, display the current file=path and abort.

```
: ?OPEN.ERROR  ( flag -- )
   dup
   if       cr seqhandle count type
   then
   abort" Open Error!" ;
```

```
CREATE FSAVE$ B/HCB ALLOT  \ A temporary handle array.
```

$FILE Open a file whose name is in an array at addr. Return a false flag if the file is opened successfully.

```
: <$FILE>       ( addr -- flag )       \ flag = true if failed to open
   fsave$ over c@ 1+ cmove \ Use temp array to try open a file.
   fsave$ >r
   r@ here over c@ 1+ cmove
   here path1 prepend.apath       \ Try opening the file in current directory.
   $hopen  dup
   if  0=        \ If failed to find fine in current directory,
     begin     r@ here    \  search all known directories.
        over c@ 1+ cmove
        npath     0=
            if  here swap
          prepend.apath
          $hopen 0=
        else      drop 0= true
        then
```

```
    until
  then   dup              \ if couldn't open, then show current directory in error message
  if  drop r@ here over c@ 1+ cmove
    here path1 prepend.apath $hopen
  then
  r>drop ;

: $FILE       ( addr -- f )
  <$FILE>
  0.0 seqhandle movepointer          \ reset to beginning of file
  loadline off                       \ reset file offset
  ibreset
  0 %!> screenchar ;                 \ -- flag
```

FILE Open a specified file name from any FPATH directory for listing, loading or editing. Any file already open is closed first.

```
: FILE  <name>       ( -- )
  gfl bl word $file ?open.error
  ." of " seqhandle endfile d. ." bytes."
  0 0 seqhandle movepointer ;        \ reset to beginning of file
```

FLOAD Open a file whose name follows, and load the source text.

```
: $FLOAD      ( addr -- flag )
  sequp             \ Nest handle stack.
  <$file> dup>r 0=
  if filepointer 2@ 2>r  \ Save current file pointer
    %off> outbuf
    <fload>         \ Load new file
```

```
    2r> filepointer 2!  \ Restore current file pointer
  then   r>                ( -- flag )
  seqdown ;          \ Unnest handle.

: FLOAD  <name>      (-- )
  bl word                    \ get file name
  $fload  ?open.error ;            \ perform the load
```

6.8. COMMAND TAIL

It is often convenient to pass information into F-PC directly on the DOS command line, after the command to execute F-PC. F83 allows the user to specify a file name to be opened under F83 after F83 is loaded. This capability is retained in F-PC. Moreover, F-PC allows you to attach a whole line of Forth commands after the file name. These Forth commands will be executed after the file is opened. This command tail gives us the capability to build rather sophisticated batch files to do a large amount of work under DOS.

In F-PC 3.5, a configuration file is used to load system configuration information. The .CFG file is loaded before the command tail is processed. HDEFAULT is modified accordingly. The name of the .CFG file is store in CFGHNDL.

DOS-LINE The address of a buffer where DOS hides the tail of the command line.

```
128 CONSTANT DOS-LINE       \ Address where DOS command tail is stored

CREATE CFGHNDL         \ Handle for the configuration file.
STRING,-T KERNEL.CFG"       \ Default configuration file is KERNEL.CFG
B/HCB ALLOT
```

DOS>TIB Move the DOS command line to the Terminal Input Buffer for interpretation.

```
: DOS>TIB   ( -- )
   dos-line count         \ Get address of the command line
   DUP #tib !
   tib swap cmove         \ Copy commands to TIB
   >in off ;          \ Character pointer
```

HDEFAULT Open a file handle for the file specified on the DOS command line at startup.

```
: HDEFAULT     ( -- )
   %off> ?loaded,              \ disable LOADED, this time ONLY
   cfghndl $fload drop           \ First loaf the configuration file. We don't care if it exists.
   seqhandle clr-hcb dos-line c@ 0<>   \ Now see if the command tail exists.
   if        dos>tib                   \ move dosline to tib
      bl word 1+ c@
      ASCII - <>            \ is - in place of file
      if        here    $file       \ can we open it?
         if     >in off \ if not just pass on
         then   loadline off
      then                \ if first word is - then skip HOPEN
   then    ;                  \ skip whole thing if the command tail line is null

' HDEFAULT IS DEFAULT
```

In a DOS command line, the first argument after F-PC or F may be a file name. If no file is necessary, a '-' must be entered after F-PC but before other Forth commands to signify that a file is not required. The Forth commands after '-' will be executed in any case. An example of a DOS command is as follows:

```
   C:> F-PC - <Forth words> <return>
```

CHAPTER 7. BASIC UTILITIES

7.1. GENERAL UTILITY WORDS

UTILS.SEQ contains a collection of miscellaneous utility words which will be needed in the subsequent files. Most of them have something to do with the screen display, but many simply do not fit into any easy category. Many of the tool words are adopted from F83.

? Display the contents in addr.

```
: ?    ( addr -- ) @ .   ;
```

TIMES Re-execute the input stream before the word TIMES a specified number of times. This seems very tricky. A command line is moved into a terminal input buffer for execution or interpretation. >IN is used to scan the line, parse words out of it and execute them. If >IN is reset to zero, the same line will be scanned and executed again. TIMES and MANY uses this property of F-PC to re-execute a command line. Used in the form:

```
   <words to execute>  n  TIMES
```

```
VARIABLE #TIMES 1 #TIMES !  \ How many times a word is executed by TIMES.

: TIMES    ( n -- )
   1 #TIMES +!          \ Increase #TIMES by 1
   #TIMES @ <         \ Is it greater than n?
   IF     1 #TIMES !        \ Yes.  All done.  Make #TIMES1 again and continue on.
```

```
   ELSE    >IN OFF     \ Not yet.  Move interpreter pointer >IN back to the

   THEN              \   beginning of line and execute same line again.

   ;
```

MANY Re-execute the input stream before the word MANY until the user presses a key. See also TIMES. Used in the form:

```
      <words to execute>  MANY
```

```
: MANY  ( -- )

   KEY? NOT          \ Key pressed?

   IF  >IN OFF       \ No.  Execute the line again.

   THEN   ;
```

ALIAS Create a word with identical function to another word. In F-PC we only have to make the code pointer field in the head to contain the same cfa. No new code is needed. Used in the following form:

```
   ' <old-word>  ALIAS  <new-word>
```

```
: ALIAS  <name> ( cfa -- )

   HEADER        \ Make alias header

   YHERE 2- Y! ;         \ Code pointer field in head
```

\UNLESS Ignore the current line of source code unless the next word is defined. Conditional compilation.

```
: \UNLESS      ( NAME --- )          \ comment out line unless name is defined

   DEFINED NIP 0=      \ Is next word already defined?

   IF     [COMPILE] \     \ No.  Execute \ which ignores this line.

   THEN    ; IMMEDIATE
```

```
' \UNLESS ALIAS \U IMMEDIATE    \ Nice examples of aliases.

' !> ALIAS =: IMMEDIATE        \ make =: the same as !>
```

USED Allow you to analyze the dictionary space you consumed by executing a line of commands, most like to be loading one or more files. It is used as:

```
USED FLOAD MYAPPL.SEQ
```

A report will be displayed on the screen showing the number bytes used in the Code, Head, and List segments.

```
CREATE DSBUF 6 ALLOT

0 DSBUF ! XSEG @ DSBUF 2+ ! 0 DSBUF 4 + !


: !USED ( --- ) \ Save the current dictionary pointers

  HERE DSBUF !

  XHERE PARAGRAPH + DSBUF 2+ !

  YHERE DSBUF 4 + ! ;


: .USED ( --- )

  CR ." CODE LIST HEAD TOTAL bytes used"

  CR HERE DSBUF @ - DUP 6 U.R 0

  XHERE PARAGRAPH + DSBUF 2+ @ - 16 *D 2DUP 8 UD.R D+

  YHERE DSBUF 4 + @ - DUP 7 U.R 0 D+ 8 UD.R CR ;


: USED ( <command_line> --- )

  !USED INTERPRET .USED ;
```

.FREE Display the free memory space above the code, list and head dictionaries.

```
: .FREE ( -- )

  ." Free Bytes:" ." CODE = "

  SP@ HERE - (U.) TYPE \ Code dictionary free space

  ." , LIST = " #LISTSEGS \ Total list segments

  XHERE DROP XSEG @ - \ List dictionary segments
```

```
   - 16 *D 1 D.R          \ Free list space in bytes
   ." , HEAD = " #HEADSEGS     \ Total head segments
   16 * YHERE - (U.) TYPE ;    \ Free head space in bytes
```

DRIVE? Display the current drive in the 'A:' format. It is needed by later utilities.

```
: DRIVE?   ( -- )
   0 25 BDOS          \ Ask DOS for current drive
   ASCII A +          \ Convert drive # to alphabet
   EMIT ." : "  ;         \ Display with a :
```

U<= Unsigned less than or equal.

```
: U<=      ( u1 u2 -- f )  U> NOT   ;
```

U>= Unsigned greater than or equal.

```
: U>=      ( u1 u2 -- f )  U< NOT   ;
```

<= Less than or equal.

```
: <=       ( n1 n2 -- f )  > NOT    ;
```

>= Greater than or equal.

```
: >=       ( n1 n2 -- f )  < NOT    ;
```

0>= Greater than or equal to zero.

```
: 0>=     ( n -- f )  0< NOT  ;
```

0<= Less than or equal to zero.

```
: 0<=     ( n  -- f )
  0> NOT  ;
```

YCOUNT Similar to COUNT, but work on a string in the Head Space, counting a name string.

```
: YCOUNT   ( a1 -- a2 n1 )
  DUP 1+        \ a2=a1+1
  SWAP YC@ ;        \ Fetch count byte
```

?ENOUGH Abort if there are less than n items on the data stack.

```
: ?ENOUGH  ( n -- )
  DEPTH 1- >        \ n must be larger than DEPTH plus itself.
  ABORT" Not enough Parameters" ;
```

7.2. SCREEN DISPLAY WORDS

This set of words are also defined in UTILS>SEQ file. They are generally used to manipulation the screen display and the cursor control. F-PC uses them to switch the screen displays rapidly, especially when the hypertext system is in action. They are normally used only by the system. User does not have to worry about them.

EEOL Erase till end of line.

TRUE VALUE ?DOSIO

```
: EEOL        ( --- )
    COLS ?DOSIO + #OUT @ - SPACES ;

DEFER CCR   ' CR IS CCR     \ Conditional carriage return
DEFER .SRCCR   ' CR IS .SRCCR   \ Print conditional carriage return
DEFER .DEFSRC          \ Nothing for now, may be set to display the source for the
   ' NOOP IS .DEFSRC       \   current definition.
DEFER .SRCDEF   ' NOOP IS .SRCDEF
VARIABLE DEFCFA         \ Hold cfa of the current word.
VARIABLE PFASAV         \ Current offset into definition.
2VARIABLE CTIME         \ Store current time
   GETTIME CTIME 2!
2VARIABLE CDATE         \ Store current date
   GETDATE CDATE 2!

$.R     Print a string and then followed by spaces to n1 columns. : $.R   ( addr len n1 -- )
   TUCK UMIN          \ addr n1 min(len, n1)
   TUCK -         \ addr min n1-min
   -ROT TYPE          \ Type the string
   SPACES ;           \ Fill with spaces to column n1.

$.L     Print a string right-justified in a field of n1 columns. : $.L   ( addr len n1 -- )
   TUCK UMIN
   TUCK - SPACES       \ Print spaces first
   TYPE ;             \ Followed by the string

DOES?   Return the pfa of the word at cfa and a flag.  The flag is true if the word at cfa is a DOES word like FORTH.
```

```
: DOES? ( cfa -- pfa f )
   DUP >BODY          \ cfa pfa --
   SWAP @REL>ABS      \ Absolute address of DOES call
   @REL>ABS           \ Absolute address of DOES inner interpreter
   ['] FORTH          \ cfa of FORTH, a DOES word
   @REL>ABS @REL>ABS   \ Same inner interpreter?
   = ;

' HEX @REL>ABS         \ Absolute address of NEST
CONSTANT 'DOCOL        \ Call it 'DOCOL as used in  figForth.
```

>.ID Search the code dictionary space below addr for a colon word. If found, display its name.

```
: >.ID     ( addr -- )
   DUP 200 U<         \ Addr cannot be less than 100H
   IF DROP EXIT THEN      \ If so, not a word.
   128 0 DO           \ Scan down the memory to find the next colon definition
      DUP @REL>ABS    \ Try next cell as an absolute address
      'DOCOL =        \ Does it look like a colon word?
      IF  LEAVE       \ Yes.  Found one.
      ELSE 1- THEN       \ No.  Try next byte.
   LOOP               \ Its unlikely that there is not a single colon word in a
                \   256 byte area.
   >NAME .ID ;        \ Display its name
```

AT Move the cursor to the column and row specified. It uses a special property of DOES> to act something like a deferred word. AT is compiled as a regular colon word, except that DOES> is compiled as (;CODE). After AT is defined, it is executed immediately. The effect is that the code field of AT is modified from a colon word to its own daughter word: CALL the routine to execute the code sequence >R 2DUP R> ... in the definition of AT after DOES>. When AT is later executed again, it does what is listed after DOES>. The interesting thing about this complicated scheme is that the two bytes after the CALL instruction in the code field of AT are left open. The parameter field was used to store the segment offset pointing to AT's colon list. If this segment offset is replaced by the segment offset of

another colon word, the sequence >R 2DUP R> PERFORM will execute that colon word before doing #LINE! and #OUT ! .

This is a convoluted way to define a deferred word with a set of prescribed functions. Don't feel bad if you do not understand it. Nobody really does. Since it worked in F83, it was made to work in F-PC.

Now, AT can be re-vectored to any desired word which will consume the column and row number from the stack and do the machine dependent cursor movement. AT will then automatically adjust #LINE and #OUT so that F-PC knows exactly where the cursor is. AT, DARK, and PAGE have machine dependent behaviors which are encapsulated in the deferred part, and a machine independent part defined after DOES>. The deferred part can be vectored to different output routines dependent on the hardware.

```
: AT       ( col row -- , 0 0 is upper left corner )
   DOES>       \ Build a CALL DODOES structure to execute the following words.
   >R 2DUP        \ At run-time move the segment offset of deferred function off stack
          \   so we can duplicate col and row.
   R> PERFORM      \ Bring back deferred word and execute it.
   #LINE !        \ Reposition the line and column
   #OUT ! ;       \   counters.

AT            \ Rewrite code field of AT so that AT calls the routine at >R.
' 2DROP IS AT          \ Init deferred execution to 2DROP, nullifying 2DUP.
```

DARK Similar to AT. Clear the CRT screen and move the cursor to the home position at the upper left corner of the screen.

```
: DARK  ( -- )
   DOES>          \ Allow it to be deferred
   PERFORM        \ Perform the deferred function first
   #LINE OFF          \ Then adjust line and column pointers
   #OUT OFF   ;
```

```
DARK                \ Rewrite cfa of DARK and make it behave like a deferred word
' NOOP IS DARK      \ Init the deferred part of DARK
```

?DARK Clear the screen when a key is pressed on the keyboard.

```
: ?DARK ( -- )
   KEY? 0=
   IF DARK CR THEN ;
```

```
DEFER AT?  ( -- col row )    \ Return the current cursor position.
DEFER -LINE          \ Delete till end-of-line
```

SAVECURSOR Save all the cursor attributes and position on the return stack so that they can be later restored.

```
: SAVECURSOR   ( -- )            \ save all of the current cursor stuff
   2R>          \ ES:IP of the word which uses SAVECURSOR
   @> ATTRIB >R            \ save attribute
   GET-CURSOR >R           \ cursor shape
   @> #OUT @> #LINE 2>R    \ and position
   2>R ;           \ Restore ES:IP so SAVECURSOR can return safely.
```

RESTCURSOR Restore cursor attributes and position.

```
: RESTCURSOR   ( -- )            \ restore all of the cursor stuff
   2R>
   2R> AT                  \ restore position
   R> SET-CURSOR           \ shape
   R> ATTRIB !             \ and attribute
    2>R ;
```

0 VALUE ?DOINGMAC Are we doing a macro?

PAGE Start a new page. A Form-Feed for printer. NOOP for screen display.

```
VARIABLE #PAGE        \ Page counter

: PAGE     ( -- )
   DOES>          \ Define prescribed function for PAGE
   PERFORM           \ Do deferred part of work
   1 #PAGE +!          \ Regular work.  increment page count.
   #LINE OFF           \ Adjust line and character counts
   #OUT OFF   ;
PAGE
```

FORM-FEED Print a form-feed character.

```
: FORM-FEED ( -- )
   CONTROL M EMIT     \ A carriage-return
   CONTROL L EMIT  ;     \ A form-feed

' FORM-FEED IS PAGE       \ Deferred function in PAGE
```

?PAGE Form-feed if line counter is not zero. This forces a page to be skipped. If line number is zero, paging is not necessary.

```
: ?PAGE ( -- )
   #LINE @       \ Form-feed only if #line is not zero.
   IF  PAGE
   THEN   ;
```

>NAME.ID Find the name of a word whose cfa is on the stack. Do a carriage-return/line-feed if near the end of a line. Also store the length of name in NLEN for later use.

```
VARIABLE NLEN          \ Store length of last word name found by >NAME.ID.
0 VALUE ?DEFATTRIB     \ True if cursor attributes are already set.

: >NAME.ID  ( cfa -- )
   >NAME          \ Find nfa of the word
   DUP YC@ 31 AND DUP \ Get the length of name string
   ?LINE          \ Do a CR-LF near end of line
   NLEN !         \ Remember the length
   ?DEFATTRIB
   IF  %.ID       \ Generic word to print a name.
   ELSE    .ID    \ Deferred to %.ID but changed to show color if
   THEN ;         \   the colorizer is turned on.

DEFER (SEE)            \ Deferred word of the decompiler

DEFER INSTALLSTUFF     \ Task chain for F-PC installation
   ' NOOP IS INSTALLSTUFF
DEFER UNINSTALLSTUFF       \ Task chain to remove F-PC
   ' NOOP IS UNINSTALLSTUFF    \   installation effects
```

GRAPHCHAR Ask the user to enter a key and return the key code and a true flag.

```
: <GRAPHDUMMY>  ( --- )
   CR ." Enter a KEY " KEY TRUE ;

DEFER GRAPHCHAR        ' <GRAPHDUMMY> IS GRAPHCHAR
```

Following is a list of words which are used to specify the attributes of characters displayed on the screen. Since different video cards require different attribute assignments, the attributes are defined for all the cards and are vectored to the proper words when the type of video card installed in the system is determined.

```
   DEFER >ATTRIB1        ' NOOP IS >ATTRIB1
   DEFER >ATTRIB2        ' NOOP IS >ATTRIB2
   DEFER >ATTRIB3        ' NOOP IS >ATTRIB3
   DEFER >ATTRIB4        ' NOOP IS >ATTRIB4
   DEFER >ATTRIB5        ' NOOP IS >ATTRIB5
   DEFER >ATTRIB6        ' NOOP IS >ATTRIB6
   DEFER >ATTRIB7        ' NOOP IS >ATTRIB7
   DEFER >ATTRIB8        ' NOOP IS >ATTRIB8

DEFER >NORM          \ Default setting of character attribute.
   ' NOOP IS >NORM
DEFER >REV           \ Default setting of reversed video
    ' NOOP IS >REV
DEFER >NORMBG        \ Default setting of normal background
   ' NOOP IS >NORMBG
DEFER DOBUTTON       \ Action when mouse button is pressed
   ' NOOP IS DOBUTTON
0 VALUE MOUSEFLG     \ Is the mouse currently turned on?
```

The following variables are used to store the current state of F-PC so that it can temporarily jump to DOS. When we return from DOS, F-PC can be restored to the state before it jumped to DOS.

```
DECIMAL
VARIABLE RESTBASE      10 RESTBASE !
VARIABLE RESTCAPS      RESTCAPS ON
VARIABLE RESTTABS      8 RESTTABS !
```

```
VARIABLE RESTLMRG          RESTLMRG OFF
VARIABLE RESTRMRG          70 RESTRMRG !
VARIABLE RESTSTAT          RESTSTAT OFF
VARIABLE STATV         STATV OFF
```

SAVESTATE Save a set of F-PC system and user variables before changing the environment.

```
: SAVESTATE ( -- )
   BASE @ RESTBASE !
   CAPS @ RESTCAPS !
   LMARGIN @ RESTLMRG !
   RMARGIN @ RESTRMRG !
   TABSIZE @ RESTTABS !
   STATV   @ RESTSTAT ! ;
```

RESTORESTATE Restore the F-PC system from the values stored in the RESTxxx variables.

```
: RESTORESTATE  ( -- )
   RESTSTAT @ STATV !
   RESTBASE @ BASE !
   RESTCAPS @ CAPS !
   RESTLMRG @ LMARGIN !
   RESTRMRG @ RMARGIN !
   RESTTABS @ TABSIZE ! ;
```

DEFAULTSTATE Initialize the set of RESTxxx variables.

```
: DEFAULTSTATE     ( -- )
   RESTSTAT ON
```

```
   10 RESTBASE !
   RESTCAPS ON
   8 RESTTABS !
   RESTLMRG OFF
   70 RESTRMRG !
   RESTORESTATE ;
```

?DOSTOP If flag is true, restore the F-PC state from the RESTxxx variables and abort to the text interpreter.

```
: ?DOSTOP   ( f -- )
   IF  RESTORESTATE
      TRUE ABORT" Stopped"
   THEN    ;
```

?KEYPAUSE   Pause. Restart F-PC only if ESC key is pressed.

```
: ?KEYPAUSE ( -- )
   KEY?
   IF  KEY 27 = ?DOSTOP
      KEY 27 = ?DOSTOP    \ Make sure the first key was not missed.
   THEN    ;
```

$>TIB   Move a string at addr into the terminal input buffer and prepare it for interpretation.

```
: $>TIB ( addr -- )
   COUNT          \ Save the count
   DUP #TIB !         \ Initialize #TIB to interpreter the string at addr
   TIB SWAP CMOVE      \ Copy string to TIB
   >IN OFF  ;         \ Init interpreter pointer
```

7.3. VOCABULARY WORDS

Source code is in the file VOCABS.SEQ. In Forth, a dictionary is the entire collection of words which can be referenced by their names for execution or compilation. The dictionary is divided in to vocabularies, which are groups of related words which can be independently referenced. A word can be uniquely identified by its name and the vocabulary it belongs to. The use of vocabularies has two main purposes: to allow the user to define and use words of the same name but in different context; and to reduced the range of the dictionary search. By grouping related words into a vocabulary, large Forth programs can be organized into modules and the undesirable side effects of interaction among modules can be minimized.

The vocabulary structure in F-PC was developed by Bill Ragsdale and included as a technical proposal in Forth-83 Standards. It was adopted in F83 and many other commercial Forth systems. Its most important feature is the use of a vocabulary stack to control the order of vocabularies to be searched by FIND. The stack has #VOC entries which can hold that many vocabularies. The top of vocabulary stack is called CONTEXT, which contains the pfa of a vocabulary which will be searched first by FIND. If a word cannot be found in this 'CONTEXT' vocabulary, the next vocabulary on the vocabulary stack will be searched. The search continues until either the word is found in a vocabulary included in the vocabulary stack or until the vocabulary stack is exhausted.

When a vocabulary word, like FORTH, EDITOR, ASSEMBLER, etc, is executed, the vocabulary replaces the existing context vocabulary. ALSO duplicates the context vocabulary and pushes it on the vocabulary stack so that it will not be replaced by the next vocabulary. F-PC 3.5 displays the vocabulary stack in the top-right corner of the screen. You should try some of the vocabulary words and watch the changes in the displayed vocabulary stack picture.

Another system variable CURRENT also contains the pfa of a vocabulary. New words are added to this vocabulary by linking its head to the top of a thread in this vocabulary. By vectoring CURRENT to various vocabularies, the user can freely add words to any vocabulary. By changing the contents of the vocabulary stack, he can access all words in the vocabularies which are prioritized dynamically.

The word set below includes tools to create vocabularies, to define the context of vocabulary search, to select the current vocabulary, and to seal the dictionary so that an end user has only a limited word set to access.

ROOT The root vocabulary that is always available in the vocabulary stack. It is generally initialized as the last vocabulary to be searched; i.e., at the bottom of the vocabulary stack. It must contain words to manipulate the vocabulary stack and words to select context and current vocabularies.

VOCABULARY ROOT

ROOT DEFINITIONS \ Following words are defined in the ROOT vocabulary

ALSO vocabulary stack -> (voc1 -- voc1 voc1)

Duplicate the CONTEXT vocabulary at the top of vocabulary stack.

```
: ALSO  ( -- )
   CONTEXT          \ Top of vocabulary stack
   DUP 2+         \ Next vocabulary
   #VOCS 2- 2*          \ Less two ends
   CMOVE>   ;        \ Push the vocabulary stack down thus duplicate the top
```

ONLY vocabulary stack -> (root .. voc1 -- root)

Clean the vocabulary stack, leaving only ROOT vocabulary.

```
: ONLY  ( -- )
   ['] ROOT >BODY        \ Address of ROOT thread table
   CONTEXT #VOCS 1- 2*     \ Erase the vocabulary stack
   2DUP ERASE
   + !          \ Put ROOT at the bottom of vocabulary stack
   ROOT ;        \ Put ROOT at the top of the vocabulary stack
```

SEAL vocabulary stack -> (root .. voc1 -- voc1)

Remove all vocabularies from the vocabulary stack. Leave only voc1 as the CONTEXT vocabulary. This prevents access to any other vocabularies unless an escape word is included in the voc1 vocabulary.

```
: SEAL     ( -- )
   ' >BODY       \ Address of vocabulary whose
                 \   name follows SEAL
   CONTEXT #VOCS 2* ERASE  \ Erase vocabulary stack
```

```
   CONTEXT !   ;          \ Make the selected vocabulary the only one to
                \   be searched.
```

PREVIOUS vocabulary stack -> (voc1 voc2 -- voc1)

Pop the context vocabulary off the vocabulary stack.

```
: PREVIOUS  ( -- )
   CONTEXT DUP        \ Top of vocabulary stack
   2+ SWAP            \ Next vocabulary
   #VOCS 2- 2* CMOVE      \ Pop the vocabulary stack
   CONTEXT #VOCS 2- 2* +   \ Last vocabulary above ROOT
   OFF   ;            \ Erase it
```

FORTH vocabulary stack -> (voc1 voc2 -- voc1 FORTH)

A pseudonym of FORTH in the ROOT vocabulary, to assure FORTH will always be available.

```
: FORTH ( -- )
   FORTH ;
```

DEFINITIONS vocabulary stack -> (voc1 -- voc1), (current = voc1)

Set the current vocabulary to the same vocabulary as the context vocabulary. A pseudonym of the same word in FORTH vocabulary.

```
: DEFINITIONS   ( -- )
   DEFINITIONS   ;
```

ORDER Display the contents of the vocabulary stack and the current vocabulary. It shows the environment of the text interpreter and compiler.

```
: ORDER ( -- )
```

```
  CR ." Context: "
  CONTEXT          \ Scan vocabulary task
  #VOCS 0 DO       \   starting from CONTEXT
    DUP @ ?DUP     \ If vocabulary cfa is not 0,
    IF  BODY> >NAME .ID    \   display its name.
    THEN  2+       \ Next vocabulary
  LOOP DROP
  CR ." Current: "
  CURRENT @ BODY>       \ Cfa of CURRENT vocabulary
  >NAME .ID  ;          \ Display its name
```

VOCS Display a list of all vocabularies defined in the dictionary. In the parameter field of a vocabulary word, there is a thread table containing the last link field addresses of words at the ends of the vocabulary threads. After the thread table there is a vocabulary link field, which points to the same field in the previous vocabulary. The first vocabulary FORTH has a 0 in its vocabulary link field, terminating the vocabulary link chain. The system variable VOC-LINK contains a pointer pointing to the vocabulary link field of the last vocabulary defined. This vocabulary link chain allows new vocabularies to be added to F-PC system and old vocabularies to be removed gracefully by FORGET.

```
: VOCS    ( -- )
  CR VOC-LINK @      \ Scan the vocabulary chain
  BEGIN   DUP        \ Vocabulary link cell
    #THREADS 2* -    \ The cell is at the end of the of the vocabulary thread
          \   table.  Move back to pfa.
    BODY> >NAME .ID    \ Display vocabulary name.
    ?CR          \ New line if necessary
    @ DUP 0=     \ End of VOC-LINK?
  UNTIL   DROP  ;    \ Continue for all vocabularies.
```

HIDDEN vocabulary stack -> (voc1 voc2 -- voc1 HIDDEN)

The HIDDEN vocabulary. It sets CONTEXT to HIDDEN when executed. The HIDDEN vocabulary is where the dirty linen in F-PC is hidden. The words defined in this vocabulary are tool words needed by the system but rather useless

for day-to-day programming.

```
ONLY FORTH ALSO DEFINITIONS \ HIDDEN is defined in FORTH

VOCABULARY HIDDEN
```

BUG vocabulary stack -> (voc1 voc2 -- voc1 BUG)

The BUG vocabulary containing words to support the debugger in F-PC. It sets CONTEXT to BUG when executed.

```
VOCABULARY BUG
```

7.4. TASK CHAINS

The words in FERERS.SEQ give you the ability to build a chain of words that will be performed in sequence when a normal DEFERred word is executed. This is a very useful technique to enhance the function of a word after the word is compiled. It is convenient to add more functions to an existing word rather than define a new word and recompile all the words which made reference to the old word.

DEFERS A task chain is the word compiled in the parameter field of a deferred word. A new definition is then installed into the deferred word with IS. An example of its uses is as the following:

```
  DEFER JUNK  ' NOOP IS JUNK

  : JUNK1 DEFERS JUNK .. MYJUNK1 .. ;

  ' JUNK1 IS JUNK

  : JUNK2 DEFERS JUNK .. MYJUNK2 .. ;

  ' JUNK2 IS JUNK
```

When JUNK is executed, the word NOOP will be executed first, then MYJUNK1, followed by MYJUNK2.

```
: DEFERS   ( -- )

  ' DUP @REL>ABS     \ Find the interpreter of the following deferred word

  [ ' KEY @REL>ABS ]     \ Is it a user deferred word like
```

```
   LITERAL          \   KEY , a user deferred word?
   = >R
   >BODY @          \ Get the cfa of the word vectored
   R>           \   to by the deferred word
   IF UP @ + @ THEN       \ Adjust cfa for a user deferred  word.
   X, ; IMMEDIATE     \ Compile the vectored word in the place of the
              \   deferred word.
```

UNDEFER Remove the last entry from a task chain. It must be used on the word defined later than the word you want to remove:

```
    UNDEFER JUNK2 <enter>
```

The above command removes JUNK1 from the chain established in the above example..

```
: UNDEFER   ( -- )
   ' DUP @REL>ABS     \ Find the interpreter of the following deferred word.
   [ ' KEY @REL>ABS ]     \ Is it a user deferred word?
   LITERAL
   = >R
   >BODY         \ Convert cfa to pfa
   R> IF @ UP @ + THEN     \ Adjust for a user deferred word
   DUP @          \ Get cfa of the vectored word
   DUP @REL>ABS       \ Is it a word installed by DEFERS?
   [ ' DEFERS @REL>ABS ]
   LITERAL <>         \ No.  Can't UNDEFER
   ABORT" Can only UNDEFER through a : (colon) definition."
   >BODY @          \ Follow the task chain up
   XSEG @ + 0 @L      \ Replace end of chain by the
   SWAP ! ;          \   one next to the end.
```

7.5. F-PC DECOMPILER

Source code is in the file DECOM.SEQ. A Forth decompiler is a utility program that translates executable Forth code back into source code. This is impossible in most high level languages, since traditional compilers produce more object code than source, commonly called code expansion, which is generally an irreversible process. However, in Forth it is quite easy. The correspondence between object and source code is almost one to one, failing only in various Forth control structures and special compiling directives. This F-PC decompiler was written with modifiability in mind, so if you add your own special compiling words, it will be easy to change the decompiler to include them. This code is highly implementation dependant, and will NOT work on other Forth system. To invoke the decompiler, use the commands:

```
SEE  <name>
```

where <name> is the name of a Forth word.

This decompiler was derived from the F83 decompiler. However, F-PC was designed specifically with decompilation in mind. Many words which could be made to compile the same run-time code word were deliberately given their own code so that the decompiler can distinguish them with out ambiguity. The F-PC decompiler can thus completely recover the source code. The source code display is also enhanced to show the nested structures with proper indentation. This decompiler can almost replace the source code except it cannot recover the comments in the source code.

First, a set of words which provide formatting aids:

+TAB Increase LMARGIN to indent the following lines in a nested control structure.

```
: +TAB     ( -- )  8 LMARGIN +! ;
```

-TAB Decrease LMARGIN to close this level of nested structure.

```
: -TAB     ( -- )
   LMARGIN @
   8 - 0 MAX         \ LMARGIN should not become
   LMARGIN ! ;       \   negative.
```

CRTAB Do a CR-LF if RMARGIN is not zero.

```
: CRTAB          ( -- )  RMARGIN @ ?LINE ;

HIDDEN DEFINITIONS        \ Following words should be hidden from normal uses.
0 VALUE DECOMSEG       \ Hold the base segment address of a colon list under
            \   decompilation.
0 VALUE ?DEBUG         \ Are we in the debugging mode?
19 VALUE SPLIT-L#         \ Row number where screen is split into two windows.

: SRCEEOLCR    EEOL CRLF ;     \ Erase rest of the current line and more to next line.

: INIT-SPLIT    ( --- )             \ initialize the split line as 6 lines up
   DEFERS INITSTUFF
   ROWS DUP 4 / - =: SPLIT-L# ;    \ Use only top quarter of the screen for decompiling.

' INIT-SPLIT IS INITSTUFF
```

DECOMSEG@ Fetch an integer at the offset addr in the decompiler segment.

```
: DECOMSEG@ ( addr -- )
   DECOMSEG SWAP @L ;
```

ASSOCIATIVE: An associative memory word. It must be followed by a table of values to be looked up. At run-time, the values stored in this table are searched for a match. If one if found, the index to that value is returned. If no match is found, then the number of entries, i.e., max index + 1 is returned. This is the inverse of an array.

```
: ASSOCIATIVE:  ( n -- )
  CONSTANT          \ Pretend to define a constant which is the array size.
  DOES>   ( n -- index )     \ Match n against array elements
```

```
  DUP @          \ n pfa count --
  -ROT DUP @ 0           \ count n pfa count 0 --
  DO   2+
    2DUP @ =       \ count n pfa' f --
    IF  2DROP DROP  \ Find a match,
      I 0 0   LEAVE   \   leave with the index
    THEN
  LOOP   2DROP   ;       \ Leave index or n+1
```

In principle, decompiling a colon word is straightforward because words in a colon definition are compiled into a list of code field address. We just have to find the name field addresses of the words in this list and display their names. This is done by .WORD . However, there are lots of other structures compiled into a colon definition, like literals, string literals, and control structures. F-PC is designed so that the decompiler can recover all these structures and display them faithfully, identical to the source code. Therefore, it needs a set of words which can deal with each and every type of these structures other than code field addresses.

The decompiler locates the list of a colon definition and uses DECOMSEG to hold the segment pointer to the first word in the list, and a simulated IP to scan the list. IP always points to the word currently being decompiled. It is incremented by 2 or whatever it takes to point to the next word to continue the decompilation until the end of the list is reached.

In F-PC 3.5, the decompiler and the debugger are integrated more closely and both are simplified and enhanced as a result.

.WORD Display the name of a word, and bump the simulated IP by 2.

```
: .WORD ( IP -- IP' )
  DUP DECOMSEG@       \ Get the next cfa in list
  >NAME YC@ 64 AND       \ Is it an immediate word?
  IF  DUP YC@ 31 AND  \ Yes.  Add the phrase
    10 + ?LINE     \   [COMPILE] which causes
    ." [COMPILE] "     \   immediate words to be
  THEN             \   compiled.
```

```
   DUP DECOMSEG@       \ In all cased, display the
   >NAME.ID            \   name of this word.
   2+  ;
```

(LIT+) Skip a literal word and the literal value.

```
: (LIT+)   ( IP -- IP+4 )
   6 ?LINE        \ CR if not enough room
   4 + ;          \ Skip the literal
```

.LIT Display the value of an in-line integer literal.

```
: .LIT  ( IP -- IP' )
   (LIT+)          \ Skip the literal structure
   DUP 2-          \ Back up to the literal value
   DECOMSEG@ . ;       \ Fetch and display it
```

.IF Display the IF structure.

```
: .IF   ( IP -- IP' )
   CRTAB           \ Start a new line with proper tabs.
   ." IF "         \ Display IF
   (LIT+)          \ Skip the branch address
   TAB +TAB ;          \ Do a tab now.  Indent next lines
```

.ELSE Display the ELSE structure.

```
: .ELSE ( IP -- IP' )
   -TAB            \ Move up a level of indentation
```

```
   CRTAB ." ELSE "        \ Display heading ELSE
   (LIT+)             \ Skip over the branch address
   TAB +TAB ;         \ Tab and indent next lines
```

.DO Display the DO structure.

```
: .DO      ( IP -- IP' )
   CRTAB ." DO  "      \ Display heading DO
   (LIT+)             \ Skip DO address
   TAB +TAB ;         \ Tab and indent next lines
```

.?DO Display the ?DO structure.

```
: .?DO     ( IP -- IP' )
   CRTAB ." ?DO  "        \ Similar to DO
   (LIT+) TAB +TAB ;
```

.LOOP Close a LOOP structure.

```
: .LOOP ( IP -- IP' )
   -TAB               \ Move up a level of indentation
   CRTAB ." LOOP "        \ Display heading LOOP
   (LIT+)             \ Skip the LOOP address
   TAB ;              \ Tab and continue
```

.+LOOP Close a +LOOP structure.

```
: .+LOOP    ( IP -- IP' )
   -TAB CRTAB
```

```
   ." +LOOP "
   (LIT+) TAB ;
```

.WHILE Display a WHILE structure.

```
: .WHILE    ( IP -- IP' )
   -TAB CRTAB          \ Similar to the ELSE structure
   ." WHILE "
   (LIT+) TAB +TAB ;
```

.REPEAT Close an indefinite loop structure.

```
: .REPEAT   ( IP -- IP' )
   -TAB CRTAB          \ Similar to LOOP
   ." REPEAT "
   (LIT+) TAB ;
```

.UNTIL Close an indefinite loop structure.

```
: .UNTIL    ( IP -- IP' )
   -TAB CRTAB          \ Similar to LOOP
   ." UNTIL "
   (LIT+) TAB ;
```

.AGAIN Close an infinite loop structure.

```
: .AGAIN    ( IP -- IP' )
   -TAB CRTAB          \ Similar to LOOP
   AGAIN "
```

```
   (LIT+) TAB ;
```

.BEGIN Begin an indefinite or infinite loop structure.

```
: .BEGIN    ( IP -- IP' )
   CRTAB 2+          \ Skip only the BEGIN word
   ." BEGIN "        \ Display trade mark
   TAB +TAB ;        \ Tab and indent next lines
```

.THEN Close a conditional branch structure.

```
: .THEN    ( IP -- IP' )
   -TAB CRTAB        \ Recover a level of indentation
   2+            \ Skip over THEN only
   ." THEN "         \ Display
   TAB ;             \ Tab and continue
```

In F-PC 3.5, the CASE structure is included in the decompiler. The following displaying words are added to take care of the case structure:

```
: .CASE    ( IP -- IP' )   CRTAB ." CASE " 2+ TAB ;

: .OF      ( IP -- IP' )   CRTAB ." OF " (LIT+) TAB +TAB ;

: .ENDOF    ( IP -- IP' )   -TAB CRTAB ." ENDOF " (LIT+) TAB ;

: .ENDCASE  ( IP -- IP' )   CRTAB ." ENDCASE " 2+ TAB ;
```

.QUOTE Handle a special case of COMPILE <word-name>, which compiles the cfa of COMPILE and the cfa of word-name.

```
: .QUOTE    ( IP -- IP' )
   .WORD          \ Display COMPILE
   .WORD ;        \ Display word to be compiled
```

."X$" Print a string at offset addr in the decompiler segment, and adjust addr to the end of the string, while aligning it. Prepend a " and a space, and append a " and a space to the string.

```
: ."X$"    ( addr -- addr+len )
   DUP "" EMIT SPACE      \ Print " and a space
   DECOMSEG SWAP      \ seg:addr
   2DUP C@L 1+ >R     \ Get string count
   ?CS: "BUF R@ CMOVEL \ Copy string to "BUF in code seg
   R> DUP 1 AND +     \ Adjust to cell boundary
   +            \ addr+len
   "BUF COUNT TYPE        \ Display the string
   "" EMIT SPACE ;    \   followed by " and space
```

.STRING Display a word with an in-line string argument.

```
: .STRING." ( IP -- IP' )
   2+           \ Skip the string word
   DECOMSEG OVER C@L   \ Get string count
   5 + ?LINE          \ CR if needed
   "." EMIT           \ Print a '.'
   ."X$" ;            \ Print the string
```

.STRING" Display an in-line quote string defined by " . The string is compiled in the Code Space.

```
: .STRING"  ( IP -- IP' )
```

```
   2+ DUP 2+ SWAP      \ IP+4 IP+2 --
   DECOMSEG@           \ Fetch the string address in the Code Segment
   DUP C@ 4 + ?LINE        \ New line if needed
   "" EMIT SPACE       \ Print opening "
   COUNT TYPE          \ Print the string
   "" EMIT SPACE ;     \ Terminating "
```

.STRING"" Display a double quote string which is compiled in-line like a dot-quote string in the List Segment.

```
: .STRING"" ( IP -- IP' )
   2+              \ Bump IP
   DECOMSEG            \ Get segment pointer
   OVER C@L 5 + ?LINE      \ New line?
   "" EMIT ."X$" ;         \ Print string from List Space
```

.ABORT" Display ABORT" and its in-line error message.

```
: .ABORT"   ( IP -- IP' )
   2+ DUP
   DECOMSEG@
   C@ 10 + ?LINE
   ." ABORT" ."X$" ;
```

.(;CODE) Continue to decompile a defining word if the inner interpreter is defined in high level. If the inner interpreter is in low level code, stop decompiling.

```
: .(;CODE)  ( IP -- IP' )
   .WORD               \ Decompile itself
   DOES?               \ Next word is DOES> ?
   IF  ." DOES> "          \ Yes.  Show it and continue.
```

```
   ELSE    DROP          \ No.  Stop decompiling.
     FALSE         \ Replace IP by 0.
   THEN  ;
```

.UNNEST Reach the end of a colon definition. Display semicolon and replace IP with 0, which stops decompilation.

```
: .UNNEST   ( IP -- IP' )
   ." ; "              \ Show semicolon
   DROP   0   ;         \ Stop decompiling
```

.FINISH Display the current word and quit. Used by words which also terminate a colon definition.

```
: .FINISH     ( IP -- IP' )
   .WORD            \ Display word name
   DROP   0   ;         \ Stop decompiling
```

EXECUTION-CLASS This table lists all the special cases that must be decompiled differently from ordinary Forth words like DUP and +, etc. At runtime, if the simulated IP points to a word in this group, the corresponding index from this table will be returned, and placed upon the stack. If there is no match, then the last index + 1 is returned.

```
27 ASSOCIATIVE: EXECUTION-CLASS ( cfa -- index )
  ( 0 )'  (LIT)      ,        ( 1 )'  ?BRANCH     ,
  ( 2 )'  BRANCH     ,        ( 3 )'  (LOOP)      ,
  ( 4 )'  (+LOOP)    ,        ( 5 )'  (DO)        ,
  ( 6 )'  COMPILE    ,        ( 7 )'  (.")        ,
  ( 8 )'  (ABORT")   ,        ( 9 )'  (;CODE)     ,
  ( 10 )'  UNNEST    ,        ( 11 )'  (")        ,
  ( 12 )'  (?DO)     ,        ( 13 )'  (;USES)    ,
  ( 14 )'  ?UNTIL    ,        ( 15 )'  ?WHILE     ,
  ( 16 )'  DOAGAIN   ,        ( 17 )'  DOREPEAT   ,
```

```
( 18 )' DOBEGIN    ,      ( 19 )' DOTHEN     ,
( 20 )' (X")       ,      ( 21 )' <'>        ,
( 22 )' (IS)       ,      ( 23 )' (OF)       ,
( 24 )' DOENDOF    ,      ( 25 )' DOCASE     ,
( 26 )' DOENDCASE  ,
```

.EXECUTION-CLASS This giant case statement handles the special case decompiling needed. Each entry corresponds to an entry in the previous EXECUTION-CLASS associative table. The function of each of these words is to decompile the current word that the simulated IP is pointing to, and advance the simulated IP accordingly. If no match is found in the table, .WORD is used.

```
: .EXECUTION-CLASS  ( index -- )
   0 MAX 21 MIN       \ Check range of index
   EXEC:              \ Index into the following table and execute select word.
   ( 0 )   .LIT       ( 1 )   .IF
   ( 2 )   .ELSE      ( 3 )   .LOOP
   ( 4 )   .+LOOP     ( 5 )   .DO
   ( 6 )   .QUOTE     ( 7 )   .STRING."
   ( 8 )   .ABORT"    ( 9 )   .(;CODE)
   ( 10 )  .UNNEST    ( 11 )  .STRING"
   ( 12 )  .?DO       ( 13 )  .FINISH
   ( 14 )  .UNTIL     ( 15 )  .WHILE
   ( 16 )  .AGAIN     ( 17 )  .REPEAT
   ( 18 )  .BEGIN     ( 19 )  .THEN
   ( 20 )  .STRING""  ( 21 )  .[']
   ( 22 )  .IS        ( 23 )  .OF
   ( 24 )  .ENDOF     ( 25 )  .CASE
   ( 26 )  .ENDCASE   ( 27 )  .WORD   ;
```

.PFA Decompiles a colon definition which contains a list of code field addresses. This is the colon word decompiler. TOPCR and other values are added in F-PC 3.5 to support the debugger.

```
0 VALUE PFALINE

0 VALUE DIDPFA

0 VALUE TOPCRS

0 VALUE DUMMYCRS

0 VALUE #EMPTY


: TOPCR       ( --- )
   DUMMYCRS
   IF        DECR> DUMMYCRS
      OFF> #OUT
   ELSE    #LINE @ SPLIT-L# 2- >=
      IF        SPLIT-L# 1- SAVE!> ROWS     \ save ROWS and set to split
         0 2 AT                    \ move to third line
          -LINE                    \ scroll upper portion
         RESTORE> ROWS             \ restore ROWS
         0 SPLIT-L# 2- AT          \ move to split line
      ELSE    SRCEEOLCR
      THEN
   THEN    INCR> TOPCRS ;


: .PFA     ( list-segment -- )
   >BODY   @ +XSEG        \ Absolute segment address of list
   =: DECOMSEG 0       \ Put it in DECOMSEG for decompiler
   SAVESTATE          \ Save current system parameters
   8 LMARGIN !        \ Define field of decompiled word
   COLS 10 - RMARGIN !
   #LINE @ =: TOPCRS
   SAVE> CR
```

```
?DEBUG
IF  ['] TOPCR IS CR
   #EMPTY =: DUMMYCRS
THEN
BEGIN             \ Scan through the list of cfa's
   ?CR            \ New line if needed
   DUP PFASAV @ OVER =     \ If the list pointer agrees with address saved in
   IF     >ATTRIB4        \   PFASAV, show name in reverse video.
   THEN    DECOMSEG@       \ Get the cfa to be decompiled
   EXECUTION-CLASS         \ Which class?
   .EXECUTION-CLASS        \ Display it.
   >NORM           \ Restore normal video display
   OFF> ?DEFATTRIB
   DUP 0= KEY? OR
   ?DEBUG
   IF     #LINE @ SPLIT-L# 2- >=  \ hit bottom
     IF     DIDPFA
       IF     PFALINE SPLIT-L# 2- >=
         IF     PFALINE 10 -
           =: #EMPTY
         ELSE    TRUE OR
         THEN
         OFF> DIDPFA
       THEN
     THEN
   THEN
   PFALINE 12 < IF OFF> #EMPTY THEN
UNTIL   DROP           \ If not, decompile next cfa.
RESTORESTATE ;         \ Restore environment
```

The colon definitions are the most complicated type of Forth words. However, there are many other types of Forth words which have to be decompiled differently. The other types are simple and can be decompiled easily, except the code words and words defined by defining words defined by the user. (Does it make sense?) The following words display word types other than colon definitions.

.IMMEDIATE This indicates whether the current word is immediate or not.

```
: .IMMEDIATE   ( cfa -- )
   >NAME YC@ 64 AND       \ Test the immediate bit in name
   IF  ." IMMEDIATE"      \ Indicate it if so.
   THEN   ;
```

.CONSTANT Decompile a constant, and prints its value.

```
: .CONSTANT ( cfa -- )
   DUP >BODY ?        \ Print value in pfa
   ." CONSTANT "          \ Show type
   >NAME.ID   ;          \ Show name
```

.VALUE Decompile a VALUE word and print its value. A VALUE word is a cross between a constant and a variable. It is used as a constant, but its value can be changed like a variable.

```
: .VALUE    ( cfa -- )
   DUP >BODY ?        \ Identical to a constant
   ." VALUE "            \   except type name
   >NAME.ID   ;
```

.VARIABLE Decompile a variable, showing its address and value.

```
: .VARIABLE ( cfa -- )
   DUP C@ 232 =           \ A variable has CALL >NEXT
   IF  DUP >BODY .    \ Display pfa
      ." VARIABLE "      \ Show type
      DUP >NAME.ID    \ Show name
      ." Value = "       \ Show current value
      >BODY ?
   ELSE              \ If not CALL, unknown type.
   >NAME.ID  THEN ;       \ Show name only.
```

.: Decompile a high level colon definition.

```
: .:       ( cfa -- )
   ." : "              \ Show type first
   DUP >NAME .ID       \ Show name
   CR TAB .PFA   ;     \ Decompile word list
```

.DOES> Decompile a word defined by a CREATE DOES> word.

```
: .DOES>    ( cfa -- )
   DUP>R
   BODY> @REL>ABS      \ Get the segment address
   DUP R@ 2+ =         \ Self defining word?
   IF  R@ @ >NAME .ID  \ Yes, dig out the action word and show it.
   ELSE    DUP >.ID       \ Else, find and display the name
   THEN    R>DROP
   ." DOES> "          \ Show inner interpreter mark
   .PFA   ;           \ Decompile inner interpreter
```

.USER-VARIABLE Decompile a user variable, showing the offset from the base of the user area and its current value.

```
: .USER-VARIABLE    ( cfa -- )
   DUP >BODY ?        \ Show offset
   ." USER VARIABLE "      \ Show type
   DUP >NAME.ID       \ Show name
   ." Value = " >IS  ?   ;     \ Show value
```

.DEFER Tell the user that this is a deferred word and decompile its current definition.

```
: .DEFER    ( cfa -- )
   ." DEFERRED "       \ Show type
   DUP >NAME.ID       \ Show name
   ." IS "            \ Show vectored name
   >IS @ (SEE)  ;         \ Decompile vectored word
```

.USER-DEFER Tell the user that this is a user deferred word and decompile its current definition.

```
: .USER-DEFER   ( cfa -- )
   ." USER DEFERRED "      \ Similar to .DEFER
   DUP >NAME.ID
   ." IS "  >IS @ (SEE)  ;
```

.OTHER Decompile a word whose category is not known. Code words are recognized, as are words defined by defining words. The run-time portion of a word defined by a defining word is decompiled, since the parameter field is determined by the CREATE portion and cannot be deciphered. If all else fails, the word is listed as UNKNOWN.

```
: .OTHER    ( cfa -- )
   DUP >NAME.ID       \ Show name first
   DUP C@  232 <>      \ cfa doesn't contain a call for a code routine
```

```
  IF  DROP          \ Must be a code word
    ." is Code"       \ Show type
    EXIT          \   and give up.
  THEN
  DUP DOES?         \ Is this a DOES> word?
  IF  .DOES>        \ Yes.  Decompile as such.
    DROP EXIT
  THEN   2DROP
  ." is Unknown"  ;     \ Failed.  Tell the truth.
```

DEFINITION-CLASS Categorize different classes of words that the F-PC decompiler will handle. For each class, determined by the type of defining word used, the code field is identical. Thus the standard classes are recognized.

```
7 ASSOCIATIVE: DEFINITION-CLASS ( cfa -- index )
  ( 0 ) '  QUIT @REL>ABS ,      ( 1 ) ' #VOCS @REL>ABS ,
  ( 2 ) '  STATE @REL>ABS ,     ( 3 ) ' BASE @REL>ABS ,
  ( 4 ) '  CR @REL>ABS ,    ( 5 ) ' EMIT @REL>ABS ,
  ( 6 ) '  DECOMSEG @REL>ABS ,
```

.DEFINITION-CLASS A case structure containing routines that handle the decompilation of every class of words. The most useful, and of course the most common one is .: which decompiles colon definitions. If the class is not recognized, check to see if it is a CODE word or perhaps defined by a high level CREATE DOES> word.

```
: .DEFINITION-CLASS ( index -- )
  0 MAX 7 MIN EXEC:
  ( 0 )  .:            ( 1 )  .CONSTANT
  ( 2 )  .VARIABLE       ( 3 )  .USER-VARIABLE
  ( 4 )  .DEFER          ( 5 )  .USER-DEFER
  ( 6 )  .VALUE          ( 7 )  .OTHER    ;
```

((SEE)) Take an arbitrary code field address and decompile it based upon its definition class. Upon completion, it indicates whether or not the word is immediate.

```
: ((SEE))  ( cfa -- )
   SAVE>ATTRIB
   CR  DUP DUP @REL>ABS  \ Find its absolute address
   DEFINITION-CLASS      \ Determine its type
   .DEFINITION-CLASS     \  and display its definition
   .IMMEDIATE        \ Show immediacy
   RESTORE> ATTRIB ;

' ((SEE)) IS (SEE)        \ (SEE) is vectored to ((SEE)) for possible enhancement
FORTH DEFINITIONS
```

SEE The user interface of the decompiler. To decompile something type:

```
    SEE <word-name>
```

```
: SEE  ( -- )
   '              \ Find cfa of following word
   (SEE) ;        \ Decompile it
```

The F-PC debugger also uses this decompiler to display the contents of a colon definition to help the debugging process. As the debugger single steps through a colon definition, the word just about to be executed is high-lighted in the decompiled definition. It is very useful in giving the user a bird's eye view of the word under debugging. The following words support the debugger.

SHOWSRC Show the source for the current debugging word.

```
: SHOWSRC  ( -- )
   savecursor
    0 0 AT       \ Start at the top of screen
```

```
   ['] SRCEEOLCR IS CR
   ON> ?DEBUG
   DEFCFA  @ (SEE)     \ Decompile using debugging mode
   OFF> ?DEBUG
   KEY? 0=
   IF         #LINE @ SPLIT-L# 1- MIN
      SPLIT-L# 1- SWAP
      ?DO     CR EEOL \ Clear rest of screen
      LOOP
   THEN
   ['] CRLF IS CR
   0 SPLIT-L# 1- AT >ATTRIB4   \ Highlight the help menu
   ."  C-cont, D-done, F-forth, N-nest, Q-quit, S-skipto, U-unnest, X-source-on/off"
   EEOL >NORM          \
   restcursor ;
```

SRCCR Source CR for the debugger sub-screen scroll.

```
: SRCCR ( -- )
   0 SPLIT-L# AT       \ Place cursor at the top of lower window
   -LINE               \ Delete that line and scroll the screen up by a line
   0 ROWS 1- AT ;      \ Place cursor a lower left corner of screen
' SRCCR IS .SRCCR
```

SRCON Enable source printing during debugging.

```
: SRCON ( -- )
   ['] showsrc is .defsrc      \ Show decompiled source code
   ['] SRCCR   IS CCR ;        \ Special CR
```

SRCOFF Disable source printing during debugging.

```
: SRCOFF    ( -- )
   ['] noop    is .defsrc
   ['] CRLF    IS CCR ;
```

7.6. DUMP

Memory dumping is more complicated in F-PC than in F83 because it has to address the entire memory space in the AT/PC. The regular DUMP in F83 is retained to dump memory in the Code Segment. Special words XDUMP and YDUMP are defined to dump memory in the List Segment and the Head Segment. All these variations are derived from the generic dump utility word LDUMP which uses a seg:addr pair to specify the memory area to be dumped. A variable DUMPSEG is used to define the dump segment to facilitate accessing the extended memory range.

The dump utility gives you a formatted hex dump, 16 bytes to a line, with the ASCII text corresponding to the bytes on the right hand side of the screen. DUMPSEG is set accordingly.

VARIABLE DUMPSEG   Store the segment address from where memory contents are shown.

DUMPC@ Fetch a byte from the dump segment, given the offset address on the stack.

```
: %DUMPC@   ( addr -- b )
   DUMPSEG @          \ Get the dump segment address
   SWAP               \ Move offset address to top
   C@L ;              \ Fetch a byte with long fetch

DEFER DUMPC@    ' %DUMPC@ IS DUMPC@
```

.2W Print n in two digits followed by a space.

```
: .2W    ( n -- )
   0 <# # # #>       \ Convert two digits only
   TYPE SPACE  ;     \ Type them with a trailing space
```

D.2W Dump the contents of an array of n bytes from addr, in two digit pairs.

```
: D.2W   ( addr len -- )
   BOUNDS        \ Scan the range of memory
   ?DO I DUMPC@      \ Fetch byte from dump segment
      .2W        \ Display one byte at a time
   LOOP  ;
```

EMIT. Emit an ASCII character char. If the char is not printable then print a ".".

```
: EMIT. ( char -- )
   127 AND          \ Eliminate most significant bit
   DUP BL 126 BETWEEN \ Is it a control character?
   NOT IF
      DROP ASCII .      \ Yes.  Substitute with a period
   THEN EMIT ;       \ Print it
```

DLN Dump a line, consisting of 16 bytes of hex data followed by 16 bytes of ASCII string data.

```
: DLN    ( addr -- )
   CR  ." |"       \ New line
   DUMPSEG @ 0       \ First display dump segment
   <# ASCII : HOLD       \  Starting with segment:
```

```
    # # # # #> TYPE
  DUP 0             \ Follow by starting address
  <# # # # # #> TYPE
  ."  | " 8 2DUP D.2W     \ Dump 8 bytes with a separator
  OVER + 8 D.2W ." |"        \ Next 8 bytes with separator
  16 BOUNDS ?DO          \ Dump contents in ASCII form
    I DUMPC@ EMIT.
  LOOP  ." |" ;          \ Field terminator
```

.HEAD Print the header line for DUMP.

```
: .HEAD ( addr len -- addr' len' )
  OVER 15 AND
  CR ." -----------"
  49 0 DO ." -" LOOP ." ? 16 0 DO ." -" LOOP ." ?
  CR ." - SEG:OFF - "
  16 0 DO DUP I + 15 AND 2 .R SPACE LOOP ." -"
  16 0 DO DUP I + 15 AND 1 .R LOOP DROP  ." -"
  CR ." ----------"
  49 0 DO ." -" LOOP ." -" 16 0 DO ." ? LOOP ." -" ;

: .FOOT ( --- )
  CR ." ----------"
  49 0 DO ." -" LOOP ." -" 16 0 DO ." ? LOOP ." -" CR ;
```

LDUMP The generic dump word. Dump len bytes of a memory area starting from seg:addr. The dump segment is also stored in DUMPSEG, so that you can continue dumping using DU. The dumped object code is enclosed in a nicely formed box with dividing lines, very pretty on the screen.

```
: LDUMP ( seg addr len -- )
```

```
   ROT DUMPSEG !      \ Save dump segment in DUMPSEG
   BASE @ -ROT        \ Save current base
   HEX .HEAD BOUNDS   \ Display head in hex
   DO  I DLN  KEY? ?LEAVE  \ Dump one line at a time
   16 +LOOP           \ Until completion or a key
   BASE !  CR ;       \ Restore BASE
```

DUMP Dump the contents of a memory array from address addr for length len in the Code Segment. This is the most commonly used dump word because most data and buffers are in the Code Segment, and it is compatible with the DUMP in F83.

```
: DUMP  ( addr len -- )
   ?CS:               \ Make CS the dump segment
   -ROT LDUMP ;       \ Dump Code Segment memory
```

YDUMP Dump an area of n bytes in the Head Segment.

```
: YDUMP ( addr n -- )
   YSEG @         \ Make Head Segment the dump segment
   -ROT LDUMP ;   \ Then dump
```

XDUMP Dump an area of n bytes in the List Segment. The segment address is relative to the base segment address XSEG pointing to the beginning of the List Segment.

```
: XDUMP ( seg-offset n -- )
   SWAP XSEG @ +      \ Get the absolute segment address for the dump segment
   0 ROT LDUMP ;      \ Dump
```

DU Dump 64 bytes from the dump segment starting at an offset address addr. Return the address of next line so that DU can be executed again.

```
: DU            ( addr -- addr+64 )
  DUP 64 DUMPSEG @       \ addr 64 seg --
  -ROT LDUMP         \ Dump 64 bytes
  64 +   ;             \ Address of next line
```

7.7. CASE

This case structure was originally reported by Charles Eaker in Forth Dimensions, Vol. 2, No. 3, Sep/Oct 1980. It was adapted to F-PC by Bob Smith. The most critical part (OF) is coded in assembly and moved into the F-PC Kernel. It is repeated here for completeness.

A case structure is a control structure which allows multiple branches selected by a value. It must be defined inside a colon definition. A case statement must be constructed as follows:

```
  CASE ( value -- )
    value1 OF  func1   ENDOF
    value2 OF  func2   ENDOF
    value3 OF  func3   ENDOF
      ...
    otherwise-func DROP
  ENDCASE
```

CASE and ENDCASE enclose the case statement. OF and ENDOF enclose a branch. Value is passed into the case structure on the data stack. OF compares value with the number before it. If the two numbers are equal, the function after OF is executed until ENDOF, and the case terminated with value removed from the stack. If the numbers are not equal, value is passed to the next OF for comparison. The otherwise-func can still make use of value if none of the branches was executed. If it does not use value, DROP discard the value and the case structure is terminated.

CASE is extremely powerful for multiple tests and multiple branches which are difficult to code with the IF-ELSE-THEN structure. F-PC uses it extensively in the system. However, if values for comparison are sequential, the EXEC: word is much faster, and more efficient in memory. See EXEC:

(OF) Primitive case comparison word, compiled by OF. Performs a comparison between n1 and n2.

```
CODE (OF)   ( n1 n2 -- n1 ) or ( n1 n1 -- )
   POP AX        \ Pop n2
   MOV DI, SP
   CMP AX, 0 [DI]     \ Compare n2 and n1
   0<> IF           \ Not equal,
      MOV ES: IP, 0 [IP]  \   jump to the next case.
      NEXT
   THEN             \ n1=n2
   ADD SP, # 2       \ Drop n1 from the data stack
   ADD IP, # 2       \ Execute the OF clause
   NEXT
   END-CODE
```

CASE Start a multiple branch control structure which allows an arbitrary value to specify a function to be performed.

```
: CASE  ( n --  n )
   COMPILE DOCASE
   <MARK         \ Simply put a place mark here
   0            \ Mark the data stack as the bottom of address list for
   ; IMMEDIATE       \   ENDCASE to resolve
```

OF Start a conditional branch phrase in a case structure.

```
: OF      ( -- )
   COMPILE (OF)       \ Compile run-time code (OF)
   >MARK ;           \ Compile a dummy forward branch address and leave its
```

```
            \   address on stack for ENDOF to resolve
   IMMEDIATE          \ Must execute immediately
```

ENDOF Close a conditional branch phrase in the a case structure.

```
: ENDOF ( -- )
   COMPILE DOENDOF        \ Compile an unconditional forward branch to ENDCASE
   >MARK          \ Leave address for ENDCASE
   SWAP >RESOLVE       \ Resolve branch address after OF
   ;   IMMEDIATE       \ Do it now
```

NRESOLVE Primitive used by ENDCASE to resolve the previous case references.

```
: NRESOLVE  ( 0 n1 n2 ... n -- )
   BEGIN             \ Place current IP in all locations on stack till the 0 marker
      DUP        \ Is it a valid address?
      IF  >RESOLVE 0  \ Yes.  Resolve forward branch
      ELSE    -1         \ No.  End of list.  Terminate.
      THEN
   UNTIL   DROP ;      \ Clean up the stack
```

ENDCASE Resolve all branches in a case structure and close the structure.

```
: ENDCASE   ( -- )
   NRESOLVE          \ Resolve all forward branches
   DROP ;         \ Discard marker left by CASE
   IMMEDIATE
```

7.8. SET DIRECTORY PATH

PATHSET.SEQ has a word set to select and set the directory path to files. PATHSET checks the file contained in a handle. It it does not contain a path, then apply the current drive and path to the handle. It allows F-PC to use the full path/file name in DOS to specify files to be accessed.

flhndl A value that returns the address of the handle we are currently working on.

```
0 value flhndl              \ Will be plugged later
```

?drive.extract Given a handle, return drive-value of the drive specified in handle. Return the current drive if no drive was specified in handle.

```
: ?drive.extract       ( handle --- drive-value )
   dup >nam 1+ c@      \ Get the second character
   ascii : =           \ Is it a : ?
   if  dup>r >nam c@       \ Get the drive character
      bl or 96 -       \ Convert to drive number, A: is 1
      r@ count 2- >r       \ Length less drive specification
      dup 2+ swap      \ name+3 name+1
      r> cmove         \ Copy name over the drive spec
      r@ c@ 2- r@ c!       \ Reduce name length by 2
      r> count + off       \ NULL fill the end
   else                    \ No drive specified
      drop 0 25 bdos 1+    \ Ask system the current drive
   then    ;
```

?drive.prepend Given a drive-value and a handle, prepend the drive-value into handle, as a letter followed by ":".

```
: ?drive.prepend    ( drive-value handle --- )
```

```
   over 0=            \ Drive number =0 ?
   if  2drop          \ Yes.  Use default drive.
   else               \ A drive is specified.
      dup>r count >r      \ Name address
      dup 2+ r> cmove>    \ Move name down by 2 characters
      64 + r@ >nam c!     \ Insert drive name
      ascii :
      r@ >nam 1+ c!       \   with a colon
      r@ c@ 2+ r> c!      \ Adjust character count
   then    ;


HANDLE pathhndl     \ A handle that contains the default drive path.
```

prepend.path Prepend the default drive and path to a handle, and return a Boolean flag that tells if we were successful.

```
: prepend.path  ( handle --- f1 )
   pathhndl clr-hcb
   dup =: flhndl ?drive.extract >r
   flhndl >nam c@ '\' <> dup                    \ leave if got path
   if     drop
      '\' pathhndl 1+ c! 64 pathhndl c!
      pathhndl 2+ r@ pdos dup 0=
      if     drop
         pathhndl 1+ 64 0           \ determine path length
         do     dup i + c@ 0= if i pathhndl c! leave then
         loop    drop pathhndl c@ 1 >
         if     '\' pathhndl count + c!
            pathhndl c@ 1+ pathhndl c!
         then    flhndl c@ pathhndl c@ + 62 > dup 0=
         if     drop
```

```
            flhndl 1+ pathhndl c@ over + flhndl c@ cmove>
            pathhndl count flhndl 1+ swap cmove
            pathhndl c@ flhndl c@ + flhndl c!
            false
          then
        then
      then
      r> flhndl ?drive.prepend 0 flhndl count + c! ;

' PREPEND.PATH IS PATHSET
```

7.9. STRING SEARCH

SEARCH.SEQ was contributed to F-PC by Roger Bird. It is 20% faster than our earlier version and 140 bytes smaller. It may be removed in a turnkey application, but it is used by WORDS.SEQ, and ENVIRON.SEQ, which is itself used by EXEC.SEQ. It must be loaded for an interactive Forth system.

The string manipulation primitives include string comparison and searching. The string search implemented here is used in the editor to find desired strings. The only unusual thing about it is the presence of a variable called CAPS, which determines whether or not to ignore the case of the subject and pattern strings. If case is ignored then A-Z = a-z. The default is to ignore case.

Much thanks to Roger Bird for this improved implementation of SEARCH. It is about 20% faster than the previous improved version in F-PC, and is 10 times faster in the case where the string being searched for starts with a space. It is also 140 bytes smaller than the old implementation.

SEARCH Search for a source string (saddr,slen) contained in the destination string (daddr,dlen). Return a true flag if found, and an address equal to the offset into the destination string where saddr,slen was found.

```
CODE SEARCH     ( saddr slen daddr dlen -- offset found? )
   pop cx            \ dlen
```

```
pop di            \ daddr
pop bx            \ slen
pop ax            \ saddr
cmp bx, # 0               \ if search length "slen" is 0 then return true
0= if   push bx
   mov ax, # true
   1push
then
push di           \ Save daddr
push es           \ Save ES
mov es, sseg          \ ES points to string segment
push si           \ Save IP
push bp        \ Save RP
mov si, ax         \ Source address saddr
dec bx            \ Source count-1
dec cx            \ Destination count-1
mov dx, bx         \ Save a copy in DX
cmp caps # 0          \ Case sensitive?
0= if             \ Yes.  Clear mask.
   mov bp, # 0
else              \ Case insensitive.
   mov bp, # $2020 \ Set mask to two blanks
then
begin             \ Search loop
   cmp bx, cx      \ Source-len > dest-len ?
<= while          \ No.  Must do search.
   begin             \ Compare strings
      mov al, 0 [si+bx]   \ Get a byte from source
      mov ah, es: 0 [di+bx]   \ Get a byte from dest
```

```
      or ax, bp       \ Mask case bit
      cmp ah, al      \ Are they the same?
   0= while           \ Yes.
      dec bx          \ Decrement source count
      0< if           \ Source string exhausted?
         pop bp       \ Yes.  Match is found.
         pop si       \ Restore IP
         pop es       \ Restore List pointer
         pop dx       \ Destination string addr
         sub di, dx   \ Addr of found string
         push di      \ Push on stack
         mov ax, # $FFFF \ True flag
         1push        \ Push on stack
      then            \ Not yet done
   repeat             \ Compare next byte
   inc di             \ Move on the next source byte
   dec cx             \ Decrement destination count
   mov bx, dx         \ Restore source count
repeat                \ Fail to find a match
pop bp                \ Restore RP
pop si                \ Restore IP
pop es                \ Restore ES
pop dx                \ Get dest address
sub di, dx            \ Move to original daddr
push di               \ Push on stack
sub ax, ax            \ Make false flag
1push                 \   and push on stack
end-code
```

7.10. WORDS

WORDS is used in F83 to display the names of words in the context vocabulary. This is a very important utility in Forth to allow the user to inspect the words currently available to be used for execution or compilation. F-PC greatly enhances the usefulness of WORDS by allowing it to take a string argument to trigger some very interesting functions. If WORDS is followed by an arbitrary string, all the words in all vocabularies containing that string pattern will be reported. This is very useful to identify a group of words whose spelling is not known for sure.

If the string argument is *.* , then all words in all vocabularies are displayed. CODE.* allows only code words to be searched and reported. :.* allows only colon words. Other patterns allowed are VARIABLE.* , CONSTANT.* , UVARIAVLE.* , DEFER.* , and UDEFER.* for obvious purposes. TOTAL.* causes only the word count to be reported.

Since F-PC has a very large dictionary, these conditional searching words are more important here than in other smaller Forth systems. If you are using a color monitor, F-PC can display different types of words in different color. The colorization can be turn on or off, by executing the word COLORIZEON or COLORIZEOFF.

Here is a list of variables used in displaying word lists:

```
VYET            \ Did we print this vocabulary yet?
VADDR           \ Vocabulary name address
TOTALWORDS          \ Accumulator for the total number of names.
```

.VYET Print the name of a vocabulary if we have not printed it yet.

```
: .VYET ( -- )
   VYET @ IF EXIT THEN \ If VYET is true, this vocabulary was printed already.  Exit.
   VADDR @ CR        \ Otherwise, print the vocabulary
   ." --[ " .ID ."  ]--"      \   name first.
   VYET ON       \ Make sure doing it only once
   CR LMARGIN @ SPACES  ;  \ New line with left margin
```

```
DEFER W.ID ' .ID IS W.ID        \ .ID is deferred to accommodate the colorizer.

CREATE W$ 64 ALLOT      \ Word name is store here temporarily to match substrings.

   W$ 64 ERASE
```

?INNAME A substring pattern is stored in the word buffer at HERE. Search the word name at nfa for this substring. Return a true if the substring is contained in the word name.

```
: ?INNAME   ( nfa -- f )

   @> YSEG SWAP        \ seg:addr of the name field of this word

   ?CS: HERE 31 CMOVEL \ Copy word name to HERE

   $C0 HERE CRESET                 \ mask to real count only

   $80 HERE COUNT + DUP OFF    \ clear end of name to NULL

   1- CRESET          \ mast high bit of last char

   W$  COUNT          \ First pattern to be searched.

   HERE COUNT SEARCH NIP

   W$ 32 + COUNT       \ Second pattern to be searched

   HERE COUNT SEARCH NIP

   AND ;              \ Return true if word name contains both sub-strings
```

?CODENAME Return true if the word at nfa is a code definition.

```
: ?CODENAME ( nfa -- f )

   NAME>          \ Get cfa

   C@ DUP 232 <>       \ If it does not contain a CALL

   SWAP 233 <> AND ;       \   nor a JMP, then it must be a code word
```

```
0 VALUE WORDTYPE        \ Address of inner interpreter for type identification
```

?WORDTYPE Return true if the word at nfa is of the same type as shown in WORDTYPE.

```
: ?WORDTYPE ( nfa -- f )
   NAME>          \ Get cfa
   @REL>ABS       \ Get absolute address of the inner interpreter of this word
   WORDTYPE = ;   \ Compare with WORDTYPE
```

?TOTALWORD Increment the total word count. Replace nfa with a false flag.

```
: ?TOTALWORDS   ( nfa -- false )
   DROP FALSE
   TOTALWORDS INCR ;
```

?W.NAME This is a critical word in the WORDS utility in F-PC. It is a deferred word. It takes a nfa and returns a flag. If the flag is true, the name of this word should be printed. Otherwise, the word will be ignored. Depending on the search string following WORDS, ?W.NAME will be vectored to a words which selects the specific type of words and sets the proper flag. See the following words for its usage and behavior.

```
DEFER ?W.NAME   ( nfa -- flag )
```

W.NAME Print the name of a word at nfa filtered by the string at HERE. Test to see if conditions are correct, like CODE, or ALL, or WITHIN, and then print the name.

```
: <W.NAME>  ( nfa -- )
   DUP ?W.NAME     \ Call ?W.NAME to decide  whether name should be printed.
   IF  .VYET 17 ?LINE W.ID     \ Yes.  Print vocabulary and word name.
      #OUT @ COLS 16 - < \ Tab if needed
      IF TAB THEN
      TOTALWORDS INCR \ Bump word count
   ELSE    DROP          \ Do not print name
   THEN    ;
```

```
DEFER W.NAME    ' <W.NAME>  IS  W.NAME
```

.VOCWORDS Display all the matching words, i.e., words selected by ?W.NAME, in a vocabulary.

```
code largest    ( a1 n1 --- a2 n2 ) \ Return the largest element of an array of words
   pop cx                  \ count of array to search
   pop bx                  \ starting address of array
   mov ax, # 0             \ starting highest value = 0
   mov dx, bx              \ highest value addr = start address
here    cmp 0 [bx], ax
   u> if   mov dx, bx
      mov ax, 0 [bx]
   then
   add bx, # 2             \ bump to next element
   loopnz
   2push         end-code

: .VOCWORDS ( voc-addr -- )
   DUP HERE 500 +      \ Copy vocabulary threads
   #THREADS 2* CMOVE      \   to Code Segment
   BODY> >NAME VADDR ! \ Store address of vocabulary
   VYET OFF          \   in VADDR
   BEGIN
      HERE 500 +      \ Thread table
      #THREADS
      LARGEST DUP \ Find highest name on  threads
      ?KEYPAUSE
```

```
   WHILE
      DUP L>NAME      \ Go from lfa to nfa
      W.NAME     \ Print out name
      Y@ SWAP !       \ Replace thread with next lfa
   REPEAT  2DROP ;     \ Repeat for the vocabulary


DEFER ?W.TEST          \ A template to provide DODEFER
' NOOP IS ?W.TEST          \   to ?DEFER.* later
```

The following set of words ?xxxxx.* vector execution words into ?W.NAME so that ?W.NAME will be able to select a specific class of words to be printed. They test the string in the word buffer at HERE and change the behavior of ?W.NAME accordingly.

```
: ?*.*     ( -- )
   W$ 1+          \ Word buffer
   " *.*" COMP 0=      \ Is the string *.* ?
   IF  ['] NOOP       \ Yes.  Store NOOP in ?W.NAME
      IS ?W.NAME      \   allow all words to be matched
   THEN    ;


: ?CODE.*   ( -- )
   W$ 1+          \ Is the string CODE.* ?
   " CODE.*" CAPS-COMP 0=
   IF  ['] ?CODENAME   \ If so, search only code words
      IS ?W.NAME      \   by vectoring ?W.NAME to
   THEN    ;         \   ?CODENAME


: ?:.*     ( -- )
   W$ 1+          \ If the string in word buffer is
   " :.*" CAPS-COMP 0=     \   :.* ,
```

```
   IF  ['] ?*.* @REL>ABS   \   let WORDTYPE points to NEST
     =: WORDTYPE
     ['] ?WORDTYPE   \   and ?W.NAME is vectored to
     IS ?W.NAME      \   ?WORDTYPE to print colon words
   THEN    ;

: ?VARIABLE.*   ( -- )
   W$ 1+             \ For variables
   " VARIABLE.*" CAPS-COMP 0=
   IF  ['] TOTALWORDS @REL>ABS     \ WORDTYPE points to >NEXT
     =: WORDTYPE
     ['] ?WORDTYPE IS ?W.NAME    \ ?WORDTYPE is execution
   THEN    ;             \   routine in ?W.NAME

: ?UVARIABLE.*  ( -- )
   W$ 1+             \ For user variables
   " UVARIABLE.*" CAPS-COMP 0=
   IF  ['] BASE @REL>ABS      \ WORDTYPE points to
     =: WORDTYPE        \   DOUSER-VARIABLE
     ['] ?WORDTYPE
     IS ?W.NAME
   THEN    ;

: ?CONSTANT.*   ( -- )
   W$ 1+             \ For constants
   " CONSTANT.*" CAPS-COMP 0=
   IF  ['] WORDTYPE @REL>ABS   \ WORDTYPE points to
     =: WORDTYPE        \   DOCONSTANT
     ['] ?WORDTYPE
```

```
      IS ?W.NAME
   THEN    ;

: ?VALUE.*     ( --- )
   W$ 1+ " VALUE.*" CAPS-COMP 0=
   IF     ['] WORDTYPE @REL>ABS =: WORDTYPE
      ['] ?WORDTYPE IS ?W.NAME
   THEN    ;

: ?DEFER.*  ( -- )
   W$ 1+             \ For deferred words
   " DEFER.*" CAPS-COMP 0=
   IF  ['] ?W.TEST @REL>ABS    \ WORDTYPE points to
      =: WORDTYPE        \   DODEFER
      ['] ?WORDTYPE
      IS ?W.NAME
   THEN    ;

: ?UDEFER.* ( -- )
   W$ 1+             \ For user deferred words
   " UDEFER.*" CAPS-COMP 0=
   IF  ['] EMIT @REL>ABS      \ Point WORDTYPE to
      =: WORDTYPE        \   DOUSER-DEFER
      ['] ?WORDTYPE
      IS ?W.NAME
   THEN    ;

: ?TOTAL.*  ( -- )
   W$ 1+             \ For counting words
```

```
   " TOTAL.*" CAPS-COMP 0=
   IF  CR ." Not displaying, just counting the TOTAL "
      ['] ?TOTALWORDS        \ Vector ?W.NAME to word
      IS ?W.NAME          \   just incrementing count
   THEN    ;
```

WORDS Display words that match a text string. <text> is optional. If it is omitted then the CONTEXT vocabulary will be displayed. Several specific values for <text> can be included on the command line, as follows:

```
   WORDS *.*                  display all words in all vocs.
   WORDS CODE.*          display all code words.
   WORDS :.*                  display all : definitions.
   WORDS CONSTANT.*           display all constants.
   WORDS VARIABLE.*           display all variables.
   WORDS UVARIABLE.*          display all user variables.
   WORDS DEFER.*              display all deferred words.
   WORDS UDEFER.*             display all user deferred words.
   WORDS TOTAL.*              display total count of words only.
```

```
FALSE VALUE CONTEXTONLY
DEFER PREWORDS  ' NOOP  IS  PREWORDS

 : WORDS    ( -- )
   TOTALWORDS OFF      \ Clear word count
   SAVESTATE           \ Save screen state
   COLS 2- RMARGIN !       \ Set display parameters
   15 TABSIZE !
   LMARGIN OFF
   CR ."  ** Press SPACE to pause, or ESC to exit ** "
   PREWORDS
```

```
>IN @ #TIB @ <>     \ If not at the end of a line
IF  ['] ?INNAME        \ May do a substring match
   IS ?W.NAME
   BL WORD W$      \ Get first sub-string to be searched
   OVER C@ 1+ 32 MIN CMOVE
   BL WORD W$ 32 +     \ Get second sub-string to be searched
   OVER C@ 1+ 32 MIN CMOVE
   ?*.*           \ May search all words
   ?CODE.*     \ May search only code words
   ?:.*           \ May search colon words
   VARIABLE.*      \ May search variables
   ?CONSTANT.*     \ Or constants
   ?DEFER.*        \ Or deferred words
   ?VALUE.*
   ?UVARIABLE.*    \ Or user variables
   ?UDEFER.*       \ Or user deferred words
   ?TOTAL.*        \ Or just count words
   CONTEXTONLY
   FALSE =: CONTEXTONLY
   IF  CONTEXT @   \ If CONTEXTONLY was true, search only the context
      .VOCWORDS   \   vocabulary.
   ELSE    VOC-LINK @  \ Otherwise, go through all vocabularies
      BEGIN
         DUP        \ Next vocabulary link
         #THREADS 2* -   \ Head of thread table
         .VOCWORDS   \ Print all words in this  vocabulary with matching
               \   characteristics
         @ DUP 0=    \ Next vocabulary or end?
      UNTIL   DROP       \ Exit when all vocabularies are processed
```

```
    THEN
  ELSE
    ['] NOOP IS ?W.NAME     \ No string following
    FALSE =: CONTEXTONLY
    CONTEXT @ .VOCWORDS \ Display words in context
  THEN              \   vocabulary, like in F83
  CR TOTALWORDS @ U. \ Print total word counted
  ." Words displayed" CR
  RESTORESTATE ;      \ Restore screen display

ROOT DEFINITIONS          \ Make WORDS available everywhere
' WORDS ALIAS WORDS
FORTH DEFINITIONS
```

7.11. CURSOR CONTROL WORDS

IBMCURSR.SEQ contains words which position the cursor on the display screen. F-PC supports IBM color card and monochrome card, and also Hercules high resolution card. It is smart enough to be able to select the appropriate word set to handle the screen display automatically.

IBM-AT Move the cursor on the IBM display screen to the column and row specified. This is done using a BIOS level interrupt. This word is normally the deferred function of AT, you should use AT for portability.

```
CODE IBM-AT ( col row -- )
  POP AX
  POP DX
  MOV DH, AL
  XOR BH, BH
  MOV AH, # 2
```

```
   PUSH SI PUSH BP
   INT $10        \ BIOS service 16, function 2
   POP BP  POP SI
   NEXT   C;
```

IBM-AT? Return the col and row of the current cursor location on the screen.

```
CODE IBM-AT?    ( -- col row )
   XOR BH, BH
   MOV AH, # 3
   PUSH SI PUSH BP
   INT $10        \ BIOS service 16, function 3
   POP BP  POP SI
   SUB AX, AX
   MOV AL, DH
   MOV DH, AH
   2PUSH   END-CODE
```

IBM-DARK Clear the screen and place the cursor in the upper left hand corner. This is the deferred function of DARK. Use DARK for portability.

```
CODE IBM-DARK   ( -- )
   MOV AX, # 2
   PUSH SI PUSH BP
   INT $10
   SUB AH, AH
   INT $10        \ Clear the screen
   POP BP  POP SI
   NEXT   END-CODE
```

LFILLW Similar to LFILL, except filling the memory area with a 16 bit word pattern.

```
CODE LFILLW     (  seg start-addr byte-count word-pattern -- )
   CLD
   POP AX
   POP CX
   SHR CX, # 1
   POP DI
   POP BX
   MOV DX, ES
   MOV ES, BX
   REPNZ          STOSW       \ Repeat Store-Words
   MOV ES, DX
   NEXT
   END-CODE
```

NORM-DARK   If >DOSIO is true, use IBM-DARK to blank the screen. Otherwise, clear the screen by filling the screen buffer with proper word pattern. This is necessary to deal with non-IBM video cards.

```
: norm-dark     ( --- )
   ?dosio
   if         ibm-dark
   else    blank.color
      VIDEO-SEG @ 0 ROWS COLS 2* *
      attrib c@ flip $20 or LFILLW
      show.color
   then    0 0 at ;
```

IBM--LINE   Delete the current line on the IBM display. It causes all lines lower on the screen to scroll up. This is the deferred function of -LINE. Use -LINE for portability.

```
CODE IBM--LINE  ( -- )
   PUSH SI PUSH BP
   XOR BH, BH
   MOV AH, # 3
   INT   $10          \ Read Cursor Position
   MOV CH, DH
   XOR CL, CL         \ Upper left corner
   MOV DX, # ' ROWS >BODY
   DEC DH
   MOV DL, ' COLS >BODY
   DEC CL
   MOV BH, # 7
   MOV AH, # 6            \ Scroll up
   MOV AL, # 1        \ Lines to scroll
   INT 16
   POP BP  POP SI
   NEXT   END-CODE
```

>IBM Select the IBM display word set as the default display words.

```
: >IBM  ( -- )
   ['] IBM-AT   IS AT
   ['] IBM-AT?  IS  AT?
   ['] NORM-DARK IS DARK
   ['] IBM--LINE IS -LINE ;
>IBM
```

CURSOR_POS_INIT Initialize the cursor pointers in F-PC to the displayed cursor.

```
: CURSOR_POS_INIT   ( -- )
   IBM-AT?
   #LINE !          \ Row number
   #OUT ! ;         \ Column number

' CURSOR_POS_INIT IS CURSORSET
```

The following words changes the shape of the cursor. The cursor shape and characteristics are a good indicator to the current F-PC status, especially in the editor.

SET-CURSOR Set the cursor shape to the shape value n.

```
CODE SET-CURSOR ( n -- )
   POP CX        \ Cursor shape mask
   MOV AH, # 1
   PUSH SI PUSHJ BP
   INT $10
   POP BP  POP SI
   NEXT C;
```

GET-CURSOR Get the cursor shape mask SHAPE.

```
: GET-CURSOR    ( -- shape )
    0 $460 @L ;          \ The shape code is stored in the memory segment 0
```

CURSOR-OFF Turn off the cursor.

```
: CURSOR-OFF    ( -- )
   GET-CURSOR
```

```
   $2000  OR          \ Set the cursor off bit
   SET-CURSOR ;

CURSOR-ON      Turn the cursor back on.

: CURSOR-ON ( -- )
   GET-CURSOR
   $0F0F AND          \ Set cursor on pattern
   SET-CURSOR ;

: *NORM-CURSOR  ( -- )
   CROWS DUP 1- FLIP +     \ Combine rows and columns
   SET-CURSOR ;

: *BIG-CURSOR   ( -- )
   CROWS  SET-CURSOR ;

: *MED-CURSOR   ( -- )
   CROWS DUP 2/   FLIP +
   SET-CURSOR ;

DEFER NORM-CURSOR   ' *NORM-CURSOR IS NORM-CURSOR
DEFER BIG-CURSOR          ' *BIG-CURSOR IS BIG-CURSOR
DEFER MED-CURSOR    ' *MED-CURSOR IS MED-CURSOR
```

7.12. MONOCHROME VIDEO WORDS

This set of words in MONOCROM.SEQ deals with the IBM Monochrome Video Card. The words are fairly self-explanatory.

```
$07 VALUE NORMVAL                       \ VALUE FOR NORMAL VIDEO

$78 VALUE REVVAL                        \ VALUE FOR REVERSE VIDEO

$07 VALUE NORMALVAL

$78 VALUE REVERSEVAL


: >NONE ( --- )     NORMVAL ATTRIB C! ;        \ NORMAL

: >NONEBG   ( --- )     NORMVAL $F0 AND ATTRIB C@ $0F AND OR ATTRIB C! ;

: >REVERSE ( --- )     REVVAL  ATTRIB C! ;        \ REVERSE

: >UL         ( --- ) $01 ATTRIB C! ;           \ UNDERLINE

: >BOLD        ( --- )     $7F ATTRIB C! ;             \ BRIGHT

: >BOLDUL  ( --- )     $79 ATTRIB C! ;             \ BOLD UNDERLINE

: >BOLDBLNK ( --- )     $8F ATTRIB C! ;              \ BOLD BLINK

: >REVBLNK  ( --- )     $F0 ATTRIB C! ;             \ REVERSE BLINK


' >NONE    IS >NORM

' >REVERSE IS >REV

' >NONEBG  IS >NORMBG


: >MONO ( --- )           \ Select highlighting for monochrome monitor.
   ['] >UL IS >ATTRIB1

   ['] >BOLDBLNK IS >ATTRIB5

   ['] >BOLDUL DUP IS >ATTRIB2 IS >ATTRIB6

   ['] >BOLD   DUP IS >ATTRIB3  IS >ATTRIB7

   ['] >REV    DUP IS >ATTRIB4 IS >ATTRIB8

   >NORM ;


' >MONO IS INITMONO              \ Enable attributes according to video board.
```

```
' >MONO IS INITCOLOR              \ Enable attributes of color card.
```

The following words allow the user to select white-on-black or black-on-while display.

```
: invert-screen          ( --- )
   blank.color
   video-seg @ rows cols 2* * 1
   do     dup i c@l dup NORMVAL =
      if drop REVVAL over i c!l
      else    REVVAL =
         if NORMVAL over i c!l
         then
      then
   2 +loop
   drop show.color ;

: white-on-black         ( --- )
   NORMVAL NORMALVAL <>
   if        invert-screen
   then
   NORMALVAL  =: NORMVAL
   REVERSEVAL =: REVVAL
   >norm ;

: black-on-white         ( --- )
   NORMVAL REVERSEVAL <>
   if  invert-screen
   then
```

```
 REVERSEVAL =: NORMVAL

NORMALVAL  =: REVVAL

 >norm ;
```

7.13. COLOR VIDEO WORDS

Source file is COLOR.SEQ. For an IBM Color Video display Card, the following values may be used for background or foreground colors.

```
$00 CONSTANT BLACK          \ black

$01 CONSTANT BLUE           \ blue

$02 CONSTANT GREEN          \ green

$03 CONSTANT CYAN           \ cyan

$04 CONSTANT RED            \ red

$05 CONSTANT MAGENTA        \ magenta

$06 CONSTANT BROWN          \ brown

$07 CONSTANT LTGRAY         \ light gray

$08 CONSTANT DKGRAY         \ dark gray
```

All following, blink in background.

```
$09 CONSTANT LTBLUE         \ light blue

$0A CONSTANT LTGREEN        \ light green

$0B CONSTANT LTCYAN         \ light cyan

$0C CONSTANT LTRED          \ light red

$0D CONSTANT LTMAGENTA      \ light magenta

$0E CONSTANT YELLOW         \ yellow

$0F CONSTANT WHITE          \ white
```

>FG Set the Foreground color to value n1.

```
: >FG     ( n -- )
   attrib c@          \ Get attribute byte
   $F0 and +          \ Mask off background bits
   attrib c! ;        \ Put back
```

>BG Set the Background color to value n1.

```
: >BG     ( n -- )
   $10 * attrib c@            \ Move to background field
   $0F and + attrib c! ;      \ Mask and put back to STTRIB
```

COLORSET Set the current screen colors to be the default normal color combination.

```
: COLORSET     ( -- )
   attrib c@ =: normval ;
```

>BUGN Set the background to BLUE, and foreground to green.

```
: >BUGN ( -- ) BLUE  >BG   GREEN >FG ;
```

>RDWT Set the background to RED, and foreground to WHITE.

```
: >RDWT ( -- ) RED   >BG   WHITE >FG ;
```

>BUWT Set the Background to Green, and Foreground to WHITE.

```
: >BUWT    ( -- ) GREEN >BG   WHITE >FG ;
```

>COLOR Select highlighting for color monitor. Initialize all attributes for the color video card.

```
: >COLOR    ( -- )
   ['] >BUGN DUP IS >ATTRIB1   IS >ATTRIB5
   ['] >RDWT DUP IS >ATTRIB2   IS >ATTRIB6
   ['] >BUWT DUP IS >ATTRIB3   IS >ATTRIB7
   ['] >RDWT DUP IS >ATTRIB4   IS >ATTRIB8
   >NORM ;

' >COLOR IS INITCOLOR          \ Enable attributes according to video board.
```

>LCD Liquid Crystal Display, or Least Common Denominator. Your choice.

```
: >LCD        ( --- )
   ['] >REV  DUP IS >ATTRIB1 DUP IS >ATTRIB5
   DUP IS >ATTRIB2 DUP IS >ATTRIB6
   DUP IS >ATTRIB3    IS >ATTRIB7
   ['] >NORM DUP IS >ATTRIB4    IS >ATTRIB8
   >NORM ;
```

7.14. SAVE SCREENS

F-PC manages many screens of useful information to help the user while running the editor, the debugger, and many on-line help utilities. A common need is to save the current screen display, replace it with some other help screens, and later restore the original screen to continue normal work. This save screen utility in SAVESCR.SEQ provides a memory area to save up to three screenful of data. The saving process is nestable to three level deep.

```
0 value svseg          \ screen save segment
4 value svmax          \ maximum number of screens to save
4000 value svsize      \ size of each screen
0 value svtotal        \ total of saved screens
```

svinit Cold start initialization word that allocates 64K bytes of space for the screen save segment.

```
: svinit    ( -- )          \ Screen save initialization
   defers initstuff
   off> svseg
   rows cols 2* * =: svsize          \ set save size per screen
   65535. svsize um/mod nip          \ maximum number of screens
   1- svmax min =: svmax             \ limit svsize to < 64k
   svsize svmax * =: svtotal         \ adjust total arraysize
   svmax 1+ svsize * paragraph
   alloc 8 =               \ allocate the space
   if 2drop cr
      ." Could not allocate memory for Screen save routine."
   else    nip =: svseg
   then    ;
' svinit is initstuff
```

savescr Save the current contents of the CRT screen to an external segment save area. Nestable up to four (SAVMAX) times.

```
: savescr   (-- )
   svseg 0
   2dup svsize + svtotal cmovel>       \ Make a hole in buf
   blank.color
   video-seg @ 0
```

```
   svseg 0 svsize cmovel          \ & save screen in it.
   show.color ;
```

restscr Restore a saved screen from the external screen save segment. Un-nestable up to three levels.

```
: restscr  ( -- )
   svseg 0= ?exit
   blank.color
   svseg 0
   video-seg @ 0 svsize cmovel     \ Restore screen &
   show.color
   svseg svsize
   over 0 svtotal cmovel ;    \ compress buffer.
```

RECOVERSCR Make a copy of the last saved screen and then restore it.

```
: recoverscr   ( -- )
   svseg 0= ?exit               \ last saved screen
   blank.color
   svseg 0
   video-seg @ 0 svsize cmovel        \ Restore screen
   show.color ;
```

RECOVERLINE Get a copy of line n1 from the last saved screen.

```
: recoverline  ( n1 --- )
   svseg 0= if drop exit then
   dup 0 rows 1- between
   if         cols 2* * >r
```

```
            blank.color     \ Restore line n1
            svseg r@
        video-seg @ r> cols 2* cmovel
            show.color
    else        drop
    then    ;
```

7.15. TIMER WORDS

Words in TIMER.SEQ and TIMESTUFF.SEQ deal with time. As we envision that many user will use F-PC for real-time applications, a complete set of time and date manipulation words has been included. They call appropriate DOS service functions to provide the requested services. The words for getting and setting the date and time are discussed in this section.

Time delay words use the DOS time function to obtain very accurate time delays. Background processing continues, as pause is called in the wait loop. Another deferred word, PAUSE-FUNC, is also in the loop, which can be re-assigned to perform any function you may want done while the delay is occurring.

GETDATE Return d1 the 32 bit binary date from the operating system. Bytes in d1 are arranged in the order of year/month/day/day-of-week.

```
: GETDATE   ( -- d1 )
    0 0 42 OS2          \ DOS service 42
    DROP ;
```

SETDATE Given the binary date d1, set the system clock to that date.

```
: SETDATE   ( d1 -- )
    SWAP 43 OS2             \ Update date
    NIP NIP 255 =           \ d1 might be invalid
```

```
   IF  CR
      ." Invalid DATE "
   THEN ;
```

GETTIME Return d1 with bytes arranged in the order hr/min/sec/100th-sec from the operating system.

```
: <GETTIME> ( -- d1 )
   0 0 44 OS2
   DROP ;
```

```
DEFER GETTIME  ' <GETTIME> IS GETTIME
```

SETTIME Given the binary time d1, set the system clock to that time.

```
: SETTIME   ( d1 -- )
   SWAP 45 OS2
   NIP NIP 255 =
   IF  CR
      ." Invalid TIME "
   THEN ;
```

```
CREATE DTBUF 16 ALLOT       \ A buffer to store date or time string for output
```

##+ Convert the two lowest digits in n1 and add the digits to DTBUF.

```
: ##+      ( n1 -- )
   0 <# # # #>           \ Convert two digits of n1
   TUCK DTBUF COUNT + \ End of string in DTBUF
   SWAP CMOVE         \ Append two digits
```

```
   DTBUF C+! ;        \ Update string count
```

"+ Append a string at addr of n bytes to the string in DTBUF.

```
: "+       ( addr n -- )
   TUCK DTBUF COUNT + \ See comments above
   SWAP CMOVE
   DTBUF C+! ;
```

BUILD-HM Append the hour:minute string converted from n1 into DTBUF.

```
: BUILD-HM  ( n1 -- )
   SPLIT             \ Separate minute from hour
   ##+               \ Convert hour
   " :" "+           \ Insert a :
   ##+ ;             \ Convert minute
```

BUILD-SH Append the second.tick string converted from n1 to the string in DTBUF.

```
: BUILD-SH  ( n1 -- )
   " :" "+           \ Add separator
   SPLIT ##+         \ Convert seconds
   " ." "+           \ Insert separator
   ##+ ;             \ Convert ticks
```

BUILD-TIME Add the hour:minute:second.tick string to the string in DTBUF. The time is represented by d1 on the stack.

```
: BUILD-TIME   ( d1 -- )
   SWAP              \ Make it Sh and HM
```

```
   BUILD-HM          \ Convert hour:minute
   BUILD-SH ;        \ Convert second.tick
```

FORM-TIME Build up a time string in DTBUF from the time spec d1 on the stack. Return the address of DTBUF.

```
: FORM-TIME ( d1 -- addr )
   BASE @ >R DECIMAL      \ Have to used decimal base
   DTBUF OFF         \ Init null string in DTBUF
   BUILD-TIME        \ Add time string
   R> BASE !         \ Restore base
   DTBUF ;        \ Push address of DTBUF
```

.TIME Print the current time to the screen, in the format HH:MM:SS.HR, where HH is in hours, MM in minutes, SS in seconds, and HR in hundredths of a second.

```
: .TIME ( -- )
   GETTIME           \ What time is it?
   FORM-TIME         \ Convert to a time string
   COUNT TYPE ;      \ Print it
```

The time string format is fairly universal as HH:MM:SS.HR. However, the representation of month and day shows many variations. Here F-PC gives you three choices, which are quite obvious from their names:

```
: <M/D/Y>   ( d1 -- )
   SPLIT             \ Split day byte from month byte
   ##+               \ Convert month
   " /" "+           \ Add separator
   ##+               \ Convert day
   " /" "+           \ Add separator
   ##+ ;             \ Convert year
```

```
: <Y-M-D>   ( d1 -- )
   SPLIT ROT          \ day month year --
   ##+ " -" "+        \ Convert year and add /
   ##+ " -" "+        \ Convert month and add /
   ##+ ;              \ Convert day

: <D.M.Y>   ( d1 -- )
   SPLIT SWAP         \ day month year --
   ##+ " ." "+        \ Convert day
   ##+ " ." "+        \ Convert month
   ##+ ;              \ Convert year
```

We are sure that these choice will not be the one you will want to use. Thus BUILD-DATE is defined as a deferred word so that you can make the correct choice and vector BUILD-DATE to it.

```
DEFER BUILD-DATE
' <M/D/Y> IS BUILD-DATE     \ Our default choice

: M/D/Y ( -- )  ['] <M/D/Y> IS BUILD-DATE ;

: Y-M-D ( -- )  ['] <Y-M-D> IS BUILD-DATE ;

: D.M.Y ( -- )  ['] <D.M.Y> IS BUILD-DATE ;
```

DORM-DATE Convert the date spec d1 into a date string in DTBUF. Return the address of DTBUF.

```
: FORM-DATE ( d1 -- addr )
   BASE @ >R DECIMAL       \ Must use decimal base
   DTBUF OFF          \ Start with a null string
```

```
   BUILD-DATE        \ Add date string
   R> BASE !         \ Restore base
   DTBUF ;        \ Return with DTBUF address
```

.DATE Print the current date to the screen, in the format MM/DD/YY, where MM is month, DD is day, and YY is year.

```
: .DATE ( -- )
   GETDATE           \ Ask Ms. DOS what day it is.
   FORM-DATE         \ Build the date string in DTBUF
   COUNT TYPE ;      \ Print it
```

A special word set is defined in F-PC to measure elapsed time for real time experiments and for characterization of program performance.

```
VARIABLE STIME 0 ,        \ Starting time
VARIABLE TTIME 0 ,        \ Buffer to store a time spec
```

T>B Convert a DOS time spec d1 to a double integer in the units of 100th of a second, or a tick. This binary representation is used to compute time difference.

```
: T>B     ( d1 -- d2 )
   SPLIT 100 * +         \ tick+100*second
   SWAP 0 SWAP       \ ticks 0 hh:mm --
   SPLIT >R 6000 *D      \ Convert minutes
   D+ R> 1000 *          \ Convert hour
   360 *D D+ ;       \ Add then altogether
```

B>T Inverse of T>B. Convert a double integer ticks back into the DOS time specification stored in TTIME.

```
: B>T     ( d1 -- )
```

```
   0 100 UM/MOD >R        \ Extract ticks
   100 UM/MOD         \ Extract seconds
   SWAP TTIME C!      \ Store ticks
   R> 60 UM/MOD       \ Extract minutes
   SWAP TTIME 1+ C!       \ Store seconds
   60 /MOD        \ Extract hours
   TTIME 3 + C!           \ Store hours
   TTIME 2+ C! ;          \ Store minutes
```

TIME-RESET Reset the accumulated time value in the double variable STIME to zero, in effect resetting the current elapsed time to zero. This word is used at the beginning of a sequence of operations you want to time. The word .ELAPSED is used at the end of the operations to print the elapsed time since the last TIME-RESET.

```
: TIME-RESET    ( -- )
   GETTIME T>B            \ Get time in binary form
   STIME 2! ;             \   and store in STIME
```

TIME-ELAPSED Get current time and find the difference from the time in STIME in units of ticks, 100th of a second.

```
: TIME-ELAPSED  ( -- d1 )
   GETTIME T>B            \ Get current time in ticks
   STIME 2@           \ Last ticks
   D- ;               \ Difference in ticks
```

B>SEC Convert the double integer ticks to seconds. Overflow occurs at about 18 hours.

```
: B>SEC ( d1 -- n1)
   0 100 UM/MOD DROP      \ Discard overflow
   100 UM/MOD NIP ;       \ Retain only seconds
```

.ELAPSED Print the elapsed time since the last TIME-RESET was performed.

```
<.ELAPSED>  ( -- )
   TIME-ELAPSED       \ Get elapsed ticks
   B>T                \ Convert ticks to DOS time
   TTIME 2@ FORM-TIME \ Convert to HH:MM:SS:HR spec
   COUNT TYPE ;       \ Print the time string

: .ELAPSED  ( -- )
   CR ." Elapsed time   =  "
   <.ELAPSED> ;
```

TIMER Perform the Forth words following on the same command line, and when they finish execution, print the elapsed time used for their execution.

```
: TIMER ( words -- )
   TIME-RESET         \ Reset timer
   INTERPRET          \ Execute following words
   .ELAPSED ;         \ Display elapsed time
```

SEC-ELAPSED Return n1 the seconds that have elapsed since TIME-RESET.

```
: SEC-ELAPSED      ( -- n1 )
   TIME-ELAPSED       \ Elapsed ticks
   B>SEC ;        \ Seconds within 18 hours
```

10TH-ELAPSED Return n1 the tenths of a second that have elapsed since the last TIME-RESET was performed.

```
: 10TH-ELAPSED  ( -- n1 )
   TIME-ELAPSED       \ Elapsed ticks
```

```
   0 100 UM/MOD DROP
   10 UM/MOD NIP ;     \ Retain 10th of seconds
```

PAUSE-FUNC A deferred word set to a function that is to be performed while waiting for timing to complete.

```
DEFER PAUSE-FUNC       ' NOOP IS PAUSE-FUNC
```

SECONDS Wait n1 seconds.

```
: SECONDS   ( n1 -- )
   STIME 2@ 2>R >R        \ Save starting time
   TIME-RESET         \ Clear STIME for counting
   BEGIN
      PAUSE  PAUSE-FUNC   \ Pause for task switching
      R@ SEC-ELAPSED > 0= \ Elapsed seconds > n1 ?
   UNTIL   R>DROP      \ If so, exit.
   2R> STIME 2! ;      \ Restore STIME
```

TENTHS Wait n1 tenths of a second.

```
: TENTHS    ( n1 -- )
   STIME 2@ 2>R >R        \ Similar to SECONDS
   TIME-RESET
   BEGIN   PAUSE
      PAUSE-FUNC
      R@ 10TH-ELAPSED     \ Except count 10th of seconds
      > 0=
   UNTIL   R>DROP
   2R> STIME 2! ;
```

MINUTES Wait n1 minutes.

```
: MINUTES   ( n1 -- )
   0 MAX   0
   ?DO   60 SECONDS       \ Let SECONDS do the work
   LOOP ;
```

HOURS   Wait n1 hours.

```
: HOURS ( n1 -- )
   0 MAX   0
   ?DO 60 MINUTES      \ Let MINUTES do it
   LOOP ;
```

0COMPILER   Start the timer to measure time required to compile files. F-PC was optimized using this tools to measure its performance.

```
: 0COMPILER ( -- )
   TOTALLINES OFF      \ Clear compiled line counter
   TIME-RESET ;          \ Clear timer
```

.COMPSTAT   Display the compiling status of the compiler. It shows total compile time, total line compiled, and the compile speed in lines per minute.

```
: .COMPSTAT ( -- )
   cr .elapsed           \ Time used
   cr ." Total lines compiled  =   "
   totallines @ 5 u.r         \ Lines compiled
   cr ." Compiled lines/minute = "
```

```
   totallines @ time-elapsed       \ compute line/min
   over 32000 u> or
   if  " Too much time has elapsed to calculate LPM"
      2drop
   else    >R 60 10000 R@ */MOD    \ Elapsed minutes
      10 * SWAP DUP 10 >      \ It is just a simple
      IF  >R 100 100 100 \   divide.
         r> r> SWAP */ */
         10 /MOD  SWAP 4 >
         IF 1+ THEN +
      ELSE    r>drop DROP
      THEN
      2DUP 100 /
      SWAP 100 / *
      3276 >
      IF  10 / 1000 */
         10 *D 5 D.R     \ Print lines/min
      ELSE
         1000 */ 5 u.R
      THEN
   then    ;
```

TILLKEY Wait up to n1 seconds for a keystroke to continue execution.

```
: TILLKEY   ( n1 -- )
   KEY?
   IF  DROP EXIT  THEN        \ Leave if a key is pressed
   CR ."  Waiting, press SPACEBAR to continue.."
   0 MAX 0
```

```
?DO KEY?          \ Leave at any key
   ?LEAVE
   1 SECONDS      \ Wait a second
LOOP    KEY?
IF  KEY 3 =       \ If key is Break
   ABORT" Quitting "   \   then abort
THEN ;
```

7.16. COMMENTS

Source code is in COMMENT.SEQ. All comment words in F83, like (, .(, \, \S are preserved and they behave similarly in F-PC. However, a file in F-PC is similar to a block in F83 and \S stops the compilation of the rest of the file. To accommodate documentation which spans over many lines, additional comment words are defined in F-PC. Multiple line comment in F-PC starts with COMMENT: and terminates at COMMENT; . It looks like:

```
COMMENT:
<text>
<more text>
...
<even more text>
COMMENT;
```

To print multiple line comments during compiling,

```
.COMMENT:
...
COMMENT;
```

.COMMENT; starts a group of lines that are to be printed to the terminal, until a terminating "COMMENT;" is found.

Plenty of documentation tools are provided in F-PC. There is no excuse not using them to beautify your program.

```
' <run> value crunsave

?EXEC   Abort if still compiling.

: ?exec    ( -- )
   state @ abort" Can not be used while Compiling!" ;

?LOADING    Abort if not loading.

: ?loading     ( --- )
   loading @ 0=
   if crunsave is run
      true abort" Use ONLY while loading"
   then    ;

variable comment$

<comment:>  Primitive used by "comment:".

: <comment:>    ( -- )
   loading @ 0=              \ Abort if not loading.
   if crunsave is run
      true abort" COMMENT: missing a COMMENT; while loading"
   then
   bl word                \ get a word
   1+ comment$ @ count caps-comp 0=    \ terminating string?
   if crunsave is run\ then back to <RUN>
      crunsave execute           \ Resume loading.
```

```
   then    [compile] \ ;          \ Otherwise ignore text.
```

comment: Start a multi-line comment. Patches <comment:> into RUN, it then reads and throws away lines until "comment;" is encountered.

```
: comment:  ( -- )
   ?loading
   [compile] \        \ Ignore this line
   " comment;"           \ Terminating string
   ">$ comment$ !         \ Make COMMENT$ pointing to it
   @> run   !> crunsave       \ Save current interpreter and switch to <COMMENT:>.
   ['] <comment:> is run ;     \ Make the interpreter to run <COMMENT:>, which skips
              \   all lines till COMMENT;
```

/* Similar to COMMENT: . Skip all lines until */ . This is an attempt to fool C and PL/I users to think that they are reading something they know. In fact it is true, because comments are universal in all programming languages. They might even be in English.

```
: /*       ( -- )
   ?loading
   [compile] \        \ See COMMENT:
   " */" ">$ comment$ !
   @> run   !> crunsave       \ Save current interpreter and switch to <COMMENT:>.
   ['] <comment:> is run ;
```

<.comment:> A primitive used by ".comment:".

```
: <.comment:>   ( -- )        \ Similar to <COMMENT:>
   loading @ 0=          \ Abort if not loading
   if  crunsave is run
      true abort" .COMMENT: missing a COMMENT; while loading"
```

```
   then
   bl word 1+          \ Compare next word with
   " comment;" caps-comp 0=    \   COMMENT;
   if  crunsave is run     \ If equal, resume loading
   else    key? 0=         \ Not equal,
      if  cr tib      \   print this line
         #tib @
         78 min type
      then
   then    [compile] \ ;       \   and then ignore it
```

.comment: Start a multi-line print statement. All lines following the ".comment:" line will be printed to the display until a "comment;" is encountered.

```
: .comment: ( -- )
   ?loading
   [compile] \         \ Ignore this line
   @> run  !> crunsave
   ['] <.comment:> is run ;        \ Display comments until  COMMENT;
```

Jeff made an interesting comment to Chuck Moore in the 1988 Forth Convention at Anaheim, that Chuck defined the comments backwards. Jeff's assertion was that comments should not be enclosed between parentheses. It is the source code that should be enclosed inside the parentheses, embedded in running text. Glen Haydon implemented the concept for F-PC:

```
: } ( --- )                 \ Skip all text between } and { .
   ?loading
   " {" ">$ comment$ !
   @> run !> crunsave
   ['] <comment:> is run <comment:>
   ; immediate
```

```
' }    alias \\                \ new char to start F-PC source files with by Tom Zimmer.

' noop alias {                  \ make this a dummy word that does nothing
```

Files that contain source code, and normal forth type imbedded comments will work as before. You can use "\" or "comment:" and "comment;" to create comments as if nothing has changed. Files that start out with "\\", rather than the normal "\" in the first line, specify files that are going to contain comments except for those lines contained in "{" and "}" which will be treated as Forth source. Here is an example:

```
\\ TEXTFILE.SEQ    This file contains comments except for { } notation.

This is a file containing comments and documentation except as follows

{   <Forth-words> ...   }

We can write a dessertation about the algorithms and implementation.

{   <More-Forth-words> ...  }

More comments to end of file, automatically drops out of comment mode at the end of the file.
```

7.17. CONDITONAL COMPILATION

Conditional compilation words are also defined in the COMMENT.SEQ file. Their function is to allow part of the source file to be loaded depending upon the condition exists at the loading time. For example, if a floating point word exists in the dictionary, the entire floating point package must have been loaded. Consequently, the floating point package needs not be reloaded. The condition can be tested during loading, and the loading sequence does not have to be specified precisely before you start loading files.

```
<#if>       A primitive used by #IF.  Ignore lines until #ELSE or #ENDIF.

: <#if>     ( -- )
   loading @ 0=
   if crunsave is run
      true abort" #IF missing a #THEN while loading"
```

```
  then
  begin   bl word dup c@ 0= >r 1+
    dup " #endif" caps-comp 0= over
        " #then"  caps-comp 0= or swap
        " #else"  caps-comp 0= or dup
    if crunsave is run
      >r crunsave execute r>
     then   r> or
  until   ;
```

#ELSE Start a false multi-line structure.

```
: #else    ( -- )
  ?loading                \ you must be loading
  ?exec                   \ but NOT compiling
  @> run !> crunsave              \ Save previous value of RUN.
  ['] <#if> is run\ Ignore following lines
  <#if> ;       immediate         \   and rest of this line.
```

#if Starts a multi-line compiler directive. If the Boolean flag passed to #IF is true, then the lines following #IF will be executed. If the flag is false, then the lines following #IF up to the #ENDIF or #ELSE, will be ignored. Here is an example of how #IF works:

```
  TRUE   ( -- f1 )
  #IF    True flag passed, this comment needs no preceding \
    cr .( This is printed due to TRUE before #IF.)
  #ENDIF

  FALSE  ( -- f1 )
  #IF
```

```
      .( This is NOT printed due to FALSE flag in #IF.)
   #ENDIF


: #if      ( f -- )
   ?loading
   ?exec
   0=
   if @> run !> crunsave  \ Save previous value of RUN.
      ['] <#if> is run       \ Ignore following lines
      <#if>          \   and rest of this line.
   then    ;      immediate

#endif  Just a noop for the #if TRUE case.  Ends a multi-line Compiler directive.  See #IF.

: #endif    ( -- )  ?loading  ; immediate

: #then     ( -- )  ?loading  ; immediate
```

Here is an example of how the interpreted compiler directives work.

```
   cr cr .( press a T for true )
   key bl or 116 =         \ wait for a key
   #if .( TRUE)
   #else .( FALSE)
   #then .(  always )
```

7.6. DUMP

Memory dumping is more complicated in F-PC than in F83 because it has to address the entire memory space in the AT/PC. The regular DUMP in F83 is retained to dump memory in the Code Segment. Special words XDUMP and YDUMP are defined to dump memory in the List Segment and the Head Segment. All these variations are derived from the generic dump utility word LDUMP which uses a seg:addr pair to specify the memory area to be dumped. A variable DUMPSEG is used to define the dump segment to facilitate accessing the extended memory range.

The dump utility gives you a formatted hex dump, 16 bytes to a line, with the ASCII text corresponding to the bytes on the right hand side of the screen. DUMPSEG is set accordingly.

VARIABLE DUMPSEG Store the segment address from where memory contents are shown.

DUMPC@ Fetch a byte from the dump segment, given the offset address on the stack.

```
: %DUMPC@   ( addr -- b )
   DUMPSEG @          \ Get the dump segment address
   SWAP               \ Move offset address to top
   C@L ;              \ Fetch a byte with long fetch

DEFER DUMPC@    ' %DUMPC@ IS DUMPC@
```

.2W Print n in two digits followed by a space.

```
: .2W      ( n -- )
   0 <# # # #>        \ Convert two digits only
   TYPE SPACE   ;     \ Type them with a trailing space
```

D.2W Dump the contents of an array of n bytes from addr, in two digit pairs.

```
: D.2W    ( addr ln -- )
   BOUNDS        \ Scan the range of memory
   ?DO I DUMPC@      \ Fetch byte from dump segment
      .2W       \ Display one byte at a time
   LOOP  ;
```

EMIT. Emit an ASCII character char. If the char is not printable then print a ".".

```
: EMIT. ( char -- )
   127 AND          \ Eliminate most significant bit
   DUP BL 126 BETWEEN \ Is it a control character?
   NOT IF
      DROP ASCII .      \ Yes.  Substitute with a period
   THEN EMIT ;       \ Print it
```

DLN Dump a line, consisting of 16 bytes of hex data followed by 16 bytes of ASCII string data.

```
: DLN     ( addr -- )
   CR  ."  |"       \ New line
   DUMPSEG @ 0       \ First display dump segment
   <# ASCII : HOLD       \  Starting with segment:
      # # # # #> TYPE
   DUP 0            \ Follow by starting address
   <# # # # # # #> TYPE
   ."  | " 8 2DUP D.2W    \ Dump 8 bytes with a separator
   OVER + 8 D.2W ." |"       \ Next 8 bytes with separator
   16 BOUNDS ?DO         \ Dump contents in ASCII form
```

```
    I DUMPC@ EMIT.
  LOOP  ." |" ;          \ Field terminator
```

.HEAD Print the header line for DUMP.

```
: .HEAD ( addr len -- addr' len' )
  OVER 15 AND
  CR ." -----------"
  49 0 DO ." -" LOOP ." ? 16 0 DO ." -" LOOP ." ?
  CR ." - SEG:OFF - "
  16 0 DO DUP I + 15 AND 2 .R SPACE LOOP ." -"
  16 0 DO DUP I + 15 AND 1 .R LOOP DROP  ." -"
  CR ." ----------"
  49 0 DO ." -" LOOP ." -" 16 0 DO ." ? LOOP ." -" ;

: .FOOT ( --- )
  CR ." ----------"
  49 0 DO ." -" LOOP ." -" 16 0 DO ." ? LOOP ." -" CR ;
```

LDUMP The generic dump word. Dump len bytes of a memory area starting from seg:addr. The dump segment is also stored in DUMPSEG, so that you can continue dumping using DU. The dumped object code is enclosed in a nicely formed box with dividing lines, very pretty on the screen.

```
: LDUMP ( seg addr len -- )
  ROT DUMPSEG !      \ Save dump segment in DUMPSEG
  BASE @ -ROT        \ Save current base
  HEX .HEAD BOUNDS       \ Display head in hex
  DO  I DLN  KEY? ?LEAVE  \ Dump one line at a time
  16 +LOOP           \ Until completion or a key
  BASE !  CR ;          \ Restore BASE
```

DUMP Dump the contents of a memory array from address addr for length len in the Code Segment. This is the most commonly used dump word because most data and buffers are in the Code Segment, and it is compatible with the DUMP in F83.

```
: DUMP  ( addr len -- )
   ?CS:              \ Make CS the dump segment
   -ROT LDUMP ;          \ Dump Code Segment memory
```

YDUMP Dump an area of n bytes in the Head Segment.

```
: YDUMP ( addr n -- )
   YSEG @         \ Make Head Segment the dump segment
   -ROT LDUMP ;          \ Then dump
```

XDUMP Dump an area of n bytes in the List Segment. The segment address is relative to the base segment address XSEG pointing to the beginning of the List Segment.

```
: XDUMP ( seg-offset n -- )
   SWAP XSEG @ +      \ Get the absolute segment address for the dump segment
   0 ROT LDUMP ;      \ Dump
```

DU Dump 64 bytes from the dump segment starting at an offset address addr. Return the address of next line so that DU can be executed again.

```
: DU              ( addr -- addr+64 )
   DUP 64 DUMPSEG @       \ addr 64 seg --
   -ROT LDUMP         \ Dump 64 bytes
   64 +   ;              \ Address of next line
```

7.7. CASE

This case structure was originally reported by Charles Eaker in Forth Dimensions, Vol. 2, No. 3, Sep/Oct 1980. It was adapted to F-PC by Bob Smith. The most critical part (OF) is coded in assembly and moved into the F-PC Kernel. It is repeated here for completeness.

A case structure is a control structure which allows multiple branches selected by a value. It must be defined inside a colon definition. A case statement must be constructed as follows:

```
CASE ( value -- )
   value1 OF  func1   ENDOF
   value2 OF  func2   ENDOF
   value3 OF  func3   ENDOF
     ...
   otherwise-func DROP
ENDCASE
```

CASE and ENDCASE enclose the case statement. OF and ENDOF enclose a branch. Value is passed into the case structure on the data stack. OF compares value with the number before it. If the two numbers are equal, the function after OF is executed until ENDOF, and the case terminated with value removed from the stack. If the numbers are not equal, value is passed to the next OF for comparison. The otherwise-func can still make use of value if none of the branches was executed. If it does not use value, DROP discard the value and the case structure is terminated.

CASE is extremely powerful for multiple tests and multiple branches which are difficult to code with the IF-ELSE-THEN structure. F-PC uses it extensively in the system. However, if values for comparison are sequential, the EXEC: word is much faster, and more efficient in memory. See EXEC:

(OF) Primitive case comparison word, compiled by OF. Performs a comparison between n1 and n2.

```
CODE (OF)   ( n1 n2 -- n1 ) or ( n1 n1 -- )
   POP AX        \ Pop n2
   MOV DI, SP
```

```
   CMP AX, 0 [DI]     \ Compare n2 and n1
   0<> IF             \ Not equal,
      MOV ES: IP, 0 [IP] \   jump to the next case.
      NEXT
   THEN               \ n1=n2
   ADD SP, # 2        \ Drop n1 from the data stack
   ADD IP, # 2        \ Execute the OF clause
   NEXT
   END-CODE
```

CASE Start a multiple branch control structure which allows an arbitrary value to specify a function to be performed.

```
: CASE  ( n --  n )
   COMPILE DOCASE
   <MARK          \ Simply put a place mark here
   0              \ Mark the data stack as the bottom of address list for
   ; IMMEDIATE        \   ENDCASE to resolve
```

OF Start a conditional branch phrase in a case structure.

```
: OF       ( -- )
   COMPILE (OF)       \ Compile run-time code (OF)
   >MARK ;            \ Compile a dummy forward branch address and leave its
                \   address on stack for ENDOF to resolve
   IMMEDIATE          \ Must execute immediately
```

ENDOF Close a conditional branch phrase in the a case structure.

```
: ENDOF ( -- )
```

```
   COMPILE DOENDOF        \ Compile an unconditional forward branch to ENDCASE
   >MARK          \ Leave address for ENDCASE
   SWAP >RESOLVE      \ Resolve branch address after OF
   ;  IMMEDIATE      \ Do it now
```

NRESOLVE Primitive used by ENDCASE to resolve the previous case references.

```
: NRESOLVE  ( 0 n1 n2 ... n -- )
   BEGIN            \ Place current IP in all locations on stack till the 0 marker
      DUP        \ Is it a valid address?
      IF  >RESOLVE 0  \ Yes.  Resolve forward branch
      ELSE   -1         \ No.  End of list.  Terminate.
      THEN
   UNTIL   DROP ;      \ Clean up the stack
```

ENDCASE Resolve all branches in a case structure and close the structure.

```
: ENDCASE   ( -- )
   NRESOLVE          \ Resolve all forward branches
   DROP ;         \ Discard marker left by CASE
   IMMEDIATE
```

7.8. SET DIRECTORY PATH

PATHSET.SEQ has a word set to select and set the directory path to files. PATHSET checks the file contained in a handle. It it does not contain a path, then apply the current drive and path to the handle. It allows F-PC to use the full path/file name in DOS to specify files to be accessed.

flhndl A value that returns the address of the handle we are currently working on.

```
0 value flhndl              \ Will be plugged later
```

?drive.extract Given a handle, return drive-value of the drive specified in handle. Return the current drive if no drive was specified in handle.

```
: ?drive.extract       ( handle --- drive-value )
   dup >nam 1+ c@      \ Get the second character
   ascii : =           \ Is it a : ?
   if  dup>r >nam c@       \ Get the drive character
      bl or 96 -       \ Convert to drive number, A: is 1
      r@ count 2- >r       \ Length less drive specification
      dup 2+ swap      \ name+3 name+1
      r> cmove         \ Copy name over the drive spec
      r@ c@ 2- r@ c!       \ Reduce name length by 2
      r> count + off       \ NULL fill the end
   else                    \ No drive specified
      drop 0 25 bdos 1+    \ Ask system the current drive
   then    ;
```

?drive.prepend Given a drive-value and a handle, prepend the drive-value into handle, as a letter followed by ":".

```
: ?drive.prepend    ( drive-value handle --- )
   over 0=             \ Drive number =0 ?
   if  2drop           \ Yes.  Use default drive.
   else                \ A drive is specified.
      dup>r count >r       \ Name address
      dup 2+ r> cmove>     \ Move name down by 2 characters
      64 + r@ >nam c!      \ Insert drive name
```

```
      ascii :
      r@ >nam 1+ c!       \   with a colon
      r@ c@ 2+ r> c!      \ Adjust character count
   then    ;
```

HANDLE pathhndl   \ A handle that contains the default drive path.

prepend.path   Prepend the default drive and path to a handle, and return a Boolean flag that tells if we were successful.

```
: prepend.path  ( handle --- f1 )
   pathhndl clr-hcb
   dup =: flhndl ?drive.extract >r
   flhndl >nam c@ '\' <> dup                    \ leave if got path
   if     drop
      '\' pathhndl 1+ c! 64 pathhndl c!
      pathhndl 2+ r@ pdos dup 0=
      if     drop
         pathhndl 1+ 64 0           \ determine path length
         do     dup i + c@ 0= if i pathhndl c! leave then
         loop    drop pathhndl c@ 1 >
         if     '\' pathhndl count + c!
            pathhndl c@ 1+ pathhndl c!
         then    flhndl c@ pathhndl c@ + 62 > dup 0=
         if     drop
            flhndl 1+ pathhndl c@ over + flhndl c@ cmove>
            pathhndl count flhndl 1+ swap cmove
            pathhndl c@ flhndl c@ + flhndl c!
            false
         then
      then
```

```
  then
  r> flhndl ?drive.prepend 0 flhndl count + c! ;

' PREPEND.PATH IS PATHSET
```

7.9. STRING SEARCH

SEARCH.SEQ was contributed to F-PC by Roger Bird. It is 20% faster than our earlier version and 140 bytes smaller. It may be removed in a turnkey application, but it is used by WORDS.SEQ, and ENVIRON.SEQ, which is itself used by EXEC.SEQ. It must be loaded for an interactive Forth system.

The string manipulation primitives include string comparison and searching. The string search implemented here is used in the editor to find desired strings. The only unusual thing about it is the presence of a variable called CAPS, which determines whether or not to ignore the case of the subject and pattern strings. If case is ignored then A-Z = a-z. The default is to ignore case.

Much thanks to Roger Bird for this improved implementation of SEARCH. It is about 20% faster than the previous improved version in F-PC, and is 10 times faster in the case where the string being searched for starts with a space. It is also 140 bytes smaller than the old implementation.

SEARCH Search for a source string (saddr,slen) contained in the destination string (daddr,dlen). Return a true flag if found, and an address equal to the offset into the destination string where saddr,slen was found.

```
CODE SEARCH     ( saddr slen daddr dlen -- offset found? )
   pop cx           \ dlen
   pop di           \ daddr
   pop bx           \ slen
   pop ax           \ saddr
   cmp bx, # 0              \ if search length "slen" is 0 then return true
   0= if   push bx
      mov ax, # true
```

```
   1push
then
push di            \ Save daddr
push es            \ Save ES
mov es, sseg           \ ES points to string segment
push si            \ Save IP
push bp         \ Save RP
mov si, ax          \ Source address saddr
dec bx             \ Source count-1
dec cx             \ Destination count-1
mov dx, bx          \ Save a copy in DX
cmp caps # 0           \ Case sensitive?
0= if              \ Yes.  Clear mask.
   mov bp, # 0
else               \ Case insensitive.
   mov bp, # $2020 \ Set mask to two blanks
then
begin              \ Search loop
   cmp bx, cx      \ Source-len > dest-len ?
<= while           \ No.  Must do search.
   begin           \ Compare strings
      mov al, 0 [si+bx]   \ Get a byte from source
      mov ah, es: 0 [di+bx]   \ Get a byte from dest
      or ax, bp       \ Mask case bit
      cmp ah, al      \ Are they the same?
   0= while           \ Yes.
      dec bx           \ Decrement source count
      0< if          \ Source string exhausted?
         pop bp      \ Yes.  Match is found.
```

```
            pop si      \ Restore IP
            pop es      \ Restore List pointer
            pop dx      \ Destination string addr
            sub di, dx  \ Addr of found string
            push di     \ Push on stack
            mov ax, # $FFFF \ True flag
            1push       \ Push on stack
          then          \ Not yet done
        repeat          \ Compare next byte
        inc di        \ Move on the next source byte
        dec cx        \ Decrement destination count
        mov bx, dx      \ Restore source count
    repeat            \ Fail to find a match
    pop bp            \ Restore RP
    pop si            \ Restore IP
    pop es            \ Restore ES
    pop dx            \ Get dest address
    sub di, dx        \ Move to original daddr
    push di           \ Push on stack
    sub ax, ax        \ Make false flag
    1push             \   and push on stack
    end-code
```

7.10. WORDS

WORDS is used in F83 to display the names of words in the context vocabulary. This is a very important utility in Forth to allow the user to inspect the words currently available to be used for execution or compilation. F-PC greatly enhances the usefulness of WORDS by allowing it to take a string argument to trigger some very interesting functions.

If WORDS is followed by an arbitrary string, all the words in all vocabularies containing that string pattern will be reported. This is very useful to identify a group of words whose spelling is not known for sure.

If the string argument is *.* , then all words in all vocabularies are displayed. CODE.* allows only code words to be searched and reported. :.* allows only colon words. Other patterns allowed are VARIABLE.* , CONSTANT.* , UVARIAVLE.* , DEFER.* , and UDEFER.* for obvious purposes. TOTAL.* causes only the word count to be reported.

Since F-PC has a very large dictionary, these conditional searching words are more important here than in other smaller Forth systems. If you are using a color monitor, F-PC can display different types of words in different color. The colorization can be turn on or off, by executing the word COLORIZEON or COLORIZEOFF.

Here is a list of variables used in displaying word lists:

```
VYET            \ Did we print this vocabulary yet?
VADDR           \ Vocabulary name address
TOTALWORDS          \ Accumulator for the total number of names.
```

.VYET Print the name of a vocabulary if we have not printed it yet.

```
: .VYET ( -- )
   VYET @ IF EXIT THEN \ If VYET is true, this vocabulary was printed already.  Exit.
   VADDR @ CR       \ Otherwise, print the vocabulary
   ." --[ " .ID ."  ]--"     \   name first.
   VYET ON       \ Make sure doing it only once
   CR LMARGIN @ SPACES  ;  \ New line with left margin

DEFER W.ID ' .ID IS W.ID      \ .ID is deferred to accommodate the colorizer.
CREATE W$ 64 ALLOT     \ Word name is store here temporarily to match substrings.
   W$ 64 ERASE
```

?INNAME A substring pattern is stored in the word buffer at HERE. Search the word name at nfa for this substring. Return a true if the substring is contained in the word name.

```
: ?INNAME   ( nfa -- f )
   @> YSEG SWAP       \ seg:addr of the name field of this word
   ?CS: HERE 31 CMOVEL \ Copy word name to HERE
   $C0 HERE CRESET                \ mask to real count only
   $80 HERE COUNT + DUP OFF    \ clear end of name to NULL
   1- CRESET          \ mast high bit of last char
   W$  COUNT          \ First pattern to be searched.
   HERE COUNT SEARCH NIP
   W$ 32 + COUNT      \ Second pattern to be searched
   HERE COUNT SEARCH NIP
   AND ;              \ Return true if word name contains both sub-strings
```

?CODENAME Return true if the word at nfa is a code definition.

```
: ?CODENAME ( nfa -- f )
   NAME>          \ Get cfa
   C@ DUP 232 <>      \ If it does not contain a CALL
   SWAP 233 <> AND ;      \   nor a JMP, then it must be a code word
```

```
0 VALUE WORDTYPE       \ Address of inner interpreter for type identification
```

?WORDTYPE Return true if the word at nfa is of the same type as shown in WORDTYPE.

```
: ?WORDTYPE ( nfa -- f )
   NAME>          \ Get cfa
   @REL>ABS           \ Get absolute address of the inner interpreter of this word
   WORDTYPE = ;       \ Compare with WORDTYPE
```

?TOTALWORD Increment the total word count. Replace nfa with a false flag.

```
: ?TOTALWORDS   ( nfa -- false )
   DROP FALSE
   TOTALWORDS INCR ;
```

?W.NAME This is a critical word in the WORDS utility in F-PC. It is a deferred word. It takes a nfa and returns a flag. If the flag is true, the name of this word should be printed. Otherwise, the word will be ignored. Depending on the search string following WORDS, ?W.NAME will be vectored to a words which selects the specific type of words and sets the proper flag. See the following words for its usage and behavior.

```
DEFER ?W.NAME   ( nfa -- flag )
```

W.NAME Print the name of a word at nfa filtered by the string at HERE. Test to see if conditions are correct, like CODE, or ALL, or WITHIN, and then print the name.

```
: <W.NAME>  ( nfa -- )
   DUP ?W.NAME     \ Call ?W.NAME to decide  whether name should be printed.
   IF  .VYET 17 ?LINE W.ID     \ Yes.  Print vocabulary and word name.
      #OUT @ COLS 16 - < \ Tab if needed
      IF TAB THEN
      TOTALWORDS INCR \ Bump word count
   ELSE    DROP          \ Do not print name
   THEN    ;

DEFER W.NAME    ' <W.NAME>  IS  W.NAME
```

.VOCWORDS   Display all the matching words, i.e., words selected by ?W.NAME, in a vocabulary.

```
code largest    ( a1 n1 --- a2 n2 ) \ Return the largest element of an array of words
   pop cx                  \ count of array to search
   pop bx                  \ starting address of array
   mov ax, # 0             \ starting highest value = 0
   mov dx, bx              \ highest value addr = start address
here    cmp 0 [bx], ax
   u> if   mov dx, bx
      mov ax, 0 [bx]
   then
   add bx, # 2             \ bump to next element
   loopnz
   2push         end-code


: .VOCWORDS ( voc-addr -- )
   DUP HERE 500 +      \ Copy vocabulary threads
   #THREADS 2* CMOVE       \   to Code Segment
   BODY> >NAME VADDR ! \ Store address of vocabulary
   VYET OFF          \   in VADDR
   BEGIN
      HERE 500 +      \ Thread table
      #THREADS
      LARGEST DUP \ Find highest name on  threads
      ?KEYPAUSE
   WHILE
      DUP L>NAME      \ Go from lfa to nfa
      W.NAME      \ Print out name
      Y@ SWAP !       \ Replace thread with next lfa
   REPEAT  2DROP ;     \ Repeat for the vocabulary
```

```
DEFER ?W.TEST          \ A template to provide DODEFER

' NOOP IS ?W.TEST          \   to ?DEFER.* later
```

The following set of words ?xxxxx.* vector execution words into ?W.NAME so that ?W.NAME will be able to select a specific class of words to be printed. They test the string in the word buffer at HERE and change the behavior of ?W.NAME accordingly.

```
: ?*.*     ( -- )
   W$ 1+          \ Word buffer
   " *.*" COMP 0=      \ Is the string *.* ?
   IF  ['] NOOP       \ Yes.  Store NOOP in ?W.NAME
      IS ?W.NAME      \   allow all words to be matched
   THEN    ;

: ?CODE.*   ( -- )
   W$ 1+          \ Is the string CODE.* ?
   " CODE.*" CAPS-COMP 0=
   IF  ['] ?CODENAME   \ If so, search only code words
      IS ?W.NAME      \   by vectoring ?W.NAME to
   THEN    ;         \   ?CODENAME

: ?:.*     ( -- )
   W$ 1+          \ If the string in word buffer is
   " :.*" CAPS-COMP 0=     \   :.* ,
   IF  ['] ?*.* @REL>ABS   \   let WORDTYPE points to NEST
      =: WORDTYPE
      ['] ?WORDTYPE   \   and ?W.NAME is vectored to
      IS ?W.NAME      \   ?WORDTYPE to print colon words
   THEN    ;
```

```
: ?VARIABLE.*   ( -- )
   W$ 1+            \ For variables
   " VARIABLE.*" CAPS-COMP 0=
   IF  ['] TOTALWORDS @REL>ABS     \ WORDTYPE points to >NEXT
      =: WORDTYPE
      ['] ?WORDTYPE IS ?W.NAME    \ ?WORDTYPE is execution
   THEN    ;             \   routine in ?W.NAME

: ?UVARIABLE.*  ( -- )
   W$ 1+            \ For user variables
   " UVARIABLE.*" CAPS-COMP 0=
   IF  ['] BASE @REL>ABS       \ WORDTYPE points to
      =: WORDTYPE        \   DOUSER-VARIABLE
      ['] ?WORDTYPE
      IS ?W.NAME
   THEN    ;

: ?CONSTANT.*   ( -- )
   W$ 1+            \ For constants
   " CONSTANT.*" CAPS-COMP 0=
   IF  ['] WORDTYPE @REL>ABS   \ WORDTYPE points to
      =: WORDTYPE        \   DOCONSTANT
      ['] ?WORDTYPE
      IS ?W.NAME
   THEN    ;

: ?VALUE.*     ( --- )
   W$ 1+ " VALUE.*" CAPS-COMP 0=
   IF     ['] WORDTYPE @REL>ABS =: WORDTYPE
```

```
      ['] ?WORDTYPE IS ?W.NAME
   THEN    ;

: ?DEFER.*  ( -- )
   W$ 1+             \ For deferred words
   " DEFER.*" CAPS-COMP 0=
   IF  ['] ?W.TEST @REL>ABS    \ WORDTYPE points to
      =: WORDTYPE        \   DODEFER
      ['] ?WORDTYPE
      IS ?W.NAME
   THEN    ;

: ?UDEFER.* ( -- )
   W$ 1+             \ For user deferred words
   " UDEFER.*" CAPS-COMP 0=
   IF  ['] EMIT @REL>ABS      \ Point WORDTYPE to
      =: WORDTYPE        \   DOUSER-DEFER
      ['] ?WORDTYPE
      IS ?W.NAME
   THEN    ;

: ?TOTAL.*  ( -- )
   W$ 1+             \ For counting words
   " TOTAL.*" CAPS-COMP 0=
   IF  CR ." Not displaying, just counting the TOTAL "
      ['] ?TOTALWORDS        \ Vector ?W.NAME to word
      IS ?W.NAME         \   just incrementing count
   THEN    ;
```

WORDS Display words that match a text string. <text> is optional. If it is omitted then the CONTEXT vocabulary will be displayed. Several specific values for <text> can be included on the command line, as follows:

```
  WORDS *.*                 display all words in all vocs.
  WORDS CODE.*          display all code words.
  WORDS :.*                 display all : definitions.
  WORDS CONSTANT.*          display all constants.
  WORDS VARIABLE.*          display all variables.
  WORDS UVARIABLE.*         display all user variables.
  WORDS DEFER.*             display all deferred words.
  WORDS UDEFER.*            display all user deferred words.
  WORDS TOTAL.*             display total count of words only.

FALSE VALUE CONTEXTONLY
DEFER PREWORDS  ' NOOP  IS  PREWORDS

: WORDS   ( -- )
  TOTALWORDS OFF     \ Clear word count
  SAVESTATE          \ Save screen state
  COLS 2- RMARGIN !      \ Set display parameters
  15 TABSIZE !
  LMARGIN OFF
  CR ."  ** Press SPACE to pause, or ESC to exit ** "
  PREWORDS
  >IN @ #TIB @ <>     \ If not at the end of a line
  IF  ['] ?INNAME        \ May do a substring match
    IS ?W.NAME
    BL WORD W$      \ Get first sub-string to be searched
    OVER C@ 1+ 32 MIN CMOVE
    BL WORD W$ 32 +     \ Get second sub-string to be searched
```

```
   OVER C@ 1+ 32 MIN CMOVE
   ?*.*          \ May search all words
   ?CODE.*     \ May search only code words
   ?:.*          \ May search colon words
   VARIABLE.*      \ May search variables
   ?CONSTANT.*     \ Or constants
   ?DEFER.*       \ Or deferred words
   ?VALUE.*
   ?UVARIABLE.*    \ Or user variables
   ?UDEFER.*       \ Or user deferred words
   ?TOTAL.*       \ Or just count words
   CONTEXTONLY
   FALSE =: CONTEXTONLY
   IF  CONTEXT @   \ If CONTEXTONLY was true, search only the context
     .VOCWORDS   \   vocabulary.
   ELSE    VOC-LINK @  \ Otherwise, go through all vocabularies
     BEGIN
       DUP        \ Next vocabulary link
       #THREADS 2* -   \ Head of thread table
       .VOCWORDS   \ Print all words in this  vocabulary with matching
            \   characteristics
       @ DUP 0=    \ Next vocabulary or end?
     UNTIL   DROP       \ Exit when all vocabularies are processed
   THEN
ELSE
   ['] NOOP IS ?W.NAME     \ No string following
   FALSE =: CONTEXTONLY
   CONTEXT @ .VOCWORDS \ Display words in context
THEN              \   vocabulary, like in F83
```

```
   CR TOTALWORDS @ U. \ Print total word counted
   ." Words displayed" CR
   RESTORESTATE ;      \ Restore screen display

ROOT DEFINITIONS           \ Make WORDS available everywhere
' WORDS ALIAS WORDS
FORTH DEFINITIONS
```

7.11. CURSOR CONTROL WORDS

IBMCURSR.SEQ contains words which position the cursor on the display screen. F-PC supports IBM color card and monochrome card, and also Hercules high resolution card. It is smart enough to be able to select the appropriate word set to handle the screen display automatically.

IBM-AT Move the cursor on the IBM display screen to the column and row specified. This is done using a BIOS level interrupt. This word is normally the deferred function of AT, you should use AT for portability.

```
CODE IBM-AT ( col row -- )
   POP AX
   POP DX
   MOV DH, AL
   XOR BH, BH
   MOV AH, # 2
   PUSH SI PUSH BP
   INT $10        \ BIOS service 16, function 2
   POP BP  POP SI
   NEXT   C;
```

IBM-AT? Return the col and row of the current cursor location on the screen.

```
CODE IBM-AT?    ( -- col row )
   XOR BH, BH
   MOV AH, # 3
   PUSH SI PUSH BP
   INT $10        \ BIOS service 16, function 3
   POP BP  POP SI
   SUB AX, AX
   MOV AL, DH
   MOV DH, AH
   2PUSH   END-CODE
```

IBM-DARK Clear the screen and place the cursor in the upper left hand corner. This is the deferred function of DARK. Use DARK for portability.

```
CODE IBM-DARK   ( -- )
   MOV AX, # 2
   PUSH SI PUSH BP
   INT $10
   SUB AH, AH
   INT $10        \ Clear the screen
   POP BP  POP SI
   NEXT   END-CODE
```

LFILLW Similar to LFILL, except filling the memory area with a 16 bit word pattern.

```
CODE LFILLW     (  seg start-addr byte-count word-pattern -- )
   CLD
   POP AX
   POP CX
```

```
   SHR CX, # 1
   POP DI
   POP BX
   MOV DX, ES
   MOV ES, BX
   REPNZ        STOSW      \ Repeat Store-Words
   MOV ES, DX
   NEXT
   END-CODE
```

NORM-DARK If >DOSIO is true, use IBM-DARK to blank the screen. Otherwise, clear the screen by filling the screen buffer with proper word pattern. This is necessary to deal with non-IBM video cards.

```
: norm-dark    ( --- )
   ?dosio
   if        ibm-dark
   else   blank.color
      VIDEO-SEG @ 0 ROWS COLS 2* *
      attrib c@ flip $20 or LFILLW
      show.color
   then   0 0 at ;
```

IBM--LINE Delete the current line on the IBM display. It causes all lines lower on the screen to scroll up. This is the deferred function of -LINE. Use -LINE for portability.

```
CODE IBM--LINE  ( -- )
   PUSH SI PUSH BP
   XOR BH, BH
   MOV AH, # 3
   INT   $10        \ Read Cursor Position
```

```
   MOV CH, DH
   XOR CL, CL         \ Upper left corner
   MOV DX, # ' ROWS >BODY
   DEC DH
   MOV DL, ' COLS >BODY
   DEC CL
   MOV BH, # 7
   MOV AH, # 6           \ Scroll up
   MOV AL, # 1        \ Lines to scroll
   INT 16
   POP BP  POP SI
   NEXT   END-CODE
```

>IBM Select the IBM display word set as the default display words.

```
: >IBM  ( -- )
   ['] IBM-AT   IS AT
   ['] IBM-AT?  IS  AT?
   ['] NORM-DARK IS DARK
   ['] IBM--LINE IS -LINE ;
>IBM
```

CURSOR_POS_INIT Initialize the cursor pointers in F-PC to the displayed cursor.

```
: CURSOR_POS_INIT   ( -- )
   IBM-AT?
   #LINE !           \ Row number
   #OUT ! ;          \ Column number
```

' CURSOR_POS_INIT IS CURSORSET

The following words changes the shape of the cursor. The cursor shape and characteristics are a good indicator to the current F-PC status, especially in the editor.

SET-CURSOR Set the cursor shape to the shape value n.

```
CODE SET-CURSOR ( n -- )
   POP CX         \ Cursor shape mask
   MOV AH, # 1
   PUSH SI PUSHJ BP
   INT $10
   POP BP  POP SI
   NEXT C;
```

GET-CURSOR Get the cursor shape mask SHAPE.

```
: GET-CURSOR    ( -- shape )
    0 $460 @L ;          \ The shape code is stored in the memory segment 0
```

CURSOR-OFF Turn off the cursor.

```
: CURSOR-OFF    ( -- )
   GET-CURSOR
   $2000  OR         \ Set the cursor off bit
   SET-CURSOR ;
```

CURSOR-ON Turn the cursor back on.

```
: CURSOR-ON ( -- )
```

```
   GET-CURSOR
   $0F0F AND          \ Set cursor on pattern
   SET-CURSOR ;

: *NORM-CURSOR  ( -- )
   CROWS DUP 1- FLIP +     \ Combine rows and columns
   SET-CURSOR ;

: *BIG-CURSOR   ( -- )
   CROWS  SET-CURSOR ;

: *MED-CURSOR   ( -- )
   CROWS DUP 2/   FLIP +
   SET-CURSOR ;

DEFER NORM-CURSOR   ' *NORM-CURSOR IS NORM-CURSOR
DEFER BIG-CURSOR          ' *BIG-CURSOR IS BIG-CURSOR
DEFER MED-CURSOR    ' *MED-CURSOR IS MED-CURSOR
```

7.12. MONOCHROME VIDEO WORDS

This set of words in MONOCROM.SEQ deals with the IBM Monochrome Video Card. The words are fairly self-explanatory.

```
$07 VALUE NORMVAL                    \ VALUE FOR NORMAL VIDEO
$78 VALUE REVVAL                     \ VALUE FOR REVERSE VIDEO
$07 VALUE NORMALVAL
$78 VALUE REVERSEVAL
```

```
: >NONE ( --- )    NORMVAL ATTRIB C! ;        \ NORMAL
: >NONEBG  ( --- )    NORMVAL $F0 AND ATTRIB C@ $0F AND OR ATTRIB C! ;
: >REVERSE ( --- )    REVVAL  ATTRIB C! ;        \ REVERSE
: >UL        ( --- ) $01 ATTRIB C! ;          \ UNDERLINE
: >BOLD       ( --- )    $7F ATTRIB C! ;             \ BRIGHT
: >BOLDUL  ( --- )    $79 ATTRIB C! ;             \ BOLD UNDERLINE
: >BOLDBLNK ( --- )    $8F ATTRIB C! ;              \ BOLD BLINK
: >REVBLNK  ( --- )    $F0 ATTRIB C! ;             \ REVERSE BLINK

' >NONE   IS >NORM
' >REVERSE IS >REV
' >NONEBG  IS >NORMBG

: >MONO ( --- )          \ Select hilighting for monochrome monitor.
   ['] >UL IS >ATTRIB1
   ['] >BOLDBLNK IS >ATTRIB5
   ['] >BOLDUL DUP IS >ATTRIB2 IS >ATTRIB6
   ['] >BOLD   DUP IS >ATTRIB3  IS >ATTRIB7
   ['] >REV    DUP IS >ATTRIB4 IS >ATTRIB8
   >NORM ;

' >MONO IS INITMONO              \ Enable attributes according to video board.

' >MONO IS INITCOLOR             \ Enable attributes of color card.
```

The following words allow the user to select white-on-black or black-on-while display.

```
: invert-screen          ( --- )
```

```
   blank.color
   video-seg @ rows cols 2* * 1
   do     dup i c@l dup NORMVAL =
     if drop REVVAL over i c!l
     else   REVVAL =
       if NORMVAL over i c!l
       then
     then
   2 +loop
   drop show.color ;

: white-on-black          ( --- )
   NORMVAL NORMALVAL <>
   if        invert-screen
   then
   NORMALVAL  =: NORMVAL
   REVERSEVAL =: REVVAL
   >norm ;

: black-on-white          ( --- )
   NORMVAL REVERSEVAL <>
   if  invert-screen
   then
    REVERSEVAL =: NORMVAL
   NORMALVAL  =: REVVAL
    >norm ;
```

7.13. COLOR VIDEO WORDS

Source file is COLOR.SEQ. For an IBM Color Videod isplay Card, the following values may be used for background or forground colors.

```
$00 CONSTANT BLACK            \ black
$01 CONSTANT BLUE             \ blue
$02 CONSTANT GREEN            \ green
$03 CONSTANT CYAN             \ cyan
$04 CONSTANT RED              \ red
$05 CONSTANT MAGENTA          \ magenta
$06 CONSTANT BROWN            \ brown
$07 CONSTANT LTGRAY           \ light gray
$08 CONSTANT DKGRAY           \ dark gray
```

All following, blink in background.

```
$09 CONSTANT LTBLUE           \ light blue
$0A CONSTANT LTGREEN          \ light green
$0B CONSTANT LTCYAN           \ light cyan
$0C CONSTANT LTRED            \ light red
$0D CONSTANT LTMAGENTA        \ light magenta
$0E CONSTANT YELLOW           \ yellow
$0F CONSTANT WHITE            \ white
```

>FG Set the Forground color to value n1.

```
: >FG      ( n -- )
   attrib c@          \ Fet attribute byte
   $F0 and +          \ Mask off background bits
```

```
   attrib c! ;        \ Put back
```

>BG Set the Background color to value n1.

```
: >BG      ( n -- )
   $10 * attrib c@            \ Move to background field
   $0F and + attrib c! ;      \ Mask and put back to STTRIB
```

COLORSET Set the current screen colors to be the default normal color combination.

```
: COLORSET     ( -- )
   attrib c@ =: normval ;
```

>BUGN Set the backgound to BLUE, and forground to green.

```
: >BUGN ( -- ) BLUE  >BG   GREEN >FG ;
```

>RDWT Set the background to RED, and forground to WHITE.

```
: >RDWT ( -- ) RED   >BG   WHITE >FG ;
```

>BUWT Set the Background to Green, and Forground to WHITE.

```
: >BUWT    ( -- ) GREEN >BG   WHITE >FG ;
```

>COLOR Select highlighting for color monitor. Initialize all attributes for the color video card.

```
: >COLOR   ( -- )
   ['] >BUGN DUP IS >ATTRIB1   IS >ATTRIB5
```

```
   ['] >RDWT DUP IS >ATTRIB2   IS >ATTRIB6
   ['] >BUWT DUP IS >ATTRIB3   IS >ATTRIB7
   ['] >RDWT DUP IS >ATTRIB4   IS >ATTRIB8
   >NORM ;

' >COLOR IS INITCOLOR          \ Enable attributes according to video board.

>LCD       Liquid Crystal Display, or Least Common Denominator.   Your choice.

: >LCD        ( --- )
   ['] >REV  DUP IS >ATTRIB1 DUP IS >ATTRIB5
   DUP IS >ATTRIB2 DUP IS >ATTRIB6
   DUP IS >ATTRIB3     IS >ATTRIB7
   ['] >NORM DUP IS >ATTRIB4     IS >ATTRIB8
   >NORM ;
```

7.14. SAVE SCREENS

Source File: SAVESCR.SEQ

F-PC manages many screens of useful information to help the user while running the editor, the debugger, and many on-line help utilities. A common need is to save the current screen display, replace it with some other help screens, and later restore the original screen to continue normal work. This save screen utility provides a memory area to save up to three screenful of data. The saving process is nestable to three level deep.

```
0 value svseg          \ screen save segment
4 value svmax                  \ maximum number of screens to save
4000 value svsize                  \ size of each screen
```

```
0 value svtotal                 \ total of saved screens
```

svinit Cold start initialization word that allocates 64K bytes of space for the screen save segment.

```
: svinit    ( -- )              \ Screen save initialization
   defers initstuff
   off> svseg
   rows cols 2* * =: svsize            \ set save size per screen
   65535. svsize um/mod nip            \ maximum number of screens
   1- svmax min =: svmax               \ limit svsize to < 64k
   svsize svmax * =: svtotal           \ adjust total arraysize
   svmax 1+ svsize * paragraph
   alloc 8 =                \ allocate the space
   if 2drop cr
      ." Could not allocate memory for Screen save routine."
   else    nip =: svseg
   then    ;
' svinit is initstuff
```

savescr Save the current contents of the CRT screen to an external segment save area. Nestable up to four (SAVMAX) times.

```
: savescr   (-- )
   svseg 0
   2dup svsize + svtotal cmovel>       \ Make a hole in buf
   blank.color
   video-seg @ 0
   svseg 0 svsize cmovel           \ & save screen in it.
   show.color ;
```

restscr Restore a saved screen from the external screen save segment. Un-nestable up to three levels.

```
: restscr   ( -- )
   svseg 0= ?exit
   blank.color
   svseg 0
   video-seg @ 0 svsize cmovel     \ Restore screen &
   show.color
   svseg svsize
   over 0 svtotal cmovel ;    \ compress buffer.
```

RECOVERSCR Make a copy of the last saved screen and then restore it.

```
: recoverscr    ( -- )
   svseg 0= ?exit                \ last saved screen
   blank.color
   svseg 0
   video-seg @ 0 svsize cmovel         \ Restore screen
   show.color ;
```

RECOVERLINE Get a copy of line n1 from the last saved screen.

```
: recoverline   ( n1 --- )
   svseg 0= if drop exit then
   dup 0 rows 1- between
   if         cols 2* * >r
              blank.color     \ Restore line n1
              svseg r@
      video-seg @ r> cols 2* cmovel
```

```
        show.color
  else      drop
  then   ;
```

7.15. TIMER WORDS

Words in TIMER.SEQ and TIMESTUFF.SEQ deal with time. As we envision that many user will use F-PC for real-time applications, a complete set of time and date manipulation words has been included. They call appropriate DOS service functions to provide the requested services. The words for getting and setting the date and time are discussed in this section.

Time delay words use the DOS time function to obtain very accurate time delays. Background processing continues, as pause is called in the wait loop. Another deferred word, PAUSE-FUNC, is also in the loop, which can be re-assigned to perform any function you may want done while the delay is occurring.

GETDATE Return d1 the 32 bit binary date from the operating system. Bytes in d1 are arranged in the order of year/month/day/day-of-week.

```
: GETDATE   ( -- d1 )
  0 0 42 OS2        \ DOS service 42
  DROP ;
```

SETDATE Given the binary date d1, set the system clock to that date.

```
: SETDATE   ( d1 -- )
  SWAP 43 OS2          \ Update date
  NIP NIP 255 =        \ d1 might be invalid
  IF  CR
    ." Invalid DATE "
  THEN ;
```

GETTIME Return d1 with bytes arranged in the order hr/min/sec/100th-sec from the operating system.

```
: <GETTIME> ( -- d1 )
   0 0 44 OS2
   DROP ;
```

```
DEFER GETTIME  ' <GETTIME> IS GETTIME
```

SETTIME Given the binary time d1, set the system clock to that time.

```
: SETTIME   ( d1 -- )
   SWAP 45 OS2
   NIP NIP 255 =
   IF  CR
      ." Invalid TIME "
   THEN ;
```

```
CREATE DTBUF 16 ALLOT       \ A buffer to store date or time string for output
```

##+ Convert the two lowest digits in n1 and add the digits to DTBUF.

```
: ##+      ( n1 -- )
   0 <# # # #>            \ Convert two digits of n1
   TUCK DTBUF COUNT + \ End of string in DTBUF
   SWAP CMOVE         \ Append two digits
   DTBUF C+! ;        \ Update string count
```

"+ Append a string at addr of n bytes to the string in DTBUF.

```
: "+       ( addr n -- )
   TUCK DTBUF COUNT + \ See comments above
   SWAP CMOVE
   DTBUF C+! ;
```

BUILD-HM Append the hour:minute string converted from n1 into DTBUF.

```
: BUILD-HM  ( n1 -- )
   SPLIT            \ Separate minute from hour
   ##+              \ Convert hour
   " :" "+          \ Insert a :
   ##+ ;            \ Convert minute
```

BUILD-SH Append the second.tick string converted from n1 to the string in DTBUF.

```
: BUILD-SH  ( n1 -- )
   " :" "+          \ Add separator
   SPLIT ##+        \ Convert seconds
   " ." "+          \ Insert separator
   ##+ ;            \ Convert ticks
```

BUILD-TIME Add the hour:minute:second.tick string to the string in DTBUF. The time is represented by d1 on the stack.

```
: BUILD-TIME  ( d1 -- )
   SWAP             \ Make it Sh and HM
   BUILD-HM         \ Convert hour:minute
   BUILD-SH ;       \ Convert second.tick
```

FORM-TIME Build up a time string in DTBUF from the time spec d1 on the stack. Return the address of DTBUF.

```
: FORM-TIME ( d1 -- addr )
   BASE @ >R DECIMAL      \ Have to used decimal base
   DTBUF OFF          \ Init null string in DTBUF
   BUILD-TIME         \ Add time string
   R> BASE !          \ Restore base
   DTBUF ;        \ Push address of DTBUF
```

.TIME Print the current time to the screen, in the format HH:MM:SS.HR, where HH is in hours, MM in minutes, SS in seconds, and HR in hundredths of a second.

```
: .TIME ( -- )
   GETTIME            \ What time is it?
   FORM-TIME          \ Convert to a time string
   COUNT TYPE ;       \ Print it
```

The time string format is fairly universal as HH:MM:SS.HR. However, the representation of month and day shows many variations. Here F-PC gives you three choices, which are quite obvious from their names:

```
: <M/D/Y>   ( d1 -- )
   SPLIT              \ Split day byte from month byte
   ##+                \ Convert month
   " /" "+              \ Add separator
   ##+                \ Convert day
   " /" "+              \ Add separator
   ##+ ;              \ Convert year

: <Y-M-D>   ( d1 -- )
   SPLIT ROT          \ day month year --
```

```
   ##+ " -" "+          \ Convert year and add /
   ##+ " -" "+          \ Convert month and add /
   ##+ ;            \ Convert day

: <D.M.Y>   ( d1 -- )
   SPLIT SWAP        \ day month year --
   ##+ " ." "+          \ Convert day
   ##+ " ." "+          \ Convert month
   ##+ ;            \ Convert year
```

We are sure that these choice will not be the one you will want to use. Thus BUILD-DATE is defined as a deferred word so that you can make the correct choice and vector BUILD-DATE to it.

```
DEFER BUILD-DATE
' <M/D/Y> IS BUILD-DATE     \ Our default choice

: M/D/Y ( -- )  ['] <M/D/Y> IS BUILD-DATE ;

: Y-M-D ( -- )  ['] <Y-M-D> IS BUILD-DATE ;

: D.M.Y ( -- )  ['] <D.M.Y> IS BUILD-DATE ;
```

DORM-DATE Convert the date spec d1 into a date string in DTBUF. Return the address of DTBUF.

```
: FORM-DATE ( d1 -- addr )
   BASE @ >R DECIMAL      \ Must use decimal base
   DTBUF OFF         \ Start with a null string
   BUILD-DATE        \ Add date string
   R> BASE !         \ Restore base
   DTBUF ;       \ Return with DTBUF address
```

.DATE Print the current date to the screen, in the format MM/DD/YY, where MM is month, DD is day, and YY is year.

```
: .DATE ( -- )
   GETDATE          \ Ask Ms. DOS what day it is.
   FORM-DATE        \ Build the date string in DTBUF
   COUNT TYPE ;     \ Print it
```

A special word set is defined in F-PC to measure elapsed time for real time experiments and for characterization of program performance.

```
VARIABLE STIME 0 ,       \ Starting time
VARIABLE TTIME 0 ,       \ Buffer to store a time spec
```

T>B Convert a DOS time spec d1 to a double integer in the units of 100th of a second, or a tick. This binary representation is used to compute time difference.

```
: T>B    ( d1 -- d2 )
   SPLIT 100 * +        \ tick+100*second
   SWAP 0 SWAP          \ ticks 0 hh:mm --
   SPLIT >R 6000 *D     \ Convert minutes
   D+ R> 1000 *         \ Convert hour
   360 *D D+ ;          \ Add then altogether
```

B>T Inverse of T>B. Convert a double integer ticks back into the DOS time specification stored in TTIME.

```
: B>T    ( d1 -- )
   0 100 UM/MOD >R      \ Extract ticks
   100 UM/MOD           \ Extract seconds
   SWAP TTIME C!        \ Store ticks
```

```
   R> 60 UM/MOD       \ Extract minutes
   SWAP TTIME 1+ C!       \ Store seconds
   60 /MOD        \ Extract hours
   TTIME 3 + C!           \ Store hours
   TTIME 2+ C! ;          \ Store minutes
```

TIME-RESET Reset the accumulated time value in the double variable STIME to zero, in effect resetting the current elapsed time to zero. This word is used at the beginning of a sequence of operations you want to time. The word .ELAPSED is used at the end of the operations to print the elapsed time since the last TIME-RESET.

```
: TIME-RESET    ( -- )
   GETTIME T>B           \ Get time in binary form
   STIME 2! ;            \   and store in STIME
```

TIME-ELAPSED Get current time and find the difference from the time in STIME in units of ticks, 100th of a second.

```
: TIME-ELAPSED  ( -- d1 )
   GETTIME T>B           \ Get current time in ticks
   STIME 2@          \ Last ticks
   D- ;              \ Difference in ticks
```

B>SEC Convert the double integer ticks to seconds. Overflow occurs at about 18 hours.

```
: B>SEC ( d1 -- n1)
   0 100 UM/MOD DROP      \ Discard overflow
   100 UM/MOD NIP ;       \ Retain only seconds
```

.ELAPSED Print the elapsed time since the last TIME-RESET was performed.

```
<.ELAPSED>  ( -- )
```

```
   TIME-ELAPSED       \ Get elapsed ticks
   B>T                \ Convert ticks to DOS time
   TTIME 2@ FORM-TIME \ Convert to HH:MM:SS:HR spec
   COUNT TYPE ;       \ Print the time string

: .ELAPSED  ( -- )
   CR ." Elapsed time   =  "
   <.ELAPSED> ;
```

TIMER Perform the Forth words following on the same command line, and when they finish execution, print the elapsed time used for their execution.

```
: TIMER ( words -- )
   TIME-RESET         \ Reset timer
   INTERPRET          \ Execute following words
   .ELAPSED ;         \ Display elapsed time
```

SEC-ELAPSED Return n1 the seconds that have elapsed since TIME-RESET.

```
: SEC-ELAPSED      ( -- n1 )
   TIME-ELAPSED       \ Elapsed ticks
   B>SEC ;        \ Seconds within 18 hours
```

10TH-ELAPSED Return n1 the tenths of a second that have elapsed since the last TIME-RESET was performed.

```
: 10TH-ELAPSED  ( -- n1 )
   TIME-ELAPSED       \ Elapsed ticks
   0 100 UM/MOD DROP
   10 UM/MOD NIP ;    \ Retain 10th of seconds
```

PAUSE-FUNC A deferred word set to a function that is to be performed while waiting for timing to complete.

```
DEFER PAUSE-FUNC       ' NOOP IS PAUSE-FUNC
```

SECONDS Wait n1 seconds.

```
: SECONDS   ( n1 -- )
   STIME 2@ 2>R >R        \ Save starting time
   TIME-RESET         \ Clear STIME for counting
   BEGIN
      PAUSE  PAUSE-FUNC   \ Pause for task switching
      R@ SEC-ELAPSED > 0= \ Elapsed seconds > n1 ?
   UNTIL   R>DROP      \ If so, exit.
   2R> STIME 2! ;      \ Restore STIME
```

TENTHS Wait n1 tenths of a second.

```
: TENTHS    ( n1 -- )
   STIME 2@ 2>R >R        \ Similar to SECONDS
   TIME-RESET
   BEGIN   PAUSE
      PAUSE-FUNC
      R@ 10TH-ELAPSED     \ Except count 10th of seconds
      > 0=
   UNTIL   R>DROP
   2R> STIME 2! ;
```

MINUTES Wait n1 minutes.

```
: MINUTES   ( n1 -- )
   0 MAX   0
   ?DO   60 SECONDS       \ Let SECONDS do the work
   LOOP ;
```

HOURS   Wait n1 hours.

```
: HOURS ( n1 -- )
   0 MAX   0
   ?DO 60 MINUTES      \ Let MINUTES do it
   LOOP ;
```

0COMPILER Start the timer to measure time required to compile files. F-PC was optimized using this tools to measure its performance.

```
: 0COMPILER ( -- )
   TOTALLINES OFF      \ Clear compiled line counter
   TIME-RESET ;          \ Clear timer
```

.COMPSTAT Display the compiling status of the compiler. It shows total compile time, total line compiled, and the compile speed in lines per minute.

```
: .COMPSTAT ( -- )
   cr .elapsed           \ Time used
   cr ." Total lines compiled  =   "
   totallines @ 5 u.r         \ Lines compiled
   cr ." Compiled lines/minute = "
   totallines @ time-elapsed       \ compute line/min
   over 32000 u> or
   if  " Too much time has elapsed to calculate LPM"
```

```
      2drop
   else    >R 60 10000 R@ */MOD    \ Elapsed mintes
      10 * SWAP DUP 10 >      \ It is just a simple
      IF  >R 100 100 100 \   divide.
         r> r> SWAP */ */
         10 /MOD  SWAP 4 >
         IF 1+ THEN +
      ELSE    r>drop DROP
      THEN
      2DUP 100 /
      SWAP 100 / *
      3276 >
      IF  10 / 1000 */
         10 *D 5 D.R     \ Print lines/min
      ELSE
         1000 */ 5 u.R
      THEN
   then    ;
```

TILLKEY Wait up to n1 seconds for a keystroke to continue execution.

```
: TILLKEY   ( n1 -- )
   KEY?
   IF  DROP EXIT  THEN         \ Leave if a key is pressed
   CR ."  Waiting, press SPACEBAR to continue.."
   0 MAX 0
   ?DO KEY?           \ Leave at any key
      ?LEAVE
      1 SECONDS       \ Wait a second
```

```
LOOP    KEY?
IF  KEY 3 =         \ If key is Break
   ABORT" Quitting "   \   then abort
THEN ;
```

7.16. COMMENTS

Source code is in COMMENT.SEQ. All comment words in F83, like (, .(, \, \S are preserved and they behave similarly in F-PC. However, a file in F-PC is similar to a block in F83 and \S stops the compilation of the rest of the file. To accommodate documentation which spans over many lines, additional comment words are defined in F-PC. Multiple line comment in F-PC starts with COMMENT: and terminates at COMMENT; . It looks like:

```
COMMENT:
<text>
<more text>
...
<even more text>
COMMENT;
```

To print multiple line comments during compiling,

```
.COMMENT:
...
COMMENT;
```

.COMMENT; starts a group of lines that are to be printed to the terminal, until a terminating "COMMENT;" is found.

Plenty of documentation tools are provided in F-PC. There is no excuse not using them to beautify your program.

' <run> value crunsave

?EXEC Abort if still compiling.

```
: ?exec    ( -- )
   state @ abort" Can not be used while Compiling!" ;
```

?LOADING Abort if not loading.

```
: ?loading    ( --- )
   loading @ 0=
   if crunsave is run
      true abort" Use ONLY while loading"
   then   ;
```

```
variable comment$
```

<comment:> Primitive used by "comment:".

```
: <comment:>   ( -- )
   loading @ 0=              \ Abort if not loading.
   if crunsave is run
      true abort" COMMENT: missing a COMMENT; while loading"
   then
   bl word              \ get a word
   1+ comment$ @ count caps-comp 0=    \ terminating string?
   if crunsave is run\ then back to <RUN>
     crunsave execute          \ Resume loading.
   then   [compile] \ ;          \ Otherwise ignore text.
```

comment: Start a multi-line comment. Patches <comment:> into RUN, it then reads and throws away lines until "comment;" is encountered.

```
: comment:  ( -- )
   ?loading
   [compile] \        \ Ignore this line
   " comment;"           \ Terminating string
   ">$ comment$ !        \ Make COMMENT$ pointing to it
   @> run   !> crunsave       \ Save current interpreter and switch to <COMMENT:>.
   ['] <comment:> is run ;     \ Make the interpreter to run <COMMENT:>, which skipps
               \   all lines till COMMENT;
```

/* Similar to COMMENT: . Skip all lines until */ . This is an attempt to fool C and PL/I users to think that they are reading something they know. In fact it is true, because comments are universal in all programming languages. They might even be in English.

```
: /*       ( -- )
   ?loading
   [compile] \        \ See COMMENT:
   " */" ">$ comment$ !
   @> run   !> crunsave       \ Save current interpreter and switch to <COMMENT:>.
   ['] <comment:> is run ;
```

<.comment:> A primitive used by ".comment:".

```
: <.comment:>   ( -- )        \ Similar to <COMMENT:>
   loading @ 0=           \ Abort if not loading
   if  crunsave is run
      true abort" .COMMENT: missing a COMMENT; while loading"
   then
   bl word 1+         \ Compare next word with
   " comment;" caps-comp 0=    \   COMMENT;
```

```
   if  crunsave is run     \ If equal, resume loading
   else    key? 0=         \ Not equal,
      if  cr tib      \   print this line
         #tib @
         78 min type
      then
   then    [compile] \ ;       \   and then ignore it
```

.comment: Start a multi-line print statment. All lines following the ".comment:" line will be printed to the display until a "comment;" is encountered.

```
: .comment: ( -- )
   ?loading
   [compile] \         \ Ignore this line
   @> run  !> crunsave
   ['] <.comment:> is run ;        \ Display comments until  COMMENT;
```

7.17. CONDITONAL COMPILATION

Conditional compilation words are also defined in the COMMENT.SEQ file. Their function is to allow part of the source file to be loaded depending upon the condition exists at the loading time. For example, if a floating point word exists in the dictionary, the entire floating point package must have been loaded. Consequently, the floating point package needs not be reloaded. The condition can be tested during loading, and the loading sequence does not have to be specified precisely before you start loading files.

<#if> A primitive used by #IF. Ignore lines until #ELSE or #ENDIF.

```
: <#if>     ( -- )
   loading @ 0=
```

```
  if crunsave is run
    true abort" #IF missing a #THEN while loading"
  then
  begin   bl word dup c@ 0= >r 1+
    dup " #endif" caps-comp 0= over
          " #then"  caps-comp 0= or swap
          " #else"  caps-comp 0= or dup
    if crunsave is run
       >r crunsave execute r>
     then   r> or
  until   ;
```

#ELSE Start a false multi-line structure.

```
: #else    ( -- )
  ?loading                \ you must be loading
  ?exec                   \ but NOT compiling
  @> run !> crunsave              \ Save previous value of RUN.
  ['] <#if> is run\ Ignore following lines
  <#if> ;       immediate         \   and rest of this line.
```

#if Starts a multi-line compiler directive. If the Boolean flag passed to #IF is true, then the lines following #IF will be executed. If the flag is false, then the lines following #IF up to the #ENDIF or #ELSE, will be ignored. Here is an example of how #IF works:

```
  TRUE   ( -- f1 )
  #IF    True flag passed, this comment needs no preceeding \
    cr .( This is printed due to TRUE before #IF.)
  #ENDIF
```

```
   FALSE   ( -- f1 )
   #IF
      .( This is NOT printed due to FALSE flag in #IF.)
   #ENDIF

: #if      ( f -- )
   ?loading
   ?exec
   0=
   if @> run !> crunsave  \ Save previous value of RUN.
      ['] <#if> is run        \ Ignore following lines
      <#if>          \   and rest of this line.
   then    ;      immediate

#endif  Just a noop for the #if TRUE case.  Ends a multi-line Compiler directive.  See #IF.

: #endif    ( -- )  ?loading  ; immediate

: #then     ( -- )  ?loading  ; immediate
```

Here is an example of how the interpreted compiler directives work.

```
   cr cr .( press a T for true )
   key bl or 116 =         \ wait for a key
   #if .( TRUE)
   #else .( FALSE)
   #then .(  always )
```